

DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way



By Ritesh Kumar Singh

Email Address: - riteshkumarsingh9559@gmail.com

LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/>

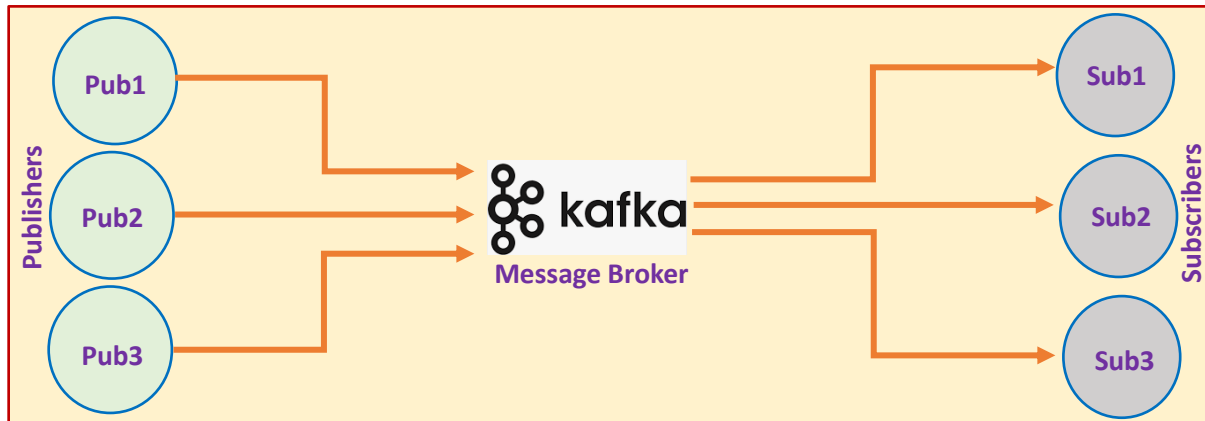
GitHub: - <https://github.com/singhritesh85>



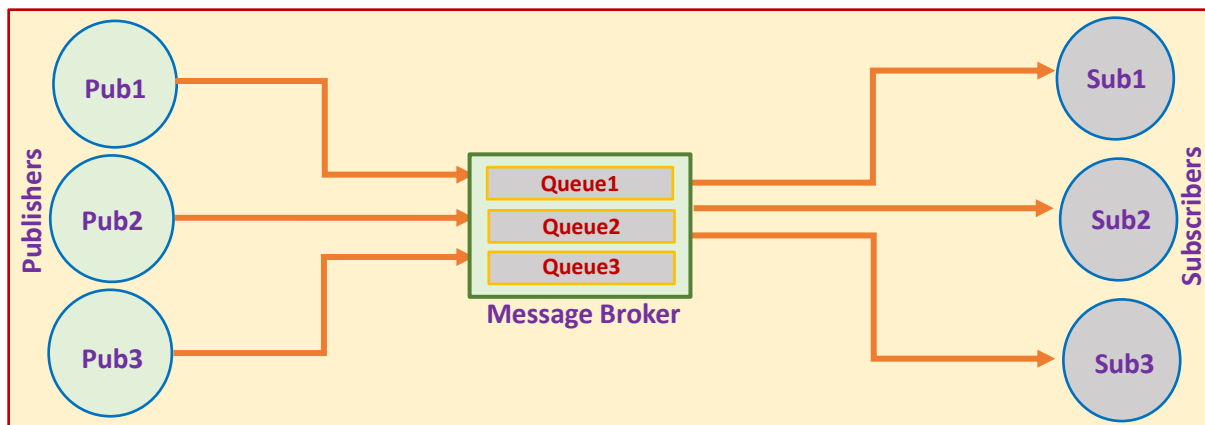
या कुन्देन्दुतुषारहारधवला या शुभ्रवस्त्रावृता
या वीणावरदण्डमण्डितकरा या श्वेतपद्मासना।
या ब्रह्माच्युत शंकरप्रभृतिभिर्देवैः सदा वन्दिता
सा मां पातु सरस्वती भगवती निःशेषजाड्यापहा ॥

DevOps Project using Kafka as MessageQueue the GitOps way Multicloud (AWS, Azure and GCP)

MQTT (Message Queuing Telemetry Transport) MQTT is a light weight publish-subscribe messaging protocol designed for high-latency, low bandwidth and unreliable networks. In DevOps it acts as highly efficient, real-time messaging transport for microservices based systems which facilitates microservice-to-microservice communication through a central broker. Publisher will publish messages which will go to message broker and message broker will route the messages to the particular subscriber.



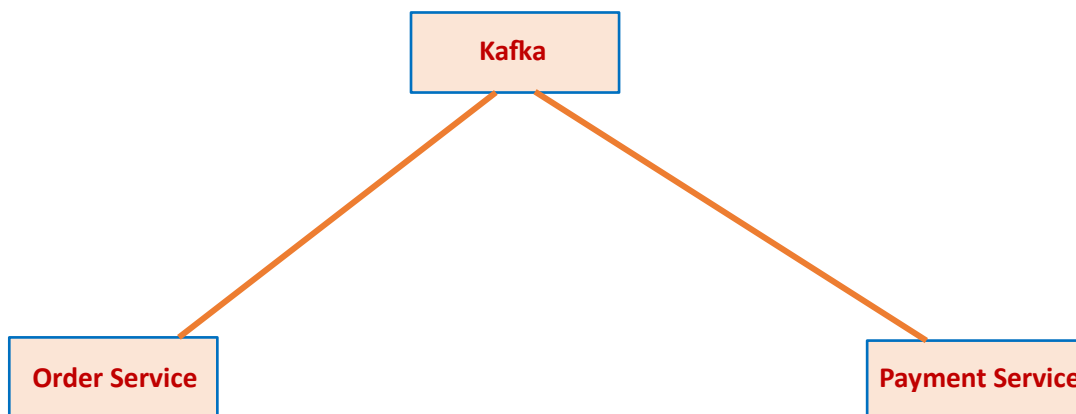
The message broker maintained several queues parallelly and subscribers are subscribed to those queues. When messages arrive, subscriber subscribed to particular queue will consume those messages.



A question may arise in readers mind that what is message or event. Answer is an activity will generate an event; as for example consider a user will login or logout this is nothing but an activity which will generate an event. Without using Kafka, the different services in a microservice are tightly coupled. Consider in a microservice there are multiple services including Order Service and Payment Service, as payment service depends on the order service and hence the Order service is tightly coupled to payment service. But when Kafka is introduced then services will communicate with each other using kafka.



In the diagram as drawn above if the Order Service fails the process cannot continue.



In the diagram drawn above the Payment service does not depend on the Order Service. The Payment service gets the information from Kafka and it is not tightly coupled with order service.

Core-Components of Kafka

1. Events (Messages): - An activity generate an event in kafka.
2. Producer: - Produces are those which publish or produce the messages or event in kafka. In the diagram drawn above order service is the producer.
3. Topic: - Topics are nothing but category of events. A producer will write into the topic and a consumer will read from the topic.
4. Partition: - In kafka inside a Topic there will be multiple partitions. As for example in the scenario explained above there is a topic called order, inside that topic there are several partitions as order-0, order-1, order-2 etc. This will help kafka to handle more traffic though parallel processing in case of more traffic.
5. Offset: - Unique sequential ID for messages. In the above example the order id generated with each order are nothing but the offset. Here the Order ID 101, 102 etc. are offset.

Order		
Oder-0	Order-1	Order-2
101	102	103
104	105	106
107	108	109

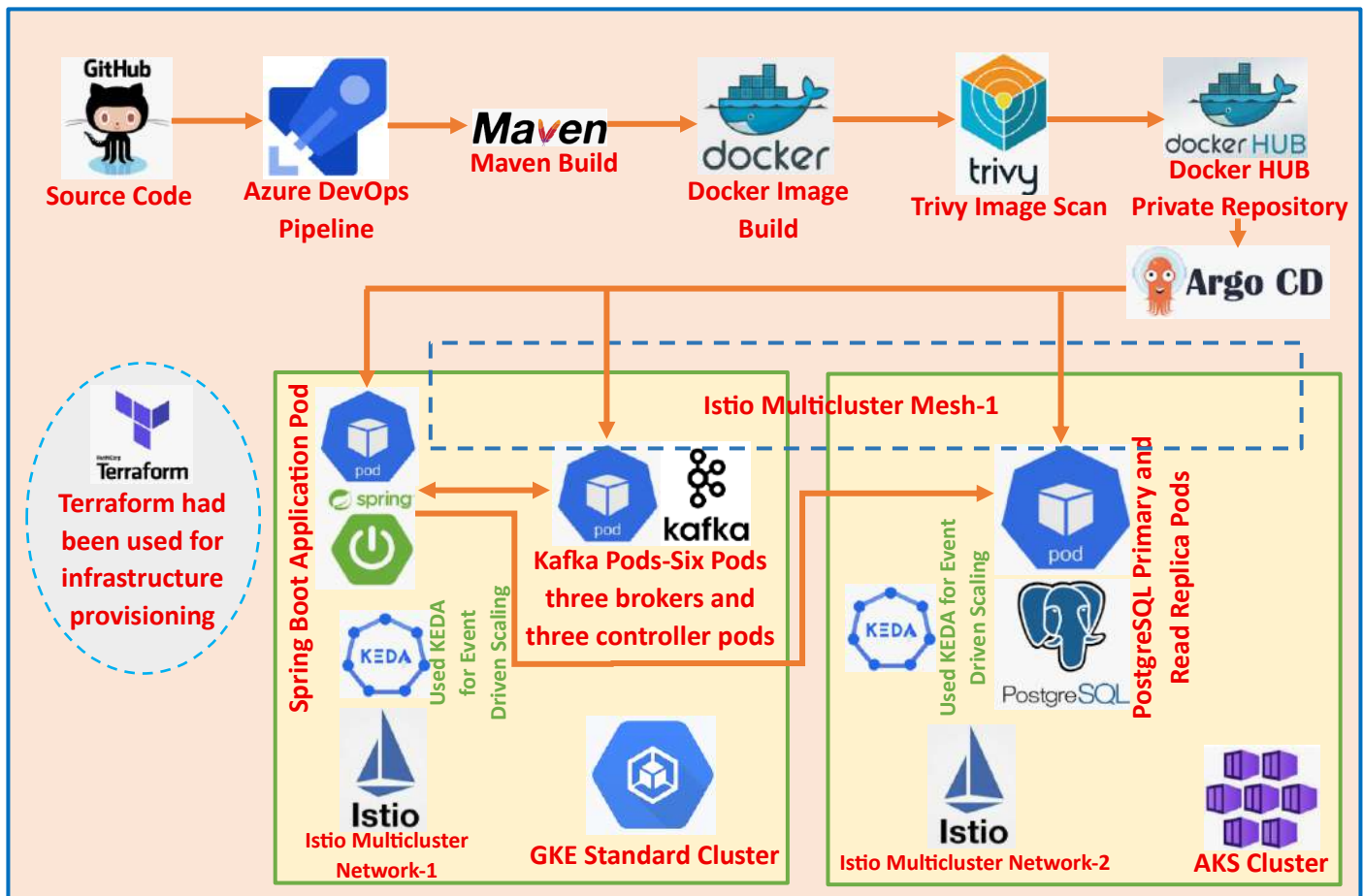
6. Subscriber: - Consumers or Subscribers are those which read from the Kafka. In the above example the payment service is the consumer.
7. Broker and Controller: - A Broker in Kafka is responsible for receives messages from publishers, stores messages into partitions, sends messages to consumers and replicate messages between brokers. However, the controller is the special broker responsible for cluster coordination.
8. Kafka Lag: - Kafka Lag is the difference between the last message produced to the topic and the last message processed by the consumer.

Apache ZooKeeper is an external distributed coordination service used by Apache Kafka for centralized metadata management and cluster coordination. KRaft is Kafka's built-in metadata

management mode based on the Raft consensus protocol, used for centralized metadata management and cluster coordination without requiring ZooKeeper. Starting from Kafka 3.3.1, KRaft became production-ready for new Kafka clusters and began replacing ZooKeeper. I can conclude here with the note that kafka is an opensource distributed event streaming platform.

Module-1

DevOps Project using Kafka as MessageQueue the GitOps way Multicloud-Multiclustere (Azure and GCP) Deployed Kafka using Bitnami Helm Chart



This DevOps Project deals with creation of infrastructure using Terraform script present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-1/terraform-aks-cluster-and-gke-cluster-public**, in this project the source code was present in the GitHub Repo <https://github.com/singhritesh85/k8s-springboot-kafka-microservice.git>. CI/CD pipelines are implemented using Azure DevOps Pipelines and Maven had been used as the Build Tool. Then Docker Image had been created and scanned using Aquasecurity Trivy. Then the scanned Docker Image was pushed to Docker Hub Private Repository and finally Spring Boot Application Pod had been created. The Order Service will take an order and stores them in PostgreSQL database and sends an order created event to Kafka. The Payment Service will receive the order created event through Kafka and process the payment and sends payment completed event to kafka. PostgreSQL database acts as persistent storage for Orders. Implemented Istio-based multiclustere connectivity between GKE Standard and AKS Kubernetes clusters. Finally, event-driven autoscaling of Kubernetes pods was implemented using KEDA.


```
[root@Terraform-Server ~]# az login
[... redacted ...]
{
  "cloudName": "AzureCloud",
  "homeTenantId": "[redacted]",
  "id": "[redacted]",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Pay-As-You-Go",
  "state": "Enabled",
  "tenantId": "[redacted]",
  "user": {
    "name": "[redacted]@gmail.com",
    "type": "user"
  }
}
```

Terraform script to provision the infrastructure is present at the path **module-1/terraform-aks-cluster-and-gke-cluster-public** in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>. Commands as written below had been used to create infrastructure using Terraform.

terraform init -----> initializes a working directory containing configuration files and installs plugins for required providers.

terraform validate -----> verify that terraform configuration file is correct or not

terraform plan -----> Check which resources are going to be created.

Then you can run the command **terraform apply -auto-approve** -----> Finally, Create the resources.

```
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [01m00s elapsed]
module.aks_cluster_and_standard_gke_cluster.azure_rm_kubernetes_cluster_node_pool.autoscale_node_pool: Still creating... [03m20s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [01m10s elapsed]
module.aks_cluster_and_standard_gke_cluster.azure_rm_kubernetes_cluster_node_pool.autoscale_node_pool: Still creating... [03m30s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [01m20s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Creation complete after 1m23s [id=projects/[redacted]/locations/us-central1/clusters/multicloud-gke-cluster/nodePools/multicloud-linux-nodepool-1]
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1: Creating...
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1: Provisioning with 'local-exec'...
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1 (local-exec): Executing: ["/bin/sh" "-c" "gcloud container clusters get-credentials multicloud-gke-cluster --location=us-central1 --project=[redacted] && chmod 600 ~/.kube/config"]
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1 (local-exec): WARNING: Accessing a Kubernetes Engine cluster requires the kubernetes com
mandline
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1 (local-exec): client [kubect1]. To install, run
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1 (local-exec): $ gcloud components install kubect1
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1 (local-exec): Fetching cluster endpoint and auth data.
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1 (local-exec): CRITICAL: ACTION REQUIRED: gke-gcloud-auth-plugin, which is needed for con
tinued use of kubect1, was not found or is not executable. Install gke-gcloud-auth-plugin for use with kubect1 by following https://cloud.google.com/kubernet
s-engine/docs/how-to/cluster-access-for-kubect1#install_plugin
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1 (local-exec): kubeconfig entry generated for multicloud-gke-cluster.
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubect1: Creation complete after 2s [id=[redacted]]
module.aks_cluster_and_standard_gke_cluster.azure_rm_kubernetes_cluster_node_pool.autoscale_node_pool: Creation complete after 3m40s [id=subscriptions/[redacted]
/resourceGroups/multicloud-rg/providers/Microsoft.ContainerService/managedClusters/multicloud-aks-cluster/agentPools/userpool]
module.aks_cluster_and_standard_gke_cluster.null_resource.kubect1_aks: Creating...
module.aks_cluster_and_standard_gke_cluster.null_resource.kubect1_aks: Provisioning with 'local-exec'...
module.aks_cluster_and_standard_gke_cluster.null_resource.kubect1_aks (local-exec): Executing: ["/bin/bash" "-c" "az account set --subscription $(az account s
how --query id | tr -d '\n') && az aks get-credentials --resource-group multicloud-rg --name multicloud-aks-cluster --overwrite-existing && chmod 600 ~/.kube/co
nfig"]
module.aks_cluster_and_standard_gke_cluster.null_resource.kubect1_aks (local-exec): WARNING: Merged "multicloud-aks-cluster" as current context in /root/.kube
/config
module.aks_cluster_and_standard_gke_cluster.null_resource.kubect1_aks: Creation complete after 1s [id=[redacted]]

Apply complete! Resources: 55 added, 0 changed, 0 destroyed.

Outputs:
azure_managed_identity_client_id_and_azure_dns_zone_nameservers = <sensitive>
```

```
[root@Terraform-Server main]# terraform output -json
{
  "azure_managed_identity_client_id_and_azure_dns_zone_nameservers": {
    "sensitive": true,
    "type": [
      "object",
      {
        "azure_managed_identity_client_id_aks": "string",
        "azuread_service_principal_client_id_gke": "string",
        "azuread_service_principal_client_secret_gke": "string",
        "dns_zone_name": "string",
        "name_servers": [
          "set",
          "string"
        ]
      }
    ],
    "value": {
      "azure_managed_identity_client_id_aks": "[REDACTED]",
      "azuread_service_principal_client_id_gke": "[REDACTED]",
      "azuread_service_principal_client_secret_gke": "[REDACTED]",
      "dns_zone_name": "singhritesh85.com",
      "name_servers": [
        "[REDACTED]",
        "[REDACTED]",
        "[REDACTED]",
        "[REDACTED]"
      ]
    }
  }
}
```

After login on k8s-managent-node for the first time run the below command (**gcloud auth login**) to authenticate and authorize GCP Account and after creation of the resources a common kubeconfig file was generated for both the GKE and AKS Cluster using the same kubeconfig file it was possible to access the kubernetes clusters (GKE and AKS Clusters).

```
[demo@demo-azure-vm ~]$ gcloud auth login
Go to the following link in your browser, and complete the sign-in prompts:
[REDACTED]

Once finished, enter the verification code provided in your browser: [REDACTED]

You are now logged in as [REDACTED]@gmail.com].
Your current project is [None]. You can change this setting by running:
$ gcloud config set project PROJECT_ID

[demo@demo-azure-vm ~]$ kubectl config get-contexts
CURRENT  NAME                                     NAMESPACE                                CLUSTER                                     AUTHINFO
gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_[REDACTED]_us-central1_multicloud-gke-cluster  gke_[REDACTED]_us
-centra11_multicloud-gke-cluster
* multicloud-aks-cluster                                multicloud-aks-cluster                                clusterUser_multicloud-rg_
multicloud-aks-cluster
[demo@demo-azure-vm ~]$ kubectl get nodes --context=gke_[REDACTED]_us-central1_multicloud-gke-cluster
NAME                                     STATUS  ROLES  AGE  VERSION
gke-multicloud-gke-c-multicloud-linux-[REDACTED]  Ready  <none>  46m  v1.34.6-gke.1307000
gke-multicloud-gke-c-multicloud-linux-[REDACTED]  Ready  <none>  46m  v1.34.6-gke.1307000
[demo@demo-azure-vm ~]$ kubectl get nodes --context=multicloud-aks-cluster
NAME                                     STATUS  ROLES  AGE  VERSION
aks-agentpool-[REDACTED]  Ready  <none>  53m  v1.34.6
aks-userpool-[REDACTED]  Ready  <none>  44m  v1.34.6
```

Create multicluster between AKS and GKE using Istio

Install Istioctl as shown in the screenshot attached below.

```
[demo@demo-azure-vm ~]$ curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.29.2 sh -
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed
100 101 100 101    0     0  2729      0 --:--:-- --:--:-- --:--:-- 2729
100 5124 100 5124    0     0 81333      0 --:--:-- --:--:-- --:--:-- 81333

Downloading istio-1.29.2 from https://github.com/istio/istio/releases/download/1.29.2/istio-1.29.2-linux-amd64.tar.gz ...
Istio 1.29.2 download complete!

The Istio release archive has been downloaded to the istio-1.29.2 directory.

To configure the istioctl client tool for your workstation,
add the /home/demo/istio-1.29.2/bin directory to your environment path variable with:
    export PATH="$PATH:/home/demo/istio-1.29.2/bin"

Begin the Istio pre-installation check by running:
    istioctl x precheck

Try Istio in ambient mode
    https://istio.io/latest/docs/ambient/getting-started/
Try Istio in sidecar mode
    https://istio.io/latest/docs/setup/getting-started/
Install guides for ambient mode
    https://istio.io/latest/docs/ambient/install/
Install guides for sidecar mode
    https://istio.io/latest/docs/setup/install/

Need more information? Visit https://istio.io/latest/docs/
```

```
[demo@demo-azure-vm ~]$ vim ~/.bashrc
```

```
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

# User specific environment
if ! [ "$PATH" =~ "$HOME/.local/bin:$HOME/bin:" ]
then
    PATH="$HOME/.local/bin:$HOME/bin:$PATH"
fi
export PATH

# Uncomment the following line if you don't like systemctl's auto-paging feature:
# export SYSTEMD_PAGER=

# User specific aliases and functions
JAVA_HOME=/usr/lib/jvm/java-17-openjdk-17.0.19.0.10-1.el8.x86_64
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/bin:/opt/apache-maven/bin:/opt/node-v16.0.0/bin:/usr/local/bin

# The next line updates PATH for the Google Cloud SDK.
if [ -f '/home/demo/google-cloud-sdk/path.bash.inc' ]; then . '/home/demo/google-cloud-sdk/path.bash.inc'; fi

# The next line enables shell command completion for gcloud.
if [ -f '/home/demo/google-cloud-sdk/completion.bash.inc' ]; then . '/home/demo/google-cloud-sdk/completion.bash.inc'; fi

PATH="$PATH:/home/demo/istio-1.29.2/bin"
```

Logout then login again then check with the command **istioctl version** that Istioctl installed or not.

```
[demo@demo-azure-vm ~]$ istioctl version
Istio is not present in the cluster: no running Istio pods in namespace "istio-system"
client version: 1.29.2
```

I created first the **istio-system** namespace in both the clusters (GKE and AKS Cluster) as shown in the screenshot attached below.

```
kubectl create ns istio-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl create ns istio-system --context=multicloud-aks-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl create ns istio-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/istio-system created
[demo@demo-azure-vm ~]$ kubectl create ns istio-system --context=multicloud-aks-cluster
namespace/istio-system created
```


To create the Root CA certificate and certificate for each of the clusters I used make command and I install the same as shown in the screenshot attached below.

```
sudo yum groupinstall 'Development Tools' -y
```

```
[demo@demo-azure-vm ~]$ sudo yum groupinstall 'Development Tools' -y
```

It will install Java-1.8 which you can uninstall later after creating the Root CA certificate and certificate for each of the clusters.

```
[demo@demo-azure-vm ~]$ java -version
openjdk version "1.8.0_492"
OpenJDK Runtime Environment (build 1.8.0_492-b09)
OpenJDK 64-Bit Server VM (build 25.492-b09, mixed mode)
```

```
mkdir certs
```

```
cd certs/
```

```
make -f ../istio-1.29.2/tools/certs/Makefile.selfsigned.mk root-ca
```

```
make -f ../istio-1.29.2/tools/certs/Makefile.selfsigned.mk multicloud-gke-cluster-cacerts
```

```
make -f ../istio-1.29.2/tools/certs/Makefile.selfsigned.mk multicloud-aks-cluster-cacerts
```

```
cd
```

```
[demo@demo-azure-vm ~]$ cd certs/
[demo@demo-azure-vm certs]$ make -f ../istio-1.29.2/tools/certs/Makefile.selfsigned.mk root-ca
generating root-key.pem
Generating RSA private key, 4096 bit long modulus (2 primes)
.....++++
.....++++
e is 65537 (0x010001)
generating root-cert.csr
generating root-cert.pem
Signature ok
subject=O = Istio, CN = Root CA
Getting Private key
[demo@demo-azure-vm certs]$ make -f ../istio-1.29.2/tools/certs/Makefile.selfsigned.mk multicloud-gke-cluster-cacerts
generating multicloud-gke-cluster/ca-key.pem
Generating RSA private key, 4096 bit long modulus (2 primes)
.....++++
.....++++
e is 65537 (0x010001)
generating multicloud-gke-cluster/cluster-ca.csr
generating multicloud-gke-cluster/ca-cert.pem
Signature ok
subject=O = Istio, CN = Intermediate CA, L = multicloud-gke-cluster
Getting CA Private Key
generating multicloud-gke-cluster/cert-chain.pem
Intermediate inputs stored in multicloud-gke-cluster/
done
rm multicloud-gke-cluster/cluster-ca.csr multicloud-gke-cluster/intermediate.conf
```

```
[demo@demo-azure-vm certs]$ make -f ../istio-1.29.2/tools/certs/Makefile.selfsigned.mk multicloud-aks-cluster-cacerts
generating multicloud-aks-cluster/ca-key.pem
Generating RSA private key, 4096 bit long modulus (2 primes)
.....++++
.....++++
e is 65537 (0x010001)
generating multicloud-aks-cluster/cluster-ca.csr
generating multicloud-aks-cluster/ca-cert.pem
Signature ok
subject=O = Istio, CN = Intermediate CA, L = multicloud-aks-cluster
Getting CA Private Key
generating multicloud-aks-cluster/cert-chain.pem
Intermediate inputs stored in multicloud-aks-cluster/
done
rm multicloud-aks-cluster/cluster-ca.csr multicloud-aks-cluster/intermediate.conf
[demo@demo-azure-vm certs]$ cd
```

```
[demo@demo-azure-vm ~]$ sudo yum remove -y java-1.8*
```

```
[demo@demo-azure-vm ~]$ java -version
openjdk version "17.0.19" 2026-04-21 LTS
OpenJDK Runtime Environment (Red_Hat-17.0.19.0.10-1) (build 17.0.19+10-LTS)
OpenJDK 64-Bit Server VM (Red_Hat-17.0.19.0.10-1) (build 17.0.19+10-LTS, mixed mode, sharing)
```

Then created the kubernetes secrets **cacerts** followed by installation of **istio-base**, **istiod** and **istio eastwest gateway** in the namespace **istio-system** in each of the kubernetes cluster (GKE Cluster and AKS Cluster) as shown in the screenshot attached below.

```
kubectl create secret generic cacerts --from-file=certs/multicloud-gke-cluster/ca-cert.pem --from-file=certs/multicloud-gke-cluster/ca-key.pem --from-file=certs/multicloud-gke-cluster/root-cert.pem --from-file=certs/multicloud-gke-cluster/cert-chain.pem -n istio-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl get ns istio-system && kubectl label ns istio-system topology.istio.io/network=network1 --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
helm repo add istio https://istio-release.storage.googleapis.com/charts
```

```
helm repo update
```

```
helm install istio-base istio/base -n istio-system --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
helm install istiod istio/istiod -n istio-system --set global.meshID=mesh1 --set meshConfig.defaultConfig.proxyMetadata.ISTIO_META_DNS_CAPTURE="true" --set pilot.env.ENABLE_MULTICLUSTER_HEADLESS="true" --set pilot.env.PILOT_ENABLE_EDS_FOR_HEADLESS_SERVICES="true" --set global.multiCluster.clusterName=multicloud-gke-cluster --set global.network=network1 --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

helm install istio-eastwestgateway istio/gateway -n istio-system --set name=istio-eastwestgateway --set networkGateway=network1 --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl get svc istio-eastwestgateway -n istio-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl create secret generic cacerts --from-file=certs/multicloud-gke-cluster/ca-cert.pem --from-file=certs/multicloud-gke-cluster/ca-key.pem --from-file=certs/multicloud-gke-cluster/root-cert.pem --from-file=certs/multicloud-gke-cluster/cert-chain.pem -n istio-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/cacerts created
[demo@demo-azure-vm ~]$ kubectl get ns istio-system && kubectl label ns istio-system topology.istio.io/network=network1 --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
istio-system      Active    20m
namespace/istio-system labeled
[demo@demo-azure-vm ~]$ helm repo add istio https://istio-release.storage.googleapis.com/charts
"istio" has been added to your repositories
[demo@demo-azure-vm ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "istio" chart repository
Update Complete. 🎉Happy Helming!🎉
[demo@demo-azure-vm ~]$ helm install istio-base istio/base -n istio-system --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

NAME: istio-base
LAST DEPLOYED: Tue May 12 15:46:23 2026
NAMESPACE: istio-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Istio base successfully installed!

To learn more about the release, try:
$ helm status istio-base -n istio-system
$ helm get all istio-base -n istio-system
```

```
[demo@demo-azure-vm ~]$ helm install istiod istio/istiod -n istio-system --set global.meshID=mesh1 --set meshConfig.defaultConfig.proxyMetadata.ISTIO_META_DNS_CAPTURE="true" --set pilot.env.ENABLE_MULTICLUSTER_HEADLESS="true" --set pilot.env.PILOT_ENABLE_EDS_FOR_HEADLESS_SERVICES="true" --set global.multiCluster.clusterName=multicloud-gke-cluster --set global.network=network1 --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME: istiod
LAST DEPLOYED: Tue May 12 15:46:29 2026
NAMESPACE: istio-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
"istiod" successfully installed!

To learn more about the release, try:
  $ helm status istiod -n istio-system
  $ helm get all istiod -n istio-system

Next steps:
* Deploy a Gateway: https://istio.io/latest/docs/setup/additional-setup/gateway/
* Try out our tasks to get started on common configurations:
  * https://istio.io/latest/docs/tasks/traffic-management
  * https://istio.io/latest/docs/tasks/security/
  * https://istio.io/latest/docs/tasks/policy-enforcement/
* Review the list of actively supported releases, CVE publications and our hardening guide:
  * https://istio.io/latest/docs/releases/supported-releases/
  * https://istio.io/latest/news/security/
  * https://istio.io/latest/docs/ops/best-practices/security/

For further documentation see https://istio.io website
```

```
[demo@demo-azure-vm ~]$ helm install istio-eastwestgateway istio/gateway -n istio-system --set name=istio-eastwestgateway --set networkGateway=network1 --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME: istio-eastwestgateway
LAST DEPLOYED: Tue May 12 15:46:33 2026
NAMESPACE: istio-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
"istio-eastwestgateway" successfully installed!

To learn more about the release, try:
  $ helm status istio-eastwestgateway -n istio-system
  $ helm get all istio-eastwestgateway -n istio-system

Next steps:
* Deploy an HTTP Gateway: https://istio.io/latest/docs/tasks/traffic-management/ingress/ingress-control/
* Deploy an HTTPS Gateway: https://istio.io/latest/docs/tasks/traffic-management/ingress/secure-ingress/
```

```
[demo@demo-azure-vm ~]$ kubectl get svc istio-eastwestgateway -n istio-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                     AGE
istio-eastwestgateway              LoadBalancer        172.19.91.239    104.186.150.186  15021:32757/TCP,15443:31578/TCP,15012:32656/TCP,15017:30596/TCP  2m5s
```

kubectl create secret generic cacerts --from-file=certs/multicloud-aks-cluster/ca-cert.pem --from-file=certs/multicloud-aks-cluster/ca-key.pem --from-file=certs/multicloud-aks-cluster/root-cert.pem --from-file=certs/multicloud-aks-cluster/cert-chain.pem -n istio-system --context=multicloud-aks-cluster

kubectl get namespace istio-system && kubectl label namespace istio-system topology.istio.io/network=network2 --context=multicloud-aks-cluster

helm repo add istio https://istio-release.storage.googleapis.com/charts

helm repo update

helm install istio-base istio/base -n istio-system --kube-context=multicloud-aks-cluster

helm install istiod istio/istiod -n istio-system --set global.meshID=mesh1 --set meshConfig.defaultConfig.proxyMetadata.ISTIO_META_DNS_CAPTURE="true" --set pilot.env.ENABLE_MULTICLUSTER_HEADLESS="true" --set pilot.env.PILOT_ENABLE_EDS_FOR_HEADLESS_SERVICES="true" --set global.multiCluster.clusterName=multicloud-aks-cluster --set global.network=network2 --kube-context=multicloud-aks-cluster

helm install istio-eastwestgateway istio/gateway -n istio-system --set name=istio-eastwestgateway --set networkGateway=network2 --kube-context=multicloud-aks-cluster

kubectl get svc istio-eastwestgateway -n istio-system --context=multicloud-aks-cluster


```
[demo@demo-azure-vm ~]$ kubectl create secret generic cacerts --from-file=certs/multicloud-aks-cluster/ca-cert.pem --from-file=certs/multicloud-aks-cluster/ca-key.pem --from-file=certs/multicloud-aks-cluster/root-cert.pem --from-file=certs/multicloud-aks-cluster/cert-chain.pem -n istio-system --context=multicloud-aks-cluster
secret/cacerts created
[demo@demo-azure-vm ~]$ kubectl get namespace istio-system && kubectl label namespace istio-system topology.istio.io/network=network2 --context=multicloud-aks-cluster
NAME      STATUS   AGE
istio-system Active   24m
namespace/istio-system labeled
[demo@demo-azure-vm ~]$ helm repo add istio https://istio-release.storage.googleapis.com/charts
"istio" already exists with the same configuration, skipping
[demo@demo-azure-vm ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "istio" chart repository
Update Complete. 🎉Happy Helming!🎉
[demo@demo-azure-vm ~]$ helm install istio-base istio/base -n istio-system --kube-context=multicloud-aks-cluster

NAME: istio-base
LAST DEPLOYED: Tue May 12 15:50:08 2026
NAMESPACE: istio-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Istio base successfully installed!

To learn more about the release, try:
  $ helm status istio-base -n istio-system
  $ helm get all istio-base -n istio-system

[demo@demo-azure-vm ~]$ helm install istiod istio/istiod -n istio-system --set global.meshID=mesh1 --set meshConfig.defaultConfig.proxyMetadata.ISTIO_META_DNS_CAPTURE="true" --set pilot.env.ENABLE_MULTICLUSTER_HEADLESS="true" --set pilot.env.PILOT_ENABLE_EDS_FOR_HEADLESS_SERVICES="true" --set global.multicluster.clusterName=multicloud-aks-cluster --set global.network=network2 --kube-context=multicloud-aks-cluster
NAME: istiod
LAST DEPLOYED: Tue May 12 15:50:13 2026
NAMESPACE: istio-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
"istiod" successfully installed!

To learn more about the release, try:
  $ helm status istiod -n istio-system
  $ helm get all istiod -n istio-system

Next steps:
* Deploy a Gateway: https://istio.io/latest/docs/setup/additional-setup/gateway/
* Try out our tasks to get started on common configurations:
  * https://istio.io/latest/docs/tasks/traffic-management
  * https://istio.io/latest/docs/tasks/security/
  * https://istio.io/latest/docs/tasks/policy-enforcement/
* Review the list of actively supported releases, CVE publications and our hardening guide:
  * https://istio.io/latest/docs/releases/supported-releases/
  * https://istio.io/latest/news/security/
  * https://istio.io/latest/docs/ops/best-practices/security/

For further documentation see https://istio.io website

[demo@demo-azure-vm ~]$ helm install istio-eastwestgateway istio/gateway -n istio-system --set name=istio-eastwestgateway --set networkGateway=network2 --kube-context=multicloud-aks-cluster
NAME: istio-eastwestgateway
LAST DEPLOYED: Tue May 12 15:50:15 2026
NAMESPACE: istio-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
"istio-eastwestgateway" successfully installed!

To learn more about the release, try:
  $ helm status istio-eastwestgateway -n istio-system
  $ helm get all istio-eastwestgateway -n istio-system

Next steps:
* Deploy an HTTP Gateway: https://istio.io/latest/docs/tasks/traffic-management/ingress/ingress-control/
* Deploy an HTTPS Gateway: https://istio.io/latest/docs/tasks/traffic-management/ingress/secure-ingress/

[demo@demo-azure-vm ~]$ kubectl get svc istio-eastwestgateway -n istio-system --context=multicloud-aks-cluster
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)                                     AGE
istio-eastwestgateway LoadBalancer 10.0.37.97    20.142.150.21:31828/TCP,15443:30606/TCP,15012:31005/TCP,15017:32304/TCP 113s
```

```
cat expose-service.yaml
```

```
apiVersion: networking.istio.io/v1
```

```
kind: Gateway
```

```
metadata:
```

```
  name: cross-network-gateway
```

```
spec:
```

```
  selector:
```

```
    istio: eastwestgateway
```

```
  servers:
```

```
    - port:
```

```
      number: 15443
```

```
      name: tls
```

```
      protocol: TLS
```

```
    tls:
```

```
      mode: AUTO_PASSTHROUGH
```

```
    hosts:
```

```
      - "*.local"
```

```
kubectl apply -f expose-service.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl apply -f expose-service.yaml --context=multicloud-aks-cluster
```

```
[demo@demo-azure-vm ~]$ cat expose-service.yaml
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: cross-network-gateway
spec:
  selector:
    istio: eastwestgateway
  servers:
    - port:
        number: 15443
        name: tls
        protocol: TLS
      tls:
        mode: AUTO_PASSTHROUGH
      hosts:
        - "*.local"
[demo@demo-azure-vm ~]$ kubectl apply -f expose-service.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
gateway.networking.istio.io/cross-network-gateway created
[demo@demo-azure-vm ~]$ kubectl apply -f expose-service.yaml --context=multicloud-aks-cluster
gateway.networking.istio.io/cross-network-gateway created
```

```
istioctl create-remote-secret --name=multicloud-gke-cluster --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | kubectl apply -f - --context=multicloud-aks-cluster
```

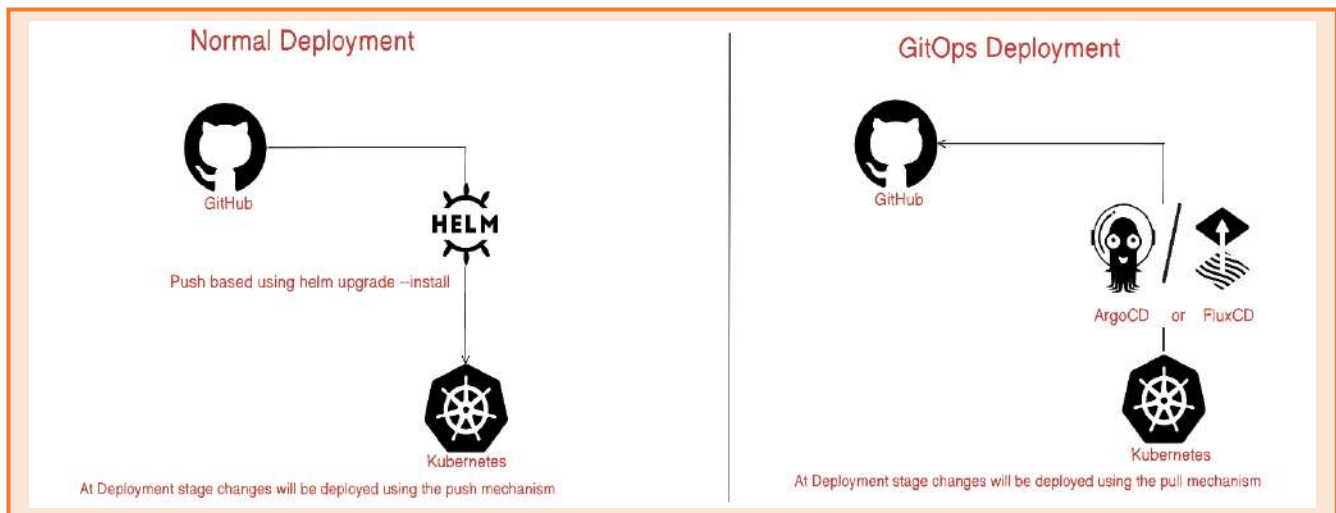
```
istioctl create-remote-secret --name=multicloud-aks-cluster --context=multicloud-aks-cluster | kubectl apply -f - --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
istioctl remote-clusters --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
istioctl remote-clusters --context=multicloud-aks-cluster
```

```
[demo@demo-azure-vm ~]$ istioctl create-remote-secret --name=multicloud-gke-cluster --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | kubectl apply -f - --context=multicloud-aks-cluster
secret/istio-remote-secret-multicloud-gke-cluster created
[demo@demo-azure-vm ~]$ istioctl create-remote-secret --name=multicloud-aks-cluster --context=multicloud-aks-cluster | kubectl apply -f - --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/istio-remote-secret-multicloud-aks-cluster created
[demo@demo-azure-vm ~]$ istioctl remote-clusters --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                SECRET                STATUS    ISTIOD
multicloud-gke-cluster  istio-system/istio-remote-secret-multicloud-aks-cluster  synced    istiod-77d7c68d8b-hvflr
multicloud-aks-cluster  istio-system/istio-remote-secret-multicloud-gke-cluster  synced    istiod-77d7c68d8b-hvflr
[demo@demo-azure-vm ~]$ istioctl remote-clusters --context=multicloud-aks-cluster
NAME                SECRET                STATUS    ISTIOD
multicloud-aks-cluster  istio-system/istio-remote-secret-multicloud-gke-cluster  synced    istiod-6b5dd87558-r2bl7
multicloud-gke-cluster  istio-system/istio-remote-secret-multicloud-aks-cluster  synced    istiod-6b5dd87558-r2bl7
```

In this project I kept the password of the PostgreSQL Pod in the Azure Keyvault Secret which will be used while creating the PostgreSQL Pod using the Helm Chart and deployment of Spring Boot Application using the Helm Chart. To use Azure Keyvault Secrets in Application Pod and in PostgreSQL Pod will be discussed later. For deployment ArgoCD had been used, here ArgoCD had been used with the concept of GitOps. In GitOps if there will be any change in the GitHub Repo then ArgoCD will pull those changes and will lead to new automated deployment. The difference between GitOps based deployment and normal deployment is, in normal deployment changes in the GitHub Repo will be pushed in the deployment stage. While in case of GitOps based deployment changes in the GitHub Repo will be pull by the ArgoCD or FluxCD.



Installation of ArgoCD in GKE Standard Cluster using the Kubernetes Manifests file is as shown in the screenshot attached below.

```
kubectl create ns argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl create ns argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/argocd created
```

```
[demo@demo-azure-vm ~]$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

Created Kubernetes Secret as shown in the screenshot attached below to access the ArgoCD using URL with HTTPS Protocol.

```
kubectl create secret tls tls-argocd-ingress --cert=d3391cd647faf661.crt --key=server.key -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl create secret tls tls-argocd-ingress --cert=d3391cd647faf661.crt --key=server.key -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/tls-argocd-ingress created
```

To install Kubernetes Gateway API controller and then to create URL to access ArgoCD follow the below procedure.

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
helm upgrade --install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric --create-namespace -n nginx-gateway --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ helm upgrade --install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric --create-namespace -n nginx-gateway --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f gateway-api.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
gateway.gateway.networking.k8s.io/argocd-gateway created
httproute.gateway.networking.k8s.io/argocd-route created
[demo@demo-azure-vm ~]$ kubectl get svc -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
argocd-applicationset-controller	ClusterIP	172.19.248.41	<none>	7000/TCP,8080/TCP	
argocd-dex-server	ClusterIP	172.19.143.235	<none>	5556/TCP,5557/TCP,5558/TCP	
argocd-gateway-nginx	LoadBalancer	172.19.80.217	34.15	80:32302/TCP,443:32334/TCP	
argocd-metrics	ClusterIP	172.19.107.152	<none>	8082/TCP	
argocd-notifications-controller-metrics	ClusterIP	172.19.91.209	<none>	9001/TCP	
argocd-redis	ClusterIP	172.19.126.216	<none>	6379/TCP	
argocd-repo-server	ClusterIP	172.19.204.37	<none>	8081/TCP,8084/TCP	
argocd-server	ClusterIP	172.19.242.191	<none>	80/TCP,443/TCP	
argocd-server-metrics	ClusterIP	172.19.186.214	<none>	8083/TCP	

Edited the configmap as shown in the screenshot attached below.

```
[demo@demo-azure-vm ~]$ kubectl edit configmap argocd-cmd-params-cm -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"ConfigMap","metadata":{"annotations":{},"labels":{"app.kubernetes.io/name":"argocd-cmd-params-cm","app.kubernetes.io/part-of":"argocd"},"name":"argocd-cmd-params-cm","namespace":"argocd"},"creationTimestamp":"2026-01-01T00:00:00Z"}
  name: argocd-cmd-params-cm
  namespace: argocd
  resourceVersion: "1"
  uid: "1"
data:
  server.insecure: "true"

[demo@demo-azure-vm ~]$ kubectl edit configmap argocd-cmd-params-cm -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
configmap/argocd-cmd-params-cm edited
```

Now, restarted the argocd-server deployment for the changes to take effect.

```
[demo@demo-azure-vm ~]$ kubectl rollout restart deployment argocd-server -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
deployment.apps/argocd-server restarted
```

kubectl edit configmap argocd-cmd-params-cm -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

data:

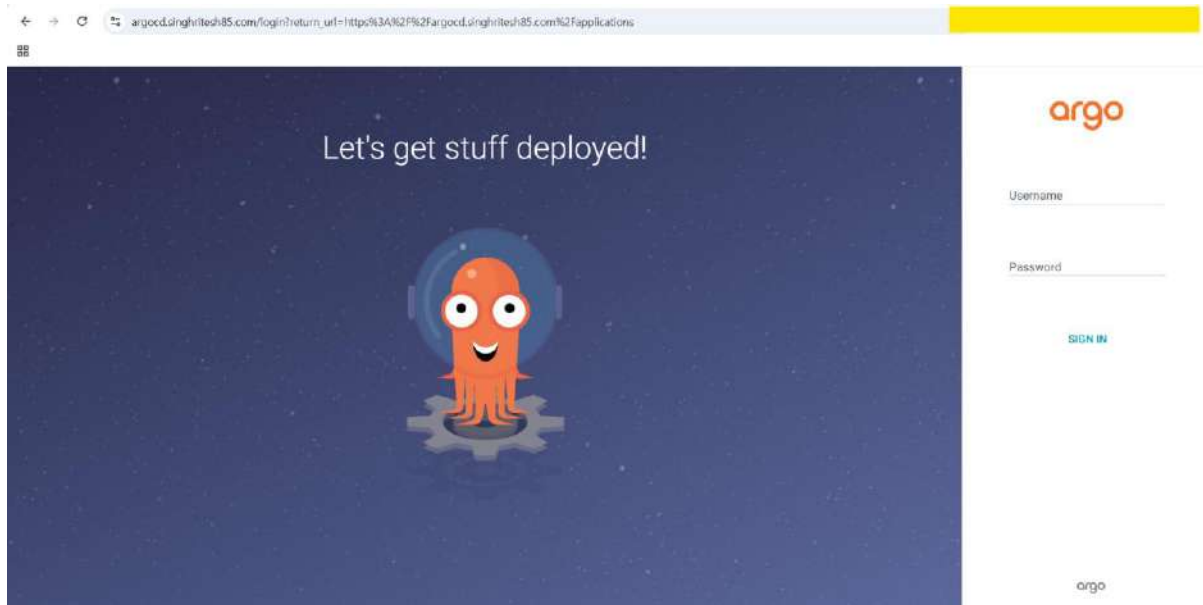
server.insecure: "true"

kubectl rollout restart deployment argocd-server -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

Then, Created the Record-Set of CNAME Type in Azure DNS Zone using the External IP of the LoadBalancer resulted from Kubernetes Gateway API for ArgoCD.

The screenshot shows the Azure Portal interface for managing DNS records. On the left, the 'Recordsets' page for the 'singhritesh85.com' DNS zone is visible, showing a table with two record sets: '@' of type 'NS' and '@' of type 'SOA'. The main area displays the 'Add record set' dialog for a new record set named 'argocd'. The dialog includes fields for Name, Type (A - IPv4 Address records), Alias record set (No), TTL (1), and IP address (34.15). The 'Add' button is highlighted in blue.

Finally, able to access the ArgoCD as shown in the screenshot attached below.



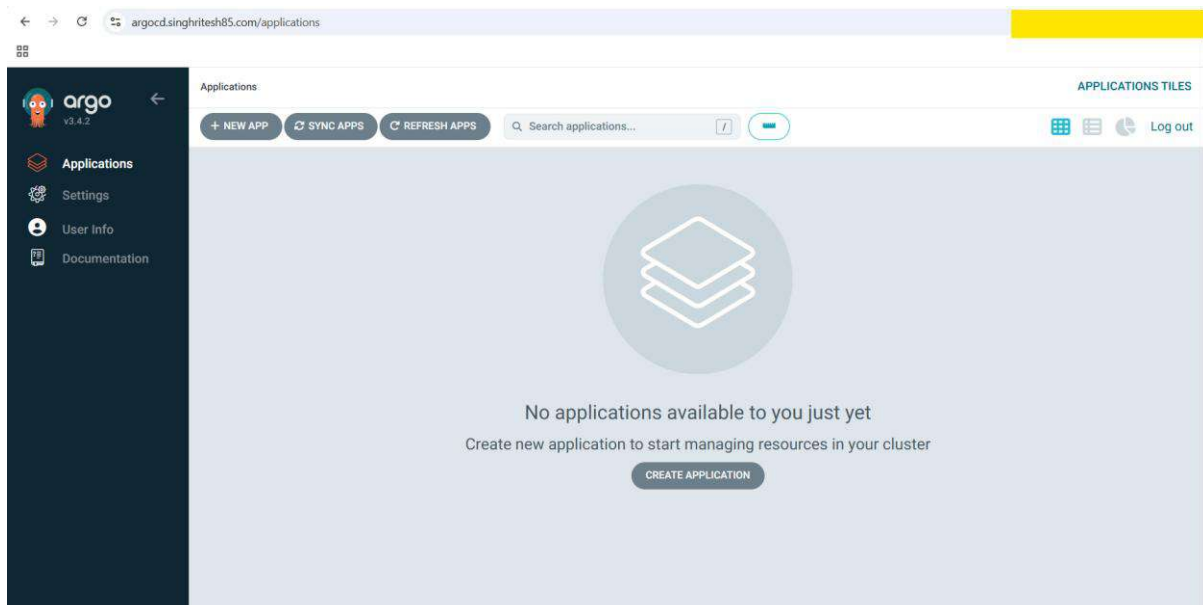
To Login into the ArgoCD for the first time you can login with user having username as admin and password you can get using the command.

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d
```

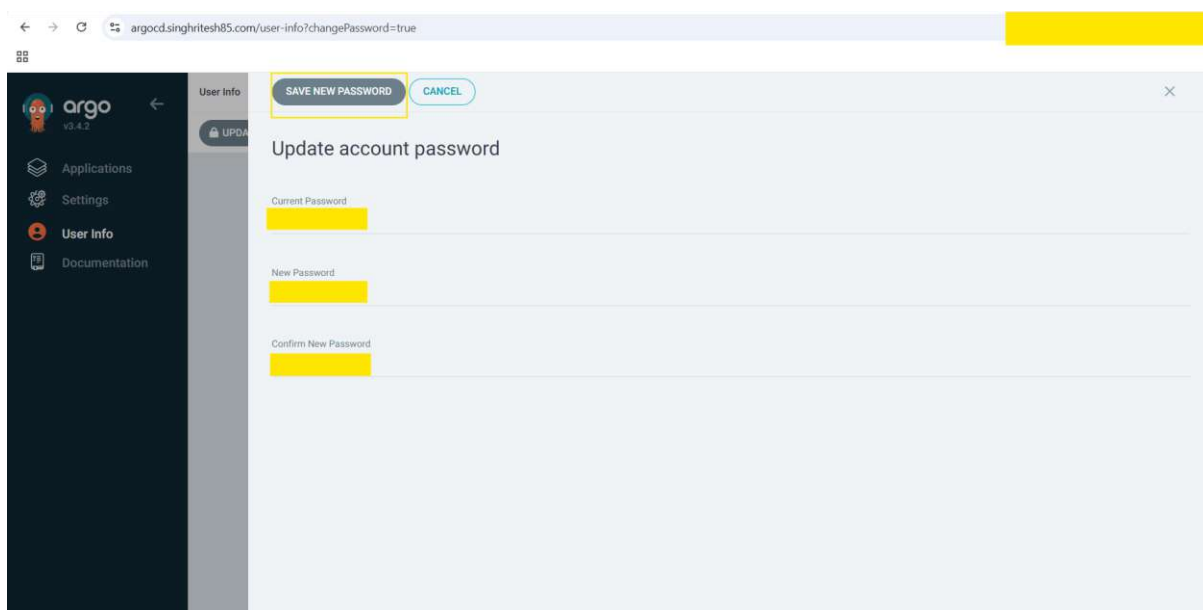
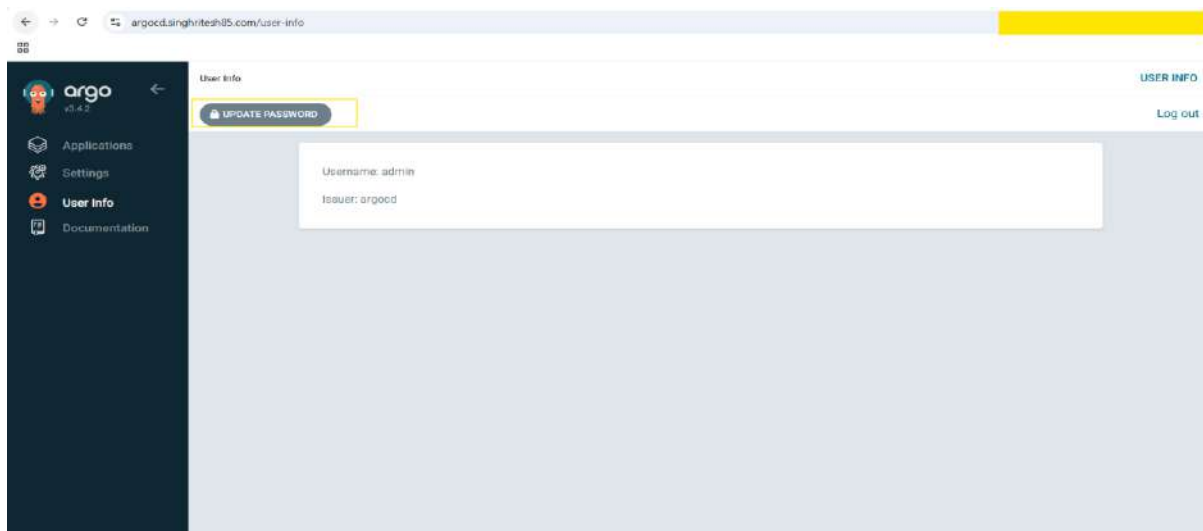
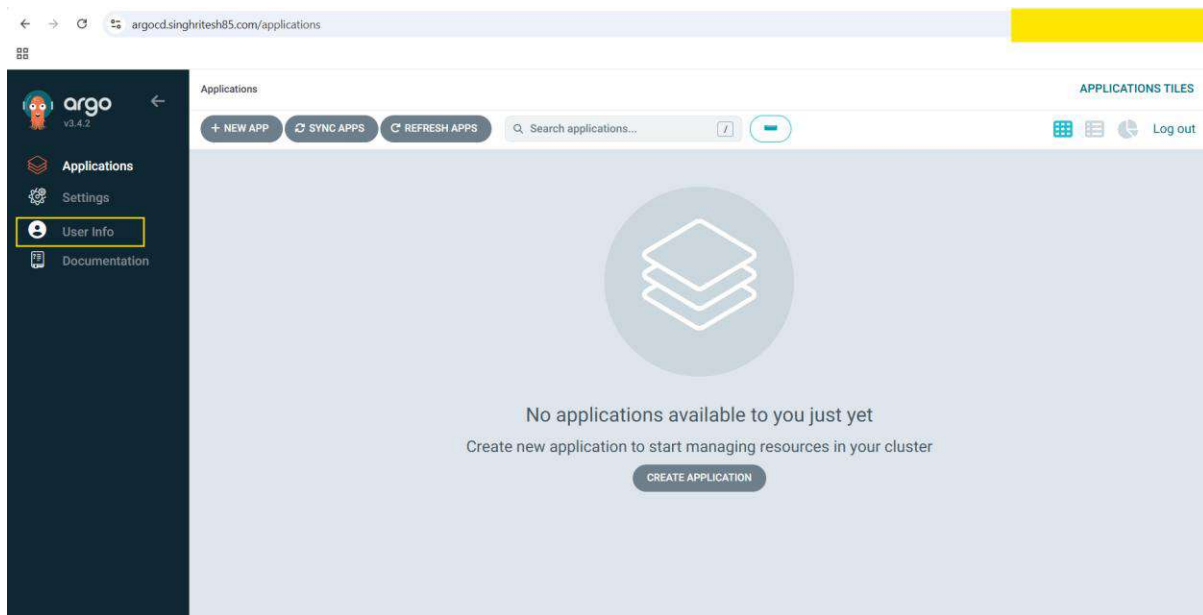
```
[demo@demo-azure-vm ~]$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d
$[REDACTED]v[demo@demo-azure-vm ~]$
```

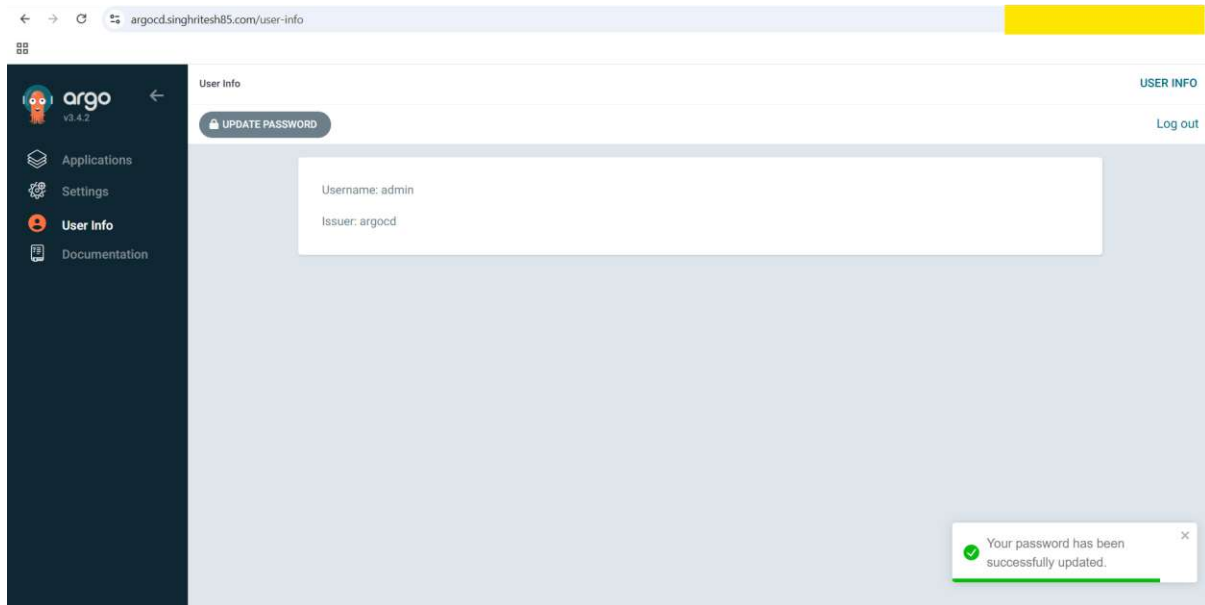
For your reference **gateway-api.yaml** file is present in the GitHub Repo

<https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-1/gateway-api.yaml**.

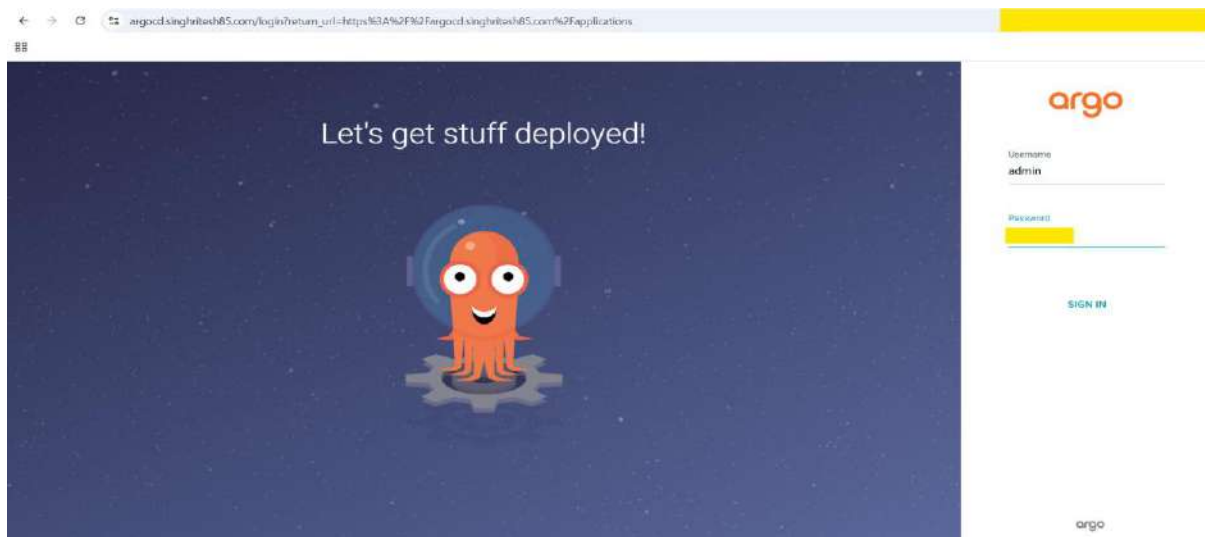


After the successful Login change the password as shown in the screenshot attached below.





Then Logout and Login again.



Install ArgoCD CLI using the command as shown below.

```
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
```

```
sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
```

```
rm argocd-linux-amd64
```

```
[demo@demo-azure-vm ~]$ curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
[demo@demo-azure-vm ~]$ sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
[demo@demo-azure-vm ~]$ rm argocd-linux-amd64
[demo@demo-azure-vm ~]$ argocd version
argocd: v3.4.2+0dc6b1b
```

Now I installed **secrets-store-csi-driver** and **secrets-store-csi-driver-provider-azure** in GKE standard cluster as shown in the screenshot attached below. It works together to mount secrets, keys, and certificates from Azure Key Vault directly into Kubernetes pods as files using CSI volumes. The **secrets-store-csi-driver** provides CSI interface for mounting external secrets and **secrets-store-csi-**

Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>

driver-provider-azure connects the CSI driver to Azure Key Vault. In AKS the Secrets store CSI Driver was already installed during its creation using Terraform through add-on as shown in the screenshot attached below.

```
key_vault_secrets_provider {
  secret_rotation_enabled = true
  secret_rotation_interval = "10s"
}
```

kubectl config get-contexts

helm repo add secrets-store-csi-driver https://kubernetes-sigs.github.io/secrets-store-csi-driver/charts

helm repo update

helm upgrade --install csi-secrets-store secrets-store-csi-driver/secrets-store-csi-driver --set syncSecret.enabled=true --set enableSecretRotation=true --set rotationPollInterval=10s --namespace kube-system --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl apply -f https://raw.githubusercontent.com/Azure/secrets-store-csi-driver-provider-azure/refs/heads/master/deployment/provider-azure-installer.yaml --namespace=kube-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl get pods -n kube-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | grep "secrets-store"

kubectl get pods -n kube-system --context=multicloud-aks-cluster | grep "secrets-store"

```
[demo@demo-azure-vm ~]$ kubectl config get-contexts
CURRENT  NAME                                     NAMESPACE                                CLUSTER                                AUTHINFO
*        gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
-        central1_multicloud-gke-cluster                                multicloud-aks-cluster                                clusterUser_multicloud-rg-aks-cluster
multicloud-aks-cluster
```

```
[demo@demo-azure-vm ~]$ helm repo add secrets-store-csi-driver https://kubernetes-sigs.github.io/secrets-store-csi-driver/charts
"secrets-store-csi-driver" has been added to your repositories
[demo@demo-azure-vm ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "secrets-store-csi-driver" chart repository
...Successfully got an update from the "istio" chart repository
Update Complete. Happy Helming!
[demo@demo-azure-vm ~]$ helm upgrade --install csi-secrets-store secrets-store-csi-driver/secrets-store-csi-driver --set syncSecret.enabled=true --set enableSecretRotation=true --set rotationPollInterval=10s --namespace kube-system --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Release "csi-secrets-store" does not exist. Installing it now.
NAME: csi-secrets-store
LAST DEPLOYED: Wed May 13 10:19:24 2026
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
The Secrets Store CSI Driver is getting deployed to your cluster.
To verify that Secrets Store CSI Driver has started, run:

  kubectl --namespace=kube-system get pods -l "app=secrets-store-csi-driver"

Now you can follow these steps https://secrets-store-csi-driver.sigs.k8s.io/getting-started/usage.html
to create a SecretProviderClass resource, and a deployment using the SecretProviderClass.
[demo@demo-azure-vm ~]$ kubectl apply -f https://raw.githubusercontent.com/Azure/secrets-store-csi-driver-provider-azure/refs/heads/master/deployment/provider-azure-installer.yaml --namespace=kube-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
serviceaccount/csi-secrets-store-provider-azure created
daemonset.apps/csi-secrets-store-provider-azure created
```

```
[demo@demo-azure-vm ~]$ kubectl get pods -n kube-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster|grep "secrets-store"
csi-secrets-store-provider-azure-2gvwq      1/1      Running    0          79s
csi-secrets-store-provider-azure-kncpz      1/1      Running    0          79s
csi-secrets-store-secrets-store-csi-driver-agkb2  3/3      Running    0          80s
csi-secrets-store-secrets-store-csi-driver-xh6np  3/3      Running    0          80s
[demo@demo-azure-vm ~]$ kubectl get pods -n kube-system --context=multicloud-aks-cluster|grep "secrets-store"
aks-secrets-store-csi-driver-15q72          3/3      Running    1 (4h14m ago)  4h14m
aks-secrets-store-csi-driver-qx4fc          3/3      Running    1 (4h8m ago)   4h8m
aks-secrets-store-provider-azure-jbs9b      1/1      Running    0            4h14m
aks-secrets-store-provider-azure-s25qr      1/1      Running    0            4h8m
```

Then created SecretProviderClass in GKE and AKS, the SecretProviderClass instructs the Secrets Store CSI Driver on how to securely fetch, mount, and sync external secrets from Azure Key Vault into Kubernetes pods as volumes.

```
[demo@demo-azure-vm ~]$ cat SecretProviderClass-gke.yaml
---
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: azure-keyvault
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    useVMManagedIdentity: "false"
    keyvaultName: "multicloud"
    tenantID: "[redacted]"
    cloudName: "AzurePublicCloud"
    objects: |
      array:
        - |
          objectName: multicloud-postgresql-server-password
          objectType: secret

  secretObjects:
  - secretName: db-credentials
    type: Opaque
    data:
      - objectName: multicloud-postgresql-server-password
        key: password
[demo@demo-azure-vm ~]$ kubectl create ns ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/ordersystem created
[demo@demo-azure-vm ~]$ kubectl label ns ordersystem istio-injection=enabled --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/ordersystem labeled
```

The Client ID and Secret used in the below command obtained from **terraform output -json** command as explained initially, while creation of the infrastructure.

```
[demo@demo-azure-vm ~]$ kubectl create secret generic secrets-store-creds --from-literal=clientId=[redacted] --from-literal=clientse
cret=[redacted] --type=Opaque --namespace ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/secrets-store-creds created
[demo@demo-azure-vm ~]$ kubectl apply -f SecretProviderClass-gke.yaml --namespace ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-clu
ster
secretproviderclass.secrets-store.csi.x-k8s.io/azure-keyvault created

[demo@demo-azure-vm ~]$ kubectl get secretproviderclass --namespace ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME          AGE
azure-keyvault 95s
```



```
[demo@demo-azure-vm ~]$ cat SecretProviderClass-and-ServiceAccount.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: postgres-sa
  namespace: postgresql
  annotations:
    azure.workload.identity/client-id: "[REDACTED]"
---
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: azure-kv-secrets
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    useVMManagedIdentity: "false"
    tenantId: "[REDACTED]"
    clientId: "[REDACTED]"
    keyvaultName: "multicloud"
    objects: |
      array:
        - |
          objectName: multicloud-postgresql-server-password
          objectType: secret

  secretObjects:
    - secretName: db-credentials
      type: Opaque
      data:
        - objectName: multicloud-postgresql-server-password
          key: password
        - objectName: multicloud-postgresql-server-password
          key: replication-password
        - objectName: multicloud-postgresql-server-password
          key: postgres-password
```

The Client ID used in the above screenshot obtained from **terraform output -json** command as explained initially, while creation of the infrastructure.

```
[demo@demo-azure-vm ~]$ kubectl create ns postgresql --context=multicloud-aks-cluster
namespace/postgresql created
[demo@demo-azure-vm ~]$ kubectl label ns postgresql istio-injection=enabled --context=multicloud-aks-cluster
namespace/postgresql labeled
[demo@demo-azure-vm ~]$ kubectl apply -f SecretProviderClass-and-ServiceAccount.yaml --namespace=postgresql --context=multicloud-aks-cluster
serviceaccount/postgres-sa created
secretproviderclass.secrets-store.csi.x-k8s.io/azure-kv-secrets created
[demo@demo-azure-vm ~]$ kubectl get secretproviderclass --namespace postgresql --context=multicloud-aks-cluster
NAME                AGE
azure-kv-secrets    27s
```

I used AKS Workload Identity for using Azure Keyvault Secrets in AKS as shown in the screenshot attached below. It establishes a trust relationship between your Kubernetes cluster and **Microsoft Entra ID** (priorly known as Azure AD), allowing pods to authenticate to Azure Key Vault.

```

### Create User Assigned Managed Identity
resource "azurerm_user_assigned_identity" "workload_id" {
  name            = "${var.prefix}-aks-workload-id"
  location        = azurerm_resource_group.vnetconnection_rg.location
  resource_group_name = azurerm_resource_group.vnetconnection_rg.name
}

### Allow User Assigned Managed Identity to Read Key Vault Secrets
resource "azurerm_role_assignment" "kv_access" {
  principal_id      = azurerm_user_assigned_identity.workload_id.principal_id
  role_definition_name = "Key Vault Secrets User"
  scope             = azurerm_key_vault.key_vault_postgresql.id
}

### Create Federated Identity Credential for AKS
resource "azurerm_federated_identity_credential" "fic" {
  name            = "${var.prefix}-postgres-federation-aks"
  user_assigned_identity_id = azurerm_user_assigned_identity.workload_id.id
  audience        = ["api://AzureADTokenExchange"]
  issuer          = azurerm_kubernetes_cluster.aks_cluster.oidc_issuer_url
  subject         = "system:serviceaccount:postgresql:postgres-sa"
}

```

↑ namespace ↑ service-account

To use Azure Keyvault Secrets in GKE Standard Cluster I used the concept of Azure AD Service Principal as shown in the screenshot attached below.

```

### Create AzureAD Application for GKE
resource "azuread_application" "gke_kv" {
  display_name = "${var.prefix}-gke-azure-kv"
}

### Create AzureAD Service Principal for GKE
resource "azuread_service_principal" "gke_kv_sp" {
  client_id = azuread_application.gke_kv.client_id
}

### Create a Password for the Azure Service Principal
resource "azuread_service_principal_password" "azuread_sp_secret" {
  service_principal_id = azuread_service_principal.gke_kv_sp.id
  display_name         = "azuread-sp-password-for-gke"
  end_date             = "2027-01-01T00:00:00Z"
}

### Allow Azure Key Vault Access to AzureAD Service Principal for GKE
resource "azurerm_role_assignment" "kv_access_sp_for_gke" {
  scope             = azurerm_key_vault.key_vault_postgresql.id
  role_definition_name = "Key Vault Secrets User"
  principal_id      = azuread_service_principal.gke_kv_sp.object_id
}

```

As I am going to use the Azure Keyvault secrets in kubernetes pod through the helm chart command line. So, whenever you rotate the credentials in Azure Keyvault secret then it will not be rotated automatically but you need to recreate the pods using the command **kubectl rollout restart**

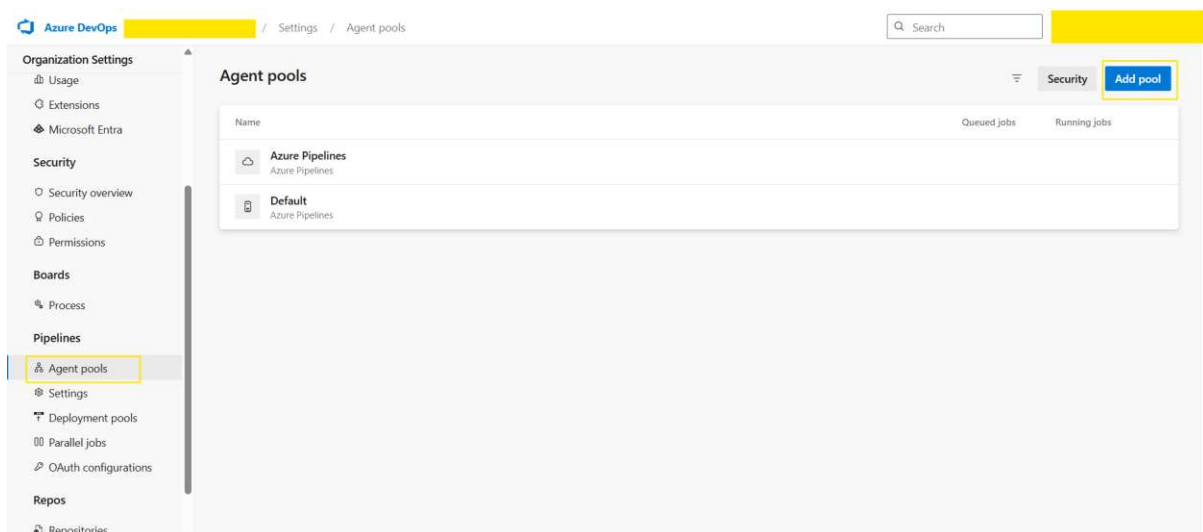
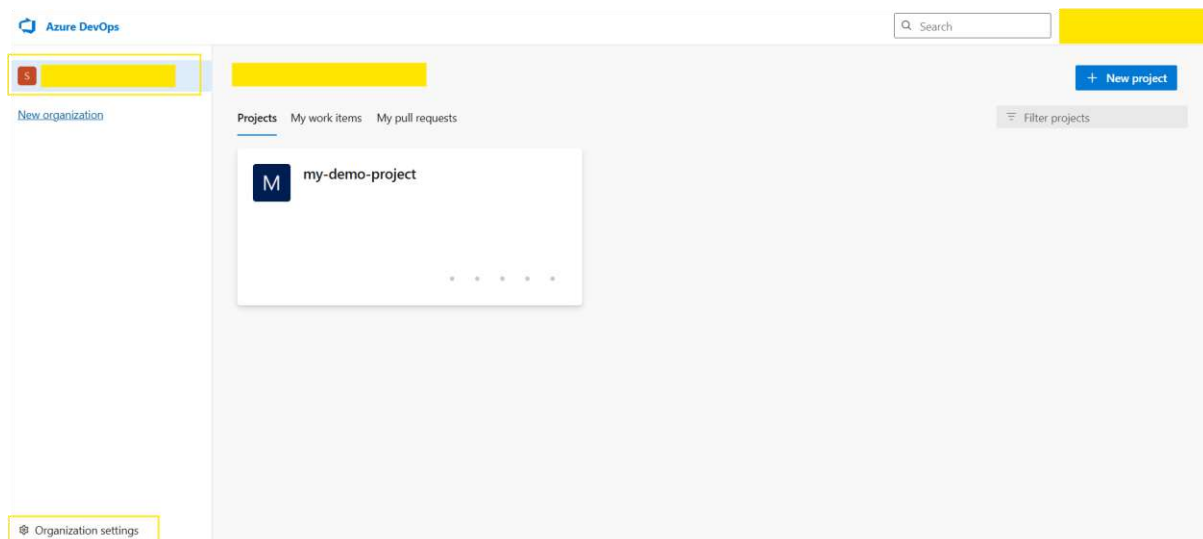
deployment <deployment-name> -n <namespace> --context=<context-name> or kubectl rollout restart statefulset <statefulset-name> -n <namespace> --context=<context-name>.

For your reference I kept the files **SecretProviderClass-gke.yaml** and **SecretProviderClass-and-ServiceAccount.yaml** present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-1/**.

For CI/CD Pipeline I used Aure DevOps and did the deployment for PostgreSQL Pods, Kafka Pods and Spring Boot Application Pods using ArgoCD through the Helm Chart. These Helm Charts are present in the GitHub Repo <https://github.com/singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary.git>, <https://github.com/singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary-only-service.git>, <https://github.com/singhritesh85/helm-repo-for-bitnami-kafka.git> and <https://github.com/singhritesh85/helm-repo-for-Order-System.git>. I had created three Azure DevOps Pipeline one for deployment of Postgresql Pods and service in AKS Cluster, another for deployment of Kafka Pods and services in GKE Standard Cluster third for deployment of Spring Boot Application Pods and Services in GKE Standard Cluster.

To install and configure Azure DevOps Self-hosted Agent follow the procedure as explained below.

Go to the Azure DevOps Organization > Organisation Settings > Agent Pools > Add Pool



Add agent pool
Agent pools are shared across an organization.

☐ Managed DevOps Pool
Reduce the effort spent in maintaining custom agents by creating a Microsoft managed pool of scalable agents. [Learn more.](#)

☒ Self-hosted
Create a pool of custom agents hosted on your own infrastructure for maximum control and flexibility. [Learn more.](#)

☐ Azure virtual machine scale set
Create a pool of custom agents based on an Azure Virtual machine scale set hosted in your own Azure subscription. [View configuration instructions.](#)

Name:

Description (optional):

☐ Markdown supported.

Pipeline permissions:
☒ Auto-provision this agent pool in all projects

Create

Agent pools

Name	Queued jobs	Running jobs
Azure Pipelines Azure Pipelines		
Default Azure Pipelines		
demo		

demo

Jobs **Agents** Details Security Settings Maintenance History Analytics

Update all agents **New agent**

Add your first agent
Manage agents and run pipeline jobs on this pool.

New agent

x64

arm

arm64

RHEL6

System prerequisites

Configure your account

Configure your account by following the steps outlined [here](#).

Download the agent

Download



Create the agent

```
~/ $ mkdir myagent && cd myagent
~/myagent$ tar zxvf ~/Downloads/vsts-agent-linux-x64-4.272.0.tar.gz
```

Configure the agent [Detailed instructions](#)

```
~/myagent$ ./config.sh
```

Optionally run the agent interactively

If you didn't run as a service above:

```
[root@demo-azure-vm opt]# mkdir myagent && cd myagent
[root@demo-azure-vm myagent]# wget https://download.agent.dev.azure.com/agent/4.272.0/vsts-agent-linux-x64-4.272.0.tar.gz
--2026-05-13 13:34:30-- https://download.agent.dev.azure.com/agent/4.272.0/vsts-agent-linux-x64-4.272.0.tar.gz
Resolving download.agent.dev.azure.com (download.agent.dev.azure.com)... 23.54.127.45, 23.54.127.47, 2600:1408:ec00:15::17d7:ca, ...
Connecting to download.agent.dev.azure.com (download.agent.dev.azure.com)[23.54.127.45]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 212447020 (203M) [application/octet-stream]
Saving to: 'vsts-agent-linux-x64-4.272.0.tar.gz'

vsts-agent-linux-x64-4.272.0.tar.gz 100%[=====] 202.68M 136MB/s in 1.5s
2026-05-13 13:34:31 (136 MB/s) - 'vsts-agent-linux-x64-4.272.0.tar.gz' saved [212447020/212447020]
```

```
[root@demo-azure-vm myagent]# tar -xvf vsts-agent-linux-x64-4.272.0.tar.gz
```

```
[root@demo-azure-vm myagent]# ls
bin config.sh env.sh externals license.html reauth.sh run-docker.sh run.sh vsts-agent-linux-x64-4.272.0.tar.gz
[root@demo-azure-vm myagent]# rm -f vsts-agent-linux-x64-4.272.0.tar.gz
```

```
[root@demo-azure-vm myagent]# chown -R demo:demo /opt/myagent/
```

```
[demo@demo-azure-vm myagent]$ sudo ./bin/installdependencies.sh
```

```
[demo@demo-azure-vm myagent]$ ./config.sh
```



```
>> End User License Agreements:
```

Building sources from a TFVC repository requires accepting the Team Explorer Everywhere End User License Agreement. This step is not required for building sources from Git repositories.

A copy of the Team Explorer Everywhere license agreement can be found at:
./opt/myagent/license.html

Enter (Y/N) Accept the Team Explorer Everywhere license agreement now? (press enter for N) >

```
>> Connect:
```

Enter server URL > https://dev.azure.com/

Enter authentication type (press enter for PAT) >

Enter personal access token >


```

Enter agent pool (press enter for default) > demo
Enter agent name (press enter for demo-azure-vm) > demo
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) >
2026-[REDACTED]: Settings Saved.

```

```

[demo@demo-azure-vm myagent]$ sudo ./svc.sh start
/etc/systemd/system/vsts.agent.[REDACTED].demo.demo.service
* vsts.agent.[REDACTED].demo.demo.service - Azure Pipelines Agent ([REDACTED],demo.demo)
   Loaded: loaded (/etc/systemd/system/vsts.agent.[REDACTED].demo.demo.service; enabled; vendor preset: disabled)
   Active: active (running) since Wed 2026-05-13 13:48:31 UTC; 12ms ago
     Main PID: 43179 (runsv.sh)
       Tasks: 2 (limit: 24526)
      Memory: 1.0M
     CGroup: /system.slice/vsts.agent.[REDACTED].demo.demo.service
             └─43179 /bin/bash /opt/myagent/runsv.sh
               └─43182 ./externals/node20_1/bin/node --version

May 13 13:48:31 demo-azure-vm systemd[1]: Started Azure Pipelines Agent ([REDACTED],demo.demo).
[demo@demo-azure-vm myagent]$ ./env.sh

```

Finally, you will Self-hosted Azure DevOps Agent will be in Online state as shown in the screenshot attached below.

Name	Last run	Current status	Agent version	Enabled
demo ● Online		Idle	4.272.0	☒ On

In this project I am using Docker Hub Private Repository to keep the Docker Image and the same Image will be pulled from the Docker Hub Private Repository for each Deployment.

Created first pipeline to deploy the PostgreSQL Pods and Services as shown in the screenshot attached below.

Azure DevOps [REDACTED] / my-demo-project / Pipelines

my-demo-project +

- Overview
- Boards
- Repos
- Pipelines
- Pipelines
- Environments
- Releases
- Library
- Task groups
- Deployment groups
- Test Plans
- Artifacts
- Project settings

Create your first Pipeline

Automate your build and release processes using our wizard, and go from code to cloud-hosted within minutes.

[Create Pipeline](#)

Azure DevOps / Pipelines

my-demo-project

Overview
Boards
Repos
Pipelines
Environments
Releases
Library
Task groups
Deployment groups
Test Plans
Artifacts
Project settings

Connect Select Configure Review

New pipeline

Where is your code?

- Azure Repos Git (YAML)
Free private Git repositories, pull requests, and code search
- GitHub (YAML)**
Home to the world's largest community of developers
- Bitbucket Cloud (YAML)
Hosted by Atlassian
- Other Git
Any generic Git repository
- Subversion
Centralized version control by Apache

Use the classic editor to create a pipeline without YAML.

Azure DevOps / Pipelines

my-demo-project

Overview
Boards
Repos
Pipelines
Environments
Releases
Library
Task groups
Deployment groups
Test Plans
Artifacts
Project settings

Connect Select Configure Review

New pipeline

Select a repository

Filter by keywords My repositories

- singhritesh85/k8s-springboot-kafka-microservice** private
1h ago
- singhritesh85/helm-repo-for-Order-System
3h ago
- singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way
3h ago
- singhritesh85/DevOps-Project-Three-Tier-WebApp-Multicloud
May 4
- singhritesh85/helm-repo-for-kafbat-kafka-ui
Apr 30
- singhritesh85/helm-repo-for-bitnami-kafka
Apr 29
- singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary-only-service
Apr 22

Azure DevOps / Pipelines

my-demo-project

Overview
Boards
Repos
Pipelines
Environments
Releases
Library
Task groups
Deployment groups
Test Plans
Artifacts
Project settings

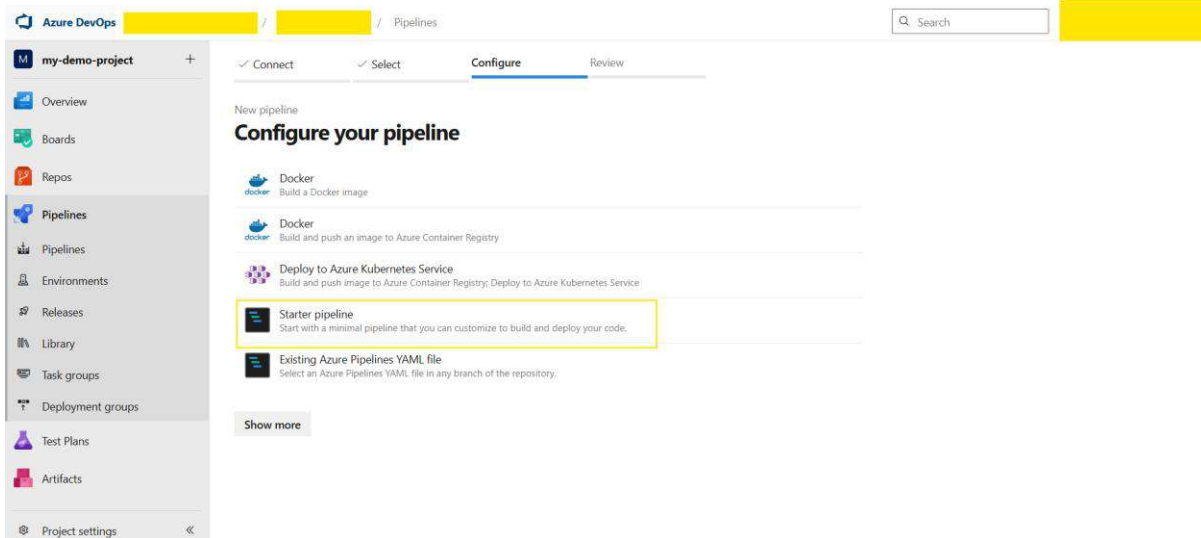
Connect Select Configure Review

New pipeline

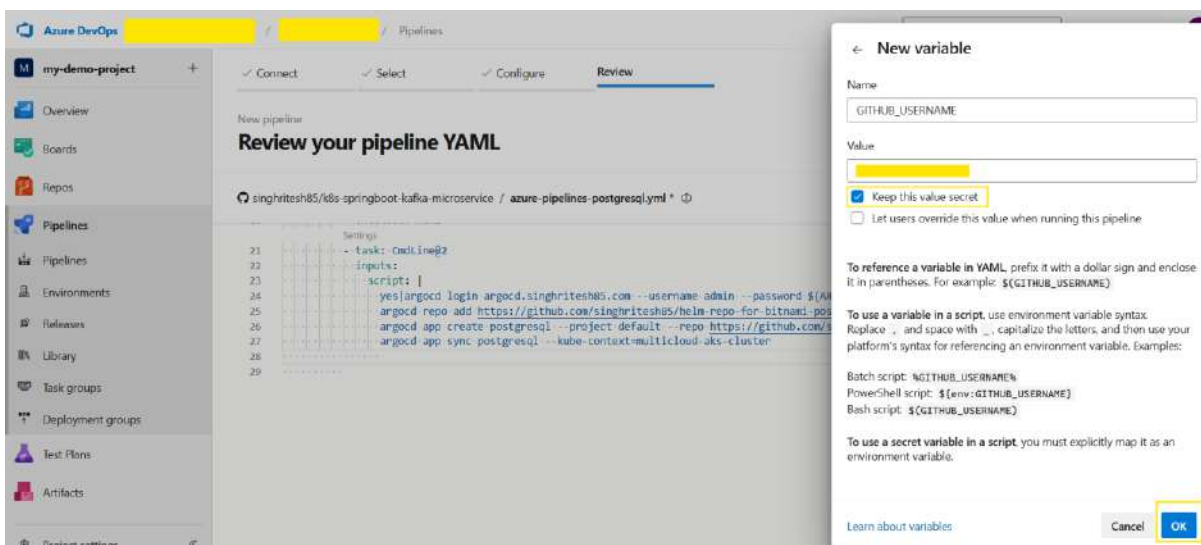
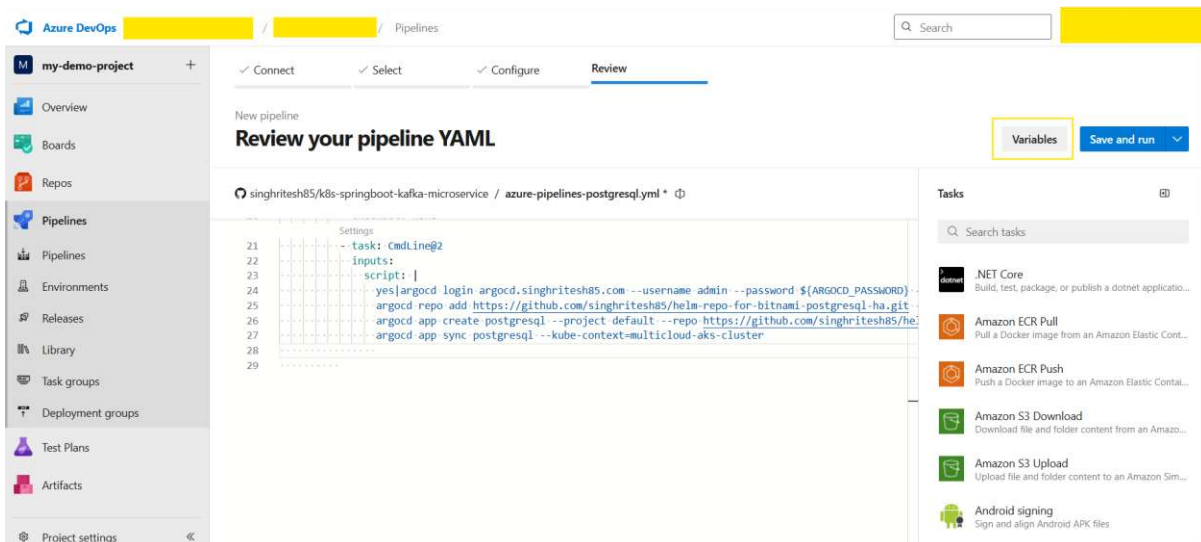
Select a repository

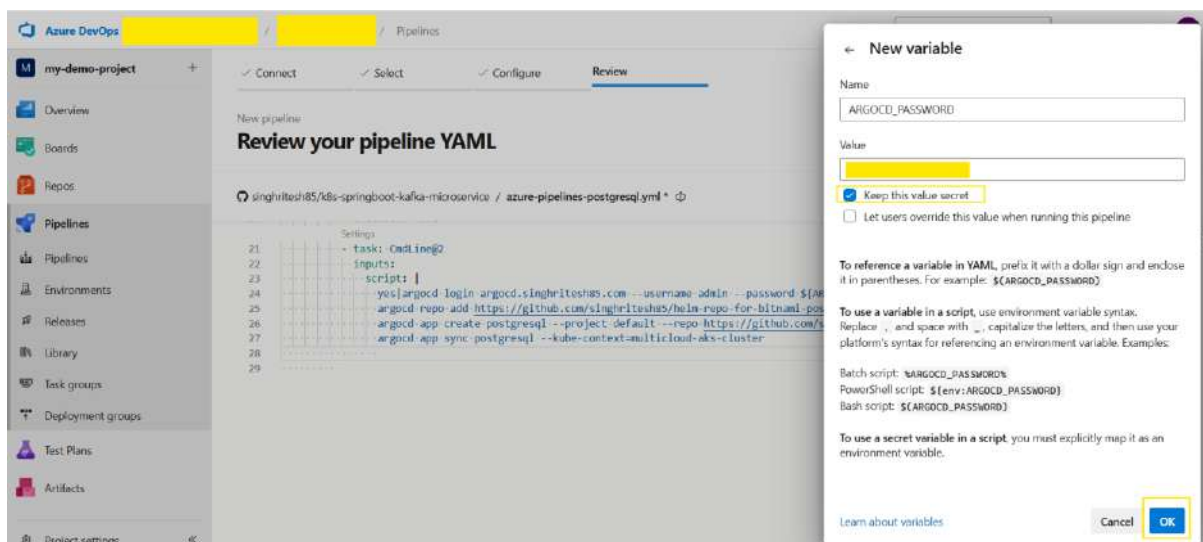
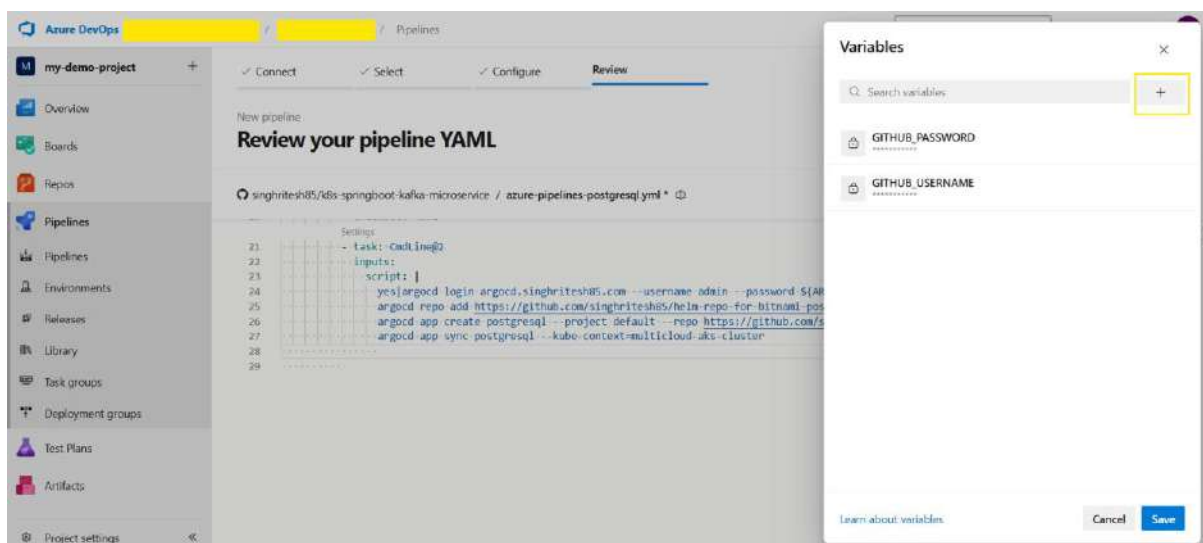
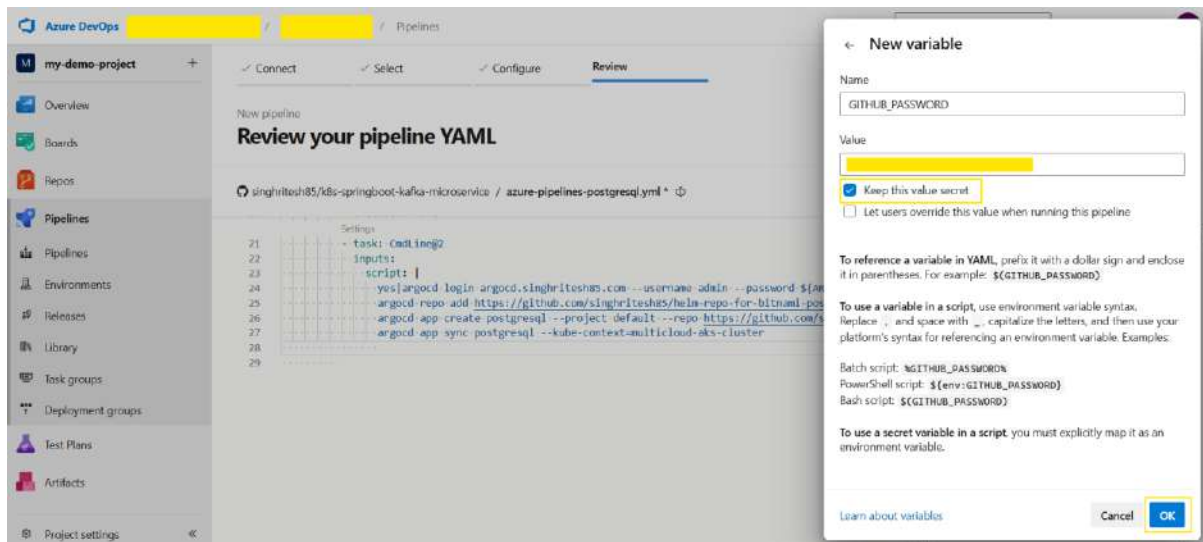
Filter by keywords My repositories

- singhritesh85/k8s-springboot-kafka-microservice** private
1h ago
- singhritesh85/helm-repo-for-Order-System
3h ago
- singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way
3h ago
- singhritesh85/DevOps-Project-Three-Tier-WebApp-Multicloud
May 4
- singhritesh85/helm-repo-for-kafbat-kafka-ui
Apr 30
- singhritesh85/helm-repo-for-bitnami-kafka
Apr 29
- singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary-only-service
Apr 22

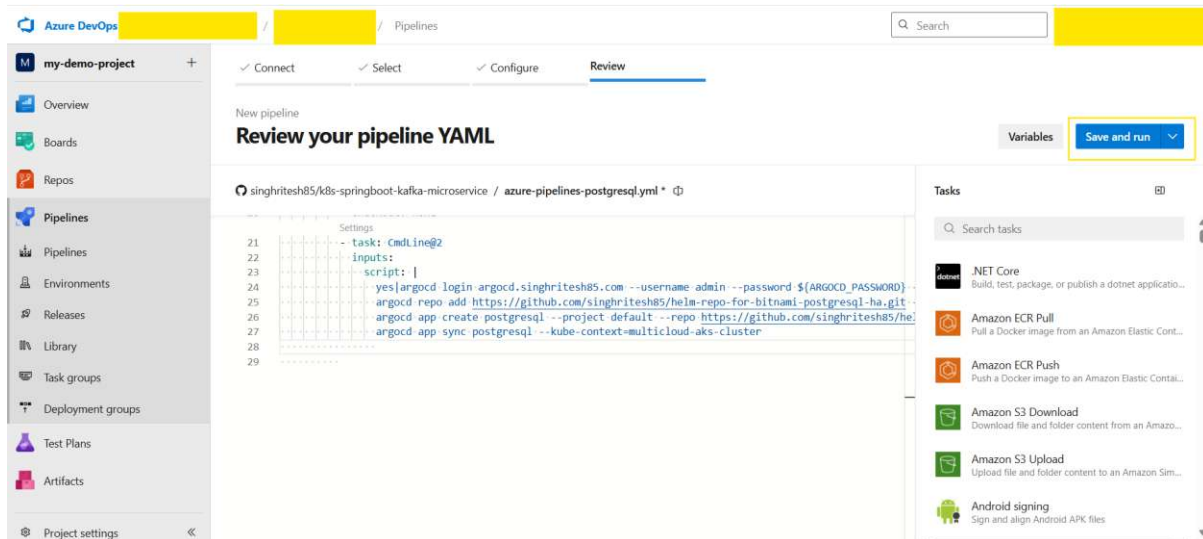


Created three variables with the name GITHUB_USERNAME, GITHUB_PASSWORD and ARGOCD_PASSWORD as shown in the screenshot attached below.





Finally, Save and Run as shown in the screenshot attached below.

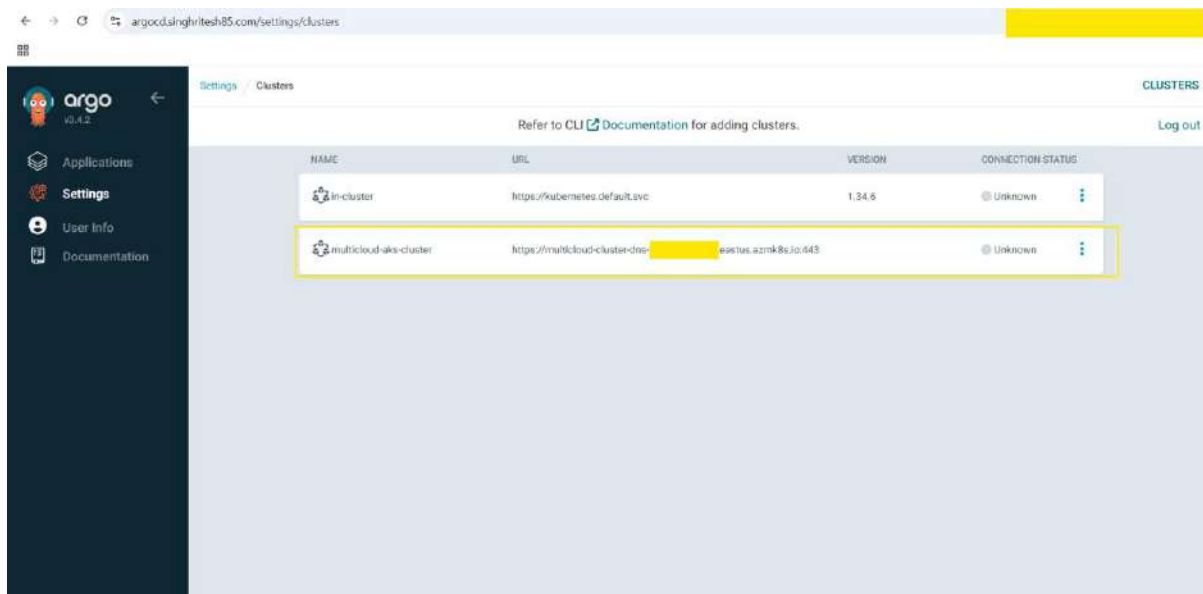


Added AKS Cluster to ArgoCD using the command as shown below.

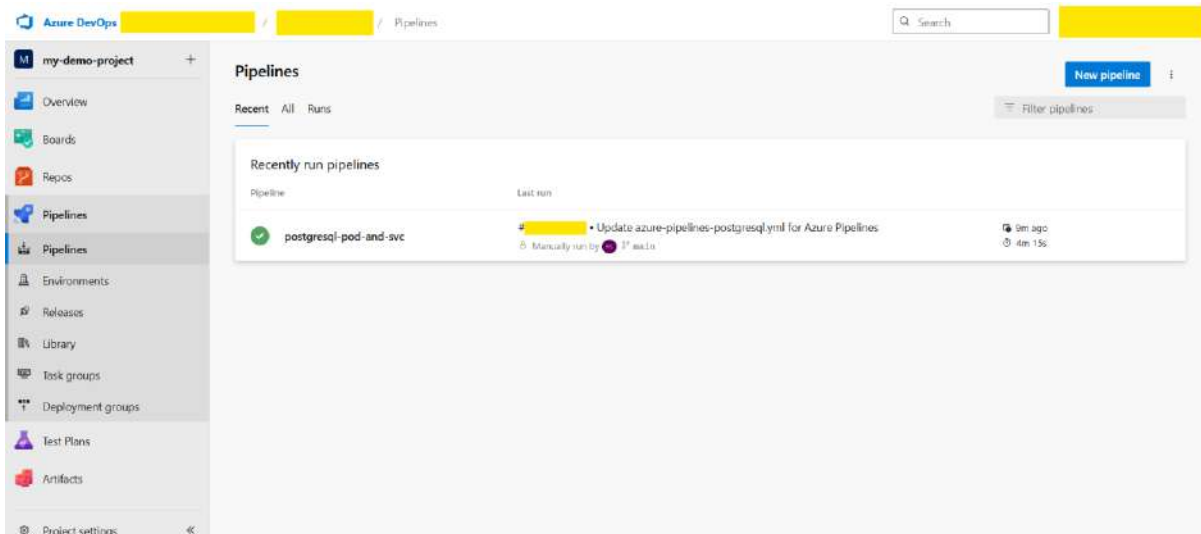
```
[demo@demo-azure-vm ~]$ kubectl config get-contexts
CURRENT  NAME                                     NAMESPACE                                     CLUSTER                                     AUTHINFO
*        gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
-        central1_multicloud-gke-cluster                                     multicloud-gke-cluster                                     clusterUser_multicloud-gke-cluster
-        multicloud-aks-cluster                                     multicloud-aks-cluster                                     clusterUser_multicloud-aks-cluster

[demo@demo-azure-vm ~]$ yes|argocd login argocd.singhritesh85.com --username admin --password Admin123 --grpc-web
WARNING: server certificate had error: error creating connection: tls: failed to verify certificate: x509: certificate signed by unknown authority. Proceed in
securely (y/n)? 'admin:login' logged in successfully
Context 'argocd.singhritesh85.com' updated
[demo@demo-azure-vm ~]$ argocd cluster add multicloud-aks-cluster
WARNING: This will create a service account 'argocd-manager' on the cluster referenced by context 'multicloud-aks-cluster' with full cluster level privileges.
Do you want to continue [y/N]? y
{"level":"info","msg":"ServiceAccount 'argocd-manager' created in namespace 'kube-system'", "time":"2026-05-13T17:28:55Z"}
{"level":"info","msg":"ClusterRole 'argocd-manager-role' created", "time":"2026-05-13T17:28:55Z"}
{"level":"info","msg":"ClusterRoleBinding 'argocd-manager-role-binding' created", "time":"2026-05-13T17:28:55Z"}
{"level":"info","msg":"Created bearer token secret 'argocd-manager-long-lived-token' for ServiceAccount 'argocd-manager'", "time":"2026-05-13T17:28:55Z"}
Cluster 'https://multicloud-cluster-dns-y63c436m.hcp.eastus.azure.com:443' added
```

The same had been verified using ArgoCD User Interface as shown in the screenshot attached below.



The first Azure DevOps Pipeline was executed successfully as shown in the screenshot attached below and pods and services had been created in AKS Cluster.



```
[demo@demo-azure-vm ~]$ kubectl get pods -n postgresql --context=multicloud-aks-cluster
NAME                                READY   STATUS    RESTARTS   AGE
postgresql-primary-0                2/2     Running   0           1m
postgresql-read-0                    2/2     Running   0           1m
postgresql-read-1                    2/2     Running   0           1m
postgresql-read-2                    2/2     Running   0           1m
[demo@demo-azure-vm ~]$ kubectl get svc -n postgresql --context=multicloud-aks-cluster
NAME                                TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
postgresql-primary                  ClusterIP       10.0.228.189    <none>           5432/TCP         1m
postgresql-primary-h1                ClusterIP       None            <none>           5432/TCP         1m
postgresql-read                      ClusterIP       10.0.212.86    <none>           5432/TCP         1m
postgresql-read-h1                  ClusterIP       None            <none>           5432/TCP         1m
```

I checked I was able to login into postgresql pod with the password present in the Azure Keyvault Secret.

```
[demo@demo-azure-vm ~]$ kubectl exec -it postgresql-primary-0 -n postgresql --context=multicloud-aks-cluster -- bash
I have no name!@postgresql-primary-0:/# psql -h localhost -d mederma -U dexter --password
Password:
psql (16.3)
Type "help" for help.

mederma=> ^D\q
could not save history to file "/.psql_history": Read-only file system
I have no name!@postgresql-primary-0:/#
exit
```

Now using the helm command line, I had created the postgresql only service in GKE Standard Cluster as shown in the screenshot attached below.

```
[demo@demo-azure-vm ~]$ kubectl create ns postgresql --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/postgresql created
[demo@demo-azure-vm ~]$
[demo@demo-azure-vm ~]$ kubectl label ns postgresql istio-injection=enabled --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/postgresql labeled

[demo@demo-azure-vm ~]$ git clone https://github.com/singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary-only-service.git
Cloning into 'helm-repo-for-bitnami-postgresql-primary-secondary-only-service'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (41/41), done.
remote: Total 42 (delta 17), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (42/42), 70.73 KiB | 14.15 MiB/s, done.
Resolving deltas: 100% (17/17), done.
[demo@demo-azure-vm ~]$ cd helm-repo-for-bitnami-postgresql-primary-secondary-only-service/
[demo@demo-azure-vm ~]$ helm upgrade --install postgresql ./ -f ./values.yaml --create-namespace --namespace postgresql --set architecture=replication,auth.secretKeys.replicationPasswordKey=replication-password,primary.service.type=ClusterIP,primary.extraVolumes[0].name=secrets-store-inline,primary.extraVolumes[0].csi.driver=secrets-store.csi.k8s.io,primary.extraVolumes[0].csi.readOnly=true,primary.extraVolumes[0].csi.volumeAttributes.secretProviderClass=azure-kv-secrets,primary.extraVolumeMounts[0].name=secrets-store-inline,primary.extraVolumeMounts[0].mountPath=/mnt/secrets-store,primary.extraVolumeMounts[0].readOnly=true,readReplicas.extraVolumes[0].name=secrets-store-inline,readReplicas.extraVolumes[0].csi.driver=secrets-store.csi.k8s.io,readReplicas.extraVolumeMounts[0].name=secrets-store-inline,readReplicas.extraVolumeMounts[0].csi.volumeAttributes.secretProviderClass=azure-kv-secrets,readReplicas.extraVolumeMounts[0].mountPath=/mnt/secrets-store,readReplicas.extraVolumeMounts[0].readOnly=true,auth.existingSecret=db-credentials,global.postgresql.auth.username=dexter,auth.userPasswordKey=password,global.postgresql.auth.database=mederma --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm helm-repo-for-bitnami-postgresql-primary-secondary-only-service]$ kubectl get all -n postgresql --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/postgresql-primary	ClusterIP	172.19.127.45	<none>	5432/TCP	16s
service/postgresql-primary-hl	ClusterIP	None	<none>	5432/TCP	16s
service/postgresql-read	ClusterIP	172.19.125.202	<none>	5432/TCP	16s
service/postgresql-read-hl	ClusterIP	None	<none>	5432/TCP	16s

```
kubectl create ns postgresql --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl label ns postgresql istio-injection=enabled --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

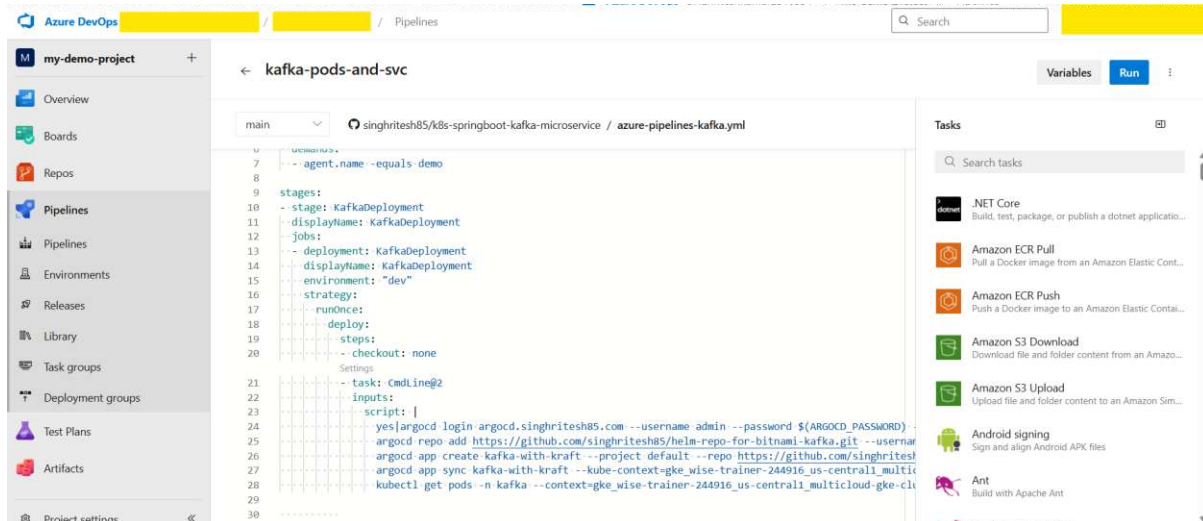
```
git clone https://github.com/singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary-only-service.git
```

```
cd helm-repo-for-bitnami-postgresql-primary-secondary-only-service
```

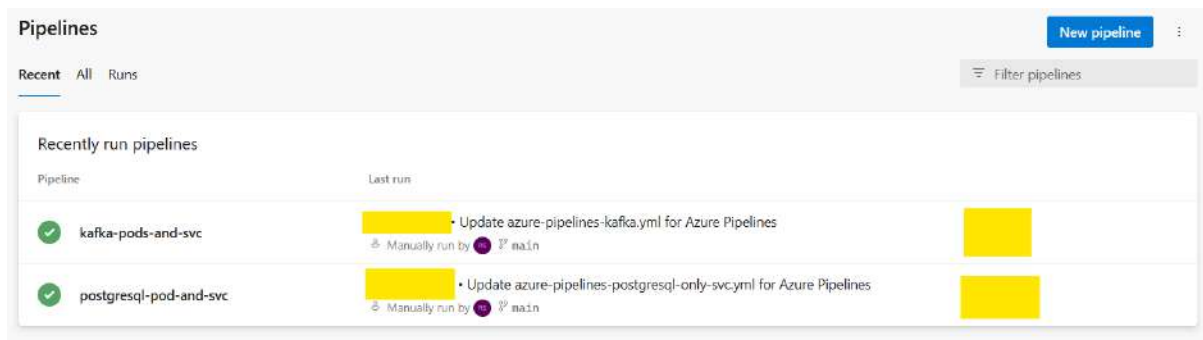
```
helm upgrade --install postgresql ./ -f ./values.yaml --create-namespace --namespace postgresql --set architecture=replication,auth.secretKeys.replicationPasswordKey=replication-password,primary.service.type=ClusterIP,primary.extraVolumes[0].name=secrets-store-inline,primary.extraVolumes[0].csi.driver="secrets-store.csi.k8s.io",primary.extraVolumes[0].csi.readOnly=true,primary.extraVolumes[0].csi.volumeAttributes.secretProviderClass=azure-kv-secrets,primary.extraVolumeMounts[0].name=secrets-store-inline,primary.extraVolumeMounts[0].mountPath=/mnt/secrets-store,primary.extraVolumeMounts[0].readOnly=true,readReplicas.extraVolumes[0].name=secrets-store-inline,readReplicas.extraVolumes[0].csi.driver="secrets-store.csi.k8s.io",readReplicas.extraVolumes[0].csi.readOnly=true,readReplicas.extraVolumes[0].csi.volumeAttributes.secretProviderClass=azure-kv-secrets,readReplicas.extraVolumeMounts[0].name=secrets-store-inline,readReplicas.extraVolumeMounts[0].mountPath=/mnt/secrets-store,readReplicas.extraVolumeMounts[0].readOnly=true,auth.existingSecret=db-credentials,global.postgresql.auth.username=dexter,auth.userPasswordKey=password,global.postgresql.auth.database=mederma --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl get all -n postgresql --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

Then created another Pipeline to deploy kafka pods and services (created three variables in this Pipeline as discussed earlier) in GKE Standard Cluster as shown in the screenshot attached below.



After successfully running this Pipeline Kafka Pods and services had been created in GKE Standard Cluster successfully as shown in the screenshot attached below.



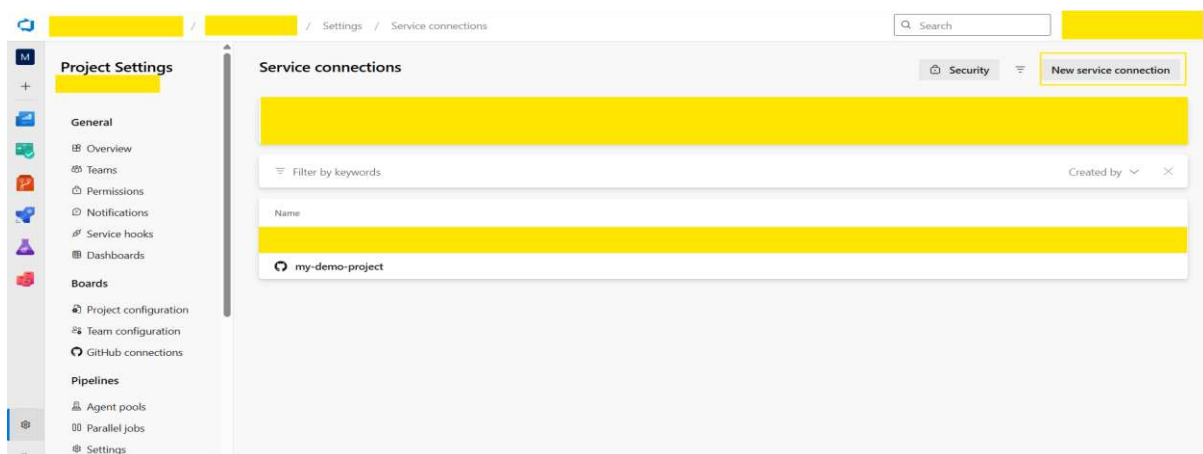
```
[demo@demo-azure-vm helm-repo-for-bitnami-postgresql-primary-secondary-only-service]$ kubectl get pods -n kafka --context=gke_wise-trainer-244916_us-central1-multi-cloud-gke-cluster
```

NAME	READY	STATUS	RESTARTS	AGE
kafka-with-kraft-broker-0	1/1	Running	0	6m2s
kafka-with-kraft-broker-1	1/1	Running	0	6m2s
kafka-with-kraft-broker-2	1/1	Running	0	6m2s
kafka-with-kraft-controller-0	1/1	Running	0	6m2s
kafka-with-kraft-controller-1	1/1	Running	0	6m2s
kafka-with-kraft-controller-2	1/1	Running	0	6m2s

```
[demo@demo-azure-vm helm-repo-for-bitnami-postgresql-primary-secondary-only-service]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1-multi-cloud-gke-cluster
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kafka-with-kraft	ClusterIP	172.19.92.119	<none>	9092/TCP	6m11s
kafka-with-kraft-broker-headless	ClusterIP	None	<none>	9094/TCP, 9092/TCP	6m11s
kafka-with-kraft-controller-headless	ClusterIP	None	<none>	9094/TCP, 9092/TCP, 9093/TCP	6m11s

Created Service Connection for Docker Hub Private Registry as shown in the screenshot attached below. Go to the Azure DevOps Organisation > Azure DevOps Project > Project Settings > Service Connections > New Service Connection.



New service connection



Choose a service or connection type



Docker Registry

[Learn more](#)

Next

New Docker Registry service connection



Registry type

☒ Docker Hub ☐ Others ☐ Azure Container Registry

Docker Registry

Docker ID

Docker Password

Email (optional)

Verify

Service connection details

Service Connection Name

Description (optional)

Security

☒ Grant access permission to all pipelines

[Learn more](#)
[Troubleshoot](#)

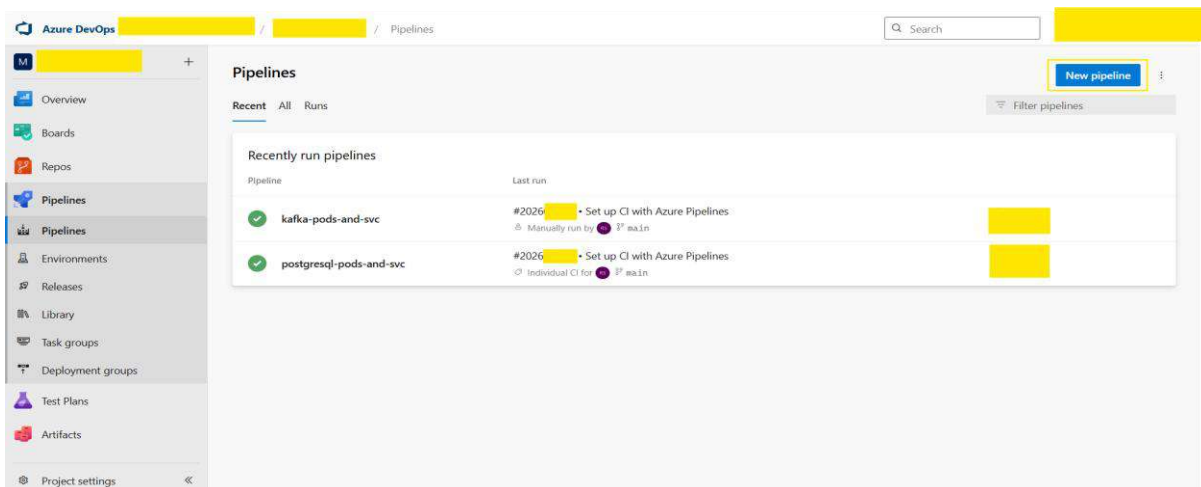
Back

Verify and save





Then created the Azure DevOps Pipeline to deploy the Spring Boot Application Pods and Service as shown in the screenshot attached below.

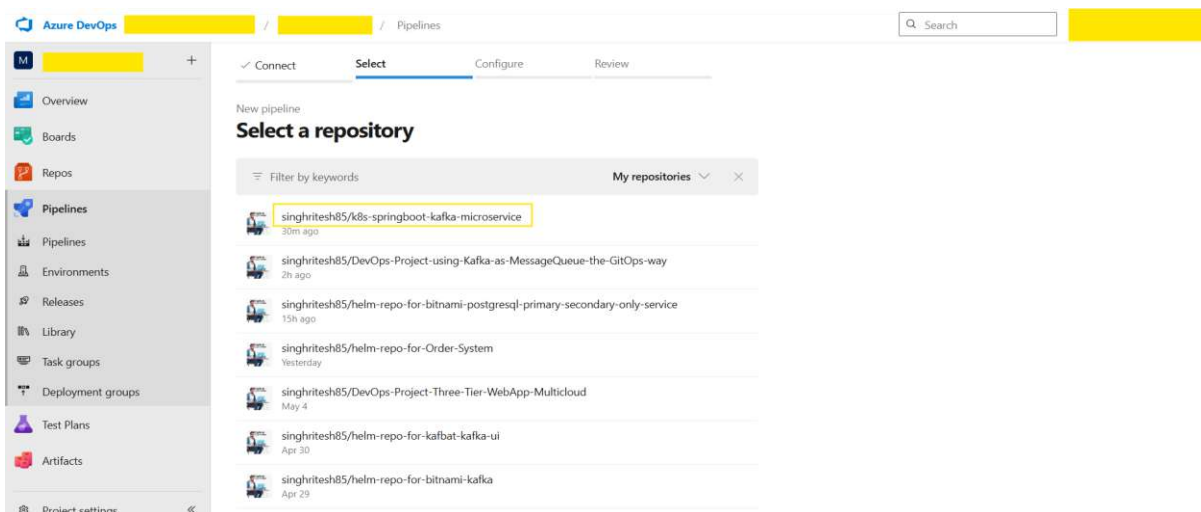


Authenticate Docker Hub Registry from k8s-management-node as shown in the screenshot attached below.

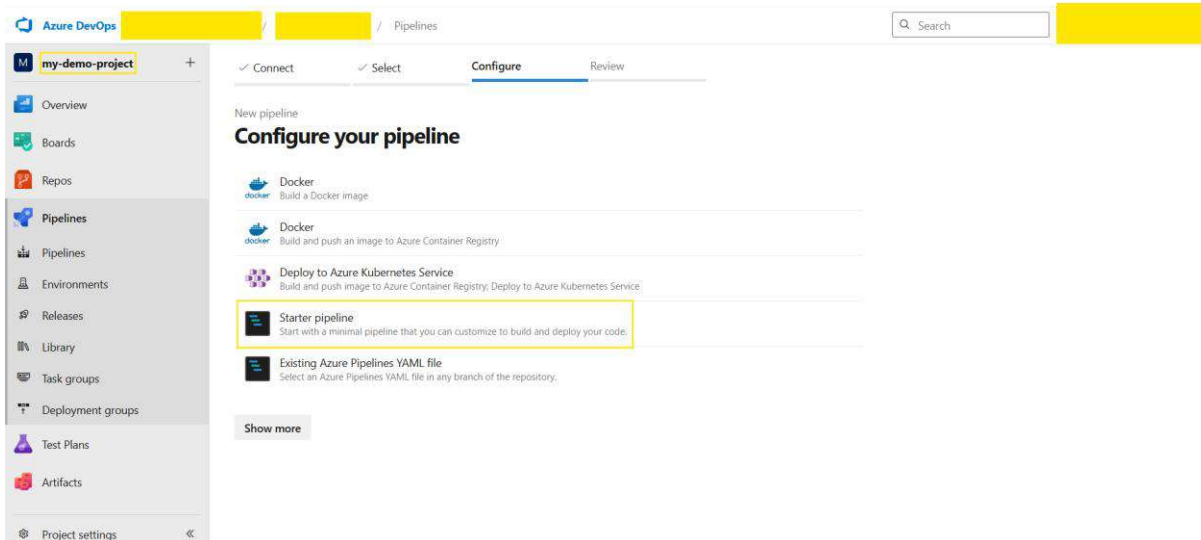
```
[demo@demo-azure-vm order-service]$ docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations using SSO. Learn more at https://docs.docker.com/go/access-tokens/

Username: [redacted]
Password: [redacted]
WARNING! Your password will be stored unencrypted in /home/demo/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

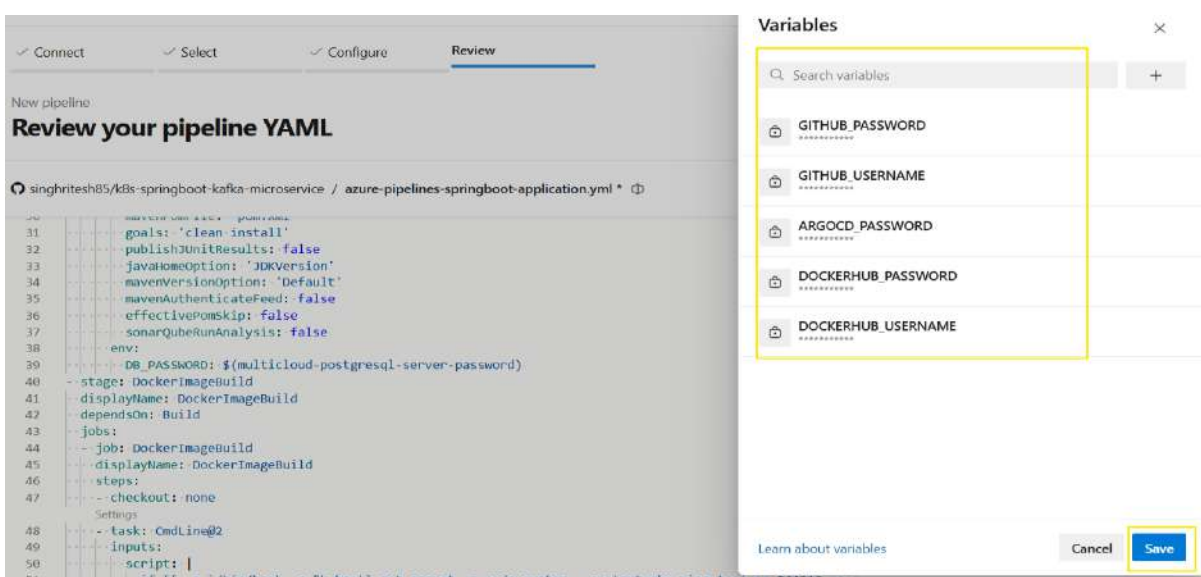
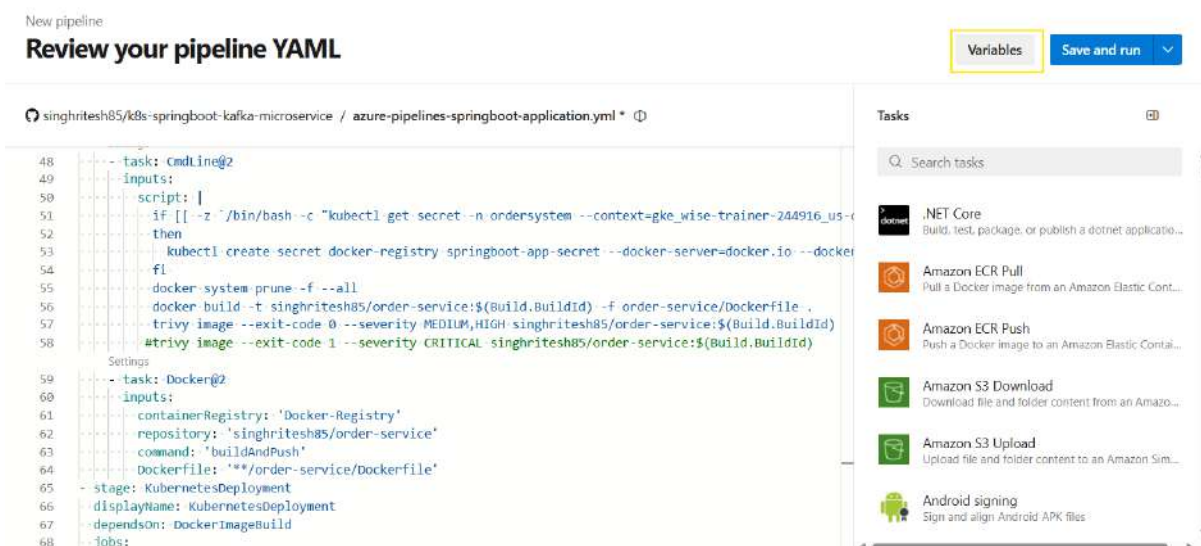
Login Succeeded
```



Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>



Before Saving and Running the Pipeline create five variables named as DOCKERHUB_USERNAME, DOCKERHUB_PASSWORD, ARGOCD_PASSWORD, GITHUB_USERNAME and GITHUB_PASSWORD in Azure DevOps Pipeline as discussed earlier.



Screenshot of all the Azure DevOps pipeline after its successful execution is as shown in the screenshot attached below.

The screenshot shows the 'Pipelines' section in Azure DevOps. Under 'Recently run pipelines', there are three entries:

- springboot-app-pod-and-svc**: Status is 'Succeeded', last run on 2026-05-14T13:14:53.232Z. Description: 'Update azure-pipelines-springboot-application.yml for Azure Pipelines'.
- kafka-pods-and-svc**: Status is 'Succeeded', last run on 2026-05-14T13:14:53.232Z. Description: 'Set up CI with Azure Pipelines'.
- postgresql-pods-and-svc**: Status is 'Succeeded', last run on 2026-05-14T13:14:53.232Z. Description: 'Set up CI with Azure Pipelines'.

```
[demo@demo-azure-vm ~]$ kubectl get all -n ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                READY   STATUS    RESTARTS   AGE
pod/order-service-folo-8f45657cf-wznnw 2/2     Running   0           60s

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/order-service-folo             ClusterIP     172.19.115.208 <none>        80/TCP      61s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/order-service-folo    1/1     1             1           61s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/order-service-folo-8f45657cf 1         1         1       60s
```

After successful deployment I checked the Application Logs as shown in screenshot attached below and found pod was running successfully.

```
[demo@demo-azure-vm ~]$ kubectl logs -f order-service-folo-8f45657cf-wznnw -n ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

2026-05-14T13:14:53.232Z DEBUG 1 --- [order-service] [ntainer@0-0-C-1] o.a.k.c.c.internals.AbstractFetch : [Consumer clientId=consumer-order-service-group-2, groupId=order-service-group] Removing pending request for fetch session: 877087600 for node: kafka-with-kraft-controller-1.kafka-with-kraft-controller-headless.kafka.svc.cluster.local:9092 (id: 1 rack: null)
2026-05-14T13:14:53.232Z DEBUG 1 --- [order-service] [ntainer@0-0-C-1] o.a.k.c.c.internals.AbstractFetch : [Consumer clientId=consumer-order-service-group-2, groupId=order-service-group] Added read_uncommitted fetch request for partition payment-events-0 at position FetchPosition{offset=0, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[kafka-with-kraft-controller-1.kafka-with-kraft-controller-headless.kafka.svc.cluster.local:9092 (id: 1 rack: null)], epoch=0}} to node kafka-with-kraft-controller-1.kafka-with-kraft-controller-headless.kafka.svc.cluster.local:9092 (id: 1 rack: null)
2026-05-14T13:14:53.232Z DEBUG 1 --- [order-service] [ntainer@0-0-C-1] o.a.k.c.c.internals.FetchSessionHandler : [Consumer clientId=consumer-order-service-group-2, groupId=order-service-group] Built incremental fetch (sessionId=877087600, epoch=39) for node 1. Added 0 partition(s), altered 0 partition(s), removed 0 partition(s), replaced 0 partition(s) out of 1 partition(s)
2026-05-14T13:14:53.232Z DEBUG 1 --- [order-service] [ntainer@0-0-C-1] o.a.k.c.c.internals.AbstractFetch : [Consumer clientId=consumer-order-service-group-2, groupId=order-service-group] Sending read_uncommitted IncrementalFetchRequest{toSend=(), toForget=(), toReplace=(), implied=payment-events-0, canUseTopicIds=true} to broker kafka-with-kraft-controller-1.kafka-with-kraft-controller-headless.kafka.svc.cluster.local:9092 (id: 1 rack: null)
2026-05-14T13:14:53.232Z DEBUG 1 --- [order-service] [ntainer@0-0-C-1] o.a.k.c.c.internals.AbstractFetch : [Consumer clientId=consumer-order-service-group-2, groupId=order-service-group] Adding pending request for node kafka-with-kraft-controller-1.kafka-with-kraft-controller-headless.kafka.svc.cluster.local:9092 (id: 1 rack: null)
2026-05-14T13:14:53.232Z DEBUG 1 --- [order-service] [ntainer@0-0-C-1] org.apache.kafka.clients.NetworkClient : [Consumer clientId=consumer-order-service-group-2, groupId=order-service-group] Sending FETCH request with header RequestHeader{apiKey=FETCH, apiVersion=16, clientId=consumer-order-service-group-2, correlationId=97, headerVersion=2} and timeout 30000 to node 1: FetchRequestData{clusterId=null, replicaId=-1, replicaState=ReplicaState{replicaId=-1, replicaEpoch=-1}, maxWaitMs=500, minBytes=1, maxBytes=52428800, isolationLevel=0, sessionId=877087600, sessionEpoch=39, topics=[], forgottenTopicsData=[], rackId=''}
2026-05-14T13:14:53.232Z DEBUG 1 --- [order-service] [ntainer@2-0-C-1] o.a.k.c.c.internals.OffsetFetcher : [Consumer clientId=consumer-order-service-group-1, groupId=order-service-group] Sending ListOffsetRequest ListOffsetRequestData{replicaId=-1, isolationLevel=0, topics=[ListOffsetTopic{name='order-events', partitions=[ListOffsetPartition{partitionIndex=0, currentLeaderEpoch=0, timestamp=-2, maxNumOffsets=1}]}]} to broker kafka-with-kraft-broker-1.kafka-with-kraft-broker-headless.kafka.svc.cluster.local:9092 (id: 101 rack: null)
```

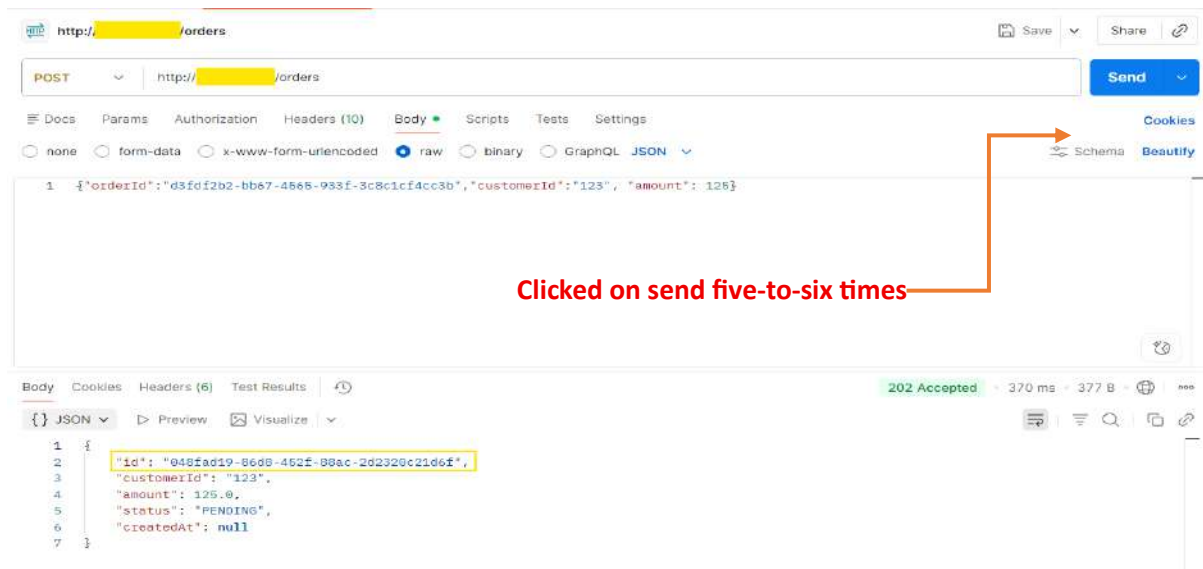
After successful deployment of all resources, I did verification of the system's functionality. I conducted an end-to-end test to check the complete event flow, from order creation to payment confirmation and status update. I edited the order service from ClusterIP to LoadBalancer and shown my findings in the screenshot attached below.

```
[demo@demo-azure-vm ~]$ kubectl edit svc order-service-folo -n ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
selector:
  app.kubernetes.io/instance: order-service
  app.kubernetes.io/name: folo
  sessionAffinity: None
type: LoadBalancer
status:
  loadBalancer: {}
```

```
[demo@demo-azure-vm ~]$ kubectl get svc -n ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
order-service-folo                  LoadBalancer   172.17.0.11     35.186.186.186   80:32375/TCP     22m
```

Created POST request using the Postman App as shown in the screenshot attached below.



You can also create the POST request using the command as shown below.

```
curl -X POST http://XXX.XXX.XXX.XXX/orders -H "Content-Type: application/json" -d
'{"orderId":"d3fdf2b2-bb67-4565-933f-3c8c1cf4cc3b","customerId":"123","amount": 125}'
```

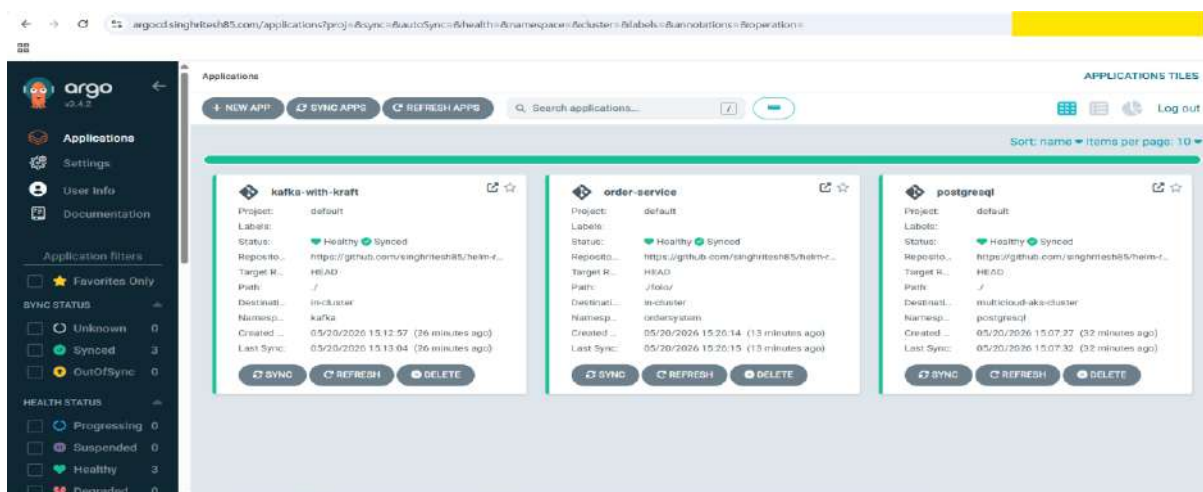
External IP of the LoadBalancer svc Order-service

1. The Order Service sends an Order Created event to Kafka.
2. The Payment Service consumes this event and sends back a Payment Completed event.
3. The Order Service then updates the order status.

```
kubectl exec -it kafka-with-kraft-broker-1 -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -- bash

I have no name!@kafka-with-kraft-broker-0:/# kafka-console-consumer.sh --topic order-events --from-beginning --bootstrap-server localhost:9092
{"orderId":"72f58988-7fa0-491e-9a5e-d6499b5268d7","customerId":"123","amount":125.0}
{"orderId":"154a8b83-6abb-485e-b50e-b8555eb5569","customerId":"123","amount":125.0}
{"orderId":"6361ea61-7bb3-4dd0-a48d-4a78dc3a8524","customerId":"123","amount":125.0}
{"orderId":"b0556f99-419e-4651-b0b6-62e1edc60fa6","customerId":"123","amount":125.0}
{"orderId":"048fad19-86d8-452f-88ac-2d2320c21d6f","customerId":"123","amount":125.0}
```

Screenshot of ArgoCD after the Deployment is as shown in the screenshot attached below.



Installation of Kafka-UI

To install kafka-ui here I used the helm chart present in the GitHub Repo <https://github.com/singhritesh85/helm-repo-for-kafbat-kafka-ui.git>. This helm chart is provided by Kafbat (priorly known as provectus) As this is a non-production environment, So I installed kafka with PLAINTEXT and later installed kafka-ui for kafka in PLAINTEXT. In **module-1** as explained earlier I used ArgoCD to deploy Postgresql Pods and Services in AKS Cluster, Kafka Pods and Services in GKE Standard Cluster and Spring Boot Application Pod and Service in GKE Standard Cluster. I used helm command-line to deploy the PostgreSQL only service in GKE Standard Cluster the reason behind it was in ArgoCD I can deploy two Applications with the same name, if you do so it will update the already existing application with the same name. But here I am using multicloud approach in which PostgreSQL Pods and Services exist in AKS Cluster and Application Pods in GKE Cluster and to communicate Application Pod with PostgreSQL Pod you must create PostgreSQL only service in the Kubernetes Cluster where Application Pods present and that is the reason I used helm command line to deploy the PostgreSQL only service. I had used helm command line to deploy kafka-ui here, if you want you can use ArgoCD as well. I had chosen Helm for this deployment because it was a simple single-release use case. ArgoCD is typically more valuable for large-scale GitOps workflows and managing multiple releases across environments.

Installation of Kafka-UI for PLAINTEXT

```
helm upgrade --install kafka-ui ./ -f ./values.yaml --set envs.config.KAFKA_CLUSTERS_0_NAME=kafka-with-kraft,envs.config.KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka-with-kraft,kafka.svc.cluster.local:9092 --namespace=kafka --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ git clone https://github.com/singhritesh85/helm-repo-for-kafbat-kafka-ui.git
Cloning into 'helm-repo-for-kafbat-kafka-ui'...
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (22/22), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 22 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (22/22), 18.77 KiB | 9.39 MiB/s, done.
Resolving deltas: 100% (3/3), done.
[demo@demo-azure-vm ~]$ cd helm-repo-for-kafbat-kafka-ui/
[demo@demo-azure-vm helm-repo-for-kafbat-kafka-ui]$ helm upgrade --install kafka-ui ./ -f ./values.yaml --set envs.config.KAFKA_CLUSTERS_0_NAME=kafka-with-kraft,envs.config.KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka-with-kraft,kafka.svc.cluster.local:9092 --namespace=kafka --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Release "kafka-ui" does not exist. Installing it now.
NAME: kafka-ui
LAST DEPLOYED: Fri May 15 09:57:29 2026
NAMESPACE: kafka
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
1. Get the application URL by running these commands:
   export POD_NAME=$(kubectl get pods --namespace kafka -l "app.kubernetes.io/name=kafka-ui,app.kubernetes.io/instance=kafka-ui" -o jsonpath="{.items[0].metadata.name}")
   echo "Visit http://127.0.0.1:8080 to use your application"
   kubectl --namespace kafka port-forward $POD_NAME 8080:8080
[demo@demo-azure-vm helm-repo-for-kafbat-kafka-ui]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
kafka-ui                            ClusterIP           172.19.245.27   <none>            80/TCP           14s
kafka-with-kraft                    ClusterIP           172.19.132.27   <none>            9092/TCP         3h13m
kafka-with-kraft-broker-headless    ClusterIP           None            <none>            9094/TCP,9092/TCP 3h13m
kafka-with-kraft-controller-headless ClusterIP           None            <none>            9094/TCP,9092/TCP,9093/TCP 3h13m
```

As the Gateway-API Controller is already installed and hence you can create the URL by directly applying the YAML file **gateway-api-kafka-ui.yaml**. For your reference this file is present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-1/**.

```
[demo@demo-azure-vm ~]$ kubectl create secret tls tls-kafka-ui-ingress --cert=693ff37a406fc433.crt --key=server.key -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f gateway-api-kafka-ui.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
gateway.gateway.networking.k8s.io/kafka-ui-gateway created
httproute.gateway.networking.k8s.io/kafka-ui-route created
[demo@demo-azure-vm ~]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kafka-ui	ClusterIP		<none>	80/TCP	
kafka-ui-gateway-nginx	LoadBalancer		34.	80:32437/TCP,443:30609/TCP	
kafka-with-kraft	ClusterIP		<none>	9092/TCP	
kafka-with-kraft-broker-headless	ClusterIP		<none>	9094/TCP,9092/TCP	
kafka-with-kraft-controller-headless	ClusterIP		<none>	9094/TCP,9092/TCP,9093/TCP	

Create Record-Set of A-Type in Azure DNS Zone by doing the entry of External IP obtained in the screenshot attached above.

Add record set

singhritesh85.com

Name

kafka-ui

.singhritesh85.com

Type

A – IPv4 Address records

Alias record set ⓘ

No

TTL *

1

TTL unit

Hours

IP address



0.0.0.0

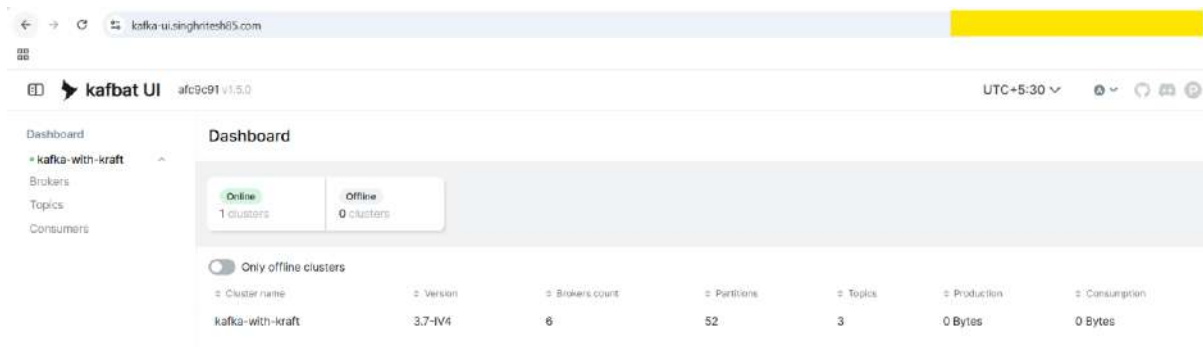
Add

Cancel

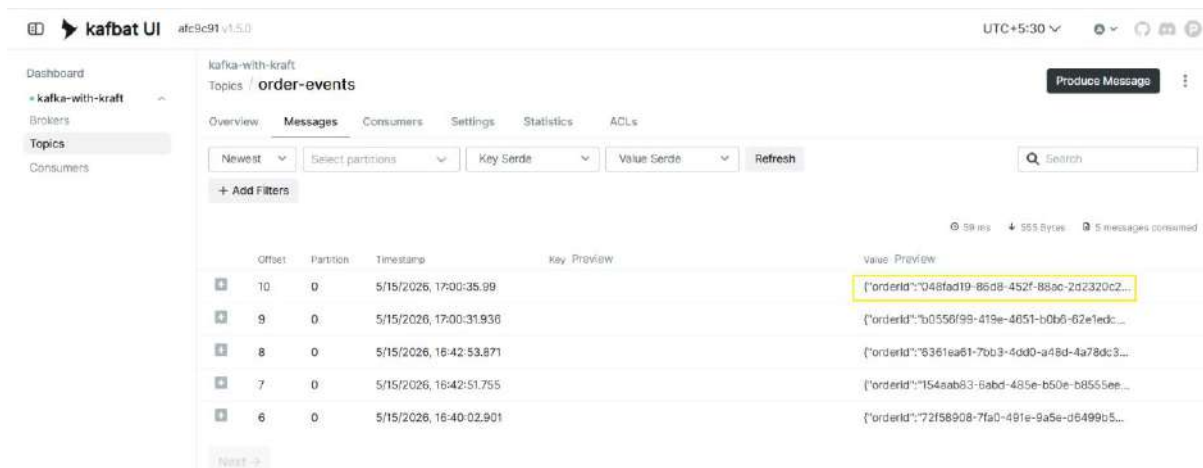


Give feedback

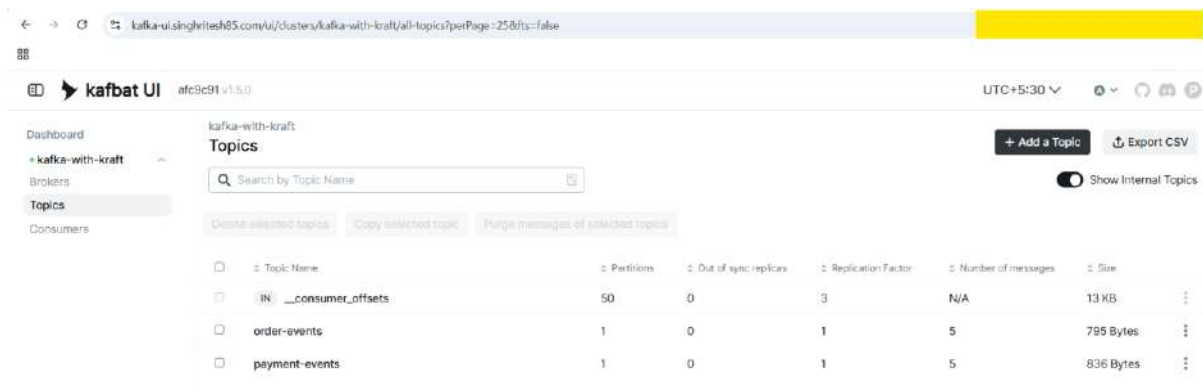
Finally, I was able to login into kafka-ui as shown in the screenshot attached below.



The same messages you can see in the topic order-events from the kafka-ui as well.



The same messages you will find in the topic payment-events and in the postgresql database mederma, inside the table orders as shown in the screenshot attached below.



```
[demo@demo-azure-vm ~]$ kubectl exec -it postgresql-primary-0 -n postgresql --context=multicloud-aks-cluster -- bash
I have no name!@postgresql-primary-0:/$ psql -h localhost -d mederma -U dexter --password
Password:
psql (16.3)
Type "help" for help.

mederma=> \l

      Name | Owner | Encoding | Locale Provider | List of databases
Collate | Ctype | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 mederma | dexter | UTF8     | libc           | en_US.UTF-8 | en_US.UTF-8 |             |             | =Tc/dexter +
                                           |             |             |             | dexter=CTc/dexter
 postgres | postgres | UTF8     | libc           | en_US.UTF-8 | en_US.UTF-8 |             |             | =c/postgres +
 template0 | postgres | UTF8     | libc           | en_US.UTF-8 | en_US.UTF-8 |             |             | postgres=CTc/postgres
 template1 | postgres | UTF8     | libc           | en_US.UTF-8 | en_US.UTF-8 |             |             | =c/postgres +
                                           |             |             |             | postgres=CTc/postgres
(4 rows)

mederma=> \c database_name
Password:
Previous connection kept
mederma=> \c mederma
Password:
You are now connected to database "mederma" as user "dexter".
mederma=> \c mederma
Password:
You are now connected to database "mederma" as user "dexter".
mederma=> \dt
      List of relations
 Schema | Name | Type | Owner
-----+-----+-----+-----
 public | orders | table | dexter
(1 row)
```

```
mederma=> select * from orders;
```

Each time you create an order it will do the entry in the database mederma, inside the table orders.

For deployment using Azure DevOps Pipeline through ArgoCD, you can see all azure-pipelines.yml file present in the GitHub Repo <https://github.com/singhritesh85/k8s-springboot-kafka-microservice.git>. The azure-pipelines.yml file to deploy spring boot application is present at the path **azure-pipelines-springboot-application.yml**.

The source code present in the GitHub Repo <https://github.com/singhritesh85/k8s-springboot-kafka-microservice.git> at the path **order-service/src/main/resources/application.yaml** contains PostgreSQL password which was stored in Azure Keyvault secret. This PostgreSQL password was not hardcoded in this file or not present in .env file in the Repo but will be taken from Azure Keyvault dynamically during the execution of Azure DevOps Pipeline using the Azure DevOps Pipeline task as shown in the screenshot attached below. To authenticate and authorize Azure DevOps Account with Azure Portal I used Azure Service Principal and the resources which I created initially using Terraform there I authorize the Service Principal to access the Keyvault secret with the Role **"Key Vault Secrets User"** as shown in the screenshot attached below.

```
### Allow Azure Key Vault Access to AzureAD Service Principal for Azure DevOps
data "azuread_service_principal" "azure_devops_sp" {
  display_name = "DevOpsDemoServicePrincipal"
}

resource "azurerm_role_assignment" "kv_secrets_user" {
  scope                = azurerm_key_vault.key_vault_postgresql.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = data.azuread_service_principal.azure_devops_sp.object_id
}
```

k8s-springboot-kafka-microservice / order-service / src / main / resources / application.yaml 

 singhritesh85 Update application.yaml

Code Blame 27 lines (24 loc) · 655 Bytes

```

1  spring:
2    application:
3      name: order-service
4
5    kafka:
6      bootstrap-servers: kafka-with-kraft.kafka.svc.cluster.local:9092
7      consumer:
8        group-id: order-service-group
9        auto-offset-reset: earliest
10
11     datasource:
12       url: jdbc:postgresql://postgresql-primary.postgresql.svc.cluster.local:5432/mederma
13       username: dexter
14       password: ${DB_PASSWORD}
15       driver-class-name: org.postgresql.Driver
16
17     jpa:
18       hibernate:
19         ddl-auto: update
20         show-sql: true
21         database-platform: org.hibernate.dialect.PostgreSQLDialect
22         properties:
23           hibernate:
24             format_sql: true
25
26     logging:
27       level:
28         org.apache.kafka: DEBUG

```

steps:

```

- task: AzureKeyVault@2
  inputs:
    azureSubscription: 'Azure DevOps Service Connection'
    KeyVaultName: 'multicloud'
    SecretsFilter: '*'
    RunAsPreJob: false

```

```

- task: Maven@4
  inputs:
    azureSubscription: 'Azure DevOps Service Connection'
    mavenPomFile: 'order-service/pom.xml'
    goals: 'clean install'
    publishJUnitResults: false
    javaHomeOption: 'JDKVersion'
    mavenVersionOption: 'Default'
    mavenAuthenticateFeed: false
    effectivePomSkip: false
    sonarQubeRunAnalysis: false

```

env:

```
DB_PASSWORD: $(multicloud-postgresql-server-password)
```

Above I installed kafka with PLAINTEXT mode using the bitnami helm chart present in the GitHub Repo <https://github.com/singhritesh85/helm-repo-for-bitnami-kafka.git> which means disabled TLS/SSL encryption and skips any client authentication which is not the approach acceptable for production environment. This approach is not acceptable for production environment as it cannot pass the SOC 2 or HIPAA compliance.

Kafka SASL_PLAINTEXT using helm (not recommended for production)

```
[demo@demo-azure-vm ~]$ cd helm-repo-for-bitnami-kafka/
```

```
[demo@demo-azure-vm helm-repo-for-bitnami-kafka]$ helm upgrade --install kafka-with-kraft ./ -f ./values.yaml --set kraft.enabled=true --set zookeeper.enabled=false --set controller.replicaCount=3,controller.persistence.enabled=true,controller.persistence.size=1Gi,controller.persistence.storageClass=standard-rwo,broker.replicaCount=3,broker.persistence.storageClass=standard-rwo,broker.persistence.enabled=true,broker.persistence.size=1Gi,controller.resources.limits.cpu=500m,controller.resources.requests.cpu=250m,controller.resources.limits.memory=512Mi,controller.resources.requests.memory=256Mi,broker.resources.limits.cpu=500m,broker.resources.requests.cpu=250m,broker.resources.limits.memory=512Mi,broker.resources.requests.memory=256Mi,listeners.client.protocol=SASL_PLAINTEXT,listeners.controller.protocol=SASL_PLAINTEXT,listeners.interbroker.protocol=SASL_PLAINTEXT,sasl.enabledMechanisms=PLAIN,sasl.interBrokerMechanism=PLAIN,sasl.controllerMechanism=PLAIN,sasl.client.users={user1},sasl.client.passwords={strong-password-123} --create-namespace --namespace=kafka --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm helm-repo-for-bitnami-kafka]$ kubectl get pods -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	READY	STATUS	RESTARTS	AGE
kafka-with-kraft-broker-0	1/1	Running	0	5m38s
kafka-with-kraft-broker-1	1/1	Running	0	5m38s
kafka-with-kraft-broker-2	1/1	Running	0	5m38s
kafka-with-kraft-controller-0	1/1	Running	0	5m38s
kafka-with-kraft-controller-1	1/1	Running	0	5m38s
kafka-with-kraft-controller-2	1/1	Running	0	5m38s

```
[demo@demo-azure-vm ~]$ cat client-sasl-plaintext.properties
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username="user1" password="strong-password-123";
[demo@demo-azure-vm ~]$ kubectl cp client-sasl-plaintext.properties kafka/kafka-with-kraft-broker-0:/tmp/client-sasl-plaintext.properties --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Defaulted container "kafka" out of: kafka, kafka-init (init)
[demo@demo-azure-vm ~]$ kubectl exec -it kafka-with-kraft-broker-0 -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -- bash
Defaulted container "kafka" out of: kafka, kafka-init (init)
I have no name!@kafka-with-kraft-broker-0:/ $ kafka-topics.sh --bootstrap-server kafka-with-kraft-kafka.svc.cluster.local:9092 --create --topic secure-test --command-config /tmp/client-sasl-plaintext.properties
Created topic secure-test.
```

```
helm upgrade --install kafka-with-kraft ./ -f ./values.yaml --set kraft.enabled=true --set zookeeper.enabled=false --set controller.replicaCount=3,controller.persistence.enabled=true,controller.persistence.size=1Gi,controller.persistence.storageClass=standard-rwo,broker.replicaCount=3,broker.persistence.storageClass=standard-rwo,broker.persistence.enabled=true,broker.persistence.size=1Gi,controller.resources.limits.cpu=500m,controller.resources.requests.cpu=250m,controller.resources.limits.memory=512Mi,controller.resources.requests.memory=256Mi,broker.resources.limits.cpu=500m,broker.resources.requests.cpu=250m,broker.resources.limits.memory=512Mi,broker.resources.requests.memory=256Mi,listeners.client.protocol=SASL_PLAINTEXT,listeners.controller.protocol=SASL_PLAINTEXT,listeners.interbroker.protocol=SASL_PLAINTEXT,sasl.enabledMechanisms=PLAIN,sasl.interBrokerMechanism=PLAIN,sasl.controllerMechanism=PLAIN,sasl.client.users={user1},sasl.client.passwords={strong-password-123} --create-namespace --namespace=kafka --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
cat client-sasl-plaintext.properties
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username="user1" password="strong-password-123";
```

Kafka SASL_SSL using helm and without using mTLS


```
[demo@demo-azure-vm helm-repo-for-bitnami-kafka]$ kubectl config get-contexts
CURRENT  NAME                                     NAMESPACE                                     CLUSTER                                     AUTHINFO
gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
* central1_multicloud-gke-cluster                                     multicloud-aks-cluster                                     multicloud-aks-cluster                                     clusterUser_multicloud-rg_
multicloud-aks-cluster
```

[demo@demo-azure-vm helm-repo-for-bitnami-kafka]\$ kubectl config use-context gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

Switched to context "gke_wise-trainer-244916_us-central1_multicloud-gke-cluster".

```
[demo@demo-azure-vm helm-repo-for-bitnami-kafka]$ kubectl config get-contexts
CURRENT  NAME                                     NAMESPACE                                     CLUSTER                                     AUTHINFO
* gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
central1_multicloud-gke-cluster                                     multicloud-aks-cluster                                     multicloud-aks-cluster                                     clusterUser_multicloud-rg_
multicloud-aks-cluster
```

Default Context = GKE Cluster

```
[demo@demo-azure-vm ~]$ kubectl create ns kafka
namespace/kafka created
[demo@demo-azure-vm ~]$
[demo@demo-azure-vm ~]$ openssl req -new -x509 -keyout ca-key -out ca-cert -days 182500 -subj "/CN=Kafka-CA" -nodes
Generating a RSA private key
.....+++++
.....+++++
writing new private key to 'ca-key'
-----
[demo@demo-azure-vm ~]$ keytool -genkey -keystore kafka.keystore.jks -alias kafka-node -validity 182500 -keyalg RSA -dname "CN=kafka-with-kraft" -storepass Dexter@123
Generating 2,048 bit RSA key pair and self-signed certificate (SHA256WithRSA) with a validity of 182,500 days
for: CN=kafka-with-kraft
[demo@demo-azure-vm ~]$ keytool -keystore kafka.keystore.jks -alias kafka-node -certreq -file cert-file -storepass Dexter@123
Certificate request was generated
[demo@demo-azure-vm ~]$ openssl x509 -req -CA ca-cert -CAkey ca-key -in cert-file -out cert-signed -days 182500 -CAcreateserial -passin pass:Dexter@123 -extfile <(printf "subjectAltName=DNS:*.kafka-with-kraft-broker-headless.kafka.svc.cluster.local,DNS:*.kafka-with-kraft-controller-headless.kafka.svc.cluster.local,DNS:*.kafka-with-kraft.kafka.svc.cluster.local")
Signature ok
subject=CN = kafka-with-kraft
Getting CA Private Key
[demo@demo-azure-vm ~]$ keytool -keystore kafka.keystore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
Certificate was added to keystore
[demo@demo-azure-vm ~]$ keytool -keystore kafka.keystore.jks -alias kafka-node -import -file cert-signed -storepass Dexter@123
Certificate reply was installed in keystore
[demo@demo-azure-vm ~]$ keytool -keystore kafka.truststore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
Certificate was added to keystore
[demo@demo-azure-vm ~]$
[demo@demo-azure-vm ~]$ kubectl create secret generic kafka-tls-secret --from-file=kafka.keystore.jks=kafka.keystore.jks --from-file=kafka.truststore.jks=kafka.truststore.jks -n kafka
secret/kafka-tls-secret created
```

```
[demo@demo-azure-vm helm-repo-for-bitnami-kafka]$ helm upgrade --install kafka-with-kraft ./ -f ./values.yaml --set kraft.enabled=true --set zookeeper.enabled=false --set controller.replicaCount=3,controller.persistence.enabled=true,controller.persistence.size=1Gi,controller.persistence.storageClass=standard-rwo,broker.replicaCount=3,broker.persistence.enabled=true,broker.persistence.size=1Gi,controller.resources.limits.cpu=500m,controller.resources.requests.cpu=250m,controller.resources.limits.memory=512Mi,controller.resources.requests.memory=256Mi,broker.resources.limits.cpu=500m,broker.resources.requests.cpu=250m,broker.resources.limits.memory=512Mi,broker.resources.requests.memory=256Mi,listeners.client.protocol=SASL_SSL,listeners.controller.protocol=SASL_SSL,listeners.interbroker.protocol=SASL_SSL,sasl.enabledMechanism=PLAIN,sasl.interbrokerMechanism=PLAIN,sasl.controllerMechanism=PLAIN,sasl.client.users={user1},sasl.client.passwords={strong-password-123},tls.type=JKS,tls.existingSecret=kafka-tls-secret,tls.keystorePassword=Dexter@123,tls.truststorePassword=Dexter@123,tls.sslClientAuth=requested --create-namespace --namespace=kafka --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm helm-repo-for-bitnami-kafka]$ kubectl get pods -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster --watch
NAME                                READY   STATUS    RESTARTS   AGE
kafka-with-kraft-broker-0           1/1     Running   0           53s
kafka-with-kraft-broker-1           1/1     Running   0           53s
kafka-with-kraft-broker-2           1/1     Running   0           53s
kafka-with-kraft-controller-0       1/1     Running   0           53s
kafka-with-kraft-controller-1       1/1     Running   0           53s
kafka-with-kraft-controller-2       1/1     Running   0           53s
```

```
[demo@demo-azure-vm ~]$ kubectl cp client-sasl-ssl.properties kafka/kafka-with-kraft-broker-0:/tmp/client-sasl-ssl.properties --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Defaulted container "kafka" out of: kafka, kafka-init (init)
[demo@demo-azure-vm ~]$ kubectl cp kafka.truststore.jks kafka/kafka-with-kraft-broker-0:/tmp/kafka.truststore.jks --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Defaulted container "kafka" out of: kafka, kafka-init (init)
[demo@demo-azure-vm ~]$ kubectl exec -it kafka-with-kraft-broker-0 -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -- bash
Defaulted container "kafka" out of: kafka, kafka-init (init)
I have no name!@kafka-with-kraft-broker-0:/$ kafka-topics.sh --bootstrap-server kafka-with-kraft.kafka.svc.cluster.local:9092 --create --topic secure-test --command-config /tmp/client-sasl-ssl.properties
Created topic secure-test.
I have no name!@kafka-with-kraft-broker-0:/$
exit
[demo@demo-azure-vm ~]$ cat client-sasl-ssl.properties
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username="user1" password="strong-password-123";
ssl.truststore.location=/tmp/kafka.truststore.jks
ssl.truststore.password=Dexter@123
ssl.endpoint.identification.algorithm=
```



```
kubectl create ns kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
openssl req -new -x509 -keyout ca-key -out ca-cert -days 182500 -subj "/CN=Kafka-CA" -nodes
```

```
keytool -genkey -keystore kafka.keystore.jks -alias kafka-node -validity 182500 -keyalg RSA -dname "CN=kafka-with-kraft" -storepass Dexter@123 -keypass Dexter@123
```

```
keytool -keystore kafka.keystore.jks -alias kafka-node -certreq -file cert-file -storepass Dexter@123 -ext "SAN=dns:localhost,dns:*.kafka-with-kraft-broker-headless.kafka.svc.cluster.local,dns:*.kafka-with-kraft-controller-headless.kafka.svc.cluster.local,dns:*.kafka-with-kraft.kafka.svc.cluster.local"
```

```
openssl x509 -req -CA ca-cert -CAkey ca-key -in cert-file -out cert-signed -days 182500 -CAcreateserial -passin pass:Dexter@123 -extfile <(printf "subjectAltName=DNS:localhost,DNS:*.kafka-with-kraft-broker-headless.kafka.svc.cluster.local,DNS:*.kafka-with-kraft-controller-headless.kafka.svc.cluster.local,DNS:*.kafka-with-kraft.kafka.svc.cluster.local")
```

```
keytool -keystore kafka.keystore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
```

```
keytool -keystore kafka.keystore.jks -alias kafka-node -import -file cert-signed -storepass Dexter@123
```

```
keytool -keystore kafka.truststore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
```

```
kubectl create secret generic kafka-tls-secret --from-file=kafka.keystore.jks=kafka.keystore.jks --from-file=kafka.truststore.jks=kafka.truststore.jks -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
helm upgrade --install kafka-with-kraft ./ -f ./values.yaml --set kraft.enabled=true --set zookeeper.enabled=false --set controller.replicaCount=3,controller.persistence.enabled=true,controller.persistence.size=1Gi,controller.persistence.storageClass=standard-rwo,broker.replicaCount=3,broker.persistence.storageClass=standard-rwo,broker.persistence.enabled=true,broker.persistence.size=1Gi,controller.resources.limits.cpu=500m,controller.resources.requests.cpu=250m,controller.resources.limits.memory=512Mi,controller.resources.requests.memory=256Mi,broker.resources.limits.cpu=500m,broker.resources.requests.cpu=250m,broker.resources.limits.memory=512Mi,broker.resources.requests.memory=256Mi,listeners.client.protocol=SASL_SSL,listeners.controller.protocol=SASL_SSL,listeners.interbroker.protocol=SASL_SSL,sasl.enabledMechanisms=PLAIN,sasl.interBrokerMechanism=PLAIN,sasl.controllerMechanism=PLAIN,sasl.client.users={user1},sasl.client.passwords={strong-password-123},tls.type=JKS,tls.existingSecret=kafka-tls-secret,tls.keystorePassword=Dexter@123,tls.truststorePassword=Dexter@123,tls.sslClientAuth=requested --create-namespace --namespace=kafka --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

cat client-sasl-ssl.properties

security.protocol=SASL_SSL

sasl.mechanism=PLAIN

sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required

username="user1" password="strong-password-123";

ssl.truststore.location=/tmp/kafka.truststore.jks

ssl.truststore.password=Dexter@123

ssl.endpoint.identification.algorithm=

kubectl cp client-sasl-ssl.properties kafka/kafka-with-kraft-broker-0:/tmp/client-sasl-ssl.properties --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl cp kafka.truststore.jks kafka/kafka-with-kraft-broker-0:/tmp/kafka.truststore.jks --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl exec -it kafka-with-kraft-broker-0 -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -- bash

kafka-topics.sh --bootstrap-server kafka-with-kraft.kafka.svc.cluster.local:9092 --create --topic secure-test --command-config /tmp/client-sasl-ssl.properties

Kafka SASL SSL using helm and mTLS (recommended for production)

```
[demo@demo-azure-vm ~]$ kubectl create ns kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/kafka created
[demo@demo-azure-vm ~]$
[demo@demo-azure-vm ~]$ openssl req -new -x509 -keyout ca-key -out ca-cert -days 182500 -subj "/CN=Kafka-CA" -nodes
Generating a RSA private key
.....+++++
.....+++++
writing new private key to 'ca-key'
-----
[demo@demo-azure-vm ~]$ keytool -genkey -keystore kafka.keystore.jks -alias kafka-node -validity 182500 -keyalg RSA -dname "CN=kafka-with-kraft" -storepass Dexter@123
Generating 2,048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 182,500 days
for: CN=kafka-with-kraft
[demo@demo-azure-vm ~]$ keytool -keystore kafka.keystore.jks -alias kafka-node -certreq -file cert-file -storepass Dexter@123 -ext SAN=dns:localhost,dns:*.kafka-with-kraft-broker-headless.kafka.svc.cluster.local,dns:*.kafka-with-kraft-controller-headless.kafka.svc.cluster.local
Certificate request self-signature ok
subject=CN = kafka-with-kraft
Getting CA Private Key
[demo@demo-azure-vm ~]$ keytool -keystore kafka.keystore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
Certificate was added to keystore
[demo@demo-azure-vm ~]$ keytool -keystore kafka.keystore.jks -alias kafka-node -import -file cert-signed -storepass Dexter@123
Certificate reply was installed to keystore
[demo@demo-azure-vm ~]$ keytool -keystore kafka.truststore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
Certificate was added to keystore
[demo@demo-azure-vm ~]$
[demo@demo-azure-vm ~]$ kubectl create secret generic kafka-tls-secret --from-file=kafka.keystore.jks=kafka.keystore.jks --from-file=kafka.truststore.jks=kafka.truststore.jks -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/kafka-tls-secret created

[demo@demo-azure-vm ~]$ helm repo add bitnami-kafka
bitnami-kafka has been added to your repository
[demo@demo-azure-vm ~]$ helm upgrade --install kafka-with-kraft ./ --values values.yaml --set kraft.enabled=true --set zookeeper.enabled=false --set controller.replicaCount=3,controller.persistence.enabled=true,controller.persistence.size=1Gi,controller.persistence.storageClass=standard-rwo,broker.replicaCount=3,broker.persistence.enabled=true,broker.persistence.size=1Gi,controller.resources.limits.cpu=500m,controller.resources.requests.cpu=250m,controller.resources.limits.memory=512Mi,controller.resources.requests.memory=256Mi,broker.resources.limits.cpu=500m,broker.resources.requests.cpu=250m,broker.resources.limits.memory=512Mi,broker.resources.requests.memory=256Mi,listeners.client.protocol=SASL_SSL,listeners.interbroker.protocol=SASL_SSL,sasl.enabledMechanisms=PLAIN,sasl.interbrokerMechanism=PLAIN,sasl.controllerMechanism=PLAIN,sasl.client.users={user1},sasl.client.passwords={strong-password-123},tls.type=JKS,tls.existingSecret=kafka-tls-secret,tls.keystorePassword=Dexter@123,tls.truststorePassword=Dexter@123,tls.sslClientAuth=required --create-namespace --namespace=kafka --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm helm-repo-for-bitnami-kafka]$ kubectl get pods -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                READY   STATUS    RESTARTS   AGE
kafka-with-kraft-broker-0          1/1     Running   0           57s
kafka-with-kraft-broker-1          1/1     Running   0           57s
kafka-with-kraft-broker-2          1/1     Running   0           57s
kafka-with-kraft-controller-0      1/1     Running   0           57s
kafka-with-kraft-controller-1      1/1     Running   0           57s
kafka-with-kraft-controller-2      1/1     Running   0           57s

[demo@demo-azure-vm ~]$ kubectl cp client-sasl-ssl.properties kafka/kafka-with-kraft-broker-0:/tmp/client-sasl-ssl.properties --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Defaulted container "kafka" out of: kafka, kafka-init (init)
[demo@demo-azure-vm ~]$ kubectl cp kafka.truststore.jks kafka/kafka-with-kraft-broker-0:/tmp/kafka.truststore.jks --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Defaulted container "kafka" out of: kafka, kafka-init (init)
[demo@demo-azure-vm ~]$ kubectl cp kafka.keystore.jks kafka/kafka-with-kraft-broker-0:/tmp/kafka.keystore.jks --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Defaulted container "kafka" out of: kafka, kafka-init (init)
[demo@demo-azure-vm ~]$ kubectl exec -it kafka-with-kraft-broker-0 -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -- bash
Defaulted container "kafka" out of: kafka, kafka-init (init)
I have no name!@kafka-with-kraft-broker-0:/$ kafka-topics.sh --bootstrap-server kafka-with-kraft.kafka.svc.cluster.local:9092 --create --topic secure-test --command-config /tmp/client-sasl-ssl.properties
Created topic secure-test.
I have no name!@kafka-with-kraft-broker-0:/$
exit
[demo@demo-azure-vm ~]$ cat client-sasl-ssl.properties
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username="user1" password="strong-password-123";
ssl.truststore.location=/tmp/kafka.truststore.jks
ssl.truststore.password=Dexter@123
ssl.keystore.location=/tmp/kafka.keystore.jks
ssl.keystore.password=Dexter@123
ssl.endpoint.identification.algorithm=
```

```
kubectl create ns kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
openssl req -new -x509 -keyout ca-key -out ca-cert -days 182500 -subj "/CN=Kafka-CA" -nodes
```

```
keytool -genkey -keystore kafka.keystore.jks -alias kafka-node -validity 182500 -keyalg RSA -dname "CN=kafka-with-kraft" -storepass Dexter@123 -keypass Dexter@123
```

```
keytool -keystore kafka.keystore.jks -alias kafka-node -certreq -file cert-file -storepass Dexter@123 -ext "SAN=dns:localhost,dns:*.kafka-with-kraft-broker-headless.kafka.svc.cluster.local,dns:*.kafka-with-kraft-controller-headless.kafka.svc.cluster.local,dns:*.kafka-with-kraft.kafka.svc.cluster.local"
```

```
openssl x509 -req -CA ca-cert -CAkey ca-key -in cert-file -out cert-signed -days 182500 -CAcreateserial -passin pass:Dexter@123 -extfile <(printf "subjectAltName=DNS:localhost,DNS:*.kafka-with-kraft-broker-headless.kafka.svc.cluster.local,DNS:*.kafka-with-kraft-controller-headless.kafka.svc.cluster.local,DNS:*.kafka-with-kraft.kafka.svc.cluster.local")
```

```
keytool -keystore kafka.keystore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
```

```
keytool -keystore kafka.keystore.jks -alias kafka-node -import -file cert-signed -storepass Dexter@123
```

```
keytool -keystore kafka.truststore.jks -alias CARoot -import -file ca-cert -storepass Dexter@123 -noprompt
```

```
kubectl create secret generic kafka-tls-secret --from-file=kafka.keystore.jks=kafka.keystore.jks --from-file=kafka.truststore.jks=kafka.truststore.jks -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
helm upgrade --install kafka-with-kraft ./ -f ./values.yaml --set kraft.enabled=true --set zookeeper.enabled=false --set controller.replicaCount=3,controller.persistence.enabled=true,controller.persistence.size=1Gi,controller.persistence.storageClass=standard-rwo,broker.replicaCount=3,broker.persistence.storageClass=standard-rwo,broker.persistence.enabled=true,broker.persistence.size=1Gi,controller.resources.limits.cpu=500m,controller.resources.requests.cpu=250m,controller.resources.limits.memory=512Mi,controller.resources.requests.memory=256Mi,broker.resources.limits.cpu=500m,broker.resources.requests.cpu=250m,broker.resources.limits.memory=512Mi,broker.resources.requests.memory=256Mi,listeners.client.protocol=SASL_SSL,listeners.controller.protocol=SASL_SSL,listeners.interbroker.protocol=SASL_SSL,sasl.enabledMechanisms=PLAIN,sasl.interBrokerMechanism=PLAIN,sasl.controllerMechanism=PLAIN,sasl.client.users={user1},sasl.client.passwords={strong-password-123},tls.type=JKS,tls.existingSecret=kafka-tls-secret,tls.keystorePassword=Dexter@123,tls.truststorePassword=Dexter@123,tls.sslClientAuth=required --create-namespace --namespace=kafka --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
cat client-sasl-ssl.properties
```

```
security.protocol=SASL_SSL
```

```
sasl.mechanism=PLAIN
```

```
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required
```

```
username="user1" password="strong-password-123";
```

```
ssl.truststore.location=/tmp/kafka.truststore.jks
```

```
ssl.truststore.password=Dexter@123
```

```
ssl.keystore.location=/tmp/kafka.keystore.jks
```

```
ssl.keystore.password=Dexter@123
```

```
ssl.endpoint.identification.algorithm=
```

```
kubectl cp client-sasl-ssl.properties kafka/kafka-with-kraft-broker-0:/tmp/client-sasl-ssl.properties --
context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl cp kafka.truststore.jks kafka/kafka-with-kraft-broker-0:/tmp/kafka.truststore.jks --
context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl cp kafka.keystore.jks kafka/kafka-with-kraft-broker-0:/tmp/kafka.keystore.jks --
context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl exec -it kafka-with-kraft-broker-0 -n kafka --context=gke_wise-trainer-244916_us-
central1_multicloud-gke-cluster -- bash
```

```
kafka-topics.sh --bootstrap-server kafka-with-kraft.kafka.svc.cluster.local:9092 --create --topic secure-
test --command-config /tmp/client-sasl-ssl.properties
```

Installation of kafka-ui for SASL SSL

To install kafka-ui for kafka with SASL_SSL using helm chart you can follow the steps as mentioned below.

```
[demo@demo-azure-vm helm-repo-for-kafbat-kafka-ui]$ helm upgrade --install kafka-ui ./ -f ./values.yaml --set envs.config.KAFKA_CLUSTERS_0_NAME=kafka-with-kraft,
envs.config.KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka-with-kraft.kafka.svc.cluster.local:9092,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SECURITY_PROTOCOL=SASL_
SSL,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SASL_MECHANISM=PLAIN,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SASL_JAAS_CONFIG='org.apache.kafka.common.security.pl
ain.PlainLoginModule required username="user1" password="strong-password-123";',envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_TRUSTSTORE_LOCATION=/ssl/kafka.tru
ststore.jks,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_TRUSTSTORE_PASSWORD=Dexter@123,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_KEYSTORE_LOCATION=/ssl/kafk
a.keystore.jks,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_KEYSTORE_PASSWORD=Dexter@123,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_ENDPOINT_IDENTIFICATION_AL
GORITHM='',volumes[0].name=ssl-truststore,volumes[0].configMap.name=kafka-ui-truststore,volumeMounts[0].name=ssl-truststore,volumeMounts[0].mountPath=/ssl,vol
umeMounts[0].readOnly=true --namespace=kafka --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Release "kafka-ui" does not exist. Installing it now.
NAME: kafka-ui
LAST DEPLOYED: Fri May 15 12:50:43 2026
NAMESPACE: kafka
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
1. Get the application URL by running these commands:
export POD_NAME=$(kubectl get pods --namespace=kafka -l "app.kubernetes.io/name=kafka-ui,app.kubernetes.io/instance=kafka-ui" -o jsonpath="{.items[0].metada
ta.name}")
echo "Visit http://127.0.0.1:8080 to use your application"
kubectl --namespace=kafka port-forward $POD_NAME 8080:8080
```

```
[demo@demo-azure-vm helm-repo-for-kafbat-kafka-ui]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kafka-ui	ClusterIP	172.19.46.130	<none>	80/TCP	72s
kafka-with-kraft	ClusterIP	172.19.82.143	<none>	9092/TCP	16m
kafka-with-kraft-broker-headless	ClusterIP	None	<none>	9094/TCP,9092/TCP	16m
kafka-with-kraft-controller-headless	ClusterIP	None	<none>	9094/TCP,9092/TCP,9093/TCP	16m

Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>


```
kubectl create configmap kafka-ui-truststore --from-file=kafka.truststore.jks=./kafka.truststore.jks --
from-file=./kafka.keystore.jks -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-
gke-cluster
```

```
helm upgrade --install kafka-ui ./ -f ./values.yaml --set
envs.config.KAFKA_CLUSTERS_0_NAME=kafka-with-
kraft,envs.config.KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka-with-
kraft,kafka.svc.cluster.local:9092,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SECURITY_PROTOC
OL=SASL_SSL,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SASL_MECHANISM=PLAIN,envs.config.
KAFKA_CLUSTERS_0_PROPERTIES_SASL_JAAS_CONFIG='org.apache.kafka.common.security.plain.Pl
ainLoginModule required username="user1" password="strong-password-
123";',envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_TRUSTSTORE_LOCATION=/ssl/kafka.trusts
tore.jks,envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_TRUSTSTORE_PASSWORD=Dexter@123,
envs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_KEYSTORE_LOCATION=/ssl/kafka.keystore.jks,en
vs.config.KAFKA_CLUSTERS_0_PROPERTIES_SSL_KEYSTORE_PASSWORD=Dexter@123,envs.config.K
AFKA_CLUSTERS_0_PROPERTIES_SSL_ENDPOINT_IDENTIFICATION_ALGORITHM='',volumes[0].name
=ssl-truststore,volumes[0].configMap.name=kafka-ui-truststore,volumeMounts[0].name=ssl-
truststore,volumeMounts[0].mountPath=/ssl,volumeMounts[0].readOnly=true --namespace=kafka
--create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl create secret tls tls-kafka-ui-ingress --cert=693ff37a406fc433.crt --key=server.key -n kafka --context=gke_wise-trainer-244916
_us-central1_multicloud-gke-cluster
secret/tls-kafka-ui-ingress created
[demo@demo-azure-vm ~]$ kubectl apply -f gateway-api-kafka-ui.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
gateway.gateway.networking.k8s.io/kafka-ui-gateway created
httproute.gateway.networking.k8s.io/kafka-ui-route created
```

```
[demo@demo-azure-vm ~]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster --watch
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)              AGE
kafka-ui                           ClusterIP           172.19.46.130   <none>           80/TCP               7m16s
kafka-ui-gateway-nginx              LoadBalancer       172.19.79.131   <pending>        80:32475/TCP,443:32488/TCP   13s
kafka-with-kraft                    ClusterIP           172.19.82.143   <none>           9092/TCP              22m
kafka-with-kraft-broker-headless    ClusterIP           None             <none>           9094/TCP,9092/TCP      22m
kafka-with-kraft-controller-headless ClusterIP           None             <none>           9094/TCP,9092/TCP,9093/TCP  22m
kafka-ui-gateway-nginx              LoadBalancer       172.19.79.131   34.70.49.21     80:32475/TCP,443:32488/TCP   38s
```

Do the entry of External IP obtained in the last step as shown in the screenshot attached above to create the A-Type Record-Set in Azure DNS Zone is as shown in the screenshot attached below. Finally, you will get the URL using which you can access the kafka-ui as shown in the screenshot attached below. For your reference I kept the file **gateway-api-kafka-ui.yaml** in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-1/**.

Add record set

singhritesh85.com



Name

kafka-ui

.singhritesh85.com

Type

A – IPv4 Address records

Alias record set ⓘ

No

TTL *

1

TTL unit

Hours

IP address

[Yellow box]



0.0.0.0

The screenshot shows the 'kafbat UI' dashboard for cluster 'kafka-with-kraft'. It displays '1 Online' and '0 Offline' clusters. A table lists the cluster details:

Cluster name	Version	Brokers count	Partitions	Topics	Production	Consumption
kafka-with-kraft	3.7-IV4	6	1	1	0 Bytes	0 Bytes

Below screenshot verifies the Kafka topic with name **secure-test** which I created earlier as shown in the screenshot attached above.

The screenshot shows the 'Topics' page in the 'kafbat UI'. It displays a table with the topic 'secure-test':

Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
secure-test	1	0	1	0	0 Bytes



Installation of Kedacore KEDA (Kubernetes Event Driven Autoscaling)

KEDA stands for Kubernetes Event Driven Autoscaling. KEDA will scale the workload automatically based on the volumes of events. To install KEDA using helm you can use the command as shown below.

```
helm repo add kedacore https://kedacore.github.io/charts
```

```
helm repo update
```

```
helm install keda kedacore/keda --namespace keda --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ helm repo add kedacore https://kedacore.github.io/charts
"kedacore" has been added to your repositories
[demo@demo-azure-vm ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "secrets-store-csi-driver" chart repository
...Successfully got an update from the "kedacore" chart repository
...Successfully got an update from the "istio" chart repository
Update Complete. 🎉Happy Helming!🎉
```

```
[demo@demo-azure-vm ~]$ helm install keda kedacore/keda --namespace keda --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl get pods --namespace keda --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	READY	STATUS	RESTARTS	AGE
keda-admission-webhooks-7d5d987497-mjhbb	1/1	Running	0	3h18m
keda-operator-658786f579-cp8d1	1/1	Running	1 (3h18m ago)	3h18m
keda-operator-metrics-apiserver-7b6fccf947-h9xmf	1/1	Running	0	3h18m

```
[demo@demo-azure-vm ~]$ kubectl get pods -n kube-system --context=multicloud-aks-cluster | grep -i "keda"
```

NAME	READY	STATUS	RESTARTS	AGE
keda-admission-webhooks-	1/1	Running	0	
keda-admission-webhooks-	1/1	Running	0	
keda-operator-	1/1	Running	1 (3h1m ago)	
keda-operator-	1/1	Running	1 (3h1m ago)	
keda-operator-metrics-apiserver-	1/1	Running	0	
keda-operator-metrics-apiserver-	1/1	Running	0	

The above screenshot shows that KEDA had been installed in both GKE standard cluster (using the helm command as explained above) and AKS Cluster (in AKS Cluster it was installed by enabling the addons during its creation using Terraform as shown in the screenshot attached below).

```
# Enable KEDA and VPA
workload_autoscaler_profile {
    keda_enabled = true
    vertical_pod_autoscaler_enabled = true
}
```

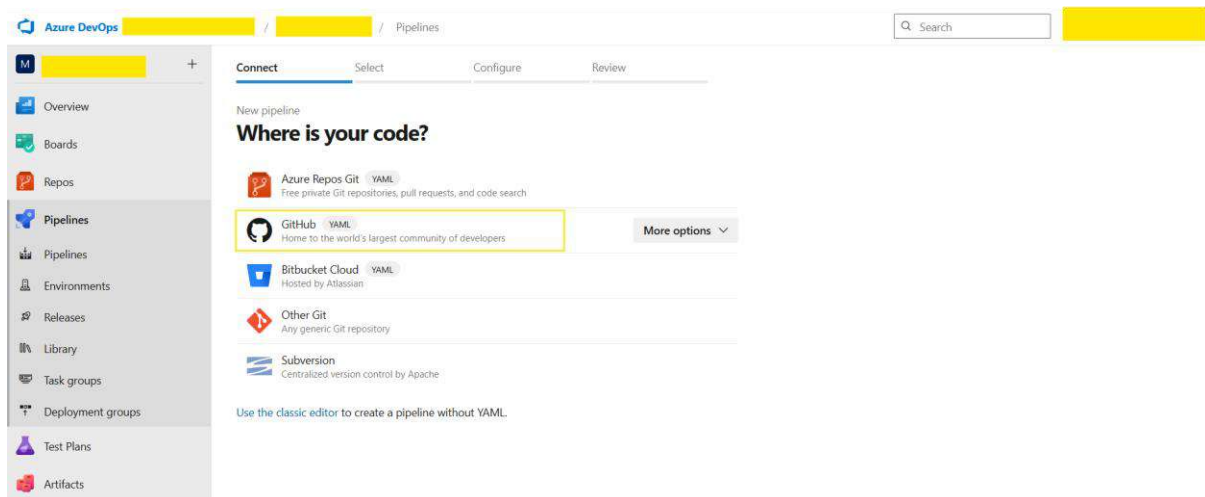
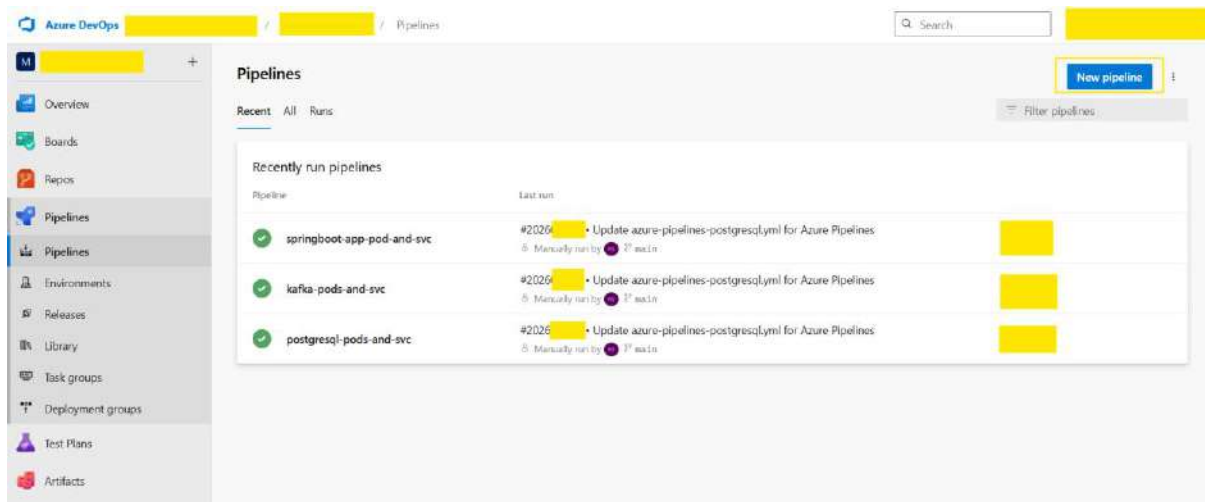
KEDA works independent of VPA, but it relies on HPA for autoscaling. As a standard practice for multicloud-multicloud setup if you want to perform autoscaling using KEDA then you must install KEDA independently on each kubernetes clusters and create ScaledObject on each kubernetes clusters independently. You can see below in both the clusters KEDA had been installed with kedacore (Open source) not with kedify (enterprise grade commercially supported version of KEDA).

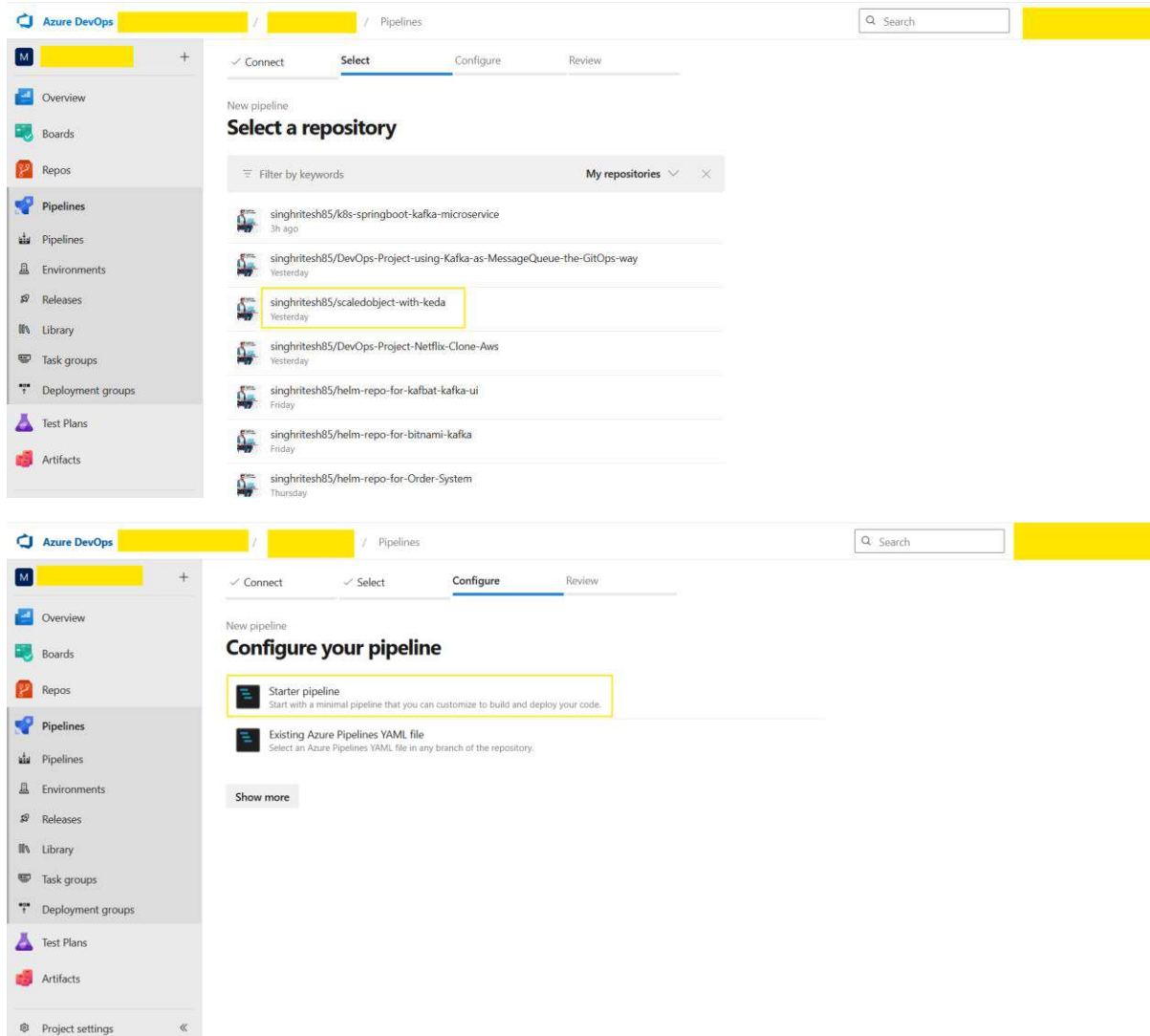
```
[demo@demo-azure-vm ~]$ kubectl get deployment keda-operator -n keda -o jsonpath='{$.spec.template.spec.containers[1].image}' --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
ghcr.io/kedacore/keda:2.19.0[demo@demo-azure-vm ~]$
[demo@demo-azure-vm ~]$ kubectl get deployment keda-operator -n kube-system -o jsonpath='{$.spec.template.spec.containers[1].image}' --context=multicloud-aks-cluster
mcr.microsoft.com/oss/v2/kedacore/keda:2.17.3-3[demo@demo-azure-vm ~]$
```

When using Kedify KEDA with **DistributedScaledObject**, you can centrally run Kedify KEDA in a GKE cluster and scale Spring Boot application pod running in an EKS cluster based on Kafka events. Similarly, you can also scale Spring Boot application pods running in the EKS cluster using PostgreSQL events originating from an AKS cluster.

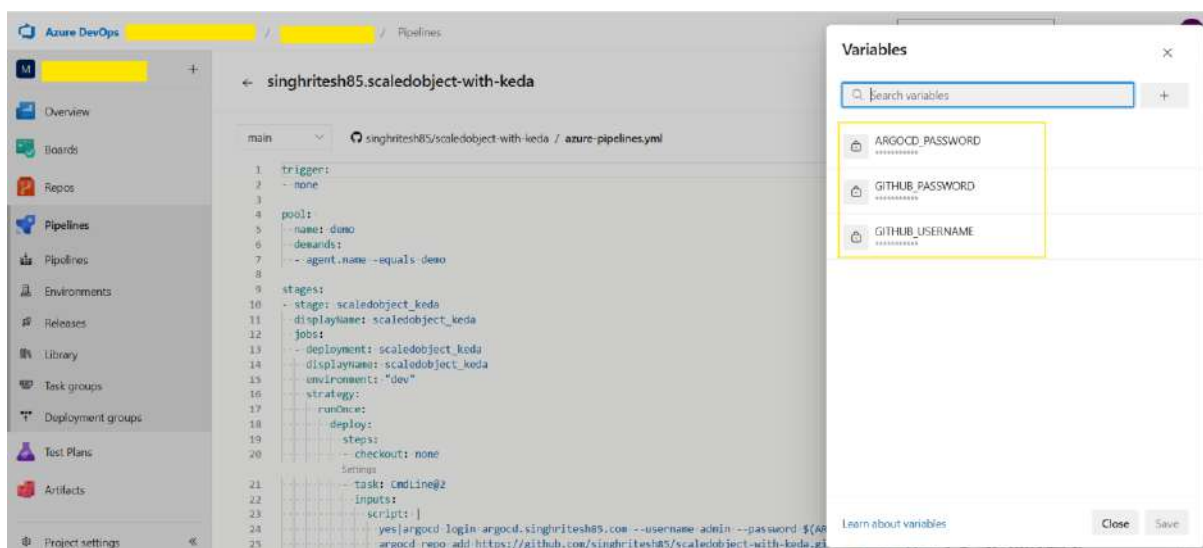
First of all, I scaled the Spring Boot Application Pods present in the GKE Cluster using Kafka Events. The Kafka and Kedacore KEDA both are installed in GKE Standard Cluster. Then scaled the PostgreSQL Pods present in AKS Cluster with KEDA installed in AKS Cluster. To achieve this, here I used the CRD (Custom Resource Definition) **ScaledObject**. I implemented these using the kubernetes manifests file present in the GitHub Repo <https://github.com/singhritesh85/scaledobject-with-keda.git> through ArgoCD and Azure DevOps Pipeline.

Created New Azure DevOps Pipeline as shown in the screenshot attached below.





Added three variables which is needed to Run this Azure DevOps Pipeline.



Finally, ran the Azure DevOps Pipeline and screenshot of Azure DevOps Pipeline and ArgoCD Application after successfully running the Pipeline is as shown in the screenshot attached below.

Pipelines New pipeline

Recent All Runs Filter pipelines

Recently run pipelines

Pipeline	Last run
singhritesh85.scaledobject-with-keda	#2026 Update azure-pipelines.yml for Azure Pipelines. Manually run by main

argocd.singhritesh85.com/applications?proj=&sync=&autoSync=&health=&namespace=&cluster=&labels=&annotations=&operation=

argocd v3.4.3

Applications

Settings User Info Documentation

Application filters

☐ Favorites Only
 SYNC STATUS
☐ Unknown 0
☒ Synced 5
☐ OutOfSync 0
 HEALTH STATUS
☐ Progressing 0
☐ Suspended 0
☒ Healthy 5
☐ Degraded 0

Applications

+ NEW APP SYNC APPS REFRESH APPS Search applications...

Path: / Destination: in-cluster Namespace: kafka Created: Last Sync: SYNC REFRESH DELETE

Path: /fols/ Destination: in-cluster Namespace: ordersystem Created: Last Sync: SYNC REFRESH DELETE

Path: / Destination: multicloud-aks-cluster Namespace: postgresql Created: Last Sync: SYNC REFRESH DELETE

scaledobject-aks

Project: default Labels: Status: Healthy Synced Repository: https://github.com/singhritesh85/scaled... Target R: HEAD Path: /trigger-authentication-scaledobject Destination: multicloud-aks-cluster Namespace: postgresql Created: (5 minutes ago) Last Sync: (a minute ago) SYNC REFRESH DELETE

scaledobject-gke

Project: default Labels: Status: Healthy Synced Repository: https://github.com/singhritesh85/scaled... Target R: HEAD Path: /scaledobject-with-keda Destination: in-cluster Namespace: ordersystem Created: (5 minutes ago) Last Sync: (2 minutes ago) SYNC REFRESH DELETE

```
[demo@demo-azure-vm ~]$ kubectl get scaledobject -n ordersystem --context=gke_wise-trainer-244916-us-central1-multicloud-gke-cluster
NAME                                SCALETARGETKIND SCALETARGETNAME MIN MAX READY ACTIVE FALLBACK PAUSED TRIGGERS AUTHENTICATIONS AGE
spring-boot-producer-scaler apps/v1.Deployment order-service-fols 1 10 True False False False kafka 24m
[demo@demo-azure-vm ~]$ kubectl get hpa -n ordersystem --context=gke_wise-trainer-244916-us-central1-multicloud-gke-cluster
NAME                                REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
keda-hpa-spring-boot-producer-scaler Deployment/order-service-fols 0/1 (avg) 1 10 1 24m
[demo@demo-azure-vm ~]$ kubectl get scaledobject -n postgresql --context=multicloud-aks-cluster
NAME                                SCALETARGETKIND SCALETARGETNAME MIN MAX READY ACTIVE FALLBACK PAUSED TRIGGERS AUTHENTICATIONS AGE
postgresql-scaler apps/v1.StatefulSet postgresql-read 3 10 True True False Unknown postgresql postgres-auth 23m
[demo@demo-azure-vm ~]$ kubectl get hpa -n postgresql --context=multicloud-aks-cluster
NAME                                REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
keda-hpa-postgresql-scaler StatefulSet/postgresql-read 1600m/1 (avg) 3 10 8 23m
```

The postgresql pods present in the namespace postgresql in AKS Cluster had been scaled as shown in the screenshot attached below.

```
[demo@demo-azure-vm ~]$ kubectl get pods -n postgresql --context=multicloud-aks-cluster --watch
NAME                                READY STATUS RESTARTS AGE
postgresql-primary-0 2/2 Running 0 4h6m
postgresql-read-0 2/2 Running 0 4h6m
postgresql-read-1 2/2 Running 0 4h5m
postgresql-read-2 2/2 Running 0 4h5m
postgresql-read-3 2/2 Running 0 4m19s
postgresql-read-4 2/2 Running 0 3m50s
postgresql-read-5 2/2 Running 0 3m16s
postgresql-read-6 2/2 Running 0 2m43s
postgresql-read-7 2/2 Running 0 2m10s
```

Consider a scenario, there is an IT Organisation in India which **production hours (not the business hours)** ranges from 9:00 AM to 6:00 PM (Monday to Friday) and during the production hours the Application Pods must be sufficient to fulfil the request from users. For the current case Application Pods maintained as 5 (as the desired count). This had been implemented using cron based scaling with KEDA as shown in the screenshot attached below. This cron job will maintain the desired count of pods as 5 between the production hours 9:00AM to 6:00PM from Monday to Friday as shown in the screenshot attached below.

```
# Cron based scaling with KEDA
- type: cron
  metadata:
    timezone: Asia/Kolkata
    start: 0 9 * * 1-5
    end: 0 18 * * 1-5
    desiredReplicas: "5"
```

```
[demo@demo-azure-vm ~]$ kubectl get hpa -n ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                REFERENCE                                TARGETS                                MINPODS  MAXPODS  REPLICAS  AGE
keda-hpa-order-service-scaler       Deployment/order-service-folo             200m/1 (avg), 0/20 (avg) + 1 more...    1         10        5         14m

[demo@demo-azure-vm ~]$ kubectl get pods -n ordersystem --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                READY  STATUS   RESTARTS  AGE
order-service-folo-                2/2    Running   0          1m
order-service-folo-                2/2    Running   0          1m
order-service-folo-                2/2    Running   0          1m
order-service-folo-                2/2    Running   0          1m
order-service-folo-                2/2    Running   0          1m
```

KEDA evaluates **all triggers independently** and then uses the **highest desired replica count** among them.

GitHub Repo: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>

Terraform Script: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>

Source Code: - <https://github.com/singhritesh85/k8s-springboot-kafka-microservice.git>

Kubernetes Manifests Files used in this project for KEDA: - <https://github.com/singhritesh85/scaledobject-with-keda.git>

azure-pipelines.yml files used in project to create the Azure DevOps Pipeline: - <https://github.com/singhritesh85/k8s-springboot-kafka-microservice.git>

<https://github.com/singhritesh85/scaledobject-with-keda.git>

Helm Charts: - <https://github.com/singhritesh85/helm-repo-for-kafbat-kafka-ui.git>

<https://github.com/singhritesh85/helm-repo-for-bitnami-kafka.git>

<https://github.com/singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary-only-service.git>

<https://github.com/singhritesh85/helm-repo-for-bitnami-postgresql-primary-secondary.git>

<https://github.com/singhritesh85/helm-repo-for-Order-System.git>

References: - <https://github.com/k8s-space/k8s-springboot-kafka-microservice.git>

<https://medium.com/@devripper133127/migration-of-an-event-driven-architecture-to-kubernetes-b62691c5a858>

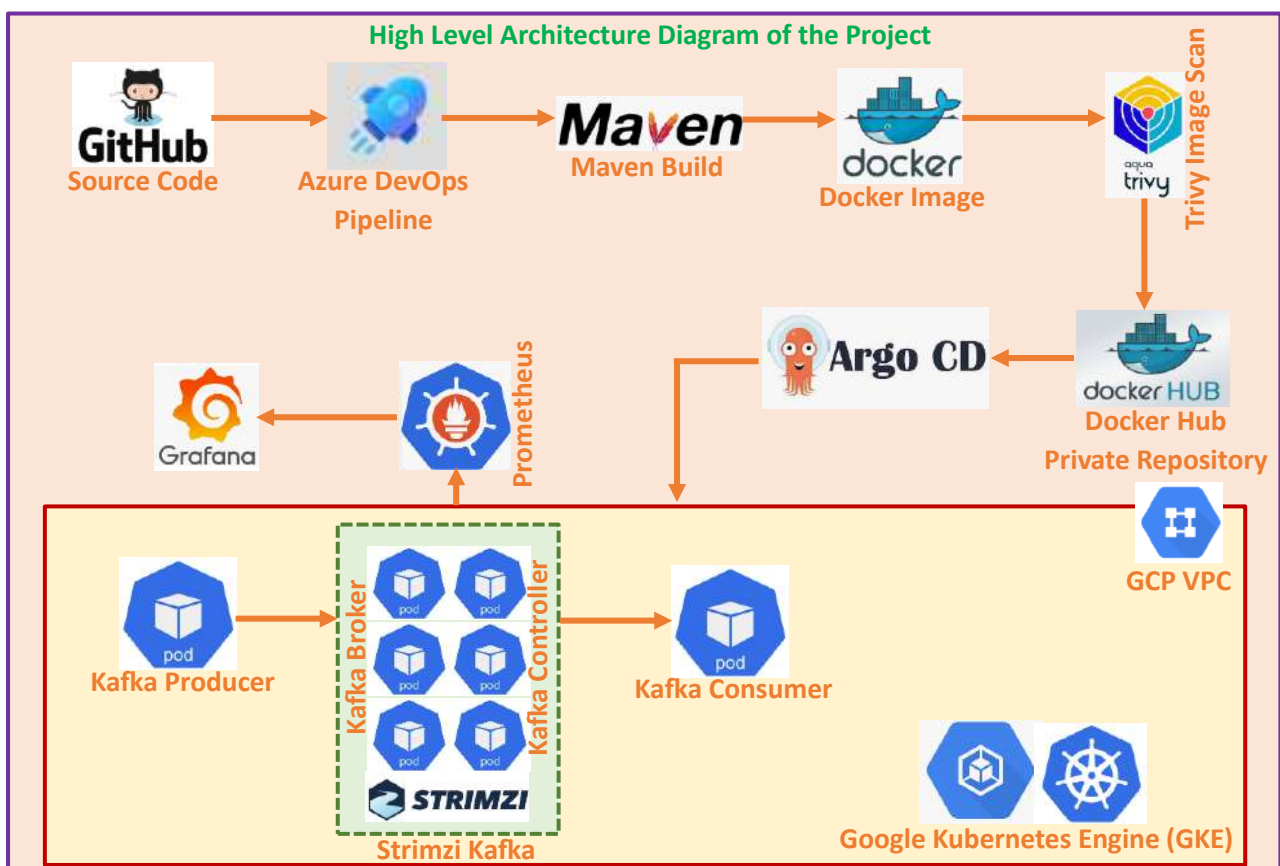
Module-2

DevOps Project using Strimzi Kafka as Message Queue with GitOps



High Level Architecture Diagram for Publisher/Subscriber Model using Kafka

Above high-level architecture diagram is for Publisher/Subscriber Model using Kafka as Message Queue. I had already explained it in starting of Module-1. In CNCF projects are classified into four stages: - Sandbox, Incubating, Graduated and Archived. Sandbox project is the early-stage project that have been accepted by CNCF to provide a place for experimentation and innovation. Incubating Projects are those CNCF Projects that have achieved growing adoption, governance and maturity. They are increasingly used in production environments by multiple organizations. **Strimzi (as on 2nd June 2026) is an example of Incubating CNCF Project.** Graduated CNCF Project achieved highest maturity level in CNCF Project. These CNCF Projects are mature, stable, well-governed, and widely adopted in production environments. **KEDA (explained in Module-1) is an example for Graduated CNCF Project (as on 2nd June 2026).** Archived CNCF project is a previously active or sandbox-stage project that is no longer maintained, supported, or recommended for new use by the Cloud Native Computing Foundation (CNCF).



This DevOps Project deals with creation of infrastructure using Terraform and for CI/CD Pipeline Azure DevOps Pipeline (Self-hosted Agent) had been used. The Source Code or Kubernetes Manifestes files were present in the GitHub Repo and Maven had been used as the build tool. Then Docker Image had been created and scanned using Trivy and scanned Docker Image was sent to Docker Hub Private Repository. Which was later deployed to GKE Standard Cluster. In this project, Apache Kafka was deployed using Strimzi, while Kafka-UI was deployed using Kafbat (formerly known as Provectus).

On Terraform Server it is necessary to authenticate and authorize Azure and GCP account using the command as shown below.

```
curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-linux-x86_64.tar.gz
```

```
tar -xf google-cloud-cli-linux-x86_64.tar.gz
```

```
./google-cloud-sdk/install.sh
```

```
gcloud auth application-default login
```

```
gcloud auth login
```

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
```

```
sudo sh -c 'echo -e "[azure-cli]\nname=Azure\nCLI\nbaseurl=https://packages.microsoft.com/yumrepos/azure-cli\nenabled=1\nngpgcheck=1\nngpgkey=https://packages.microsoft.com/keys/microsoft.asc" > /etc/yum.repos.d/azure-cli.repo'
```

```
sudo dnf install azure-cli -y
```

```
az login
```

```
[root@Terraform-Server opt]# curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-linux-x86_64.tar.gz
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
0   0   0    0    0    0     0      0      0      0  0 0 0
curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-linux-x86_64.tar.gz
tar -xf google-cloud-cli-linux-x86_64.tar.gz
100 83.4M 100 83.4M    0    0  217M      0      0      0  217M
[root@Terraform-Server opt]# tar -xf google-cloud-cli-linux-x86_64.tar.gz
```

```
[root@Terraform-Server opt]# ./google-cloud-sdk/install.sh
```

```
[root@Terraform-Server ~]# gcloud auth application-default login
Go to the following link in your browser, and complete the sign-in prompts:
[redacted]

Once finished, enter the verification code provided in your browser: [redacted]
Credentials saved to file: [/root/.config/gcloud/application_default_credentials.json]
These credentials will be used by any library that requests Application Default Credentials (ADC).
[redacted]
Cannot find a quota project to add to ADC. You might receive a "quota exceeded" or "API not enabled" error. Run $ gcloud auth application-default set-quota-pr
ject to add a quota project.
[root@Terraform-Server ~]# gcloud auth login
Go to the following link in your browser, and complete the sign-in prompts:
[redacted]

Once finished, enter the verification code provided in your browser: [redacted]
You are now logged in as [redacted]@gmail.com.
Your current project is [None]. You can change this setting by running:
$ gcloud config set project PROJECT_ID

[root@Terraform-Server ~]# sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
[root@Terraform-Server ~]# sudo sh -c 'echo -e "[azure-cli]\nname=Azure CLI\nbaseurl=https://packages.microsoft.com/yumrepos/azure-cli\nenabled=1\nngpgcheck=1\
ngpgkey=https://packages.microsoft.com/keys/microsoft.asc" > /etc/yum.repos.d/azure-cli.repo'
[root@Terraform-Server ~]# sudo dnf install azure-cli -y

[root@Terraform-Server ~]# az login
To sign in, use a web browser to open the page https://aka.ms/DeviceLogin and enter the code [redacted] for authentication.
[redacted]
{
  "cloudName": "AzureCloud",
  "homeTenantId": "[redacted]",
  "id": "[redacted]",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Pay-As-You-Go",
  "state": "Enabled",
  "tenantId": "[redacted]",
  "user": {
    "name": "[redacted]@gmail.com",
    "type": "user"
  }
}
1
```

Terraform script to provision the infrastructure is present at the path **module-2/terraform-azure-vm-gke-cluster-public** in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>. Commands as written below had been used to create infrastructure using Terraform.

terraform init -----> initializes a working directory containing configuration files and installs plugins for required providers.

terraform validate -----> verify that terraform configuration file is correct or not

terraform plan -----> Check which resources are going to be created.

Then you can run the command **terraform apply -auto-approve** -----> Finally, Create the resources.


```

module.aks_cluster_and_standard_gke_cluster.google_container_cluster.gke_cluster: Still creating... [10m50s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_cluster.gke_cluster: Still creating... [11m00s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_cluster.gke_cluster: Still creating... [11m10s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_cluster.gke_cluster: Still creating... [11m20s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_cluster.gke_cluster: Still creating... [11m30s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_cluster.gke_cluster: Still creating... [11m40s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_cluster.gke_cluster: Creation complete after 11m50s [id=projects/[REDACTED]/locations/us-central1/clusters/multicloud-gke-cluster]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Creating...
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [80m10s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [80m20s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [80m30s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [80m40s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [80m50s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [91m00s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Still creating... [91m10s elapsed]
module.aks_cluster_and_standard_gke_cluster.google_container_node_pool.gke_linux_nodepool_1: Creation complete after 91m15s [id=projects/[REDACTED]/locations/us-central1/clusters/multicloud-gke-cluster/nodePools/multicloud-linux-nodepool-1]
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: Creating...
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: Provisioning with 'local-exec'...
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: Executing: ["/bin/sh" "-c" "gcloud container clusters get-credentials multicloud-gke-cluster --location=us-central1 --project=[REDACTED] --chmod 600 ~/.kube/config"]
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: WARNING: Accessing a Kubernetes Engine cluster requires the kubernetes commandline
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: client [kubectll]. To install, run
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: $ gcloud components install kubectll

module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: Fetching cluster endpoint and auth data.
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: CRITICAL: ACTION REQUIRED: gke-gcloud-auth-plugin, which is needed for continued use of kubectll, was not found or is not executable. Install gke-gcloud-auth-plugin for use with kubectll by following https://cloud.google.com/kubernetes-engine/docs/how-to/cluster-access-for-kubectll#install_plugin
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: kubeconfig entry generated for multicloud-gke-cluster.
module.aks_cluster_and_standard_gke_cluster.null_resource.gke_kubectll: Creation complete after 4s [id=[REDACTED]]

Apply complete! Resources: 41 added, 0 changed, 0 destroyed.

Outputs:
azuread_service_principal_client_id_azuread_service_principal_client_secret_and_azure_dns_zone_nameservers = <sensitive>

```

```

[root@Terraform-Server main]# terraform output -json
{
  "azuread_service_principal_client_id_azuread_service_principal_client_secret_and_azure_dns_zone_nameservers": {
    "sensitive": true,
    "type": [
      "object",
      {
        "azuread_service_principal_client_id_gke": "string",
        "azuread_service_principal_client_secret_gke": "string",
        "dns_zone_name": "string",
        "name_servers": [
          "set",
          "string"
        ]
      }
    ],
    "value": {
      "azuread_service_principal_client_id_gke": "[REDACTED]",
      "azuread_service_principal_client_secret_gke": "[REDACTED]",
      "dns_zone_name": "singhritesh85.com",
      "name_servers": [
        "ns1-01.azure-dns.com.",
        "ns2-01.azure-dns.net.",
        "ns3-01.azure-dns.org.",
        "ns4-01.azure-dns.info."
      ]
    }
  }
}

```

After login on k8s-managent-node for the first time run the below command (**gcloud auth login**) to authenticate and authorize GCP Account and after creation of the resources a kubeconfig file was generated for the GKE Cluster using the same kubeconfig file it was possible to access the kubernetes clusters (GKE Cluster).

```

[demo@demo-azure-vm ~]$ gcloud auth login
Go to the following link in your browser, and complete the sign-in prompts:
[REDACTED]

Once finished, enter the verification code provided in your browser: [REDACTED]

You are now logged in as [REDACTED]@gmail.com].
Your current project is [None]. You can change this setting by running:
$ gcloud config set project PROJECT_ID

```

You can use below helm commands to install Strimzi Operator, Prometheus and Grafana.

```
helm repo add strimzi https://strimzi.io/charts/
```

```
helm repo update
```

```
helm upgrade --install strimzi-kafka-operator strimzi/strimzi-kafka-operator --namespace kafka --create-namespace --version 1.0.0 --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

```
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl get pods -n monitoring --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-kube-prometheus-stack-alertmanager-0	2/2	Running	0	9m10s
kube-prometheus-stack-grafana-646476459-242v5	3/3	Running	0	9m16s
kube-prometheus-stack-kube-state-metrics-6b747b7c5f-wqqs	1/1	Running	0	9m16s
kube-prometheus-stack-operator-59c5d9f8c-ngg42	1/1	Running	0	9m16s
kube-prometheus-stack-prometheus-node-exporter-1nxq9	1/1	Running	0	9m17s
kube-prometheus-stack-prometheus-node-exporter-m17v5	1/1	Running	0	9m17s
prometheus-kube-prometheus-stack-prometheus-0	2/2	Running	0	9m9s

```
[demo@demo-azure-vm ~]$ kubectl get svc -n monitoring --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP, 9094/TCP, 9094/UDP	9m24s
kube-prometheus-stack-alertmanager	ClusterIP	172.19.112.127	<none>	9093/TCP, 8080/TCP	9m31s
kube-prometheus-stack-grafana	ClusterIP	172.19.47.103	<none>	80/TCP	9m31s
kube-prometheus-stack-kube-state-metrics	ClusterIP	172.19.64.221	<none>	8080/TCP	9m31s
kube-prometheus-stack-operator	ClusterIP	172.19.100.68	<none>	443/TCP	9m31s
kube-prometheus-stack-prometheus	ClusterIP	172.19.193.46	<none>	9090/TCP, 8080/TCP	9m31s
kube-prometheus-stack-prometheus-node-exporter	ClusterIP	172.19.245.11	<none>	9100/TCP	9m31s
prometheus-operated	ClusterIP	None	<none>	9090/TCP	9m23s

Installation of ArgoCD in GKE Standard Cluster using the Kubernetes Manifests file is as shown in the screenshot attached below.

```
kubectl create ns argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl create ns argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
namespace/argocd created
[demo@demo-azure-vm ~]$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

Created Kubernetes Secret as shown in the screenshot attached below to access the ArgoCD using URL with HTTPS Protocol.

```
kubectl create secret tls tls-argocd-ingress --cert=8d6a041500e3ad4f.crt --key=server.key -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl create secret tls tls-argocd-ingress --cert=8d6a041500e3ad4f.crt --key=server.key -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/tls-argocd-ingress created
```

To install Kubernetes Gateway API controller and then to create URL to access ArgoCD follow the below procedure.

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
helm upgrade --install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric --create-namespace -n nginx-gateway --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ helm upgrade --install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric --create-namespace -n nginx-gateway --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f gateway-api-argocd.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
gateway.gateway.networking.k8s.io/argocd-gateway created
httproute.gateway.networking.k8s.io/argocd-route created
```

```
[demo@demo-azure-vm ~]$ kubectl get svc -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
argocd-applicationset-controller	ClusterIP	172.19.231.94	<none>	7000/TCP,8080/TCP	9m10s
argocd-dex-server	ClusterIP	172.19.24.221	<none>	5556/TCP,5557/TCP,5558/TCP	9m10s
argocd-gateway-nginx	LoadBalancer	172.19.3.219	34. [REDACTED]	80:30358/TCP,443:31387/TCP	52s
argocd-metrics	ClusterIP	172.19.219.73	<none>	8082/TCP	9m10s
argocd-notifications-controller-metrics	ClusterIP	172.19.172.93	<none>	9001/TCP	9m10s
argocd-redis	ClusterIP	172.19.51.63	<none>	6379/TCP	9m9s
argocd-repo-server	ClusterIP	172.19.223.209	<none>	8081/TCP,8084/TCP	9m9s
argocd-server	ClusterIP	172.19.102.54	<none>	80/TCP,443/TCP	9m9s
argocd-server-metrics	ClusterIP	172.19.92.236	<none>	8083/TCP	9m9s

Edited the configmap as shown in the screenshot attached below.

```
[demo@demo-azure-vm ~]$ kubectl edit configmap argocd-cmd-params-cm -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: ConfigMap
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"ConfigMap","metadata":{"annotations":{},"labels":{"app.kubernetes.io/name":"argocd-cmd-params-cm","app.kubernetes.io/part-of":"argocd"},"name":"argocd-cmd-params-cm","namespace":"argocd"},"creationTimestamp":"2026-[REDACTED]"}
  labels:
    app.kubernetes.io/name: argocd-cmd-params-cm
    app.kubernetes.io/part-of: argocd
  name: argocd-cmd-params-cm
  namespace: argocd
  resourceVersion: "[REDACTED]"
  uid: [REDACTED]
data:
  server.insecure: "true"
```

```
[demo@demo-azure-vm ~]$ kubectl edit configmap argocd-cmd-params-cm -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
configmap/argocd-cmd-params-cm edited
```

Now, restarted the argocd-server deployment for the changes to take effect.

```
[demo@demo-azure-vm ~]$ kubectl rollout restart deployment argocd-server -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
deployment.apps/argocd-server restarted
```

```
kubectl edit configmap argocd-cmd-params-cm -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

data:

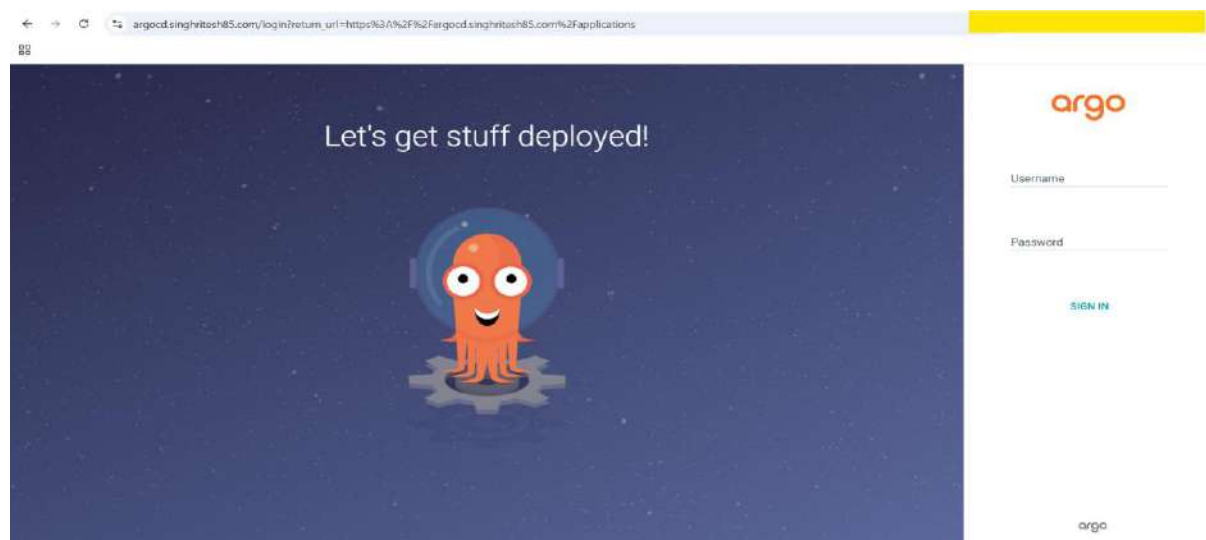
server.insecure: "true"

```
kubectl rollout restart deployment argocd-server -n argocd --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```


Then, Created the Record-Set of CNAME Type in Azure DNS Zone using the External IP of the LoadBalancer resulted from Kubernetes Gateway API for ArgoCD.

The screenshot shows the 'Add record set' dialog in the Azure Portal. The DNS zone is 'singhritesh85.com'. The record set name is 'argocd'. The type is 'A - IPv4 Address records'. The alias record set is set to 'No'. The TTL is '1' hour. The IP address is '34'. The 'Add' button is highlighted with a yellow box.

Finally, able to access the ArgoCD as shown in the screenshot attached below.

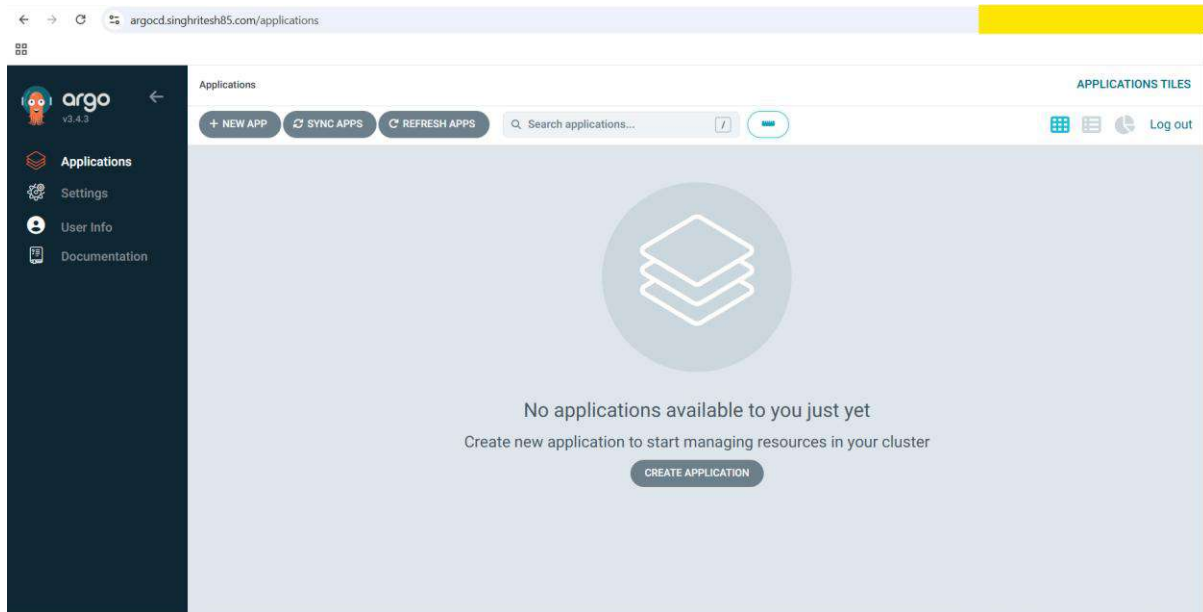


To Login into the ArgoCD for the first time you can login with user having username as admin and password you can get using the command.

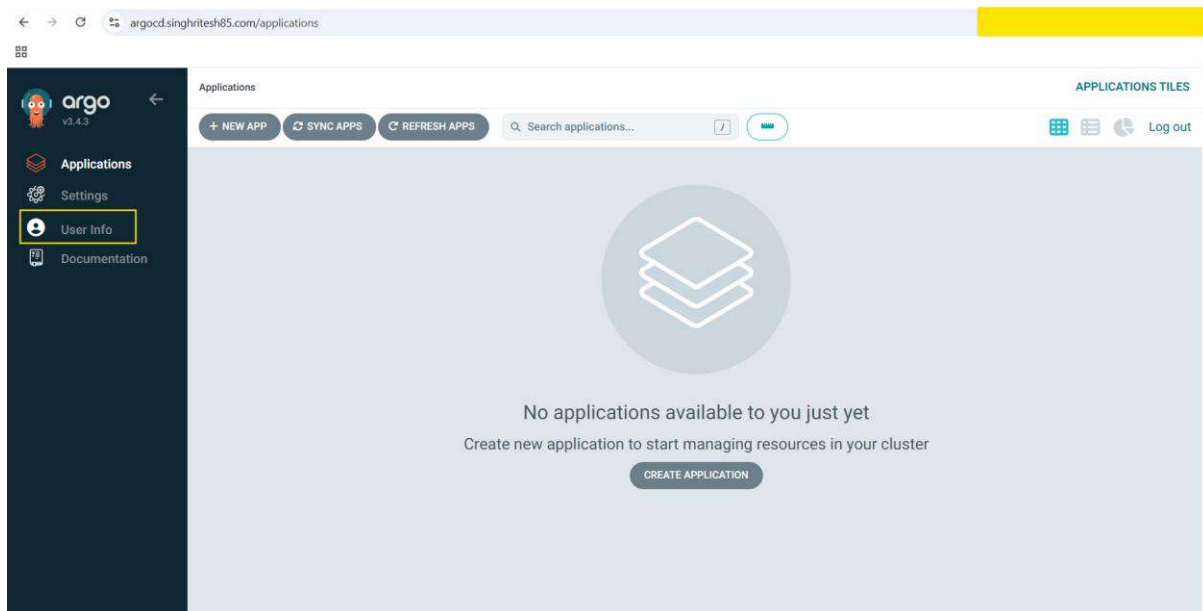
```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d
```

```
[demo@demo-azure-vm ~]$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d
[demo@demo-azure-vm ~]$
```

For your reference **gateway-api-argocd.yaml** file is present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-2/gateway-api-argocd.yaml**.



After the successful Login change the password as shown in the screenshot attached below.



argo v3.4.3

Applications
Settings
User Info
Documentation

User Info

SAVE NEW PASSWORD CANCEL

Update account password

Current Password

New Password

Confirm New Password

argo v3.4.3

Applications
Settings
User Info
Documentation

User Info

UPDATE PASSWORD

USER INFO

Log out

Username: admin
Issuer: argocd

✓ Your password has been successfully updated.

Then Logout and Login again.

argo v3.4.3

Applications
Settings
User Info
Documentation

Let's get stuff deployed!

argo

Username
admin

Password

SIGN IN

argo

Install ArgoCD CLI using the command as shown below.

```
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64

sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd

rm argocd-linux-amd64
```

```
[demo@demo-azure-vm ~]$ curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
[demo@demo-azure-vm ~]$ sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
[demo@demo-azure-vm ~]$ rm argocd-linux-amd64
```

Now I installed **secrets-store-csi-driver** and **secrets-store-csi-driver-provider-azure** in GKE standard cluster as shown in the screenshot attached below. It works together to mount secrets, keys, and certificates from Azure Key Vault directly into Kubernetes pods as files using CSI volumes. The **secrets-store-csi-driver** provides CSI interface for mounting external secrets and **secrets-store-csi-driver-provider-azure** connects the CSI driver to Azure Key Vault.

```
kubectl config get-contexts
```

```
helm repo add secrets-store-csi-driver https://kubernetes-sigs.github.io/secrets-store-csi-driver/charts
```

```
helm repo update
```

```
helm upgrade --install csi-secrets-store secrets-store-csi-driver/secrets-store-csi-driver --set syncSecret.enabled=true --set enableSecretRotation=true --set rotationPollInterval=10s --namespace kube-system --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl apply -f https://raw.githubusercontent.com/Azure/secrets-store-csi-driver-provider-azure/refs/heads/master/deployment/provider-azure-installer.yaml --namespace=kube-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
kubectl get pods -n kube-system --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | grep "secrets-store"
```

```
[demo@demo-azure-vm ~]$ kubectl config get-contexts
CURRENT  NAME                                     CLUSTER                                     AUTHINFO
*        gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster  gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

[demo@demo-azure-vm ~]$ helm repo add secrets-store-csi-driver https://kubernetes-sigs.github.io/secrets-store-csi-driver/charts
"secrets-store-csi-driver" has been added to your repositories

[demo@demo-azure-vm ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "secrets-store-csi-driver" chart repository
...Successfully got an update from the "strimzi" chart repository
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ✨Happy Helming!

[demo@demo-azure-vm ~]$ helm upgrade --install csi-secrets-store secrets-store-csi-driver/secrets-store-csi-driver --set syncSecret.enabled=true --set enableSecretRotation=true --set rotationPollInterval=10s --namespace kube-system --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
Release "csi-secrets-store" does not exist. Installing it now.
NAME: csi-secrets-store
LAST DEPLOYED: Fri May 29 06:20:06 2026
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
The Secrets Store CSI Driver is getting deployed to your cluster.

To verify that Secrets Store CSI Driver has started, run:

  kubectl --namespace=kube-system get pods -l "app=secrets-store-csi-driver"

Now you can follow these steps https://secrets-store-csi-driver.sigs.k8s.io/getting-started/usage.html
to create a SecretProviderClass resource, and a deployment using the SecretProviderClass.
```

Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>

```
[demo@demo-azure-vm ~]$ cat SecretProviderClass-gke.yaml
---
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: azure-keyvault
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    useVMManagedIdentity: "false"
    keyvaultName: "multicloud"
    tenantId: "XXXXXXXXXXXXXXXXXXXX"
    cloudName: "AzurePublicCloud"
    objects: |
      array:
      - |
        objectName: multicloud-keystore-truststore-server-password
        objectType: secret

secretObjects:
- secretName: keystore-truststore-credentials
  type: Opaque
  data:
  - objectName: multicloud-keystore-truststore-server-password
    key: password
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f secretProviderClass.gke.yaml -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secretproviderclass.secrets-store.csi.x-k8s.io/azure-keyvault created
[demo@demo-azure-vm ~]$ kubectl create secret generic secrets-store-creds --from-literal=clientid=[REDACTED] --from-literal=clientse
cret=[REDACTED] --type=Opaque --namespace kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/secrets-store-creds created

[demo@demo-azure-vm ~]$ kubectl get secretproviderclass -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                AGE
azure-keyvault      51s
```

Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>

To use Azure Keyvault Secrets in GKE Standard Cluster I used the concept of Azure AD Service Principal as shown in the screenshot attached below.

```
### Create AzureAD Application for GKE
resource "azuread_application" "gke_kv" {
  display_name = "${var.prefix}-gke-azure-kv"
}

### Create AzureAD Service Principal for GKE
resource "azuread_service_principal" "gke_kv_sp" {
  client_id = azuread_application.gke_kv.client_id
}

### Create a Password for the Azure Service Principal
resource "azuread_service_principal_password" "azuread_sp_secret" {
  service_principal_id = azuread_service_principal.gke_kv_sp.id
  display_name         = "azuread-sp-password-for-gke"
  end_date              = "2027-01-01T00:00:00Z"
}

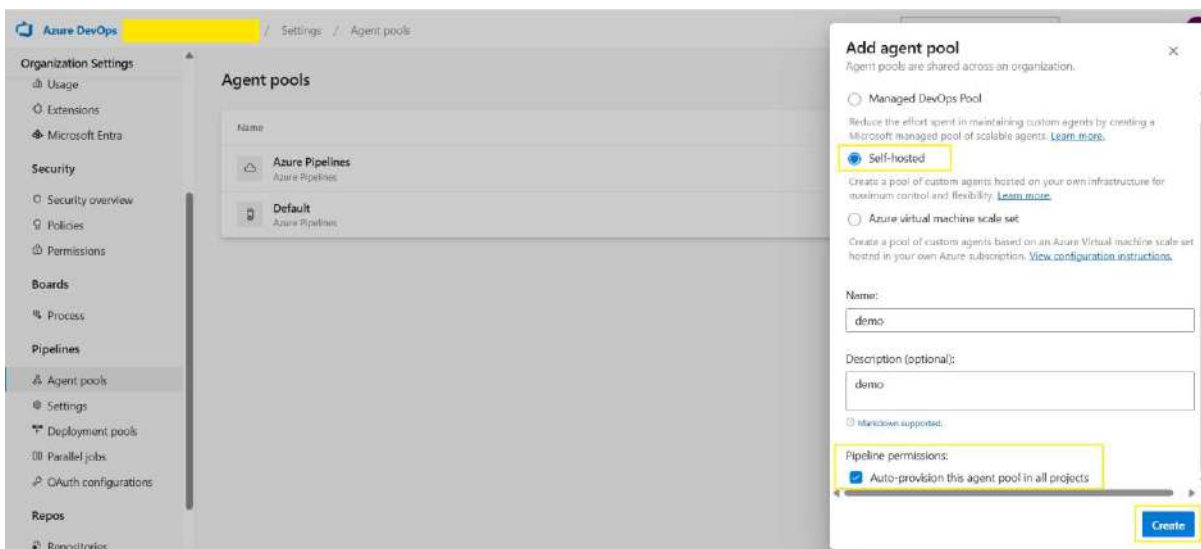
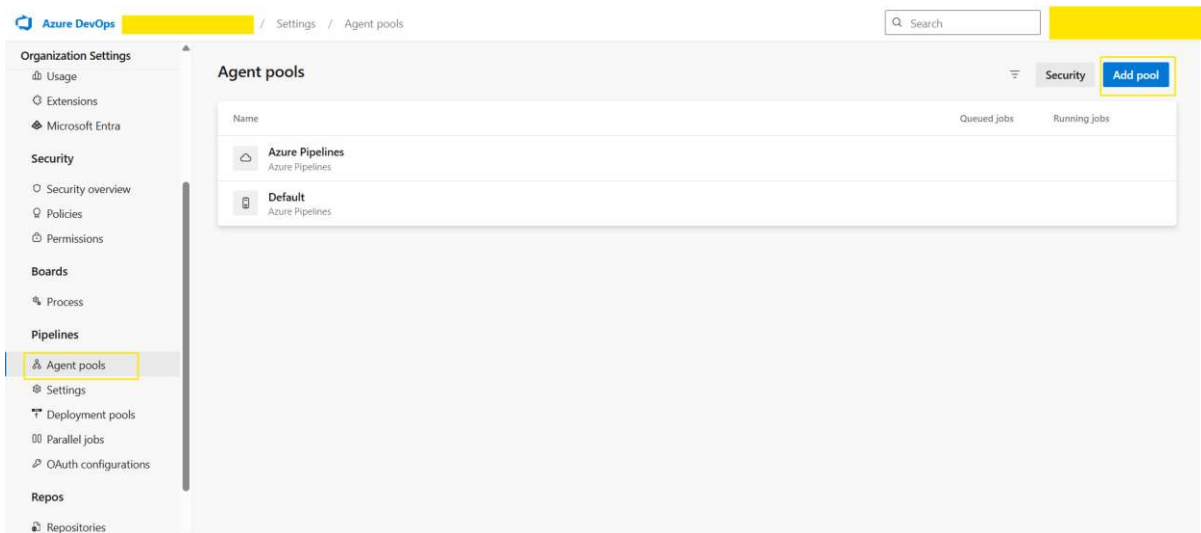
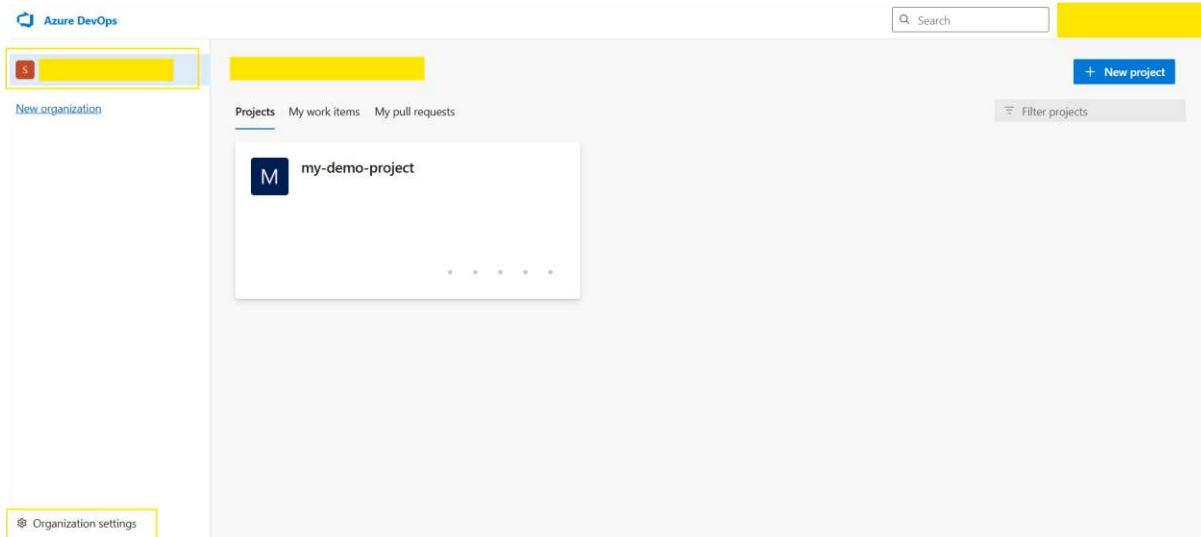
### Allow Azure Key Vault Access to AzureAD Service Principal for GKE
resource "azurerm_role_assignment" "kv_access_sp_for_gke" {
  scope                = azurerm_key_vault.key_vault_postgresql.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = azuread_service_principal.gke_kv_sp.object_id
}
```

For your reference I kept the file **SecretProviderClass-gke.yaml** present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-2/**.

For CI/CD Pipeline I used Aure DevOps and did the deployment for Strimzi-Kafka Pods and Services, Kafka-user, Kafka-topic, Kubernetes Secrets and ConfigMaps, Producer and Consumer deployments using ArgoCD. I had created four Azure DevOps Pipeline first for deployment of Strimzi Kafka Pods and services, deployment of Kafka Users and deployment of Kafka Topic, second for Kubernetes Secrets and ConfigMaps, third for deployment of kafka producer and consumers and fourth for deployment of Kafka-UI (used Kafbat formerly known as Provectus).

To install and configure Azure DevOps Self-hosted Agent follow the procedure as explained below.

Go to the Azure DevOps Organization > Organisation Settings > Agent Pools > Add Pool



Azure DevOps / Settings / Agent pools

Organization Settings

- Usage
- Extensions
- Microsoft Entra
- Security
 - Security overview
 - Policies
 - Permissions
- Boards
 - Process
- Pipelines
 - Agent pools**
 - Settings
 - Deployment pools
 - Parallel jobs
 - OAuth configurations
- Repos
 - Repositories

Agent pools

Name	Queued jobs	Running jobs
Azure Pipelines		
Default		
demo		

Azure DevOps / Settings / Agent pools / demo

Organization Settings

- Search Settings
- General
 - Overview
 - Projects
 - Users
 - Billing
 - Global notifications
 - Usage
 - Extensions
 - Microsoft Entra
- Security
 - Security overview
 - Policies
 - Permissions
- Boards

demo

Update all agents New agent

Jobs Agents Details Security Settings Maintenance History Analytics

Add your first agent

Manage agents and run pipeline jobs on this pool.

New agent

x64

arm

arm64

RHEL6

System prerequisites

Configure your account

Configure your account by following the steps outlined [here](#).

Download the agent

[Download](#)


Create the agent

```
~/ $ mkdir myagent && cd myagent
~/myagent$ tar zxvf ~/Downloads/vsts-agent-linux-x64-4.272.0.tar.gz
```

Configure the agent [Detailed instructions](#)

```
~/myagent$ ./config.sh
```

Optionally run the agent interactively

If you didn't run as a service above:

```
[root@demo-azure-vm myagent]# ls
bin config.sh env.sh externals license.html reauth.sh run-docker.sh run.sh vsts-agent-linux-x64-4.273.0.tar.gz
[root@demo-azure-vm myagent]# rm -f vsts-agent-linux-x64-4.273.0.tar.gz
[root@demo-azure-vm myagent]# chown -R demo:demo /opt/myagent/
[root@demo-azure-vm myagent]# logout
[demo@demo-azure-vm ~]$ cd /opt/myagent/
[demo@demo-azure-vm myagent]$ sudo ./bin/installdependencies.sh

[demo@demo-azure-vm myagent]$ ./config.sh


agent v4.273.0 (commit e89620c)

>> End User License Agreements:

Building sources from a TFVC repository requires accepting the Team Explorer Everywhere End User License Agreement. This step is not required for building sources from Git repositories.

A copy of the Team Explorer Everywhere license agreement can be found at:
/opt/myagent/license.html

Enter (Y/N) Accept the Team Explorer Everywhere license agreement now? (press enter for N) >

>> Connect:

Enter server URL > https://dev.azure.com/
Enter authentication type (press enter for PAT) >
Enter personal access token >
Connecting to server ...

>> Register Agent:

Enter agent pool (press enter for default) > demo
Enter agent name (press enter for demo-azure-vm) > demo
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) >
2026-05-29 08:45:59: Settings Saved.

[demo@demo-azure-vm myagent]$ sudo ./svc.sh install
Creating launch agent in /etc/systemd/system/vsts.agent. demo.demo.service
Run as user: demo
Run as uid:
gid:
/sbin/selinuxstatus
SELinux status: enabled
Created symlink /etc/systemd/system/multi-user.target.wants/vsts.agent. demo.demo.service → /etc/systemd/system/vsts.agent. demo.demo.service.

[demo@demo-azure-vm myagent]$ sudo ./svc.sh start

/etc/systemd/system/vsts.agent. demo.demo.service
● vsts.agent. demo.demo.service - Azure Pipelines Agent ( demo.demo)
   Loaded: loaded (/etc/systemd/system/vsts.agent. demo.demo.service; enabled; vendor preset: disabled)
   Active: active (running) since Fri 2026-05-29 08:45:59 UTC; 14ms ago
   Main PID: 12476 (runsv.sh)
   Tasks: 2 (limit: 24500)
   Memory: 852.0K
   CGroup: /system.slice/vsts.agent. demo.demo.service
           └─12476 /bin/bash /opt/myagent/runsv.sh
             └─12479 ./externals/node20_1/bin/node --version

May 29 08:45:59 demo-azure-vm systemd[1]: Started Azure Pipelines Agent ( demo.demo).
May 29 08:45:59 demo-azure-vm runsv.sh[12476]: .path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/bin:/opt/apache-maven/bin:/opt/node-v16.0.0-r/local/bin
Hint: Some lines were ellipsized, use -l to show in full.

[demo@demo-azure-vm myagent]$ sudo ./svc.sh status

/etc/systemd/system/vsts.agent. demo.demo.service
● vsts.agent. demo.demo.service - Azure Pipelines Agent ( demo.demo)
   Loaded: loaded (/etc/systemd/system/vsts.agent. demo.demo.service; enabled; vendor preset: disabled)
   Active: active (running) since Fri 2026-05-29 08:45:59 UTC; 2s ago
   Main PID: 12476 (runsv.sh)
   Tasks: 24 (limit: 24500)
   Memory: 70.2M
   CGroup: /system.slice/vsts.agent. demo.demo.service
           └─12476 /bin/bash /opt/myagent/runsv.sh
             └─12480 ./externals/node20_1/bin/node ./bin/AgentService.js
               └─12487 /opt/myagent/bin/Agent.Listener run --startuptype service

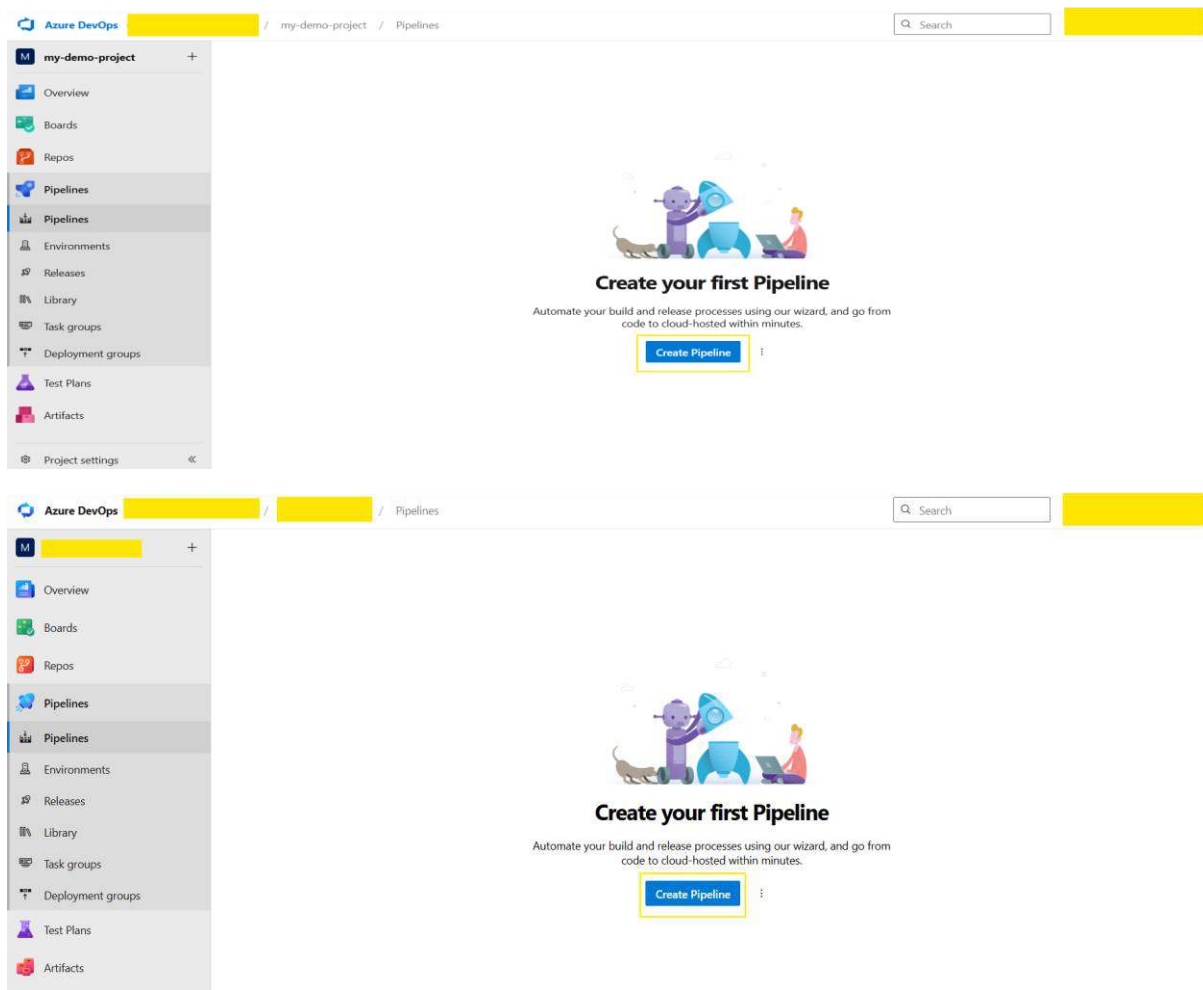
May 29 08:45:59 demo-azure-vm systemd[1]: Started Azure Pipelines Agent ( demo.demo).
May 29 08:45:59 demo-azure-vm runsv.sh[12476]: .path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/bin:/opt/apache-maven/bin:/opt/node-v16.0.0-r/local/bin
May 29 08:45:59 demo-azure-vm runsv.sh[12479]: v20.20.2
May 29 08:45:59 demo-azure-vm runsv.sh[12480]: Starting Agent listener with startup type: service
May 29 08:45:59 demo-azure-vm runsv.sh[12480]: Started listener process
May 29 08:45:59 demo-azure-vm runsv.sh[12480]: Started running service
May 29 08:46:01 demo-azure-vm runsv.sh[12480]: Scanning for tool capabilities.
May 29 08:46:01 demo-azure-vm runsv.sh[12480]: Connecting to the server.
Hint: Some lines were ellipsized, use -l to show in full.
[demo@demo-azure-vm myagent]$ ./env.sh
```

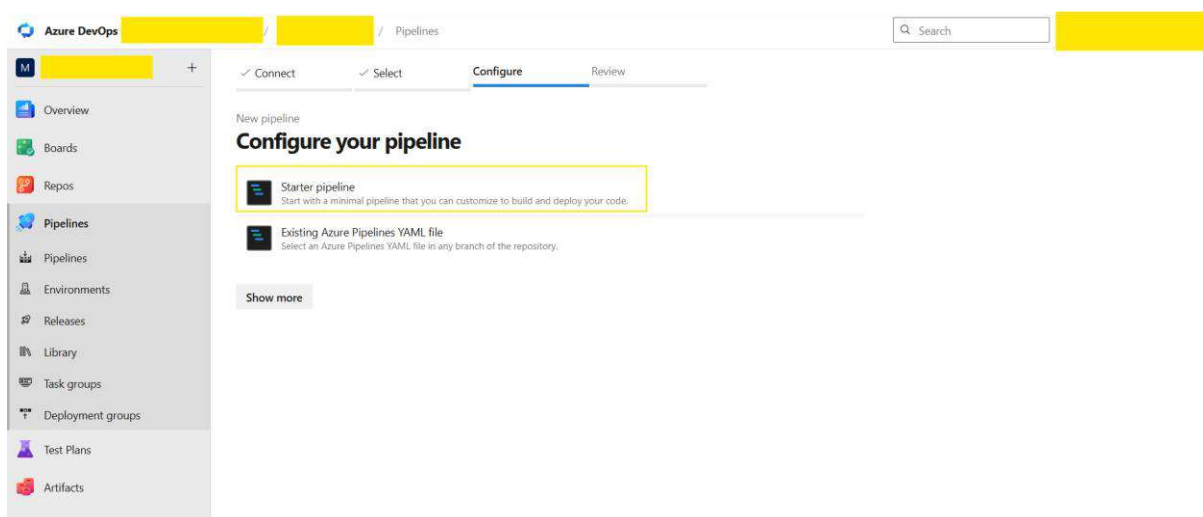
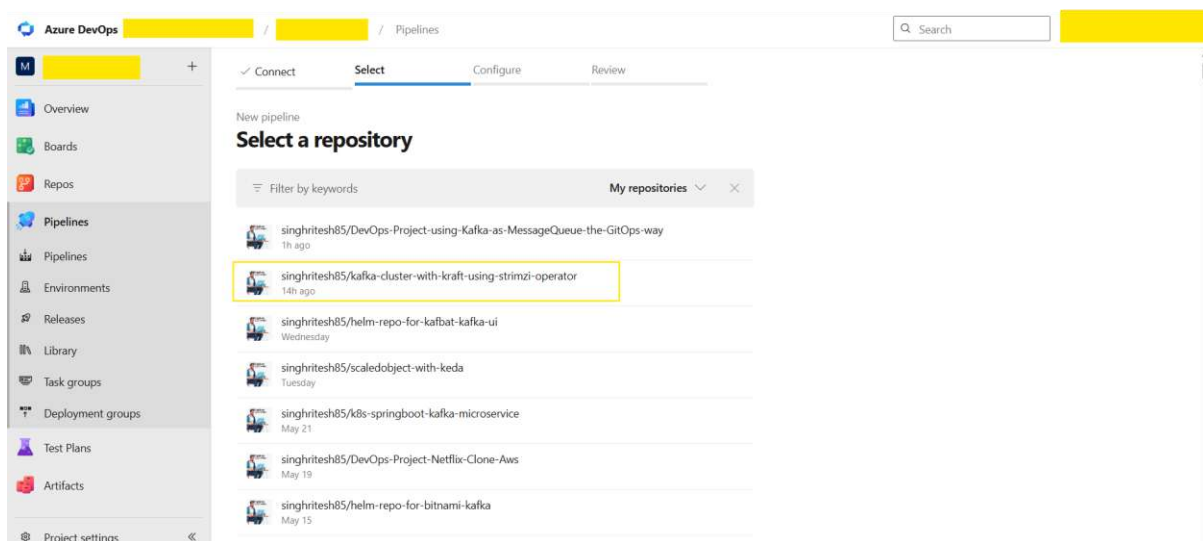
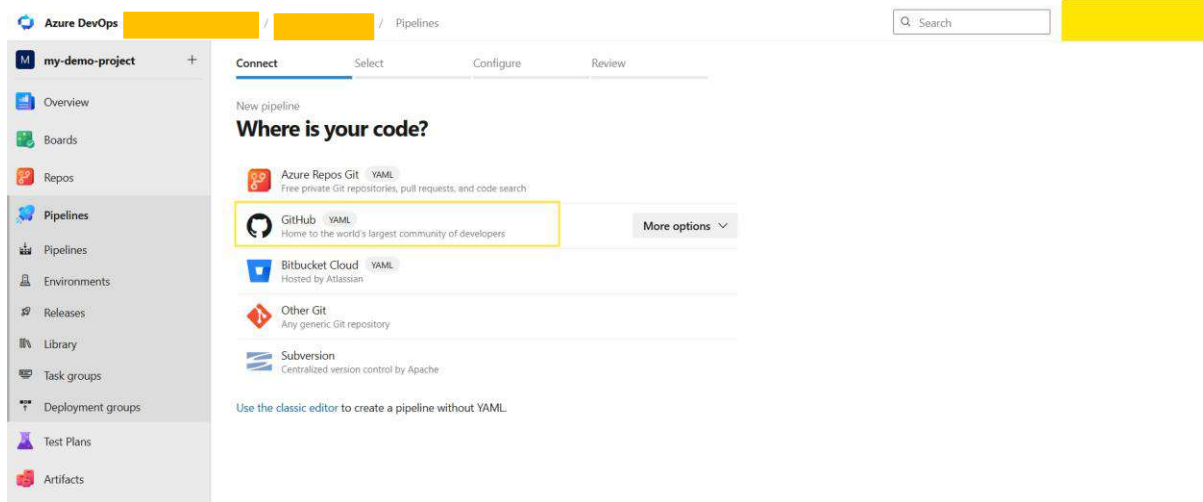
Finally, you will Self-hosted Azure DevOps Agent will be in Online state as shown in the screenshot attached below.

demo					Update all agents	New agent
Jobs Agents Details Security Settings Maintenance History Analytics						
Name	Last run	Current status	Agent version	Enabled		
demo		Idle	4.273.0	☒	On	

In this project I am using Docker Hub Private Repository to keep the Docker Image and the same Image will be pulled from the Docker Hub Private Repository for each Deployment.

Created first pipeline to deploy the Strimzi Kafka Pods and Services as shown in the screenshot attached below.





Before Saving and Running the Pipeline create three variables named as ARG OCD_PASSWORD, GITHUB_USERNAME and GITHUB_PASSWORD in Azure DevOps Pipeline as discussed earlier.

The first screenshot shows the Azure DevOps interface with the 'Review your pipeline YAML' step selected. The pipeline YAML code is displayed, and the 'Variables' button is highlighted. The code includes steps for logging in, adding repository, creating Kafka topics, and deploying the application.

The second screenshot shows the 'Variables' dialog box, which allows users to define variables for the pipeline. It includes a 'New variable' button and a link to 'Learn about variables'.

The third screenshot shows the 'New variable' dialog box, where a new variable named 'ARGOCD_PASSWORD' is being created. The 'Value' field is highlighted, and the 'Keep this value secret' checkbox is checked. The dialog also provides instructions on how to reference variables in YAML and scripts.

Azure DevOps / Pipelines

Connect Select Configure **Review**

New pipeline

Review your pipeline YAML

Q singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator / azure-pipelines-strimzi-kafka.yml * @

```

17 runonci:
18   deploy:
19     steps:
20       - checkout: none
21       - task: CmdLine@2
22         inputs:
23           script: |
24             yes|argocd login argocd.singhritesh85.com --username admin --password $(ARGOCD_PASSWORD)
25             argocd repo add https://github.com/singhritesh85/kafka-cluster-with-kraft
26             argocd app create kafka-with-kraft --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
27             #argocd app sync kafka-with-kraft --kube-context gke_wise-trainer-244916_us-central1-m
28             kubectl get pods -n kafka --context gke_wise-trainer-244916_us-central1-m
29             kubectl get svc -n kafka --context gke_wise-trainer-244916_us-central1-m
30             argocd app create kafka-topic --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
31             #argocd app sync kafka-topic --kube-context gke_wise-trainer-244916_us-central1-m
32             kubectl get kafkatopics -n kafka --context gke_wise-trainer-244916_us-central1-m
33             argocd app create kafka-user --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
34             #argocd app sync kafka-user --kube-context gke_wise-trainer-244916_us-central1-m
35             kubectl get kafkausers -n kafka --context gke_wise-trainer-244916_us-central1-m
36
37

```

Variables

Search variables

ARGOCD_PASSWORD

Learn about variables

Cancel Save

Azure DevOps / Pipelines

Connect Select Configure **Review**

New pipeline

Review your pipeline YAML

Q singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator / azure-pipelines-strimzi-kafka.yml * @

```

17 runonci:
18   deploy:
19     steps:
20       - checkout: none
21       - task: CmdLine@2
22         inputs:
23           script: |
24             yes|argocd login argocd.singhritesh85.com --username admin --password $(ARGOCD_PASSWORD)
25             argocd repo add https://github.com/singhritesh85/kafka-cluster-with-kraft
26             argocd app create kafka-with-kraft --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
27             #argocd app sync kafka-with-kraft --kube-context gke_wise-trainer-244916_us-central1-m
28             kubectl get pods -n kafka --context gke_wise-trainer-244916_us-central1-m
29             kubectl get svc -n kafka --context gke_wise-trainer-244916_us-central1-m
30             argocd app create kafka-topic --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
31             #argocd app sync kafka-topic --kube-context gke_wise-trainer-244916_us-central1-m
32             kubectl get kafkatopics -n kafka --context gke_wise-trainer-244916_us-central1-m
33             argocd app create kafka-user --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
34             #argocd app sync kafka-user --kube-context gke_wise-trainer-244916_us-central1-m
35             kubectl get kafkausers -n kafka --context gke_wise-trainer-244916_us-central1-m
36
37

```

New variable

Name

GITHUB_USERNAME

Value

☐ Keep this value secret

☐ Let users override this value when running this pipeline

To reference a variable in YAML, prefix it with a dollar sign and enclose it in parentheses. For example: \$(GITHUB_USERNAME)

To use a variable in a script, use environment variable syntax. Replace _ and space with _, capitalize the letters, and then use your platform's syntax for referencing an environment variable. Examples:

Batch script: %GITHUB_USERNAME%

PowerShell script: \$(env:GITHUB_USERNAME)

Bash script: \${GITHUB_USERNAME}

To use a secret variable in a script, you must explicitly map it as an environment variable.

Learn about variables

Cancel OK

Azure DevOps / Pipelines

Connect Select Configure **Review**

New pipeline

Review your pipeline YAML

Q singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator / azure-pipelines-strimzi-kafka.yml * @

```

17 runonci:
18   deploy:
19     steps:
20       - checkout: none
21       - task: CmdLine@2
22         inputs:
23           script: |
24             yes|argocd login argocd.singhritesh85.com --username admin --password $(ARGOCD_PASSWORD)
25             argocd repo add https://github.com/singhritesh85/kafka-cluster-with-kraft
26             argocd app create kafka-with-kraft --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
27             #argocd app sync kafka-with-kraft --kube-context gke_wise-trainer-244916_us-central1-m
28             kubectl get pods -n kafka --context gke_wise-trainer-244916_us-central1-m
29             kubectl get svc -n kafka --context gke_wise-trainer-244916_us-central1-m
30             argocd app create kafka-topic --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
31             #argocd app sync kafka-topic --kube-context gke_wise-trainer-244916_us-central1-m
32             kubectl get kafkatopics -n kafka --context gke_wise-trainer-244916_us-central1-m
33             argocd app create kafka-user --project default --repo https://github.com/singhritesh85/kafka-cluster-with-kraft
34             #argocd app sync kafka-user --kube-context gke_wise-trainer-244916_us-central1-m
35             kubectl get kafkausers -n kafka --context gke_wise-trainer-244916_us-central1-m
36
37

```

Variables

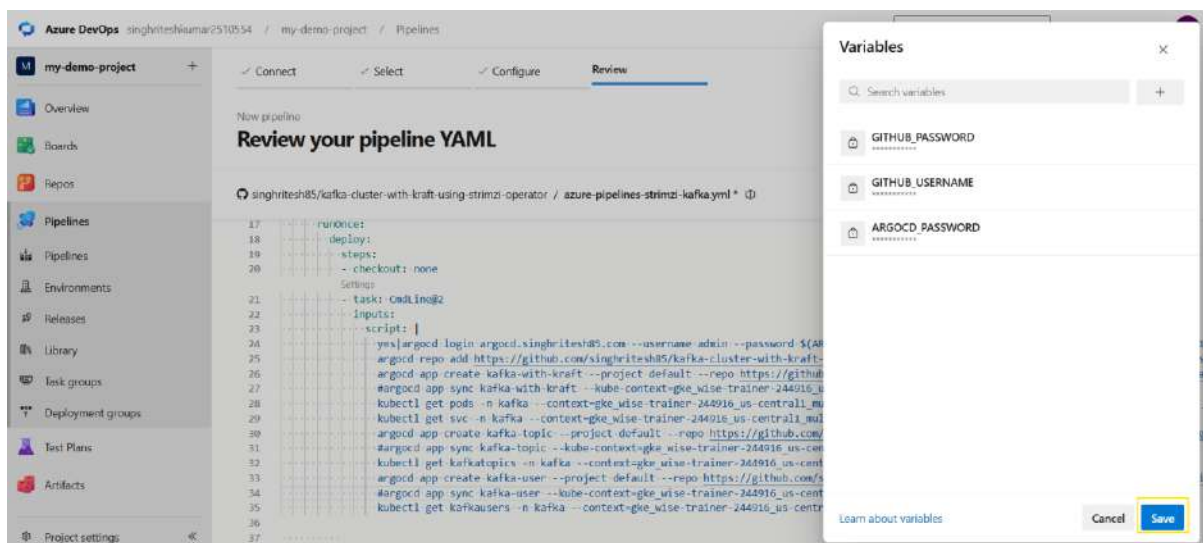
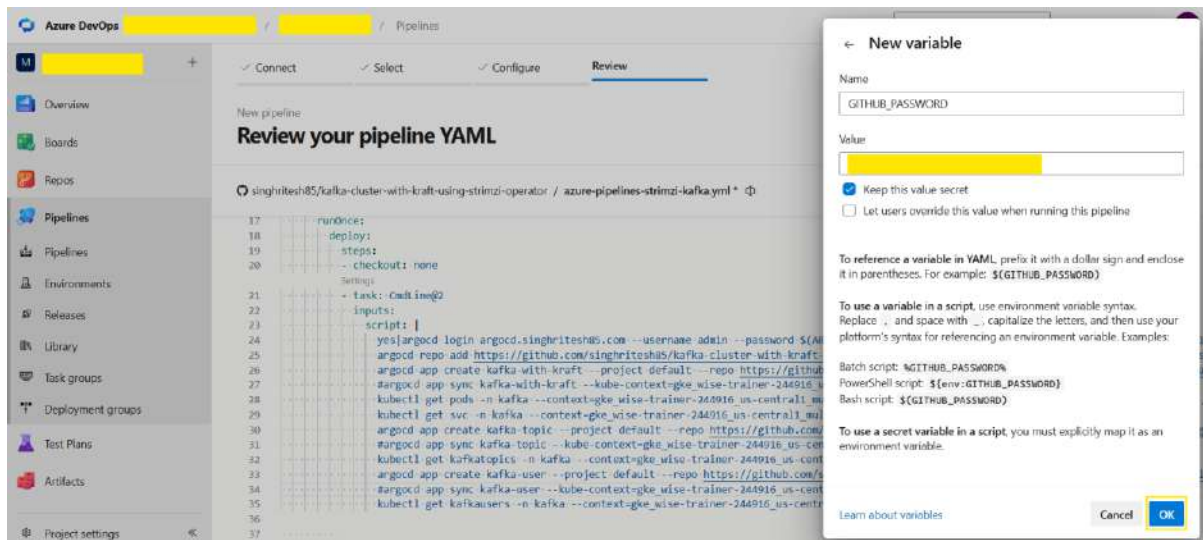
Search variables

GITHUB_USERNAME

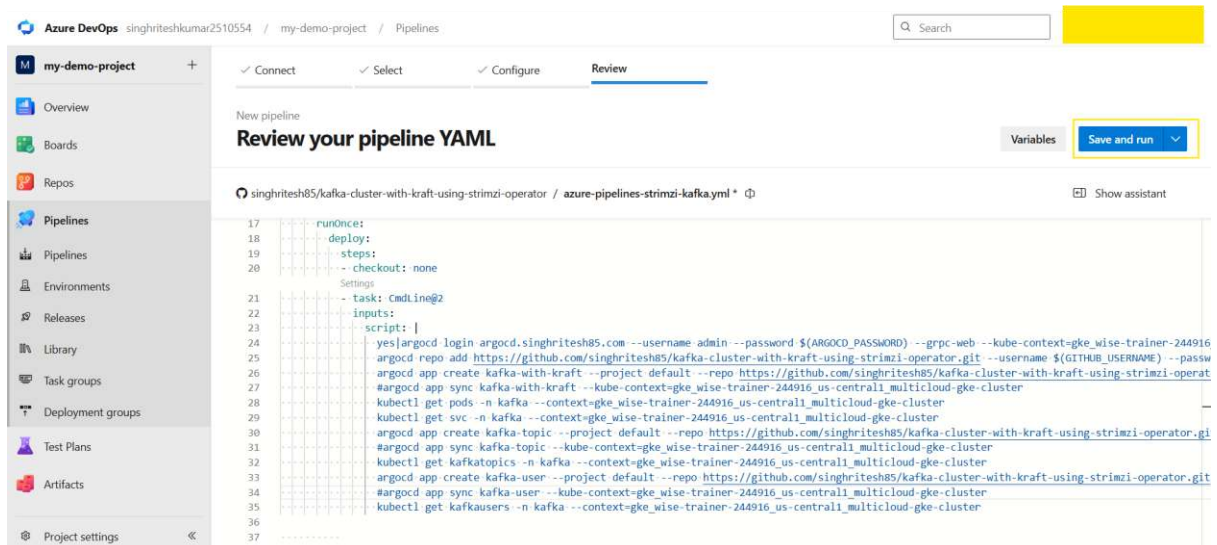
ARGOCD_PASSWORD

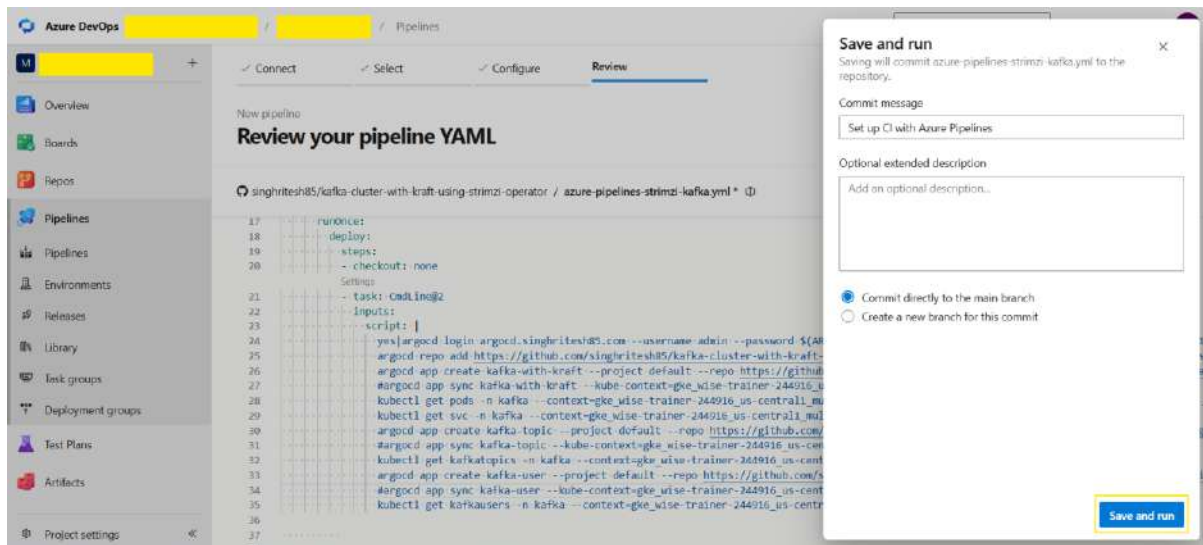
Learn about variables

Cancel Save



Now, to execute this pipeline click on **Save and Run** as shown in the screenshot attached below.



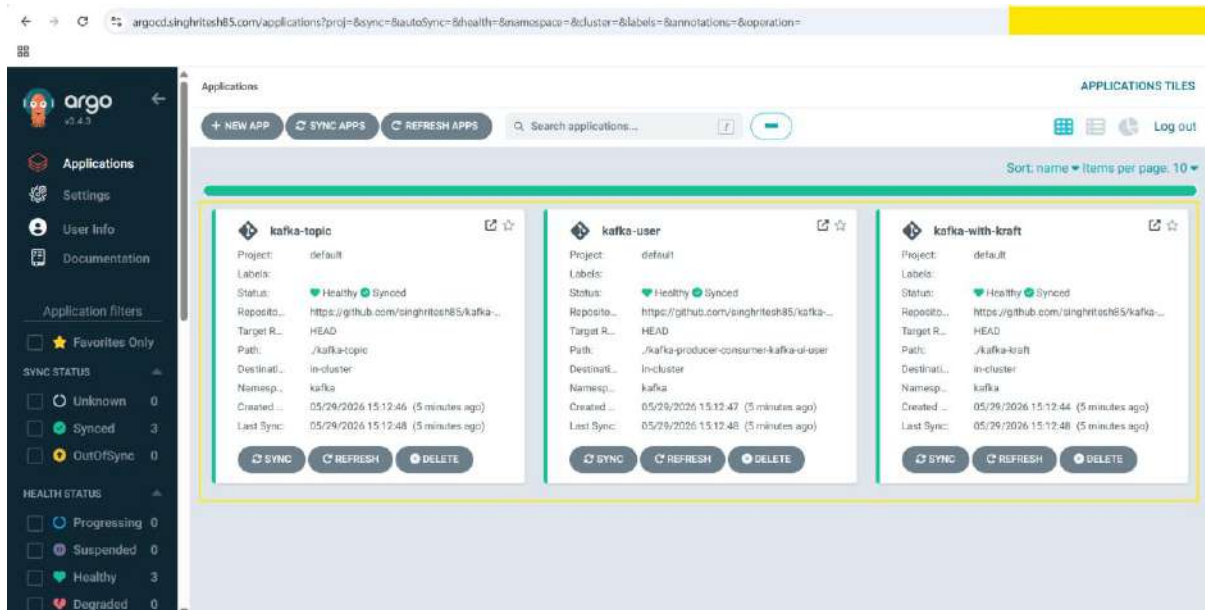


Finally, Strimzi Kafka Pods and services, kafka topic and kafka users had been created as shown in the screenshot attached below.

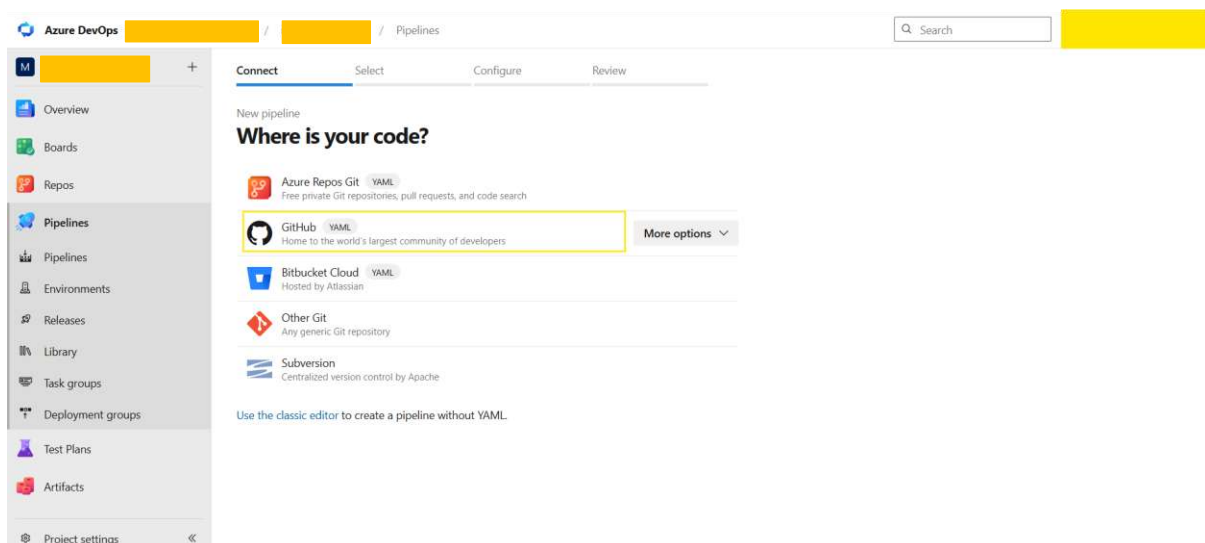
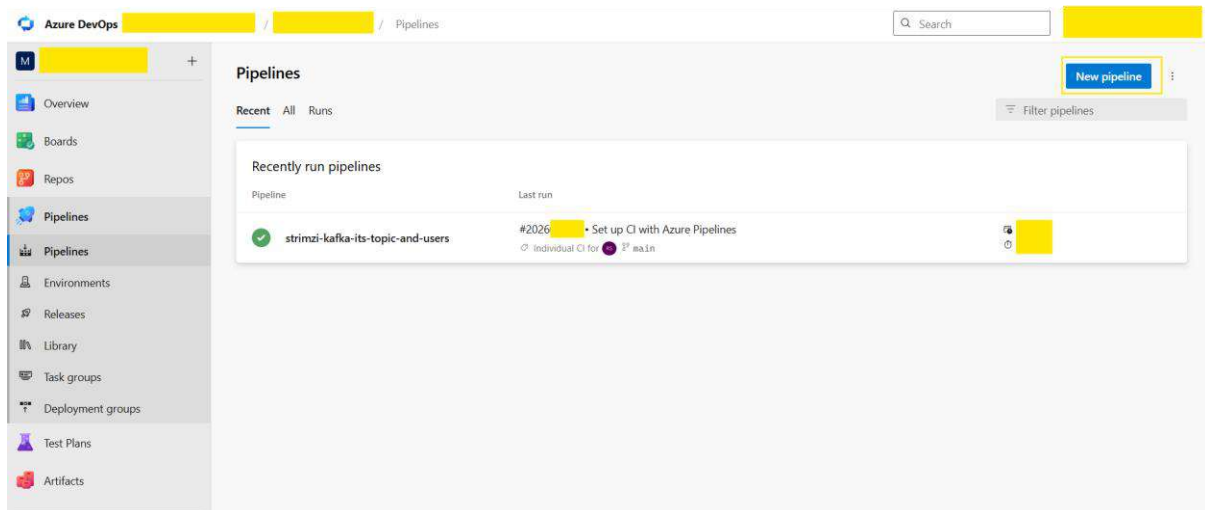
```
[demo@demo-azure-vm myagent]$ kubectl get pods -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                READY   STATUS    RESTARTS   AGE
kafka-with-kraft-cruise-control-7ccf6848b8-rk8dd    1/1     Running   0           38s
kafka-with-kraft-entity-operator-57d66b45b-19gvt     2/2     Running   0           60s
kafka-with-kraft-kafka-with-kraft-broker-0          1/1     Running   0          109s
kafka-with-kraft-kafka-with-kraft-broker-1          1/1     Running   0          109s
kafka-with-kraft-kafka-with-kraft-broker-2          1/1     Running   0          109s
kafka-with-kraft-kafka-with-kraft-controller-3       1/1     Running   0          109s
kafka-with-kraft-kafka-with-kraft-controller-4       1/1     Running   0          109s
kafka-with-kraft-kafka-with-kraft-controller-5       1/1     Running   0          109s
strimzi-cluster-operator-5b6db6bdc-cwg22            1/1     Running   0          4h45m
[demo@demo-azure-vm myagent]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
kafka-metrics                      ClusterIP          172.19.193.168  <none>           9404/TCP         2m
kafka-with-kraft-cruise-control    ClusterIP          172.19.183.42   <none>           9090/TCP         44s
kafka-with-kraft-kafka-bootstrap    ClusterIP          172.19.175.91   <none>           9091/TCP,9092/TCP,9093/TCP,9094/TCP  116s
kafka-with-kraft-kafka-brokers      ClusterIP          None            <none>           9090/TCP,9091/TCP,8443/TCP,9092/TCP,9093/TCP,9094/TCP  116s
[demo@demo-azure-vm myagent]$ kubectl get kafkatopics -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME          CLUSTER   PARTITIONS  REPLICATION FACTOR  READY
kafka-demo-topic  kafka-with-kraft  1           3                    True
[demo@demo-azure-vm myagent]$ kubectl get kafkausers -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME          CLUSTER   AUTHENTICATION  AUTHORIZATION  READY
kafka-consumer  kafka-with-kraft  tls            simple         True
kafka-producer  kafka-with-kraft  tls            simple         True
kafka-ui        kafka-with-kraft  tls            simple         True
```

The screenshot of Azure DevOps and ArgoCD UI after successfully creation of the three applications as shown below.





When you create a kafka user in Strimzi kafka, by-default secrets also get created, those secrets contain passwords keys and certificates. Using those passwords and certificates I had created Azure DevOps Pipeline to create the Kubernetes Secrets and ConfigMap.



The first screenshot shows the 'Select a repository' step in the Azure DevOps Pipelines interface. The left sidebar contains navigation links for Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, Artifacts, and Project settings. The main area shows a list of repositories under the heading 'Select a repository'. The repository 'singhritesh85/strimzi-kafka-demo' is highlighted.

The second screenshot shows the 'Configure your pipeline' step. The left sidebar is the same. The main area shows options for configuring the pipeline: Docker, Deploy to Azure Kubernetes Service, Starter pipeline (highlighted), and Existing Azure Pipelines YAML file. A 'Show more' button is at the bottom.

Finally, Save and Run the Azure DevOps Pipeline as shown in the screenshot attached below.

The screenshot shows the 'Review your pipeline YAML' step in the Azure DevOps Pipelines interface. The left sidebar is the same. The main area shows the pipeline name 'singhritesh85/strimzi-kafka-demo / azure-pipelines-kubernetes-secrets-and-configmaps.yml'. The 'Save and run' button is highlighted.

```

42
43 if [[ -z "/bin/bash -c 'kubectl get secret -n producer-consumer --context=gke_wise-trainer-244916-us-central1-multicloud-gke-cluster|grep
44 then
45     kubectl create secret generic kafka-producer-app --from-file=keystore.p12=/tmp/kafka-producer.p12 --from-file=keystore.password=/tmp/kaf
46 else
47     kubectl delete secret kafka-producer-app -n producer-consumer --context=gke_wise-trainer-244916-us-central1-multicloud-gke-cluster
48     kubectl create secret generic kafka-producer-app --from-file=keystore.p12=/tmp/kafka-producer.p12 --from-file=keystore.password=/tmp/kaf
49 fi
50 if [[ -z "/bin/bash -c 'kubectl get configmap -n producer-consumer --context=gke_wise-trainer-244916-us-central1-multicloud-gke-cluster|gr
51 then
52     kubectl create configmap kafka-consumer-app --from-file=src/kafka-consumer/src/main/resources/application.yml -n producer-consumer --con
53 else
54     kubectl delete configmap kafka-consumer-app -n producer-consumer --context=gke_wise-trainer-244916-us-central1-multicloud-gke-cluster
55     kubectl create configmap kafka-consumer-app --from-file=src/kafka-consumer/src/main/resources/application.yml -n producer-consumer --con
56 fi
57 if [[ -z "/bin/bash -c 'kubectl get secret -n producer-consumer --context=gke_wise-trainer-244916-us-central1-multicloud-gke-cluster|grep
58 then
59     kubectl create secret generic kafka-consumer-app --from-file=keystore.p12=/tmp/kafka-consumer.p12 --from-file=keystore.password=/tmp/kaf
60 else
61     kubectl delete secret generic kafka-consumer-app -n producer-consumer --context=gke_wise-trainer-244916-us-central1-multicloud-gke-clust
62     kubectl create secret generic kafka-consumer-app --from-file=keystore.p12=/tmp/kafka-consumer.p12 --from-file=keystore.password=/tmp/kaf
63 fi
  
```

Finally, ConfigMaps and Secrets had been created after successfully running the azure devops pipeline as shown in the screenshot attached below and Kubernetes Secrets and ConfigMaps had been created.


```
[demo@demo-azure-vm ~]$ kubectl get secrets -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                TYPE      DATA      AGE
kafka-consumer-app                 Opaque    4           2m43s
kafka-producer-app                 Opaque    4           2m43s
secrets-store-creds                 Opaque    2           4h44m
[demo@demo-azure-vm ~]$ kubectl get configmaps -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                DATA      AGE
kafka-consumer-app                 1           2m57s
kafka-producer-app                 1           2m58s
kube-root-ca.crt                   1           4h46m
```

For your reference I had provided those commands separately as shown in the screenshot attached below.

```
kubectl get secrets -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl describe kafkausers kafka-producer -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl describe kafkausers kafka-consumer -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl describe secret kafka-producer -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl describe secret kafka-consumer -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl get secret kafka-producer -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.user\.p12}' | base64 -d > /tmp/kafka-producer.p12

kubectl get secret kafka-producer -o jsonpath='{.data.user\.password}' -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d > /tmp/kafka-producer.password

kubectl get secret kafka-consumer -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.user\.p12}' | base64 -d > /tmp/kafka-consumer.p12

kubectl get secret kafka-consumer -o jsonpath='{.data.user\.password}' -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d > /tmp/kafka-consumer.password

kubectl get secret kafka-with-kraft-cluster-ca-cert -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.ca\.p12}' | base64 -d > /tmp/ca.p12

kubectl get secret kafka-with-kraft-cluster-ca-cert -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.ca\.password}' | base64 -d > /tmp/ca.password

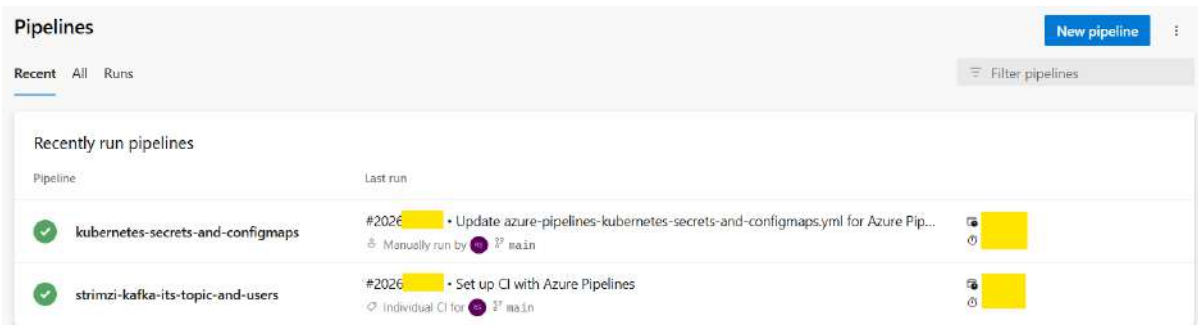
kubectl create configmap kafka-producer-app --from-file=strimzi-kafka-demo/src/kafka-producer/src/main/resources/application.yml -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl create secret generic kafka-producer-app --from-file=keystore.p12=/tmp/kafka-producer.p12 --from-file=keystore.password=/tmp/kafka-producer.password --from-file=truststore.p12=/tmp/ca.p12 --from-file=truststore.password=/tmp/ca.password -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

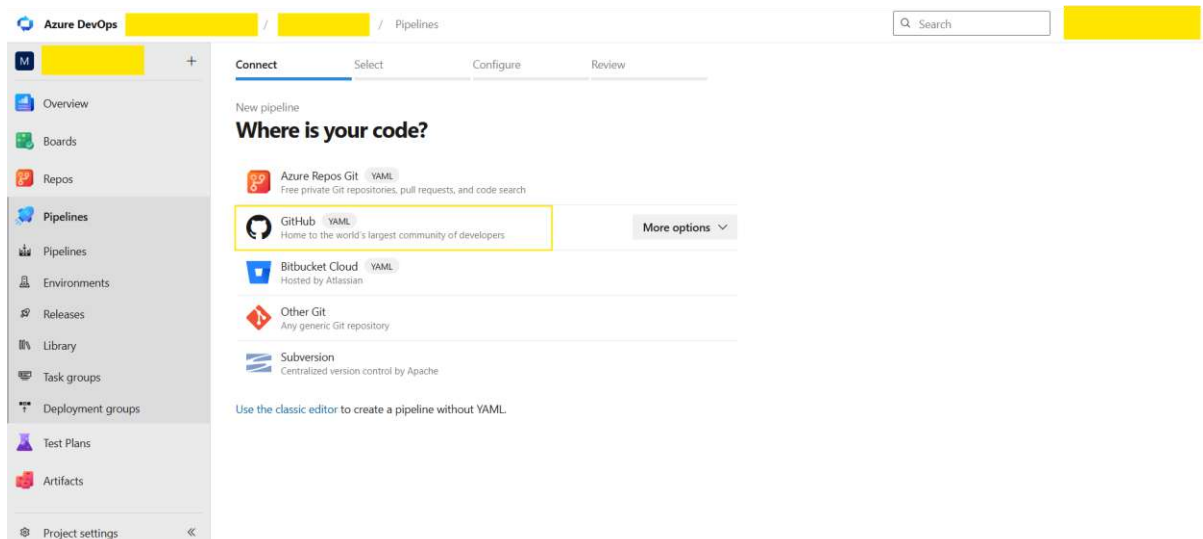
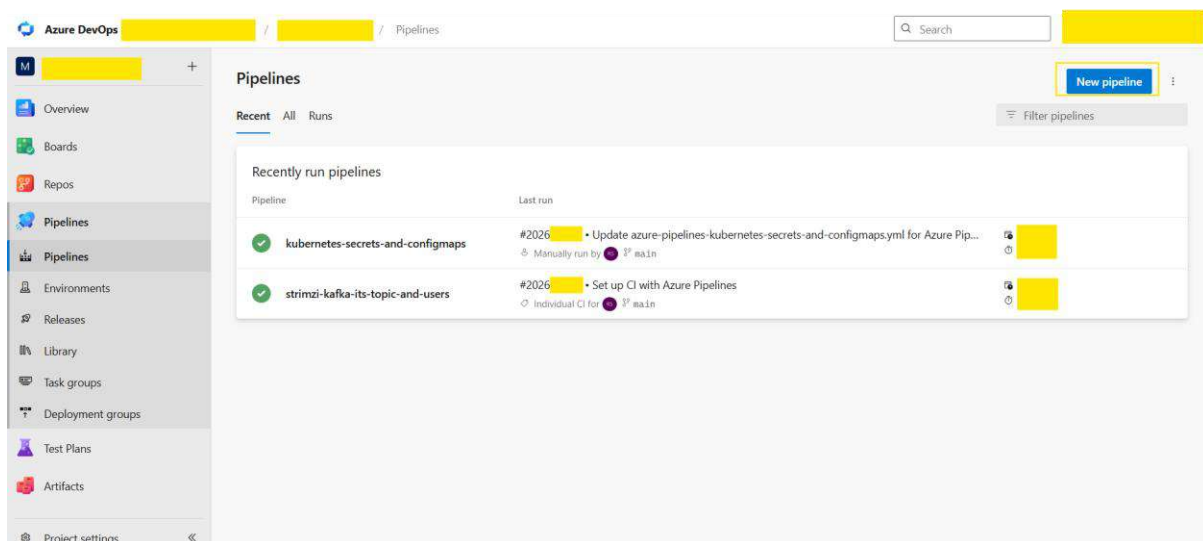
kubectl create configmap kafka-consumer-app --from-file=strimzi-kafka-demo/src/kafka-consumer/src/main/resources/application.yml -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubectl create secret generic kafka-consumer-app --from-file=keystore.p12=/tmp/kafka-consumer.p12 --from-file=keystore.password=/tmp/kafka-consumer.password --from-file=truststore.p12=/tmp/ca.p12 --from-file=truststore.password=/tmp/ca.password -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

The screenshot of Azure DevOps Pipeline after its successful execution is as shown in the screenshot attached below.

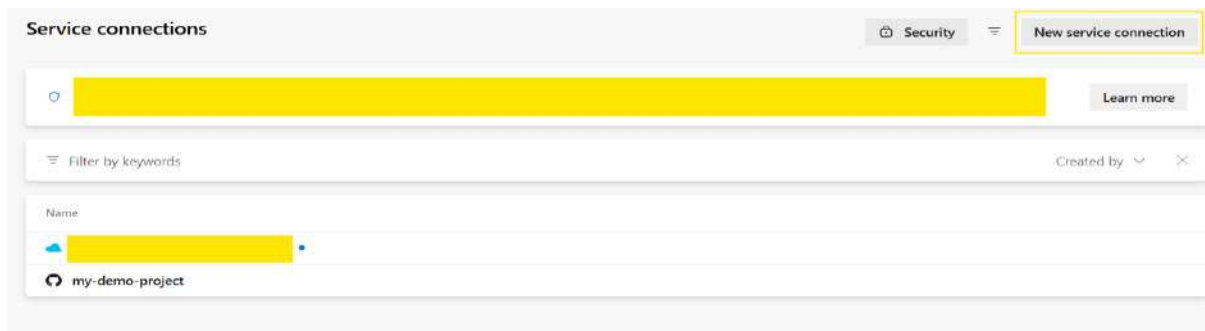
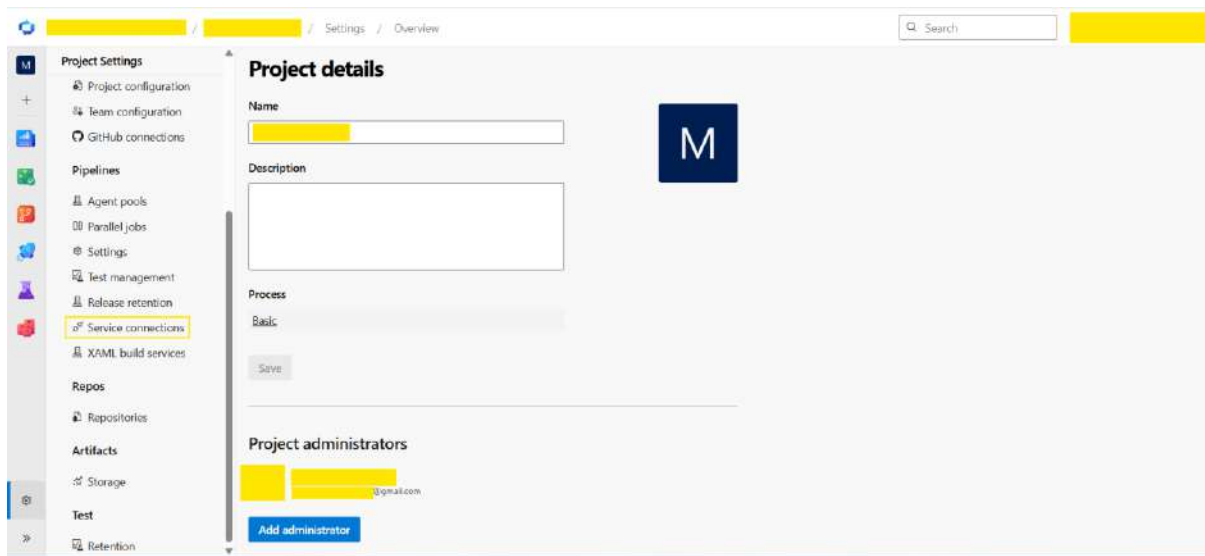
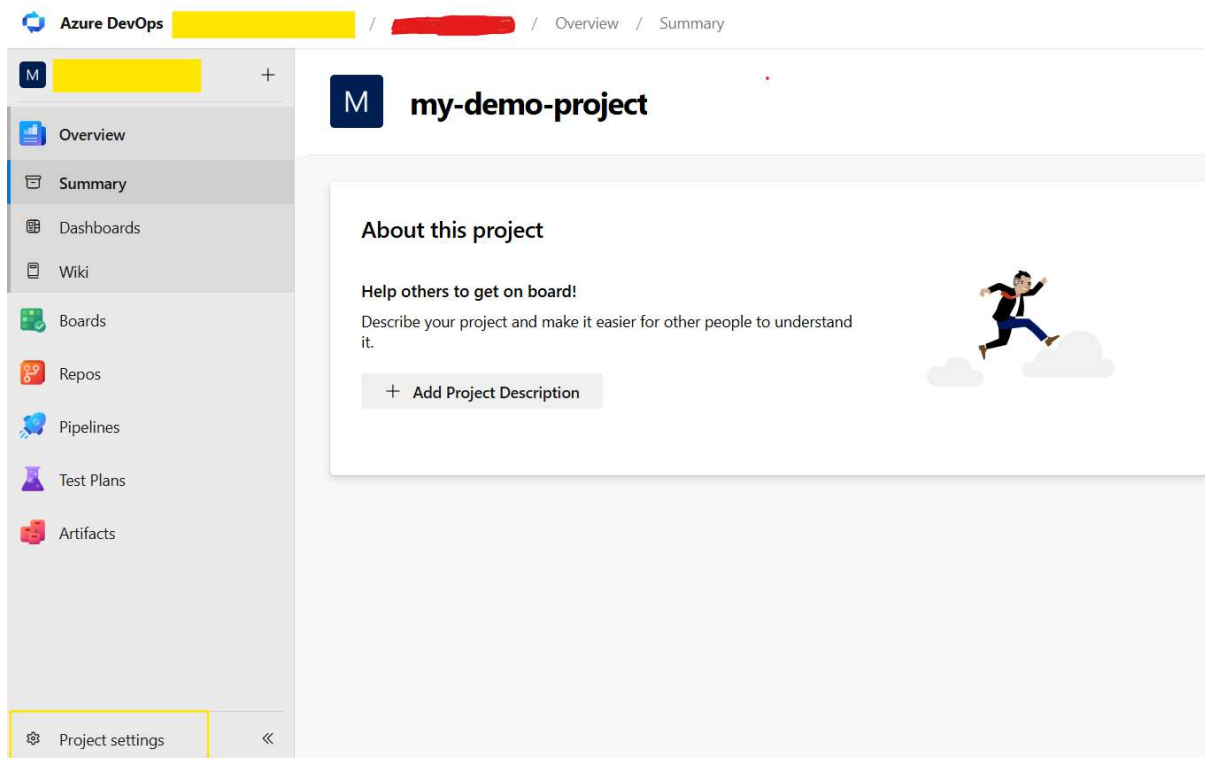


Created third Azure DevOps Pipeline for Deployment of Kafka-Producer and Kafka-Consumer App as shown in the screenshot attached below.



Created **Service Connection** for Docker Hub Private Registry as shown in the screenshot attached below.

Go to **Azure DevOps Organisation > Azure DevOps Project > Project Settings > Service Connections**



Service connections

Convert your existing Azure Resource Manager service connections which use secrets to authenticate to leverage improved security and simplified maintenance.

Filter by keywords

Name

 my-demo-project

New Docker Registry service connection

Registry type

☒ Docker Hub ☐ Others ☐ Azure Container Registry

Docker Registry

Docker ID

Docker Password

Email (optional)

Verify

Service connection details

Service Connection Name

Description (optional)

Service connections

Filter by keywords

Name

 my-demo-project

New Docker Registry service connection



Docker Password

Email (optional)

Verify

Service connection details

Service Connection Name

Description (optional)

Security

☒ Grant access permission to all pipelines

[Learn more](#) [Troubleshoot](#) [Back](#) [Verify and save](#)

Service connections

 Security [New service connection](#)

  [Learn more](#)

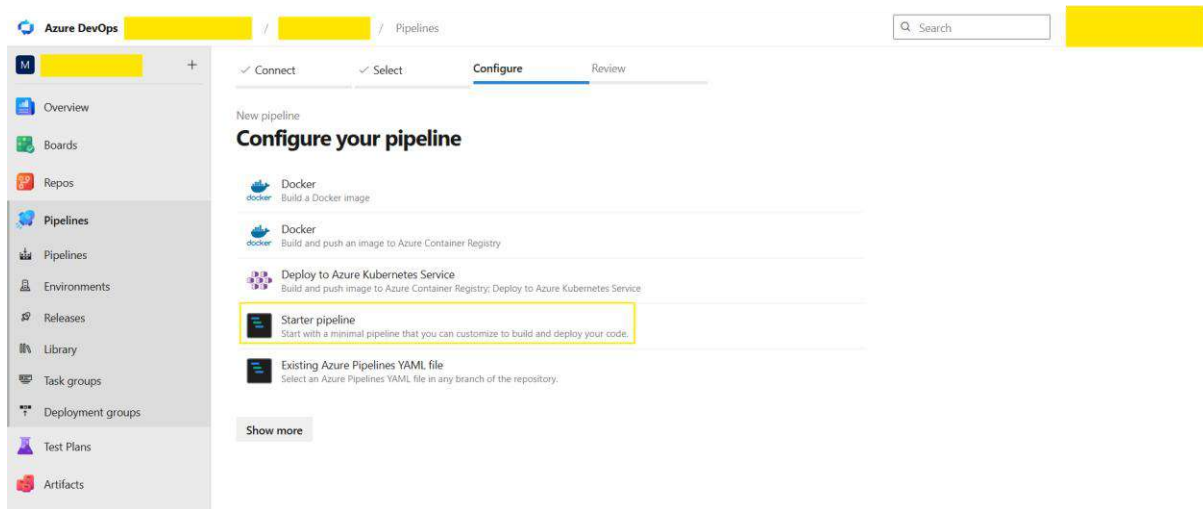
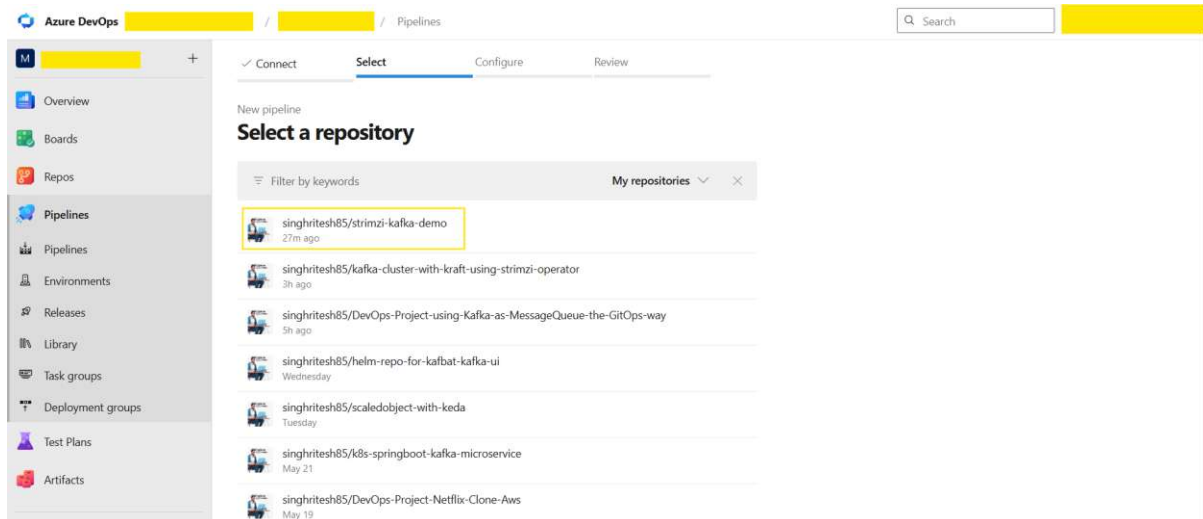
Filter by keywords Created by 

Name

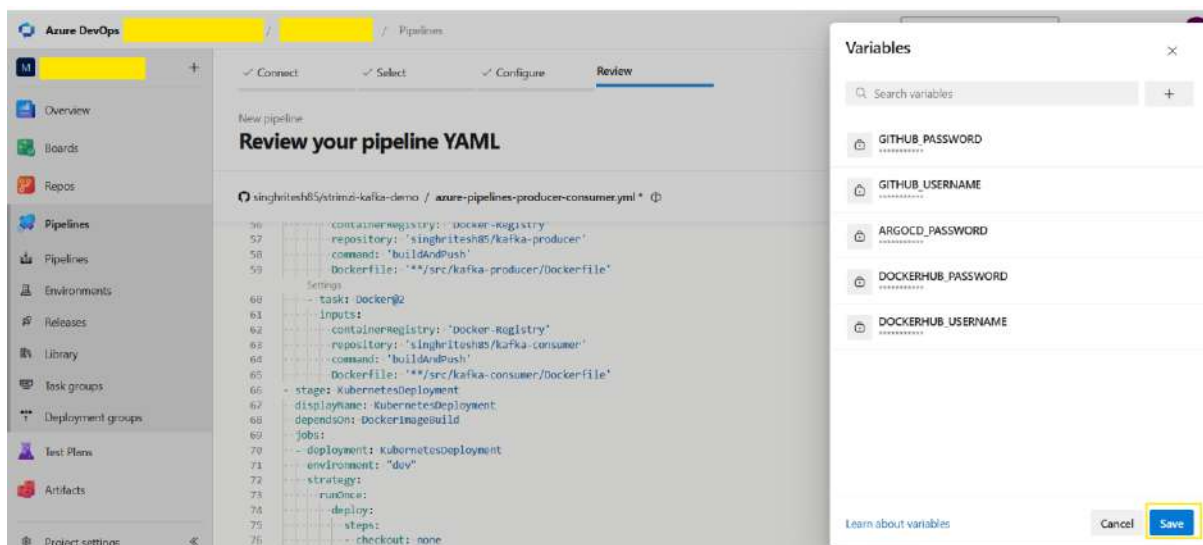
 

 Docker-Registry

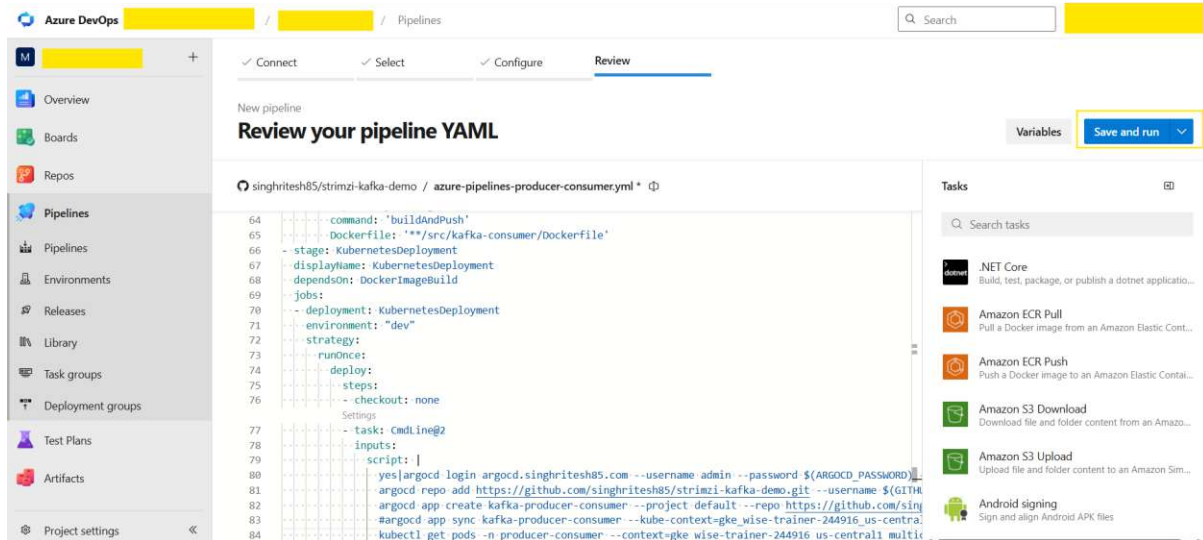
 my-demo-project



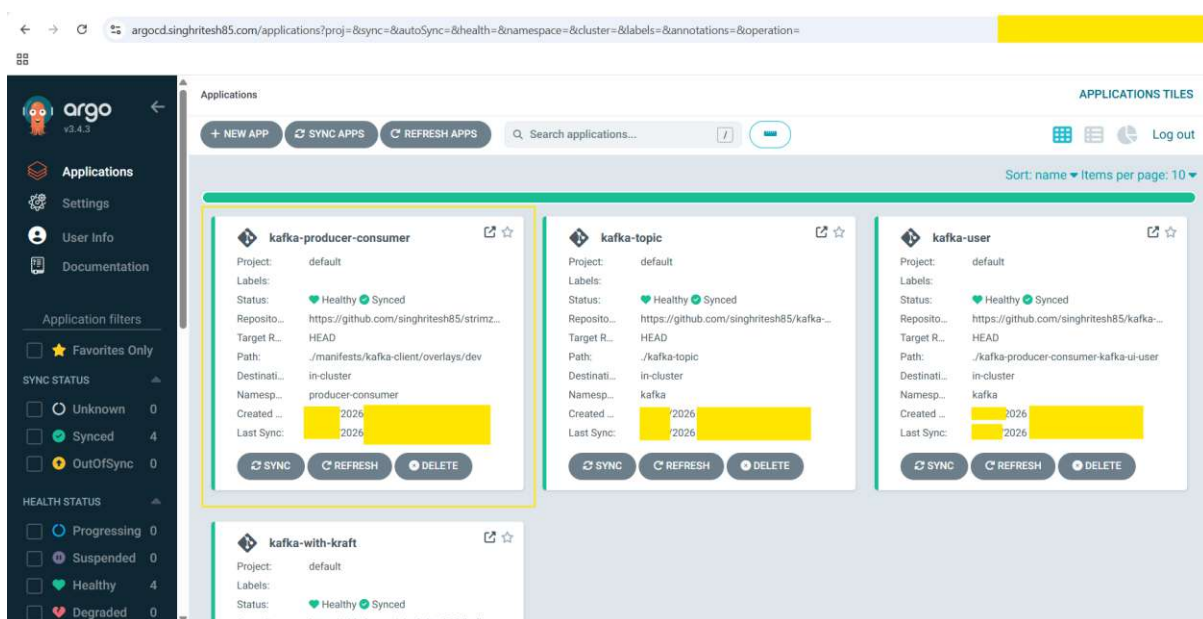
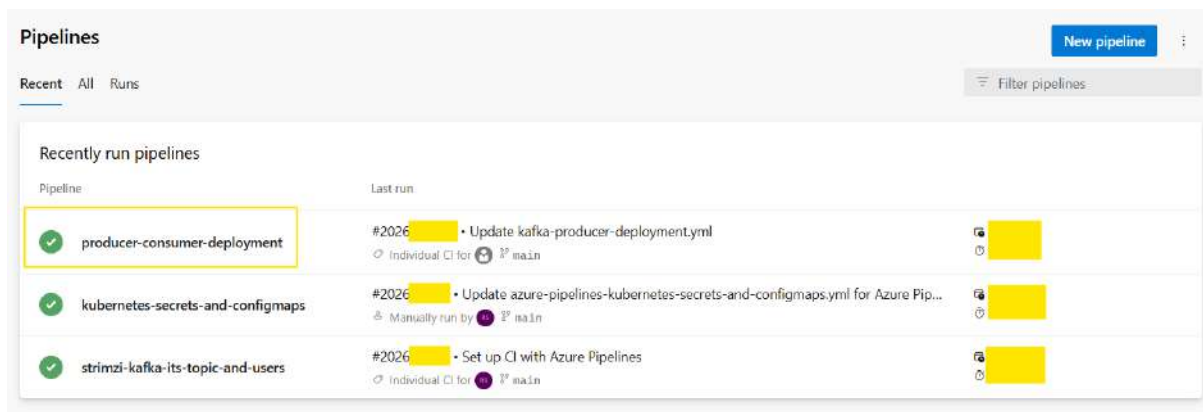
Before Saving and Running the Pipeline create five variables named as DOCKERHUB_USERNAME, DOCKERHUB_PASSWORD, ARGOCD_PASSWORD, GITHUB_USERNAME and GITHUB_PASSWORD in Azure DevOps Pipeline as discussed earlier.



Finally, Save and Run the Azure DevOps Pipeline as shown in the screenshot attached below.



Finally, ran the Azure DevOps Pipeline and screenshot of Azure DevOps Pipeline and ArgoCD Application after successfully running the Pipeline is as shown in the screenshot attached below.



The screenshot for kubernetes pods and deployments of kafka-producer and consumer is as shown in the screenshot attached below.

```
[demo@demo-azure-vm ~]$ kubectl get pods -n producer-consumer
NAME                                READY   STATUS    RESTARTS   AGE
kafka-consumer-cd78dfc7f-mhnwx     1/1     Running   0           11m
kafka-producer-59859dcdc5-8jm4t     1/1     Running   0           11m
[demo@demo-azure-vm ~]$ kubectl get deployments -n producer-consumer
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
kafka-consumer     1/1     1             1           16m
kafka-producer     1/1     1             1           16m
```

Then sent a message using the command as shown below and checked there is no error.

```
kubectl exec deployment/kafka-producer -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -- curl -X POST -d 'mon message' -H 'Content-Type: application/json' http://localhost:8080/produce/message
```

```
[demo@demo-azure-vm ~]$ kubectl exec deployment/kafka-producer -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -- curl -X POST -d 'mon message' -H 'Content-Type: application/json' http://localhost:8080/produce/message
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           % Done    %         kbytes   spent   left   left      speed
100    100    100    111    0:00:01  0:00:01 --:--:--  29
Message sent successfully
```

```
[demo@demo-azure-vm ~]$ kubectl logs -f deployment/kafka-producer -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

As shown in screenshot attached below, the consumer application displays the logs for content of the messages recorded in the kafka-demo-topic topic.

```
[demo@demo-azure-vm ~]$ kubectl logs -f deployment/kafka-consumer -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | grep -i "mon message"
2026-05-29 19:39:59.458 INFO 1 --- [ntainer#0-0-C-1] io.project.kafka.consumer.KafkaConsumer : Received record from Topic-Partition 'kafka-demo-topic-0' with Offset '1' -> Key: 'null' - Value 'mon message'
```

Now, Obtained the password for Grafana admin user using the command as shown below.

```
kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 --decode; echo
```

```
[demo@demo-azure-vm ~]$ kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 --decode; echo
r-244916_us-central1_multicloud-gke-cluster | base64 --decode; echo
```

To access Grafana using the URL through Kubernetes Gateway API, I created the kubernetes secrets.

Then applied the gateway-api YAML manifests file as shown in the screenshot attached below. For

your reference I kept this gateway-api YAML manifests file in the GitHub Repo

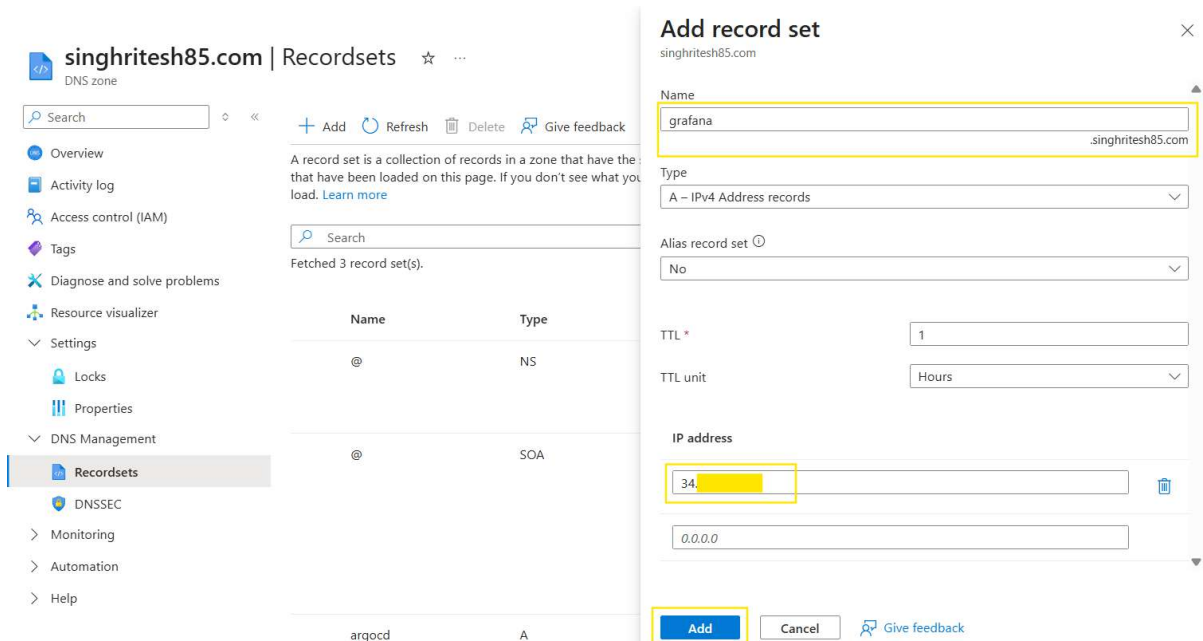
<https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>

at the path **module-2/gateway-api-grafana.yaml**. Grafana and Prometheus had been installed during initial stage using the helm commands s provided earlier.

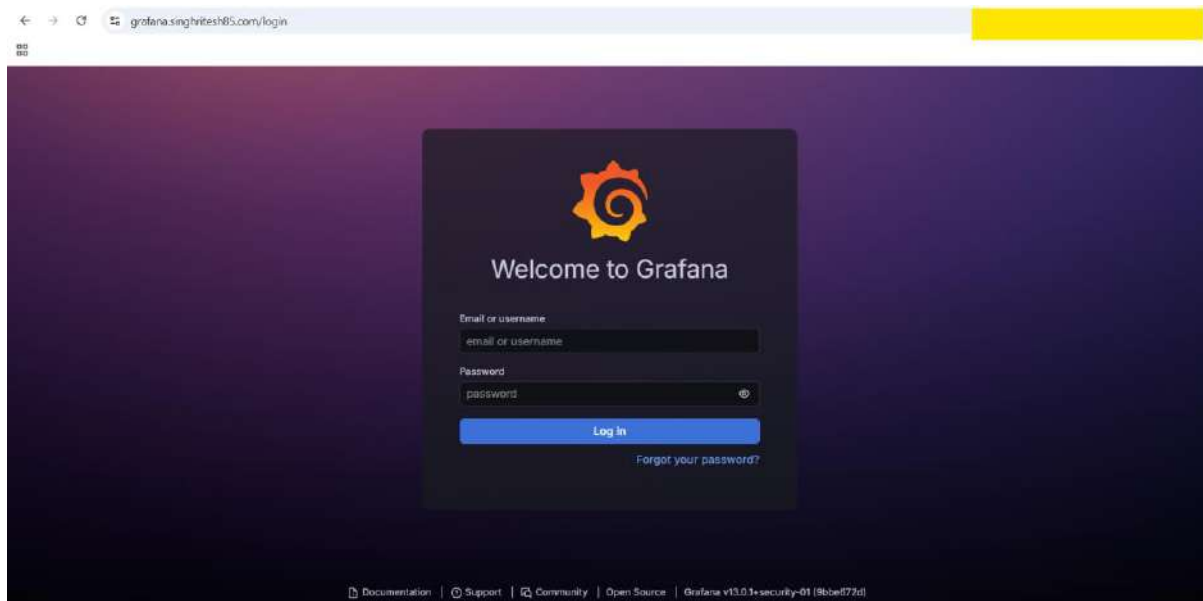
```
[demo@demo-azure-vm ~]$ kubectl create secret tls tls-grafana-ingress --cert=ld0ceb6b7fc166f1.crt --key=server.key -n monitoring --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/tls-grafana-ingress created
```

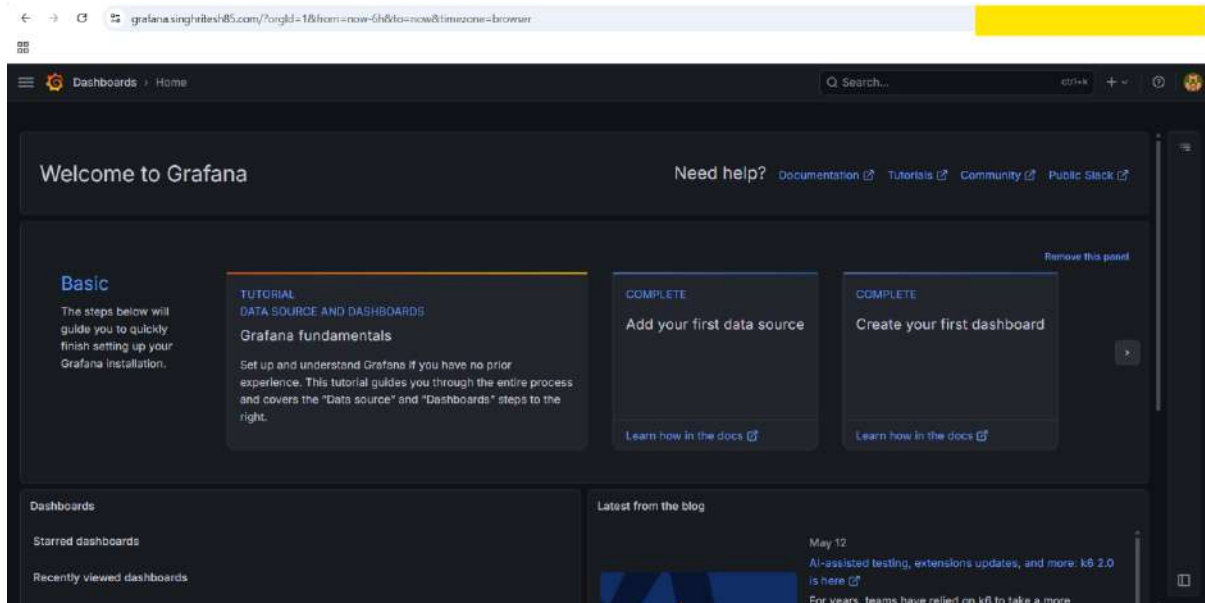
```
[demo@demo-azure-vm ~]$ kubectl apply -f gateway-api-grafana.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
gateway.gateway.networking.k8s.io/grafana-gateway created
httproute.gateway.networking.k8s.io/grafana-route created
[demo@demo-azure-vm ~]$ kubectl get svc -n monitoring --watch
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
alertmanager-operated               ClusterIP            None             <none>            9093/TCP,9094/TCP,9094/UDP             78m
grafana-gateway-nginx                LoadBalancer        172.17.0.10      <pending>         80:31629/TCP,443:32762/TCP             9s
kube-prometheus-stack-alertmanager  ClusterIP            172.17.0.11      <none>            9093/TCP,8080/TCP                     78m
kube-prometheus-stack-grafana       ClusterIP            172.17.0.12      <none>            80/TCP                                 78m
kube-prometheus-stack-kube-state-metrics ClusterIP            172.17.0.13      <none>            8080/TCP                               78m
kube-prometheus-stack-operator       ClusterIP            172.17.0.14      <none>            443/TCP                                78m
kube-prometheus-stack-prometheus    ClusterIP            172.17.0.15      <none>            9090/TCP,8080/TCP                     78m
kube-prometheus-stack-prometheus-node-exporter ClusterIP            172.17.0.16      <none>            9100/TCP                               78m
prometheus-operated                  ClusterIP            None             <none>            9090/TCP                                78m
grafana-gateway-nginx                LoadBalancer        172.17.0.10      34.123.456.78    80:31629/TCP,443:32762/TCP             34s
```

Finally, did the entry in Azure DNS Zone to create the Record-Set of A-Type as shown in the screenshot attached below.

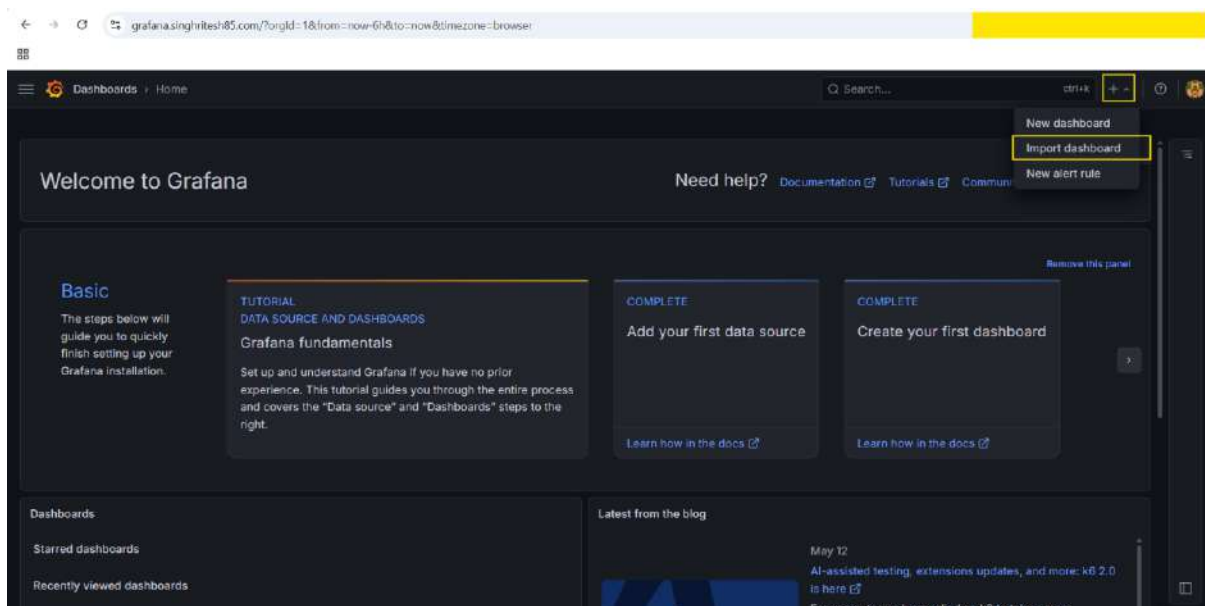


Below screenshot confirms, I was able access the Grafana User Interface and was able to login-into Grafana using the admin password obtained earlier.

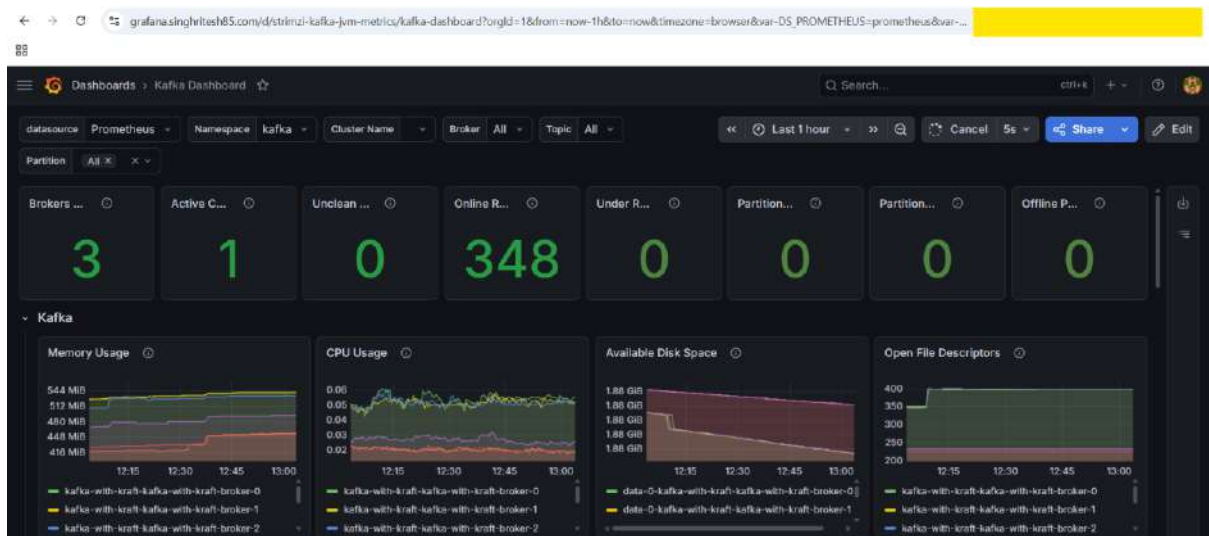
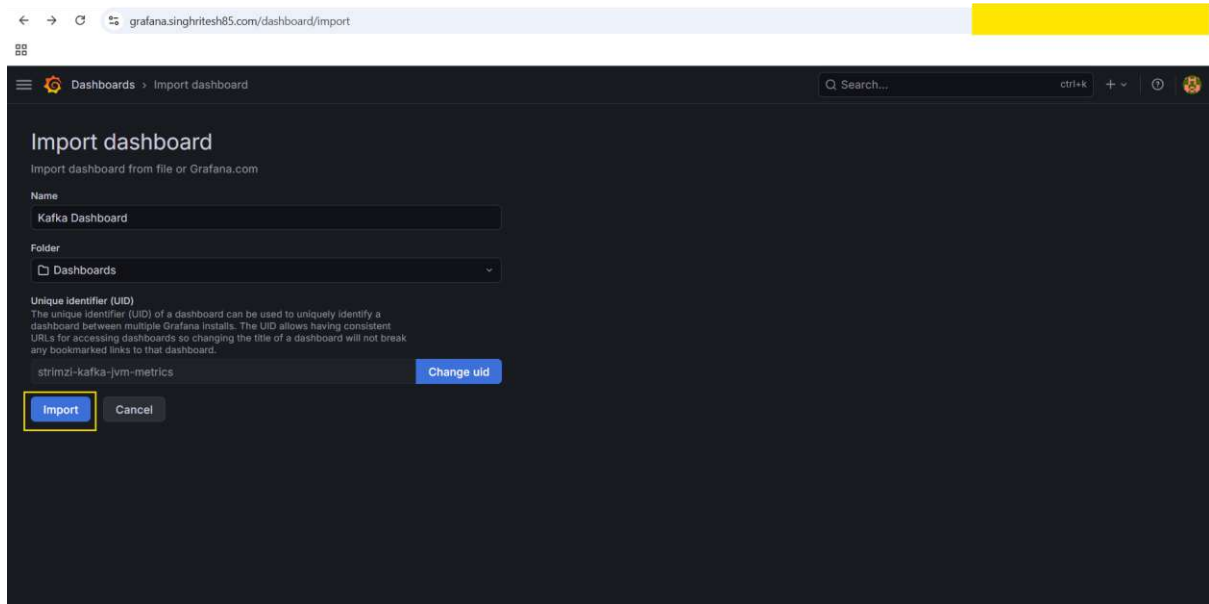
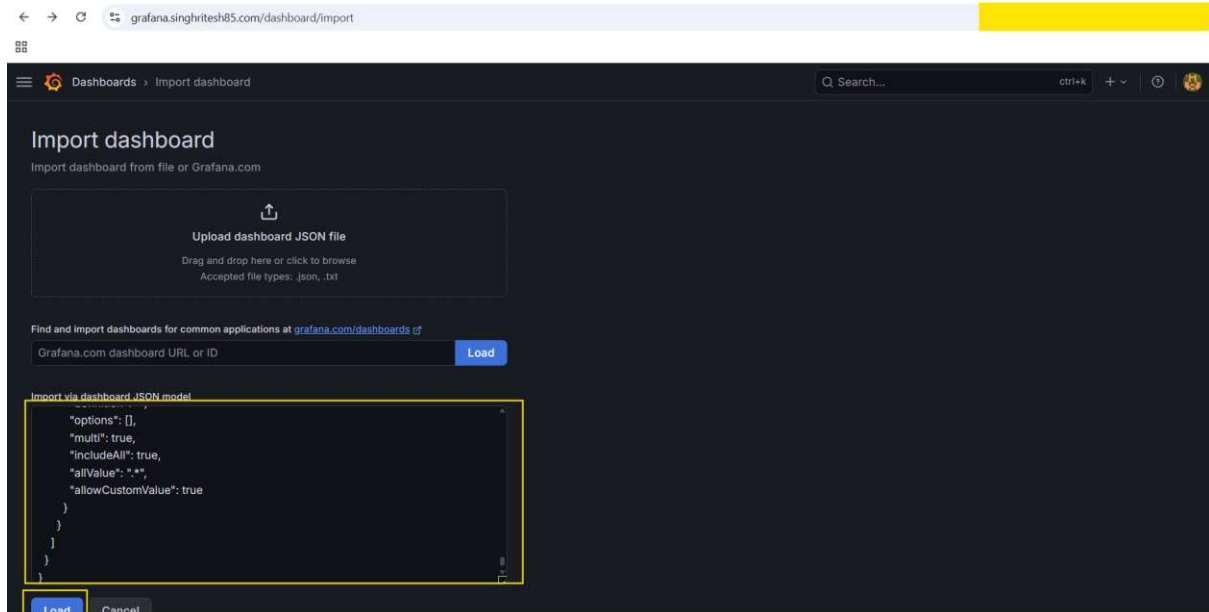




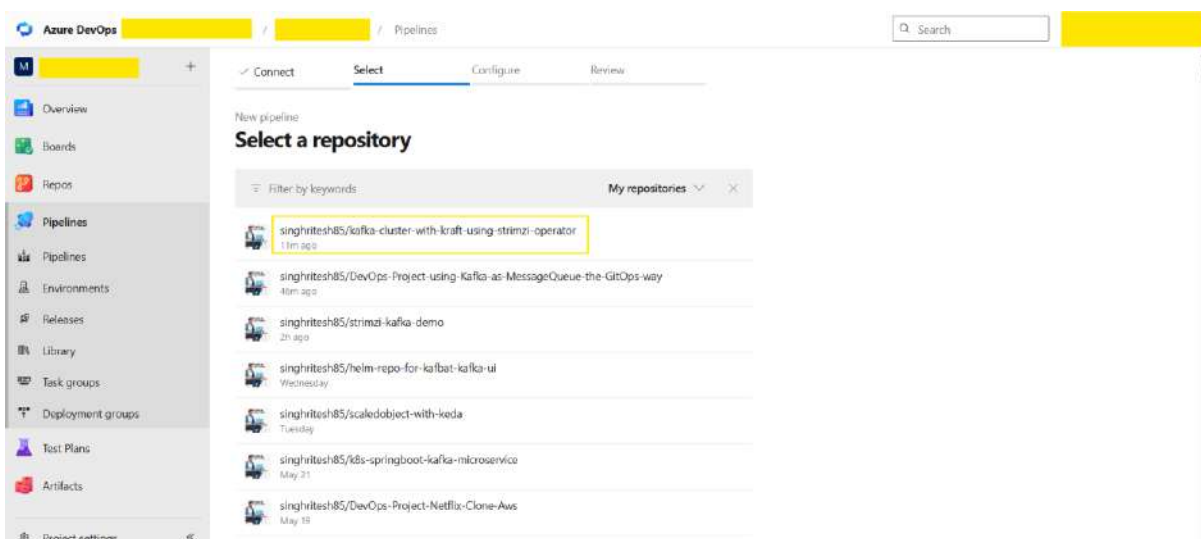
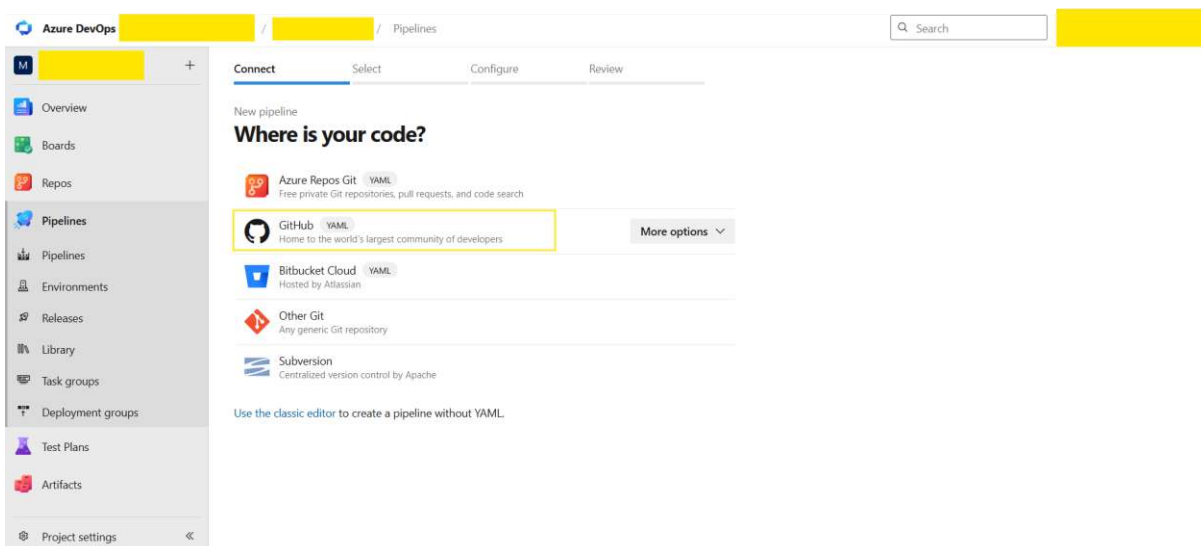
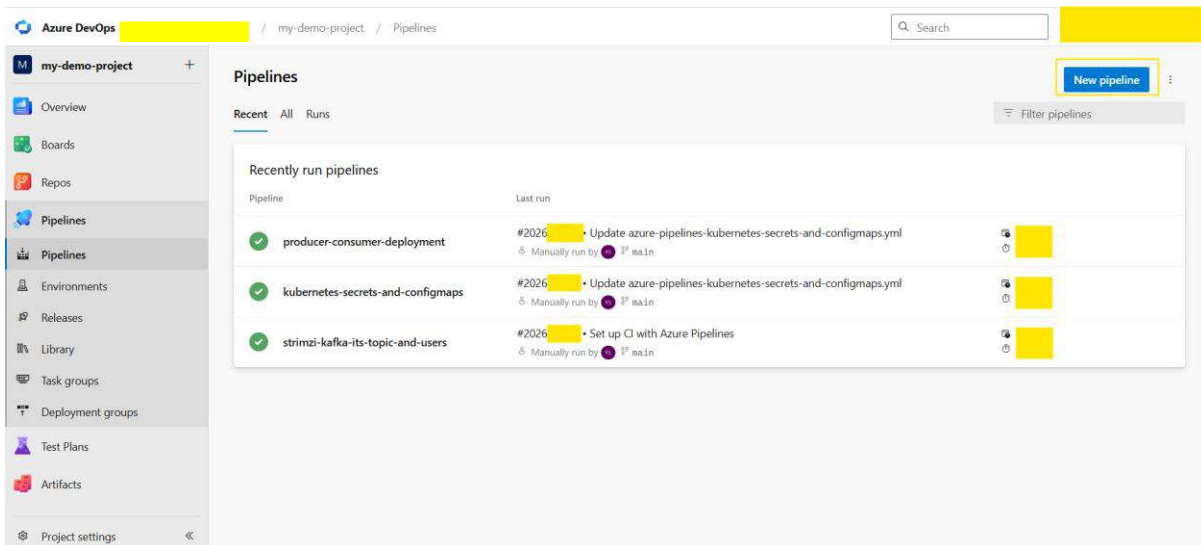
Finally, Created the Grafana Dashboard using the Grafana JSON model present in the GitHub Repo <https://github.com/singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator.git> at the path `./grafana-dashboard.json`.

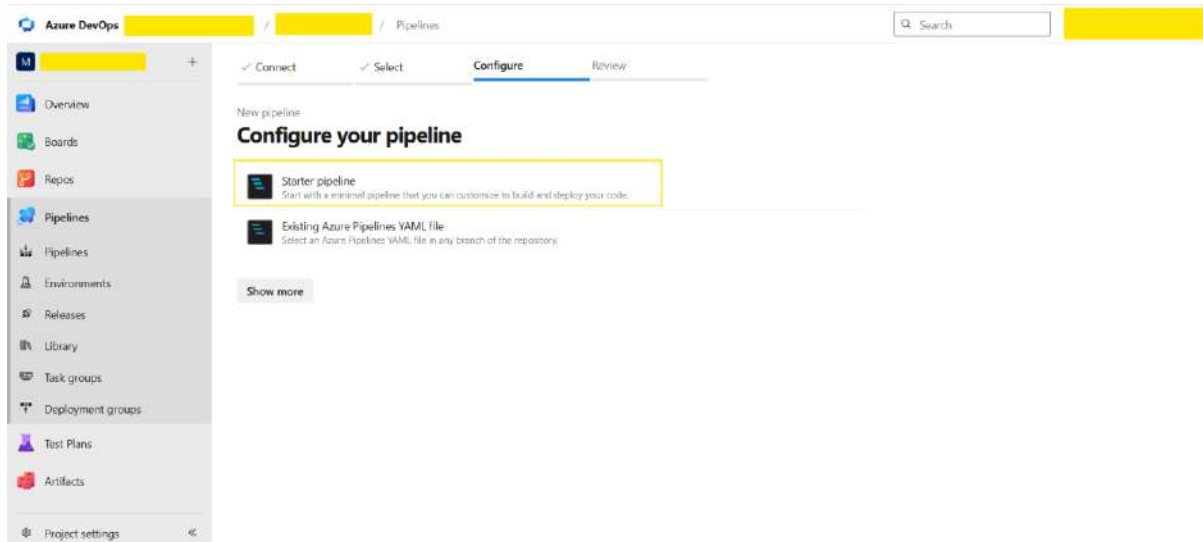


Copy and paste the JSON model then loaded the same as shown in the screenshot attached below.

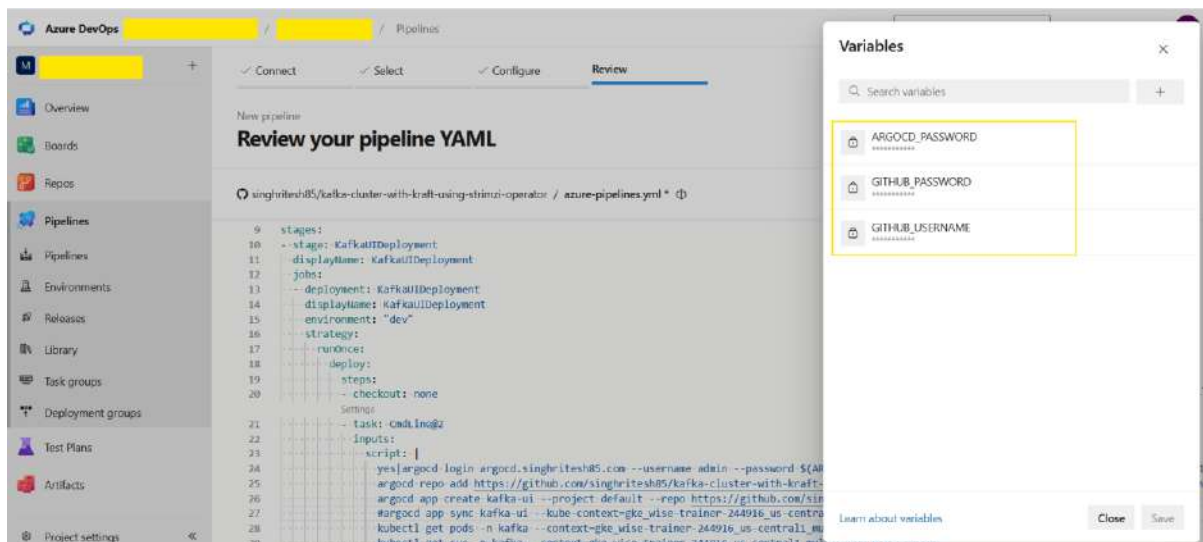


Installation of Kafka-UI using Kubernetes Manifests file present in the GitHub Repo <https://github.com/singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator.git> at the path **kafka-ui/ kafka-ui.yaml**. For deployment I used Azure DevOps Pipeline with the help of ArgoCD command line as shown in the steps below.

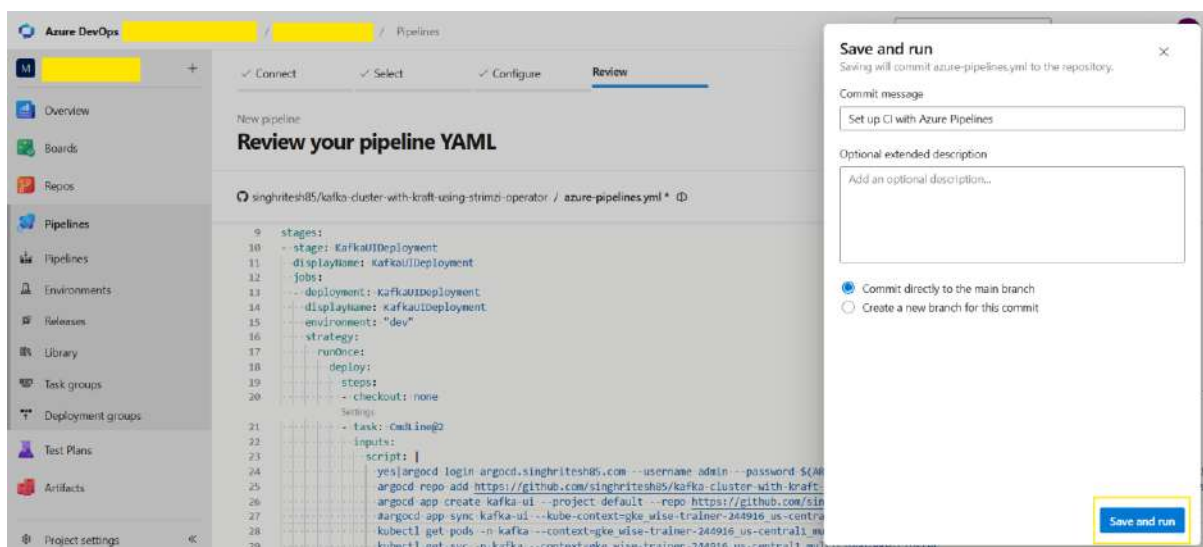




Before Saving and Running the Azure DevOps Pipeline, create three variables GITHUB_USERNAME, GITHUB_PASSWORD and ARGOCD_PASSWORD as explained earlier.



Finally, Save and Run as shown in the screenshot attached below.



Before saving and running the above shown Azure DevOps Pipeline create the Private Keys, Certificate and Kubernetes Secrets using the command as shown below.

```
kubect! get secret kafka-with-kraft-cluster-ca-cert -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.ca\.crt}' | base64 -d > /tmp/ca.crt

kubect! get kafkausers -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster

kubect! get secret kafka-ui -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.user\.p12}' | base64 -d > /tmp/user.p12

kubect! get secret kafka-ui -o jsonpath='{.data.user\.password}' -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d > /tmp/user.password

keytool -import -trustcacerts -alias strimzi-ca -file /tmp/ca.crt -keystore /tmp/truststore.jks -storepass XXXXXXXXXXXX -noprompt

keytool -importkeystore -srckeystore /tmp/user.p12 -srcstoretype PKCS12 -srcstorepass "$(cat /tmp/user.password)" -destkeystore /tmp/keystore.jks -deststoretype JKS -deststorepass XXXXXXXXXXXX -destkeypass XXXXXXXXXXXX

kubect! create secret generic kafka-ui-tls-secret --from-file=truststore.jks=/tmp/truststore.jks --from-file=keystore.jks=/tmp/keystore.jks -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

Home > Key vaults > multicloud | Secrets > multicloud-keystore-truststore-server-password

Secret Version

Properties

Created 2026, [redacted]

Updated 2026, [redacted]

Secret Identifier <https://multicloud.vault.azure.net/secrets/multicloud-keystore-trustst...>

Settings

Set activation date ☐

Set expiration date ☐

Enabled ☒ Yes ☐ No

Tags 0 tags

Secret

Content type (optional)

Hide Secret Value

Secret value 1H[redacted]t8

```
[demo@demo-azure-vm ~]$ kubectl get secret kafka-with-kraft-cluster-ca-cert -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.ca.crt}' | base64 -d > /tmp/ca.crt
[demo@demo-azure-vm ~]$ kubectl get kafkausers -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME          CLUSTER    AUTHENTICATION  AUTHORIZATION  READY
kafka-consumer  kafka-with-kraft  tls             simple         True
kafka-producer  kafka-with-kraft  tls             simple         True
kafka-ui        kafka-with-kraft  tls             simple         True
[demo@demo-azure-vm ~]$ kubectl get secret kafka-ui -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster -o jsonpath='{.data.user\.p12}' | base64 -d > /tmp/user.p12
[demo@demo-azure-vm ~]$ kubectl get secret kafka-ui -o jsonpath='{.data.user\.password}' -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster | base64 -d > /tmp/user.password
[demo@demo-azure-vm ~]$ keytool -import -trustcacerts -alias strimzi-ca -file /tmp/ca.crt -keystore /tmp/truststore.jks -storepass [REDACTED] -noprompt
Certificate was added to keystore
[demo@demo-azure-vm ~]$ keytool -importkeystore -srckeystore /tmp/user.p12 -srcstoretype PKCS12 -srcstorepass "${cat /tmp/user.password}" -destkeystore /tmp/keystore.jks -deststoretype JKS -deststorepass [REDACTED] -destkeypass [REDACTED]
Importing keystore /tmp/user.p12 to /tmp/keystore.jks...
Entry for alias kafka-ui successfully imported.
Import command completed: 1 entries successfully imported, 0 entries failed or cancelled

Warning:
The JKS keystore uses a proprietary format. It is recommended to migrate to PKCS12 which is an industry standard format using "keytool -importkeystore -srckeystore /tmp/keystore.jks -destkeystore /tmp/keystore.p12 -deststoretype pkcs12".
[demo@demo-azure-vm ~]$ kubectl create secret generic kafka-ui-tls-secret --from-file=truststore.jks=/tmp/truststore.jks --from-file=keystore.jks=/tmp/keystore.jks -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
secret/kafka-ui-tls-secret created
```

Finally, Save and Run the Azure DevOps Pipeline and after successful execution of Azure DevOps Pipeline the Screenshot of Azure DevOps Pipeline, ArgoCD UI for Application, Kafka-UI Pods and Services are as shown below.

The screenshot displays the ArgoCD web interface. The top navigation bar includes 'Pipelines' and 'New pipeline'. Below it, the 'Recent' tab shows a list of pipelines. The 'Applications' section is visible, showing a grid of application cards. Each card provides details about the application's health, sync status, and configuration. The 'kafka-ui' application is highlighted with a yellow box, indicating it is the focus of the current view.

```
[demo@demo-azure-vm ~]$ kubectl get pods -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                READY   STATUS    RESTARTS   AGE
kafka-ui-                           1/1     Running   0           1m
kafka-with-kraft-cruise-control-   1/1     Running   0           1m
kafka-with-kraft-entity-operator-   2/2     Running   0           1m
kafka-with-kraft-kafka-with-kraft-broker-0  1/1     Running   0           1m
kafka-with-kraft-kafka-with-kraft-broker-1  1/1     Running   0           1m
kafka-with-kraft-kafka-with-kraft-broker-2  1/1     Running   0           1m
kafka-with-kraft-kafka-with-kraft-controller-3  1/1     Running   0           1m
kafka-with-kraft-kafka-with-kraft-controller-4  1/1     Running   0           1m
kafka-with-kraft-kafka-with-kraft-controller-5  1/1     Running   0           1m
strimzi-cluster-operator-           1/1     Running   0           1m
[demo@demo-azure-vm ~]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
kafka-metrics                       ClusterIP            172.19.103.173   <none>            9404/TCP                              1m
kafka-ui-service                     ClusterIP            172.19.155.209   <none>            80/TCP                                1m
kafka-with-kraft-cruise-control     ClusterIP            172.19.11.142    <none>            9090/TCP                              1m
kafka-with-kraft-kafka-bootstrap    ClusterIP            172.19.139.31    <none>            9091/TCP,9092/TCP,9093/TCP,9094/TCP  1m
kafka-with-kraft-kafka-brokers      ClusterIP            None              <none>            9090/TCP,9091/TCP,8443/TCP,9092/TCP,9093/TCP,9094/TCP  1m
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f gateway-api-kafka-ui.yaml --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
gateway.gateway.networking.k8s.io/kafka-ui-gateway created
httproute.gateway.networking.k8s.io/kafka-ui-route created
[demo@demo-azure-vm ~]$ kubectl get svc -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster --watch
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
kafka-metrics                       ClusterIP            172.19.103.173   <none>            9404/TCP                              1m
kafka-ui-gateway-nginx              LoadBalancer        172.19.155.209   <pending>         80:31597/TCP,443:32131/TCP           1m
kafka-ui-service                     ClusterIP            172.19.11.142    <none>            80/TCP                                1m
kafka-with-kraft-cruise-control     ClusterIP            172.19.139.31    <none>            9090/TCP                              1m
kafka-with-kraft-kafka-bootstrap    ClusterIP            172.19.139.31    <none>            9091/TCP,9092/TCP,9093/TCP,9094/TCP  1m
kafka-with-kraft-kafka-brokers      ClusterIP            None              <none>            9090/TCP,9091/TCP,8443/TCP,9092/TCP,9093/TCP,9094/TCP  1m
kafka-ui-gateway-nginx              LoadBalancer        172.19.155.209   80:31597/TCP,443:32131/TCP  1m
```

singhritesh85.com | Recordsets

Search

+ Add Refresh Delete Give feedback

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Settings

Locks

Properties

DNS Management

Recordsets

DNSSEC

Monitoring

Automation

Help

A record set is a collection of records in a zone that have been loaded on this page. If you don't see what you load. [Learn more](#)

Search

Fetches 4 record set(s).

Name	Type
@	NS
@	SOA
argocd	A

Add record set

singhritesh85.com

Name

kafka-ui

singhritesh85.com

Type

A - IPv4 Address records

Alias record set

No

TTL *

1

TTL unit

Hours

IP address

35

0.0.0.0

Add

Cancel

Give feedback

← → ↻

kafka-ui.singhritesh85.com

UI for Apache Kafka
83b5a80 v0.22

Dashboard

kafka-with-kraft

Brokers

Topics

Consumers

ACL

Dashboard

Online 1 clusters

Offline 0 clusters

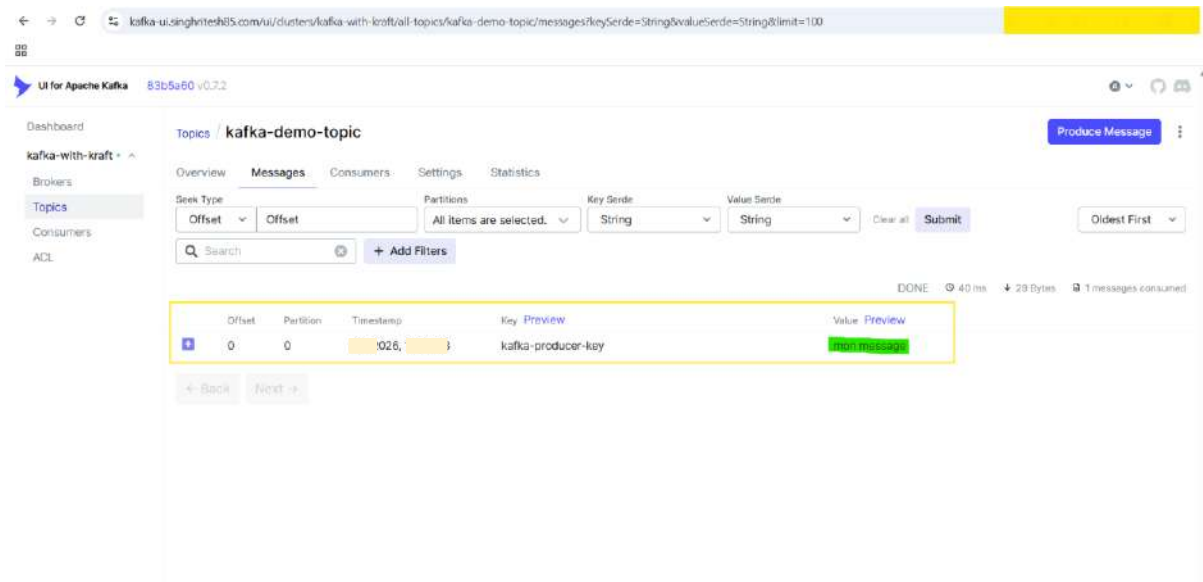
Only offline clusters

Cluster name	Version	Brokers count	Partitions	Topics	Production	Consumption
kafka-with-kraft	1.0-UNKNOWN	3	116	5	0 Bytes	0 Bytes

Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>

The gateway-api-kafka-ui.yaml kubernetes manifests file to create the Kubernetes Gateway API to access Kafka-UI using the URL is present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-2/gateway-api-kafka-ui.yaml**.

You can see the same message reflected in the kafka-ui topic as shown in the screenshot attached below.



I had used truststore and keystore password to generate the certificate used while creating the Kafka-UI. This Password was randomly generated during creation of Infrastructure using Terraform as shown in the screenshot attached below.

```
resource "random_password" "keystore_truststore_password" {
  length          = 16
  special          = true
  override_special = "_!%^@"
  min_upper       = 2
  min_lower       = 2
  min_numeric     = 2
}
```

The Kafka-UI deployment is configured to retrieve SSL-related secrets from Azure Key Vault through the Secrets Store CSI Driver and consume passwords via Kubernetes Secret references (keystore-truststore-credentials), while TLS certificates and keystore/truststore files are mounted into the pod through secure secret volumes. This ensures that sensitive SSL credentials and cryptographic artifacts are not hardcoded in the deployment configuration and are securely provided to the application at runtime.

```

volumes:
- name: tls-secrets
  secret:
    secretName: kafka-ui-tls-secret
- name: tls-kafka-ui-secrets
  csi:
    driver: secrets-store.csi.k8s.io
    readOnly: true
    volumeAttributes:
      secretProviderClass: azure-keyvault
      nodePublishSecretRef:
        name: secrets-store-creds

- name: KAFKA_CLUSTERS_0_PROPERTIES_SSL_TRUSTSTORE_PASSWORD
  valueFrom:
    secretKeyRef:
      name: keystore-truststore-credentials
      key: password

volumeMounts:
- name: tls-secrets
  mountPath: /opt/kafka-ui/secrets
  readOnly: true
- name: tls-kafka-ui-secrets
  mountPath: /opt/kafka-ui/truststore-keystore
  readOnly: true

- name: KAFKA_CLUSTERS_0_PROPERTIES_SSL_KEYSTORE_PASSWORD
  valueFrom:
    secretKeyRef:
      name: keystore-truststore-credentials
      key: password
- name: KAFKA_CLUSTERS_0_PROPERTIES_SSL_KEY_PASSWORD
  valueFrom:
    secretKeyRef:
      name: keystore-truststore-credentials
      key: password

```

I would like to mention here that for deployment of Producer-Consumer Deployment, Kustomization had been used along with ArgoCD with Kustomization I am updating the TAG NAME of Docker Image during Runtime with Build Number.

```

spec:
  containers:
  - name: kafka-consumer
    image: kafka-consumer
    ports:
    - containerPort: 8080
      name: http

spec:
  containers:
  - name: kafka-producer
    image: kafka-producer
    ports:
    - containerPort: 8080
      name: http

images:
- name: kafka-producer
  newTag: ""
- name: kafka-consumer
  newTag: ""

```

```
--kustomize-image kafka-producer=singhritesh85/kafka-producer:${Build.BuildId} --kustomize-image kafka-consumer=singhritesh85/kafka-consumer:${Build.BuildId}
```



Installation and configuration of KEDA (Kubernetes Event Driven Autoscaling)



KEDA stands for Kubernetes Event Driven Autoscaling. KEDA will scale the workload automatically based on the volumes of events. To install KEDA using helm you can use the command as shown below.

```
helm repo add kedacore https://kedacore.github.io/charts
```

```
helm repo update
```

```
helm install keda kedacore/keda --namespace keda --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ helm repo add kedacore https://kedacore.github.io/charts
"kedacore" has been added to your repositories
[demo@demo-azure-vm ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "secrets-store-csi-driver" chart repository
...Successfully got an update from the "kedacore" chart repository
...Successfully got an update from the "istio" chart repository
Update Complete. 🎉Happy Helming!🎉
```

```
[demo@demo-azure-vm ~]$ helm install keda kedacore/keda --namespace keda --create-namespace --kube-context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

```
[demo@demo-azure-vm ~]$ kubectl get pods -n keda --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	READY	STATUS	RESTARTS	AGE
keda-admission-webhooks-85957885bf-q48md	1/1	Running	0	131m
keda-operator-6bb79ff79-852xb	1/1	Running	1 (131m ago)	131m
keda-operator-metrics-apiserver-656749fd7c-vvgz7	1/1	Running	0	131m

The above screenshot shows KEDA had been installed successfully in GKE Standard Cluster. In the current project KEDA's job is to watch Kafka lag and automatically increase or decrease the number of kafka-consumer pods between 1 and 5 based on how many messages are waiting to be processed. To do so I configured KEDA Trigger Authentication and Scaled Object using Azure DevOps Pipeline and ArgoCD as shown in the screenshot attached below.

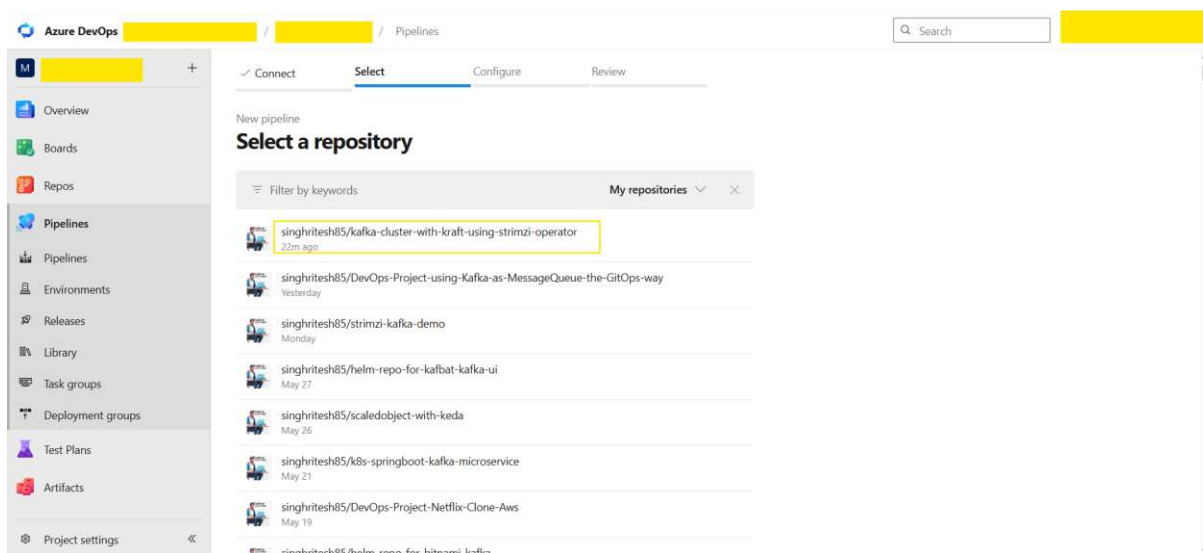
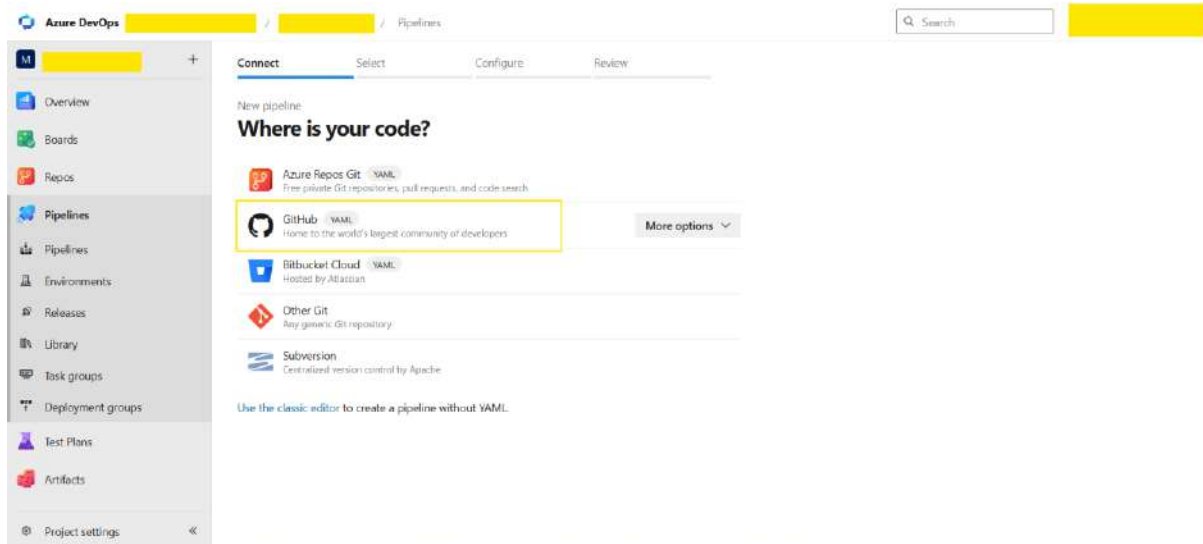
```
[demo@demo-azure-vm ~]$ kubectl get kafkausers -n kafka --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

NAME	CLUSTER	AUTHENTICATION	AUTHORIZATION	READY
kafka-consumer	kafka-with-kraft	tls	simple	True
kafka-producer	kafka-with-kraft	tls	simple	True
kafka-ui	kafka-with-kraft	tls	simple	True
keda-user	kafka-with-kraft	tls	simple	True

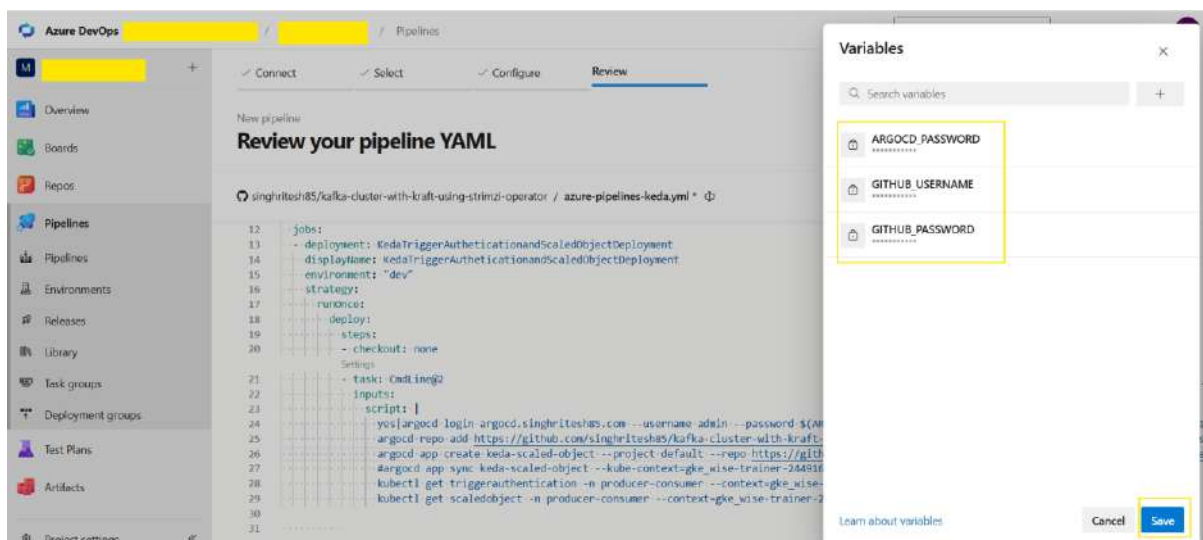
A new user keda-user had been created in the namespace kafka which will be used to watch the kafka lag.

The screenshot shows the 'Pipelines' section in Azure DevOps. Under the 'Recently run pipelines' tab, there is a list of pipelines. The pipeline named 'keda-user' is highlighted with a red arrow. The list includes the following pipelines:

Pipeline	Last run
stirzi-kafka-its-topic-and-users	#2026-11-11 10:00:00 • Update kafka-producer-consumer-kafka-ui-user.yaml Manually run by sa1n
kafka-ui	#2026-11-11 10:00:00 • Update kafka-ui.yaml Manually run by sa1n
producer-consumer-deployment	#2026-11-11 10:00:00 • Update azure-pipelines-kubernetes-secrets-and-configmaps.yaml Manually run by sa1n
kubernetes-secrets-and-configmaps	#2026-11-11 10:00:00 • Update azure-pipelines-kubernetes-secrets-and-configmaps.yaml Manually run by sa1n



Before Saving and Running it create three variables named as GITHUB_USERNAME, GITHUB_PASSWORD and ARGOCD_PASSWORD in Azure DevOps Pipeline as shown in the screenshot attached below.



After Saving and Succesfully Running it, the screenshot of Azure DevOps Pipeline and ArgoCD Application is as shown in the screenshot attchaed below.

Pipelines

Recent All Runs Filter pipelines

Recently run pipelines

Pipeline	Last run
ked-trigger-authentication-and-scaled...	#2026-11-11 10:10:10 - Set up CI with Azure Pipelines Individual CI for main
strimzi-kafka-its-topic-and-users	#2026-11-11 10:10:10 - Update kafka-producer-consumer-kafka-ui-user.yaml Manually run by main
kafka-ui	#2026-11-11 10:10:10 - Update kafka-ui.yaml Manually run by main
producer-consumer-deployment	#2026-11-11 10:10:10 - Update azure-pipelines-kubemetes-secrets-and-configmaps.yaml Manually run by main
kubernetes-secrets-and-configmaps	#2026-11-11 10:10:10 - Update azure-pipelines-kubemetes-secrets-and-configmaps.yaml Manually run by main

```
[demo@demo-azure-vm ~]$ kubectl get triggerauthentication -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                POIIDENTITY SECRET ENV VAULTADDRESS
kafka-trigger-auth                  kafka-with-kraft-cluster-ca-cert
[demo@demo-azure-vm ~]$ kubectl get scaledobject -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
NAME                                SCALETARGETKIND SCALETARGETNAME MIN MAX READY ACTIVE FALLBACK PAUSED TRIGGERS AUTHENTIFICATIONS AGE
kafka-consumer-scaler              apps/v1.Deployment kafka-consumer 1 5 True False False False kafka kafka-trigger-auth 2m38s
[demo@demo-azure-vm ~]$ kubectl describe scaledobjects kafka-consumer-scaler -n producer-consumer --context=gke_wise-trainer-244916_us-central1_multicloud-gke-cluster
```

Events:

Type	Reason	Age	From	Message
Normal	KEDAScalersStarted	67s	keda-operator	Scaler kafka is built
Normal	KEDAScalersStarted	67s	keda-operator	Started scalers watch
Normal	ScaledObjectReady	67s	keda-operator	ScaledObject is ready for scaling

argocd.singhritesh85.com/applications?proj=&sync=&autoSync=&health=&namespace=&cluster=&labels=&annotations=&operation=

ArgoCD Applications

Applications: + NEW APP + SYNC APPS + REFRESH APPS Search applications...

APPLICATIONS TILES

Log out

Application filters

SYNC STATUS

- Unknown 0
- Synced 6
- OutOfSync 0

HEALTH STATUS

- Progressing 0
- Suspended 0
- Healthy 6
- Degraded 0

Applications

kafka-user

Project: default
Labels: Healthy Synced
Repository: https://github.com/singhritesh85/kafka-...
Target R: HEAD
Path: /kafka-producer-consumer-kafka-ui-user
Destination: in-cluster
Namespace: kafka
Created: 2026-11-11 10:10:10
Last Sync: 2026-11-11 10:10:10

kafka-with-kraft

Project: default
Labels: Healthy Synced
Repository: https://github.com/singhritesh85/kafka-...
Target R: HEAD
Path: /kafka-kraft
Destination: in-cluster
Namespace: kafka
Created: 2026-11-11 10:10:10
Last Sync: 2026-11-11 10:10:10

keda-scaled-object

Project: default
Labels: Healthy Synced
Repository: https://github.com/singhritesh85/kafka-...
Target R: HEAD
Path: /trigger-authentication-scaled-object
Destination: in-cluster
Namespace: producer-consumer
Created: 2026-11-11 10:10:10
Last Sync: 2026-11-11 10:10:10

GitHub Repo: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>

Terraform Script: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>

Source Code: - <https://github.com/singhritesh85/strimzi-kafka-demo.git>

Kubernetes Manifests File: - <https://github.com/singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator.git>

azure-pipelines.yml files present in the GitHub Repo: -

<https://github.com/singhritesh85/strimzi-kafka-demo.git>

<https://github.com/singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator.git>

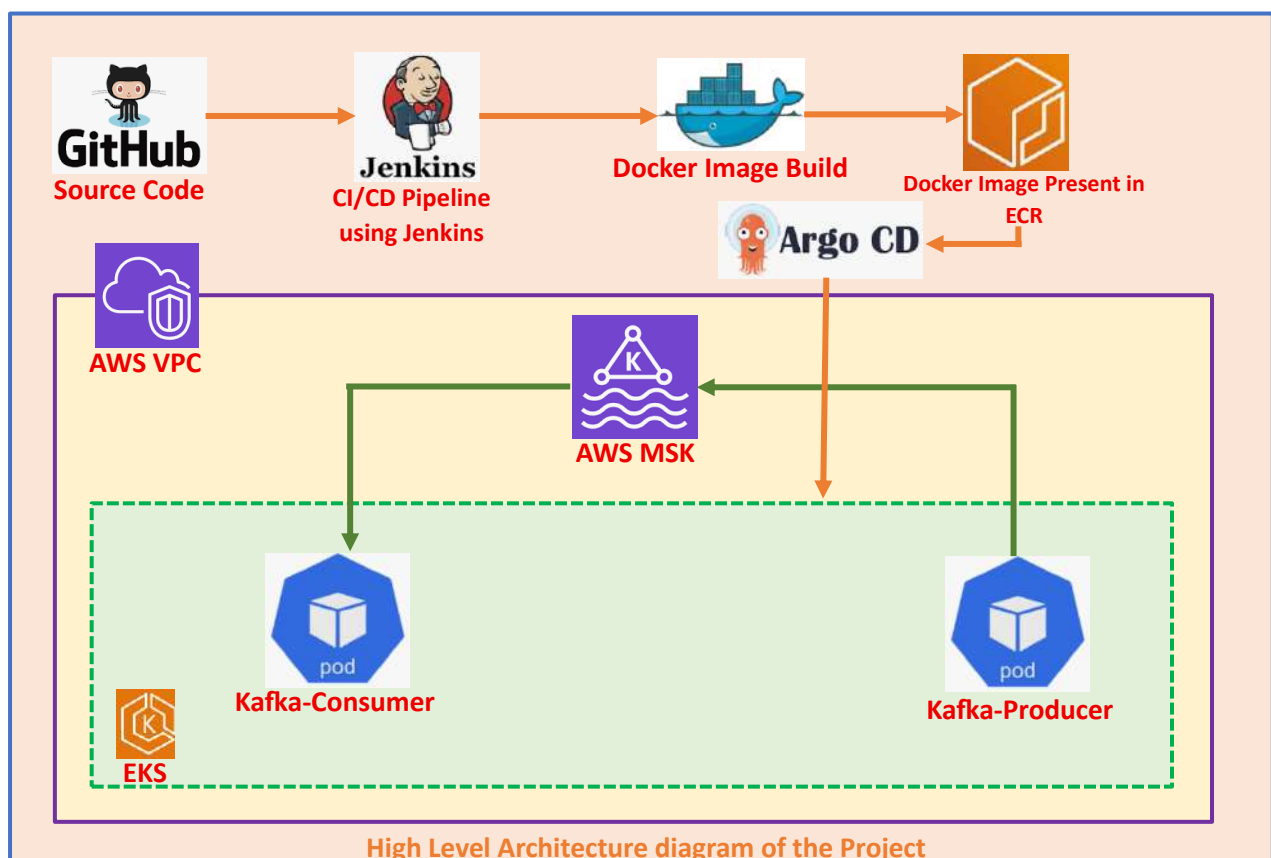
References:-

https://anissetiouajni.com/posts/accessing_a_secure_apache_kafka_managed_by_strimzi_from_springboot/

<https://github.com/atouajni/strimzi-kafka-demo.git>

Module-3

Section-1: - DevOps Project using AWS MSK as Message Queue with GitOps



This DevOps Project deals with creation of infrastructure using Terraform Script present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-3/Section-1/terraform-eks-msk** and Dockerfile along with Kubernetes Manifests files are present in the GitHub Repo <https://github.com/singhritesh85/aws-msk-producer-consumer.git>. Jenkins was used as the CI/CD pipeline tool to retrieve Dockerfile and Kubernetes manifest files from GitHub, build Docker images, and push them to a private Elastic Container Registry (ECR). ArgoCD was then used to deploy the Kafka producer and consumer applications to Amazon EKS. Additionally, ArgoCD managed the deployment of KEDA, enabling a GitOps-based deployment approach across the Kubernetes environment.

Commands as written below had been used to create infrastructure using Terraform.

terraform init -----> initializes a working directory containing configuration files and installs plugins for required providers.

terraform validate -----> verify that terraform configuration file is correct or not

terraform plan -----> Check which resources are going to be created.

Then you can run the command **terraform apply -auto-approve** -----> Finally, Created the resources.

```
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [25m20s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [25m30s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [25m40s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [25m50s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [26m00s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [26m10s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [26m20s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [26m30s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [26m40s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [26m50s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [27m00s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [27m10s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [27m20s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [27m30s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [27m40s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [27m50s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [28m00s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [28m10s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [28m20s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Still creating... [28m30s elapsed]
module.aws_eks_msk.aws_msk_cluster.msk_cluster: Creation complete after 28m40s [id=arn:aws:kafka:us-east-2:02[REDACTED]:cluster/aws-msk-cluster/[REDACTED]]
module.aws_eks_msk.aws_iam_policy.msk_access: Creating...
module.aws_eks_msk.aws_appautoscaling_target.msk_storage: Creating...
module.aws_eks_msk.aws_iam_policy.msk_access: Creation complete after 0s [id=arn:aws:iam:02[REDACTED]:policy/msk-access]
module.aws_eks_msk.aws_iam_role_policy_attachment.msk_access_role_attachment: Creating...
module.aws_eks_msk.aws_iam_role_policy_attachment.msk_access_role_attachment: Creation complete after 0s [id=aws-irsa-role/arn:aws:iam:02[REDACTED]:policy/msk-access]
module.aws_eks_msk.aws_appautoscaling_target.msk_storage: Creation complete after 0s [id=arn:aws:kafka:us-east-2:02[REDACTED]:cluster/aws-msk-cluster/[REDACTED]]
module.aws_eks_msk.aws_appautoscaling_policy.msk_storage: Creating...
module.aws_eks_msk.aws_appautoscaling_policy.msk_storage: Creation complete after 1s [id=aws-msk-cluster-storage]

Apply complete! Resources: 86 added, 0 changed, 0 destroyed.

Outputs:
aws_ec2_eks_msk_iam_role_arn_details = <sensitive>
```

The output after running the Terraform Script is as shown in the screenshot attached below.

```
[root@terraform-server main]# terraform output -json
```

```

"cluster_name": "eks-demo-cluster-dev",
"cluster_oidc_issuer_url": "https://oidc.eks.us-east-2.amazonaws.com/id/3[REDACTED]",
"cluster_security_group_id": "sg-0[REDACTED]",
"instance_id": "i-047fc23ef5bab7945",
"msk_bootstrap_brokers_sasl_iam": "[REDACTED]:9098",
com:9098,
"msk_cluster_arn": "arn:aws:kafka:us-east-2:02[REDACTED]:cluster/aws-msk-cluster/[REDACTED]",
"msk_current_version": "[REDACTED]",
"msk_iam_role_arn": "arn:aws:iam::02[REDACTED]:role/aws-irsa-role",
"private_ip": "192.168.[REDACTED]",
"private_subnet_cidrs": [
  "192.168.1.0/24",
  "192.168.2.0/24",
  "192.168.3.0/24"
],
"private_subnet_ids": [
  "subnet-0[REDACTED]",
  "subnet-0[REDACTED]",
  "subnet-0[REDACTED]"
],
"public_ip": "3.[REDACTED]",
"public_subnet_cidrs": [
  "192.168.4.0/24",
  "192.168.5.0/24",
  "192.168.6.0/24"
],
"public_subnet_ids": [
  "subnet-0[REDACTED]",
  "subnet-0[REDACTED]",
  "subnet-0[REDACTED]"
],
"vpc_arn": "arn:aws:ec2:us-east-2:02[REDACTED]:vpc/vpc-0[REDACTED]",
"vpc_cidr_block": "192.168.0.0/16",
"vpc_default_security_group_id": "sg-0[REDACTED]",
"vpc_id": "vpc-0[REDACTED]"
}
}

```

Terraform script to launch EC2 Instance and install Jenkins (with self-healing architecture), Create AWS Route53 hosted-zone, AWS ACM Certificate and Validate the AWS ACM Certificate is present at the path **module-3/Section-1/terraform-jenkins-self-healing-route53-hosted-zone-with-acm-certificate** in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>.

Commands as written below had been used to create infrastructure using Terraform.

terraform init -----> initializes a working directory containing configuration files and installs plugins for required providers.

terraform validate -----> verify that terraform configuration file is correct or not

terraform plan -----> Check which resources are going to be created.

Then you can run the command **terraform apply -auto-approve** -----> Finally, Created the resources.

At this point after executing the command **terraform apply -auto-approve** make sure you do the entry for Nameservers of the Route53 created Hosted Zone in your domain name service provider's Nameserver. After doing such an entry the terraform script will give the output as written below.

```

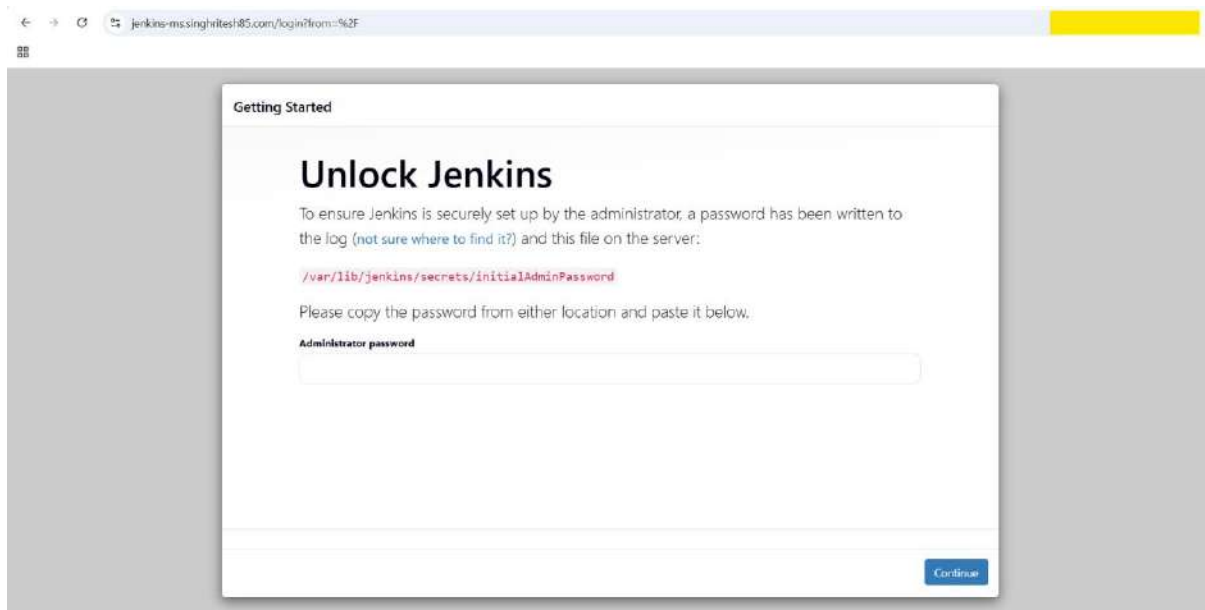
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [02m10s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [02m20s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [02m30s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [02m40s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [02m50s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [03m00s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [03m10s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Still creating... [03m20s elapsed]
module.jenkins_self_healing.aws_lb.test-application-loadbalancer: Creation complete after 3m21s [id=arn:aws:elasticloadbalancing:us-east-2:02[REDACTED]:loadba
lancer/app/jenkins-alb-[REDACTED]]
module.jenkins_self_healing.aws_lb_listener.alb_listener_front_end_HTTPS: Creating...
module.jenkins_self_healing.aws_route53_record.jenkins_record_set: Creating...
module.jenkins_self_healing.aws_lb_listener.alb_listener_front_end_HTTP: Creation complete after 0s [id=arn:aws:elasticloadbalancing:us-east-2:02[REDACTED]:li
stener/app/jenkins-alb-[REDACTED]]
module.jenkins_self_healing.aws_lb_listener.alb_listener_front_end_HTTPS: Creation complete after 1s [id=arn:aws:elasticloadbalancing:us-east-2:02[REDACTED]:li
stener/app/jenkins-alb-[REDACTED]]
module.jenkins_self_healing.aws_route53_record.jenkins_record_set: Still creating... [00m10s elapsed]
module.jenkins_self_healing.aws_route53_record.jenkins_record_set: Still creating... [00m20s elapsed]
module.jenkins_self_healing.aws_route53_record.jenkins_record_set: Still creating... [00m30s elapsed]
module.jenkins_self_healing.aws_route53_record.jenkins_record_set: Creation complete after 31s [id=[REDACTED].jenkins-ms.singhritesh85.com_A]

Apply complete! Resources: 19 added, 0 changed, 0 destroyed.

Outputs:
efs_private_ip_route53_hosted_zone_details_dns_name_of_jenkins_alb = {
  "certificate_arn" = "arn:aws:acm:us-east-2:02[REDACTED]:certificate/[REDACTED]"
  "efs_mount_target_ips" = ["192.168.[REDACTED]"]
  "hosted_zone_id" = "Z[REDACTED]"
  "hosted_zone_name_servers" = tolist([
    "ns-[REDACTED]",
    "ns-[REDACTED]",
    "ns-[REDACTED]",
    "ns-[REDACTED]"
  ])
}
"jenkins_alb_dns_name" = "jenkins-alb-[REDACTED].us-east-2.elb.amazonaws.com"

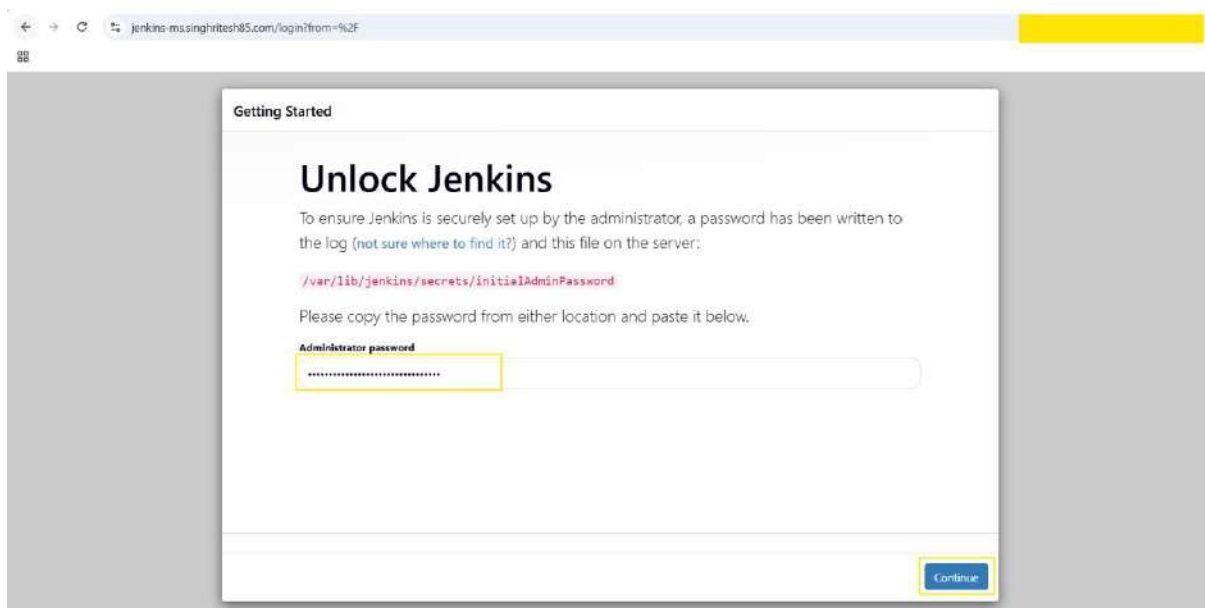
```

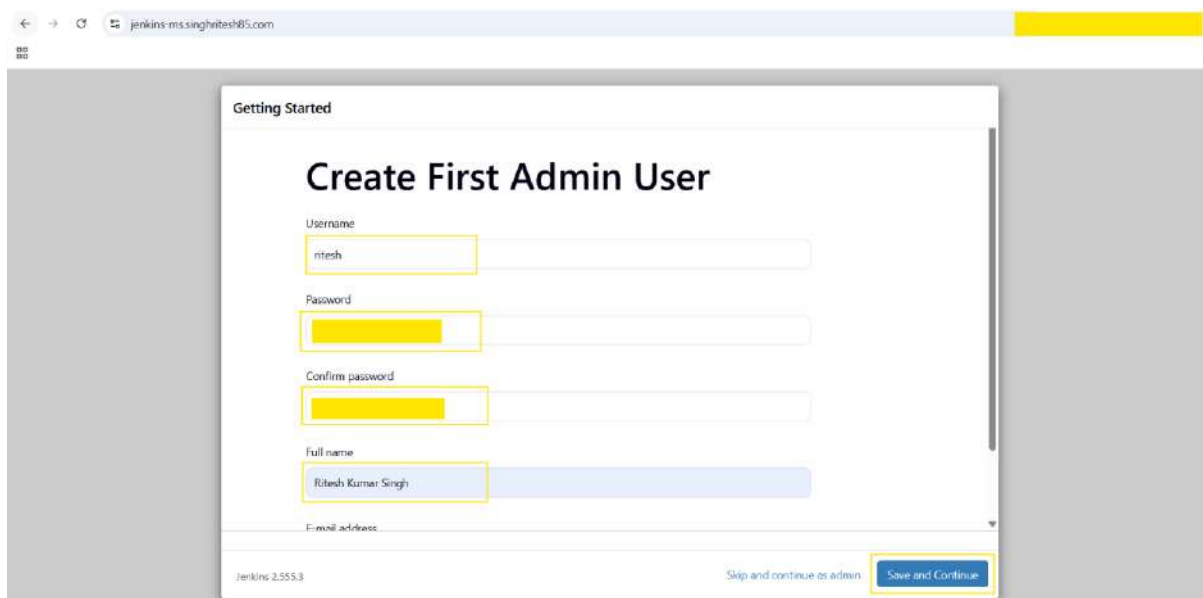
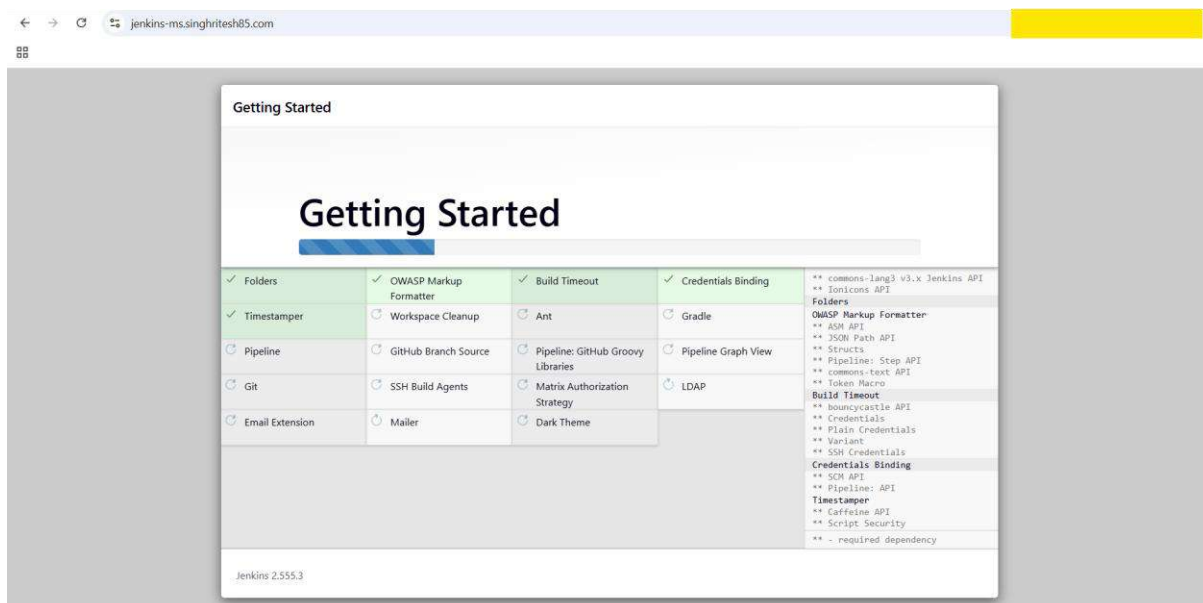
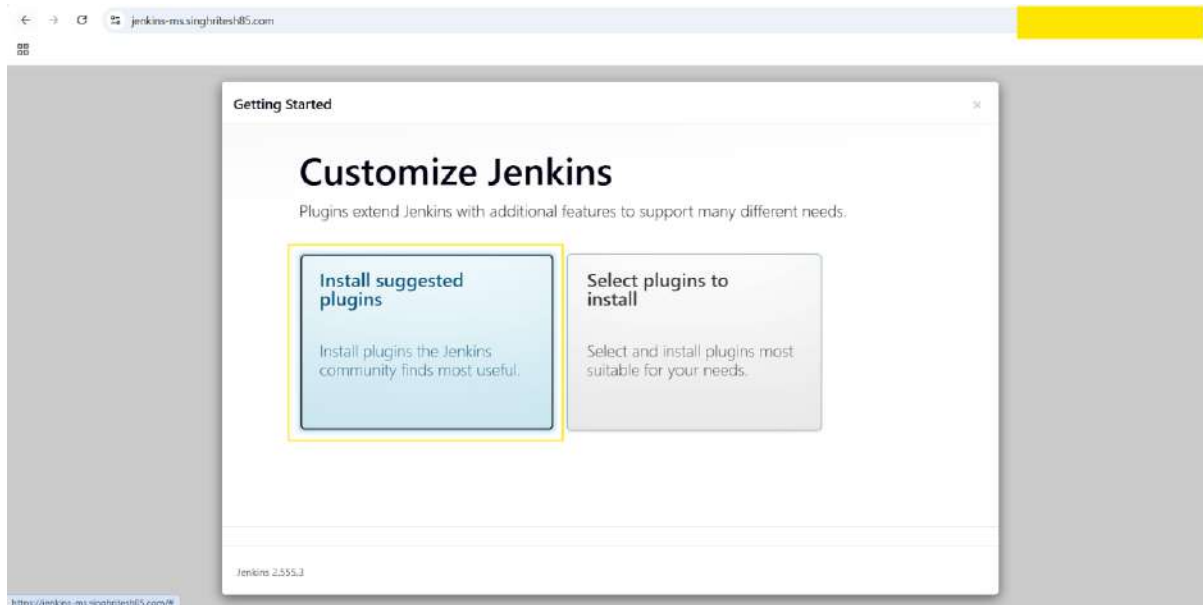
Finally, I was able to access Jenkins using the URL as shown in the screenshot attached below.

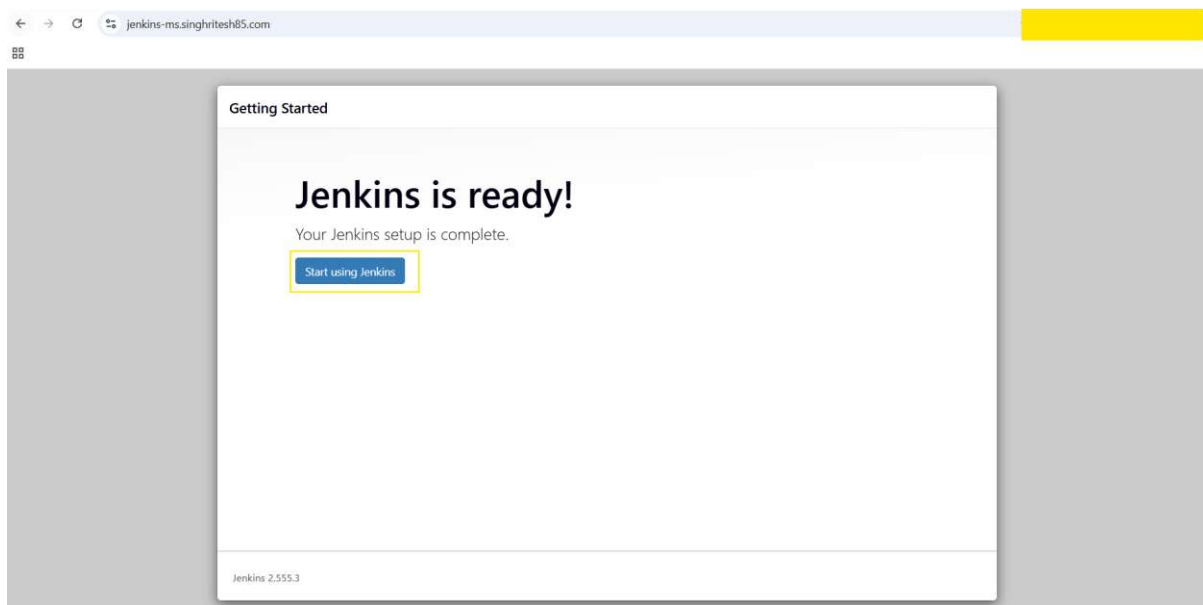
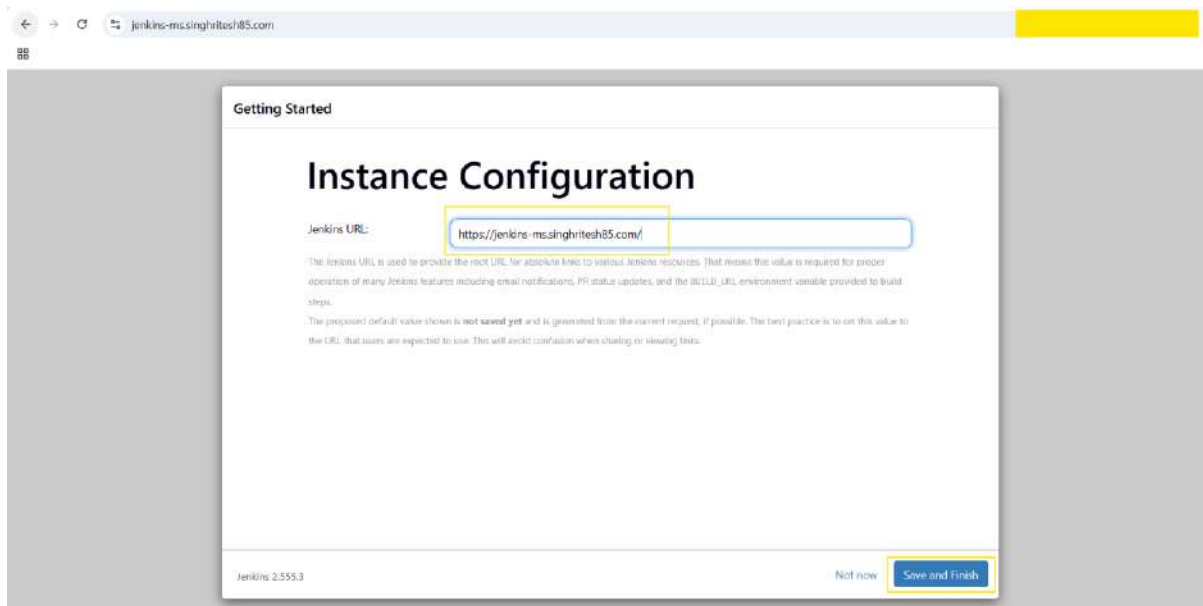


For first time login, you can get the initial-admin-password from the path **/var/lib/jenkins/secrets/initialAdminPassword** as shown in the screenshot attached below.

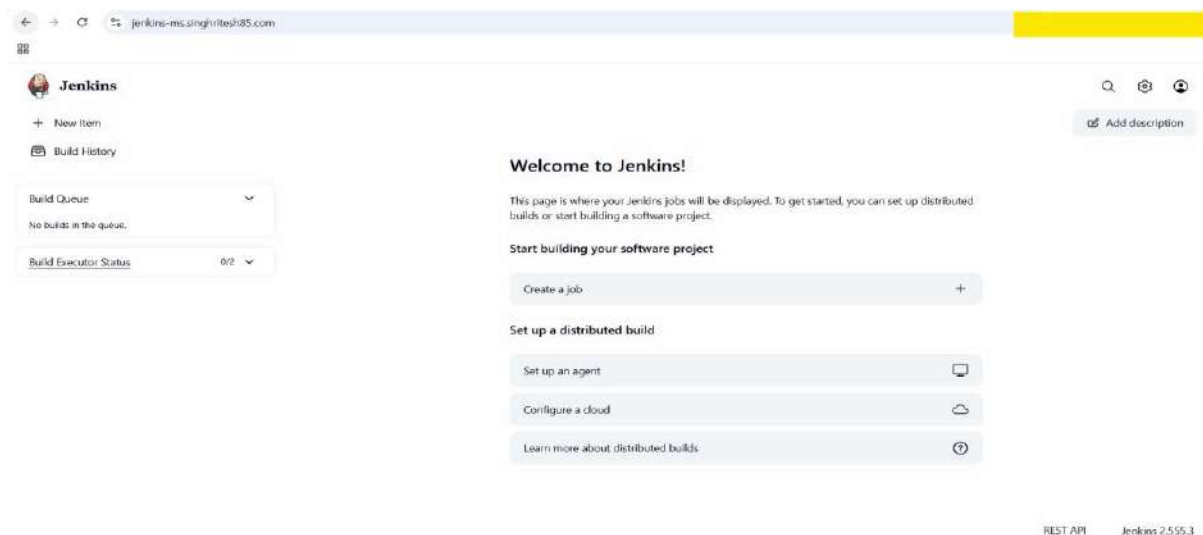
```
[dexter@ip-192-168-4-131 ~]$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword  
2b
```







Finally, Jenkins had been configured to use as shown in the screenshot attached below.



[Kafka-as-MessageQueue-the-GitOps-way.git](#). For your reference **karpenter.yaml** I had also provided below.

```
[k8s-management@k8s-management ~]$ kubectl apply -f karpenter.yaml
nodepool.karpenter.sh/general-purpose created
ec2nodeclass.karpenter.k8s.aws/default created
```

karpenter.yaml

This example NodePool will provision general purpose instances

apiVersion: karpenter.sh/v1

kind: NodePool

metadata:

name: general-purpose

annotations:

kubernetes.io/description: "General purpose NodePool for generic workloads"

spec:

template:

spec:

requirements:

- key: kubernetes.io/arch

operator: In

values: ["amd64"]

- key: kubernetes.io/os

operator: In

values: ["linux"]

- key: karpenter.sh/capacity-type

operator: In

values: ["on-demand"] #["spot"] ### You can select on-demand or spot instance depending on your requirement.

- key: karpenter.k8s.aws/instance-category

operator: In

values: ["t"] # Interested to launch t series instance #["c", "m", "r"]

- key: karpenter.k8s.aws/instance-generation

operator: Gt

values: ["1"] # Instances launched will be greater than 1 ###["2"] # Instances launched will be greater than 2

nodeClassRef:

group: karpenter.k8s.aws

```

kind: EC2NodeClass
name: default
---
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
  name: default
  annotations:
    kubernetes.io/description: "General purpose EC2NodeClass for running Amazon Linux 2023 nodes"
spec:
  role: "karpenter-eks-noderole" # replace with your karpenter noderole
  subnetSelectorTerms:
    - tags:
        karpenter.sh/discovery: "eks-demo-cluster-dev" # replace with your cluster name
  securityGroupSelectorTerms:
    - tags:
        karpenter.sh/discovery: "eks-demo-cluster-dev" # replace with your cluster name
  amiSelectorTerms:
    - alias: al2023@latest # Amazon Linux 2023

```

```

[k8s-management@k8s-management ~]$ kubectl get pods -n karpenter
NAME                                READY   STATUS    RESTARTS   AGE
karpenter-██████████-██████████    1/1     Running   0           11m
karpenter-██████████-██████████    1/1     Running   0           11m

```

Karpenter had been installed as shown in the screenshot attached above.

To install ArgoCD in EKS Cluster follow the steps as written below.

```
kubectl create ns argocd
```

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

```

[k8s-management@k8s-management ~]$ kubectl create ns argocd
namespace/argocd created
[k8s-management@k8s-management ~]$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

```

Created Kubernetes Secret as shown in the screenshot attached below to access the ArgoCD using URL with HTTPS Protocol.

```
kubectl create secret tls tls-argocd-ingress --cert=STAR_singhritesh85_com.crt --key=mykey.key -n argocd
```

```
[k8s-management@k8s-management ~]$ kubectl create secret tls tls-argocd-ingress --cert=STAR_singhritesh85_com.crt --key=mykey.key -n argocd
secret/tls-argocd-ingress created
```

To install Kubernetes Gateway API controller and then to create URL to access ArgoCD follow the below procedure.

kubectl apply -f <https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml>

helm upgrade --install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric --create-namespace -n nginx-gateway

```
[k8s-management@k8s-management ~]$ kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.4.0/standard-install.yaml
```

```
[k8s-management@k8s-management ~]$ helm upgrade --install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric --create-namespace -n nginx-gateway
```

```
[k8s-management@k8s-management ~]$ kubectl apply -f gateway-api-argocd.yaml
gateway.gateway.networking.k8s.io/argocd-gateway created
httproute.gateway.networking.k8s.io/argocd-route created
[k8s-management@k8s-management ~]$ kubectl get svc -n argocd
```

NAME	AGE	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
argocd-applicationset-controller	11m	ClusterIP	10.100.15.45	<none>	7000/TCP,80
argocd-dex-server	11m	ClusterIP	10.100.215.123	<none>	5556/TCP,55
argocd-gateway-nginx	11m	LoadBalancer	10.100.169.195	us-east-2.elb.amazonaws.com	80:31901/TCP
argocd-metrics	11m	ClusterIP	10.100.30.19	<none>	8082/TCP
argocd-notifications-controller-metrics	11m	ClusterIP	10.100.26.64	<none>	9001/TCP
argocd-redis	11m	ClusterIP	10.100.101.125	<none>	6379/TCP
argocd-repo-server	11m	ClusterIP	10.100.162.28	<none>	8081/TCP,80
argocd-server	11m	ClusterIP	10.100.176.39	<none>	80/TCP,443/TCP
argocd-server-metrics	11m	ClusterIP	10.100.208.88	<none>	8083/TCP

Edited the configmap as shown in the screenshot attached below.

```
[k8s-management@k8s-management ~]$ kubectl edit configmap argocd-cmd-params-cm -n argocd
```

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"ConfigMap","metadata":{"annotations":{},"labels":{"app.kubernetes.io/name":"argocd-cmd-params-cm","app.kubernetes.io/part-of":"argocd"},"name":"argocd-cmd-params-cm","namespace":"argocd"},"creationTimestamp":"2026-01-01T00:00:00Z"}
  labels:
    app.kubernetes.io/name: argocd-cmd-params-cm
    app.kubernetes.io/part-of: argocd
    name: argocd-cmd-params-cm
    namespace: argocd
    resourceVersion: "1"
    uid: "1"
data:
  server.insecure: "true"
```

```
[k8s-management@k8s-management ~]$ kubectl edit configmap argocd-cmd-params-cm -n argocd
Edit cancelled, no changes made.
```

Now, restarted the argocd-server deployment for the changes to take effect.


```
[k8s-management@k8s-management ~]$ kubectl rollout restart deployment argocd-server -n argocd
deployment.apps/argocd-server restarted
```

```
kubectl edit configmap argocd-cmd-params-cm -n argocd
```

data:

```
server.insecure: "true"
```

```
kubectl rollout restart deployment argocd-server -n argocd
```

Then, Created the Record-Set of A-Type in AWS Route53 Hosted Zone using the DNS Name of the LoadBalancer resulted from Kubernetes Gateway API for ArgoCD.

Route 53 > Hosted zones > singhritesh85.com > Create record

Quick create record Switch to wizard

▼ Record 1 Delete

Record name Info .singhritesh85.com Keep blank to create a record for the root domain.

Record type Info A - Routes traffic to an IPv4 address and some AWS resources

☒ Alias

Route traffic to Info Alias to Application and Classic Load Balancer

US East (Ohio)

×

Alias hosted zone ID:

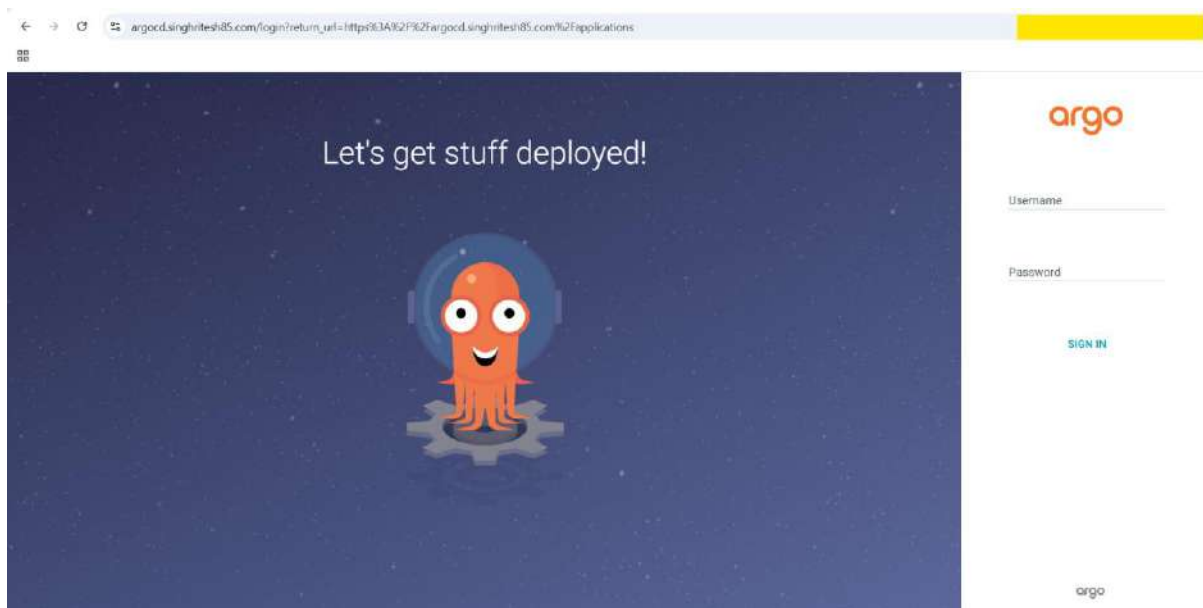
Routing policy Info Simple routing

Evaluate target health ☐ No

Add another record

Cancel Create records

Finally, able to access the ArgoCD as shown in the screenshot attached below.



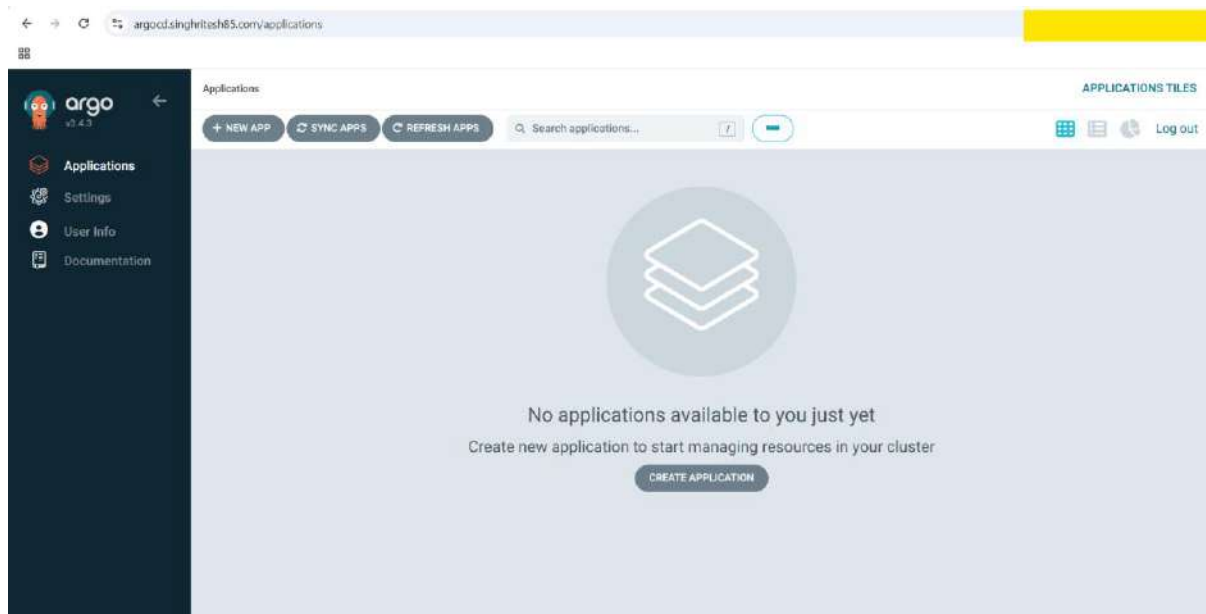
To Login into the ArgoCD for the first time you can login with user having username as admin and password you can get using the command.

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
```

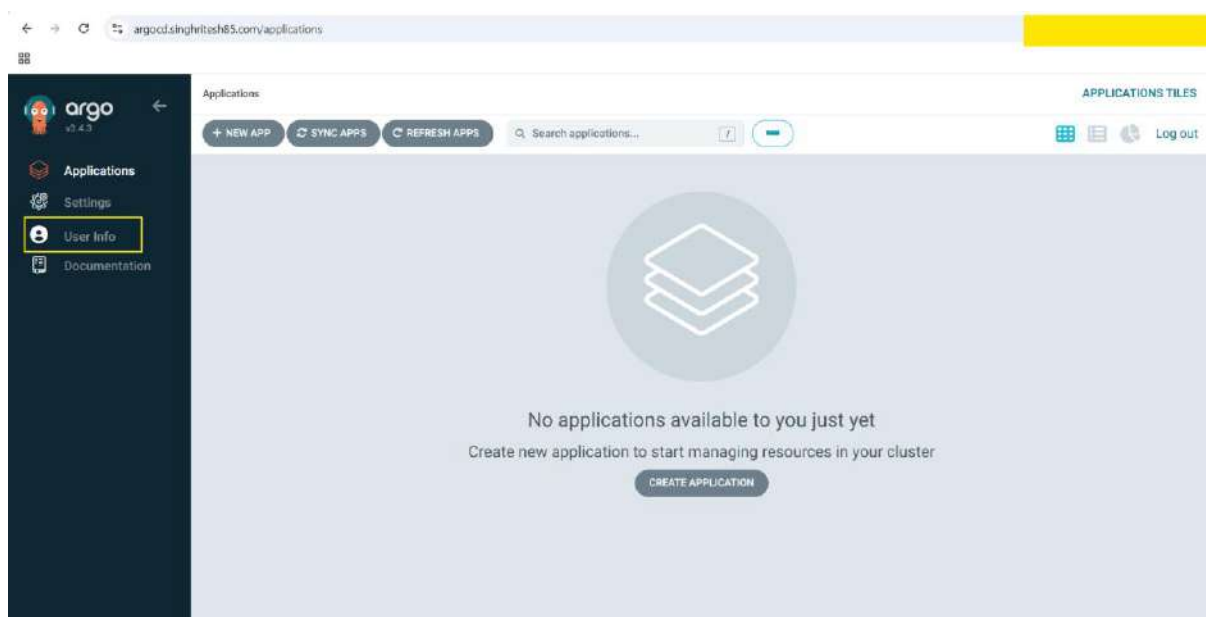
```
[k8s-management@k8s-management ~]$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
k[REDACTED]m[k8s-management@k8s-management ~]$
```

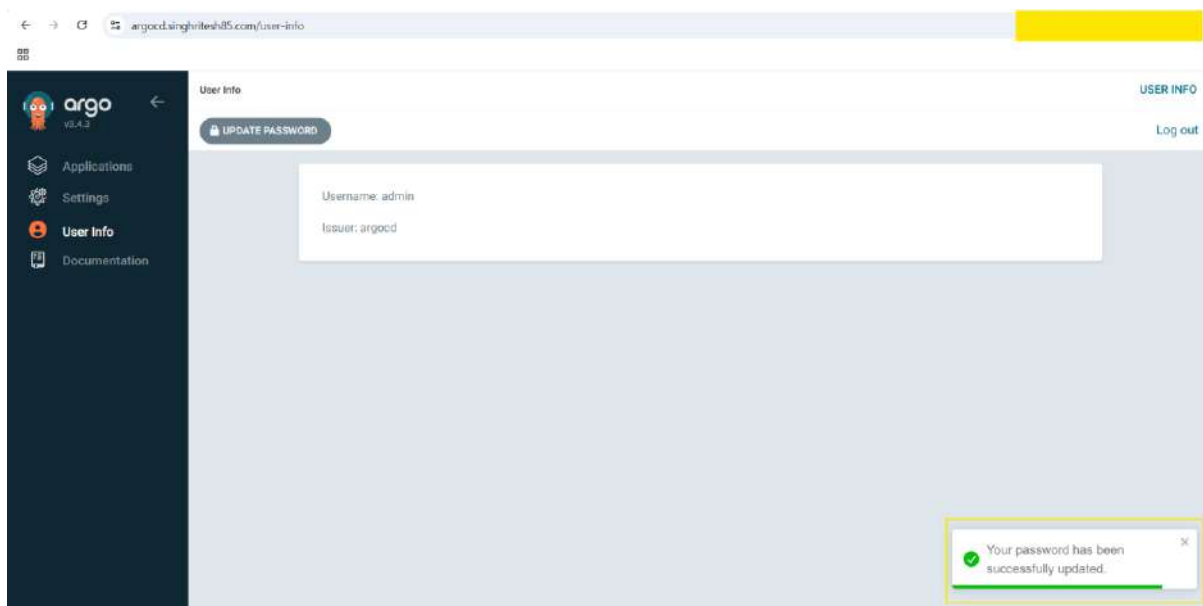
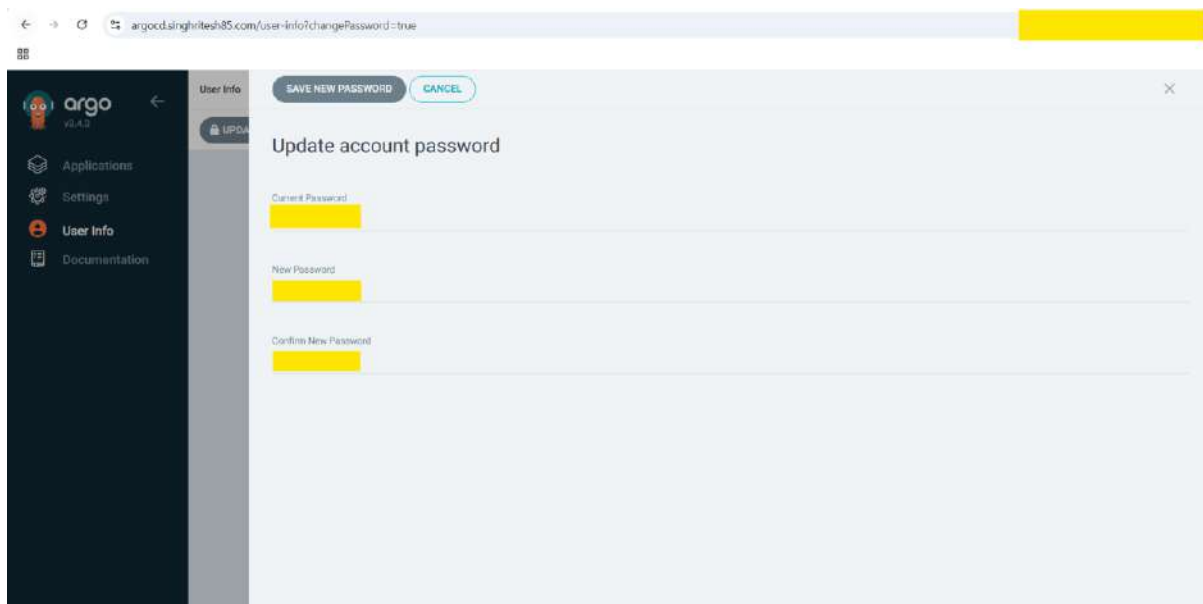
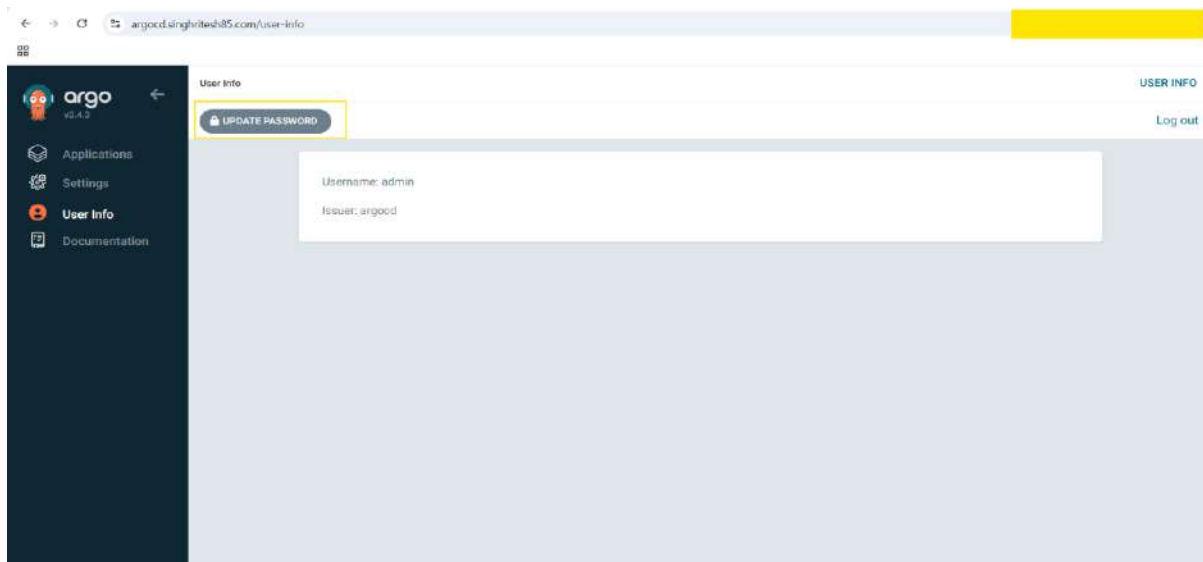
For your reference **gateway-api-argocd.yaml** file is present in the GitHub Repo

<https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>
at the path **module-3/Section-1/gateway-api-argocd.yaml**.

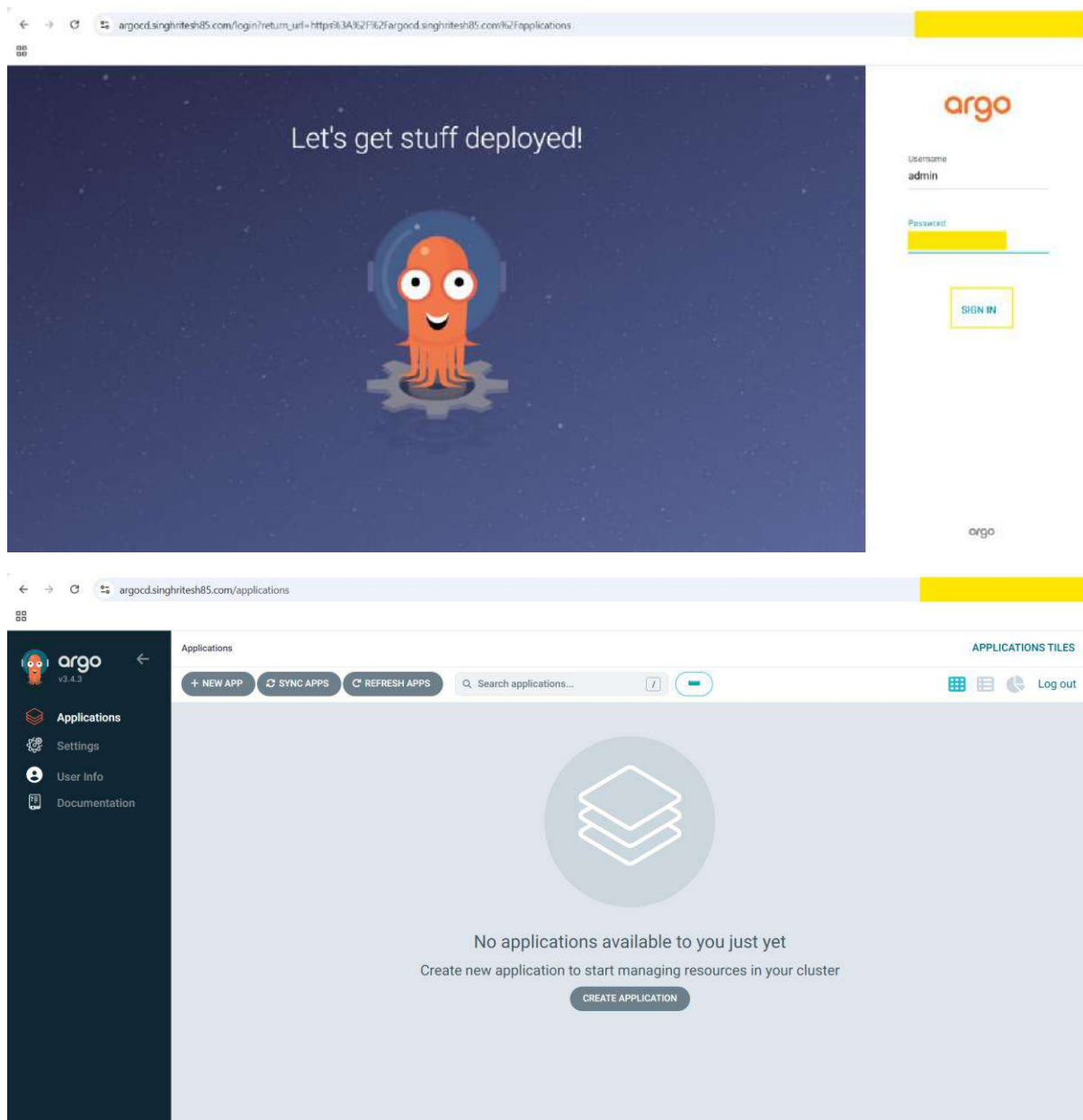


After the successful Login change the password as shown in the screenshot attached below.





Then Logout and Login again.



Install ArgoCD CLI using the command as shown below.

```
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64

sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd

rm argocd-linux-amd64
```

```
[k8s-management@k8s-management ~]$ curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
[k8s-management@k8s-management ~]$ sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
[k8s-management@k8s-management ~]$ rm argocd-linux-amd64
```

Added Slave to Jenkins as shown in the screenshot attached below.

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-in Node	Linux (amd64)	In sync	8388608.00 TB	0 B	1.87 GB	0ms
Data obtained				7 min 31 sec	7 min 31 sec	7 min 31 sec	7 min 31 sec

← → 🔍 jenkins-ms.singhritesh85.com/manage/computer/new

Jenkins / Manage Jenkins / Nodes / New node

New node

Node name
Slave-1

Type
☒ Permanent Agent
 Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create

REST API Jenkins 2.555.3

← → 🔍 jenkins-ms.singhritesh85.com/manage/computer/createItem

Jenkins / Manage Jenkins / Nodes

/home/k8s-management

Labels ?
Slave-1

Usage ?
Use this node as much as possible

Launch method ?
Launch agents via SSH

Host ?
192.168.4.233

Credentials ?
- none -

+ Add

The selected credentials cannot be found

Save

← → 🔍 jenkins-ms.singhritesh85.com/manage/computer/createItem

Jenkins / Manage Jenkins / Nodes

/home/k8s-management

Labels ?
Slave-1

Usage ?
Use this node as much as possible

Launch method ?
Launch agents via SSH

Host ?
192.168.4.233

Credentials ?
- none -

The selected credentials cannot be found

Save

Add Credentials

Select a type of credential

- ☒ Username with password
Commonly used for authentication to services like Git, APIs, or registries.
- ☐ GitHub App
- ☐ SSH Username with private key
- ☐ Secret file
- ☐ Secret text
- ☐ Certificate
Upload a PKCS#12 or PEM encoded certificate and private key.

Next

jenkins-ms.singhritesh85.com/manage/computer/createItem

Jenkins / Manage Jenkins / Nodes

/home/k8s-management

Labels ?

Slave-1

Usage ?

Use this node as much as possible

Launch method ?

Launch agents via SSH

Host ?

192.168.4.233

Credentials ?

- none -

The selected credentials cannot be found

Save

Add Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username

k8s-management

☐ Treat username as secret ?

Password

ID ?

jenkins-cred

Description ?

jenkins-cred

Create

jenkins-ms.singhritesh85.com/manage/computer/createItem

Jenkins / Manage Jenkins / Nodes

Launch method ?

Launch agents via SSH

Host ?

192.168.4.233

Credentials ?

k8s-management/***** (jenkins-cred)

Host Key Verification Strategy ?

Non verifying Verification Strategy

Advanced

Availability ?

Keep this agent online as much as possible

Save

jenkins-ms.singhritesh85.com/manage/computer/

Jenkins / Manage Jenkins / Nodes

+ New Node

Configure Monitors

Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-in Node	Linux (amd64)	In sync	8388608.00 TiB	0 B	1.87 GiB	0ms
	Slave-1	Linux (amd64)	In sync	15.97 GiB	0 B	1.87 GiB	57ms
Data obtained		0.33 sec	0.32 sec	0.27 sec	0.26 sec	0.27 sec	0.29 sec

Icons: S M L

Legend

To add Swap Space on Jenkins Master and Slave Run the below commands.

```
[root@k8s-management ~]# fallocation -l 512M /swapfile
[root@k8s-management ~]# chmod 600 /swapfile
[root@k8s-management ~]# mkswap /swapfile
Setting up swap space version 1, size = 512 MiB ( bytes)
no label, UUID=
[root@k8s-management ~]# vim /etc/fstab
[root@k8s-management ~]# swapon -a
[root@k8s-management ~]# free -mh
              total        used        free      shared  buff/cache   available
Mem:           3.7Gi         458Mi        1.2Gi         0.0Ki         2.7Gi         3.0Gi
Swap:          511Mi           0B          511Mi
[root@k8s-management ~]# cat /etc/fstab
#
UUID=          /          xfs     defaults,noatime 1 1
UUID=          /boot/efi      vfat    defaults,noatime,uid=0,gid=0,umask=0077,shortname=winnt,x-systemd.automount 0 2
/swapfile swap swap defaults 0 0

[root@ip-192-168-4-104 ~]# fallocation -l 512M /swapfile
[root@ip-192-168-4-104 ~]# chmod 600 /swapfile
[root@ip-192-168-4-104 ~]# mkswap /swapfile
Setting up swap space version 1, size = 512 MiB (536866816 bytes)
no label, UUID=
[root@ip-192-168-4-104 ~]# vim /etc/fstab
[root@ip-192-168-4-104 ~]# swapon -a
[root@ip-192-168-4-104 ~]# free -mh
              total        used        free      shared  buff/cache   available
Mem:           3.7Gi         810Mi        1.2Gi         5.0Mi         1.8Gi         2.7Gi
Swap:          511Mi           0B          511Mi
[root@ip-192-168-4-104 ~]# cat /etc/fstab
#
UUID=          /          xfs     defaults,noatime 1 1
UUID=          /boot/efi      vfat    defaults,noatime,uid=0,gid=0,umask=0077,shortname=winnt,x-systemd.automount 0 2
192.168.      /var/lib/jenkins nfs4     nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2,noresvport,_netdev 0 0
/swapfile swap swap defaults 0 0
```

```
fallocation -l 512M /swapfile
chmod 600 /swapfile
mkswap /swapfile

vim /etc/fstab
/swapfile swap swap defaults 0 0

:wq

swapon -a
free -mh
```

Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

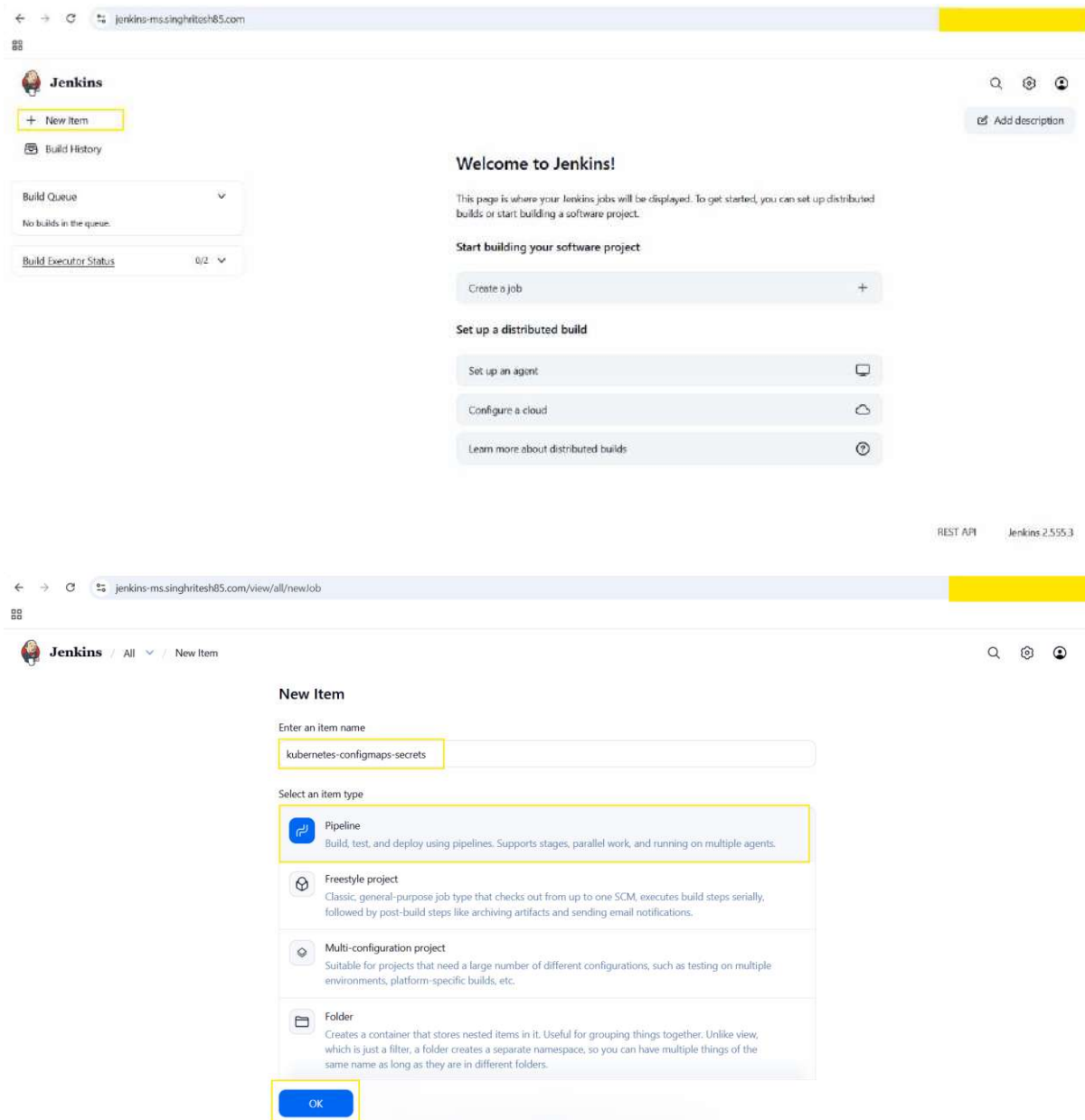
[+ New Node](#)
[Configure Monitors](#)


S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	8388608.00 TiB	512.00 MiB	1.87 GiB	0ms
	Slave-1	Linux (amd64)	In sync	15.47 GiB	512.00 MiB	1.87 GiB	23ms
Data obtained		22 sec	22 sec	22 sec	22 sec	22 sec	22 sec

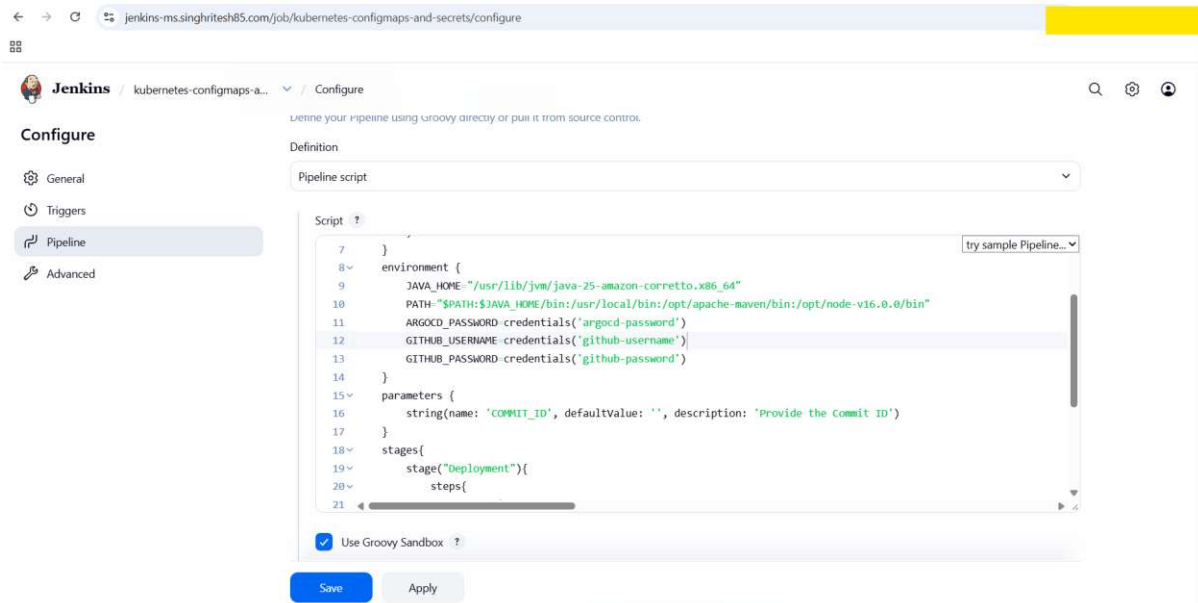
Icon: S M L

Legend

Now, created first Jenkins Pipeline to create the Kubernetes ConfigMap and Secrets as shown in the screenshot attached below.



For your reference I kept the Jenkinsfile at the path **module-3/Section-1/Jenkinsfile-Kubernetes-cm-secrets** in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>.



jenkins-ms.singhritesh85.com/job/kubernetes-configmaps-and-secrets/configure

Jenkins / kubernetes-configmaps-a... / Configure

Define your pipeline using Groovy directly or pull it from source control.

Definition: Pipeline script

Script ?

```

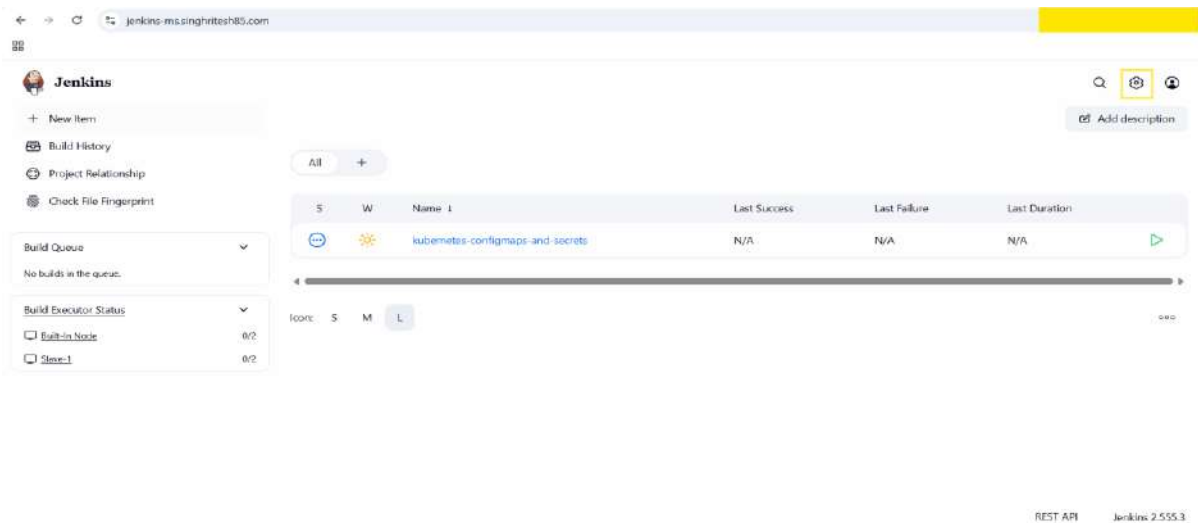
7   }
8   environment {
9       JAVA_HOME="/usr/lib/jvm/java-25-amazon-corretto.x86_64"
10      PATH="$PATH:$JAVA_HOME/bin:/usr/local/bin:/opt/apache-maven/bin:/opt/node-v16.0.0/bin"
11      ARGCD_PASSWORD=credentials('argcd-password')
12      GITHUB_USERNAME=credentials('github-username')
13      GITHUB_PASSWORD=credentials('github-password')
14   }
15   parameters {
16       string(name: 'COMMIT_ID', defaultValue: '', description: 'Provide the Commit ID')
17   }
18   stages {
19       stage("Deployment"){
20           steps{
21

```

☒ Use Groovy Sandbox ?

Save Apply

Before running this Jenkins Job create three Jenkins Credentials as shown in the screenshot attached below.



jenkins-ms.singhritesh85.com

Jenkins

+ New Item

Build History

Project Relationship

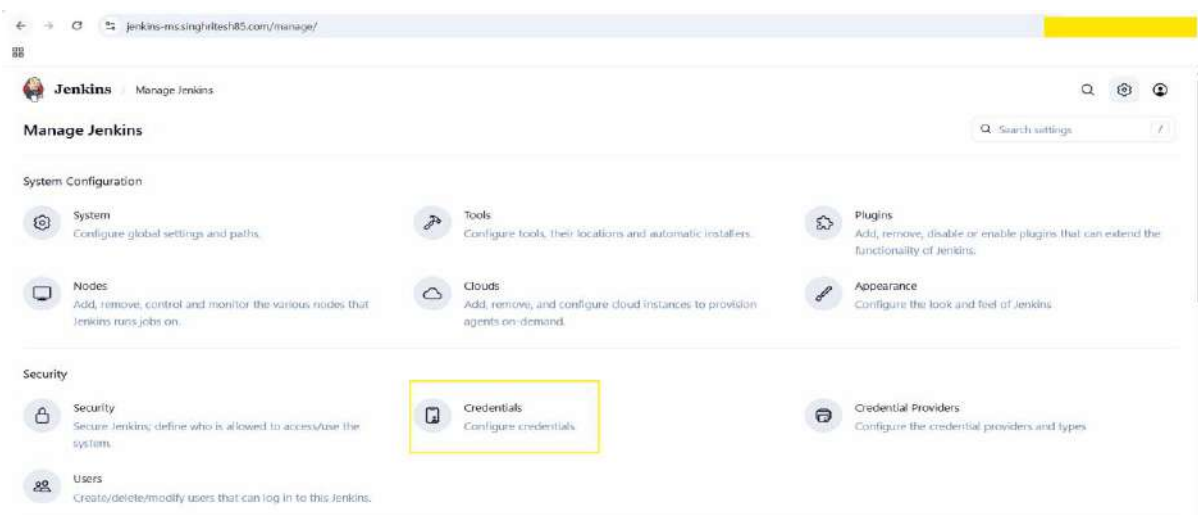
Check File Fingerprint

Build Queue: No builds in the queue.

Build Executor Status: 0/2

S	W	Name	Last Success	Last Failure	Last Duration
		kubernetes-configmaps-and-secrets	N/A	N/A	N/A

REST API Jenkins 2.555.3



jenkins-ms.singhritesh85.com/manage/

Jenkins / Manage Jenkins

Search settings

System Configuration

- System: Configure global settings and paths.
- Tools: Configure tools, their locations and automatic installers.
- Plugins: Add, remove, disable or enable plugins that can extend the functionality of Jenkins.
- Nodes: Add, remove, control and monitor the various nodes that Jenkins runs jobs on.
- Clouds: Add, remove, and configure cloud instances to provision agents on-demand.
- Appearance: Configure the look and feel of Jenkins.

Security

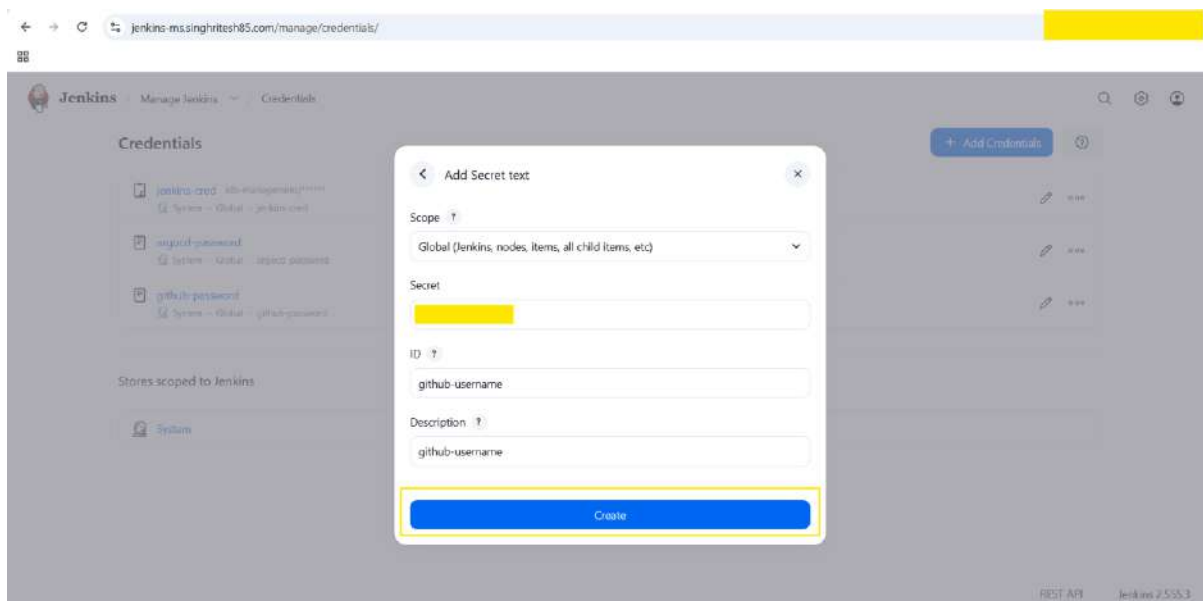
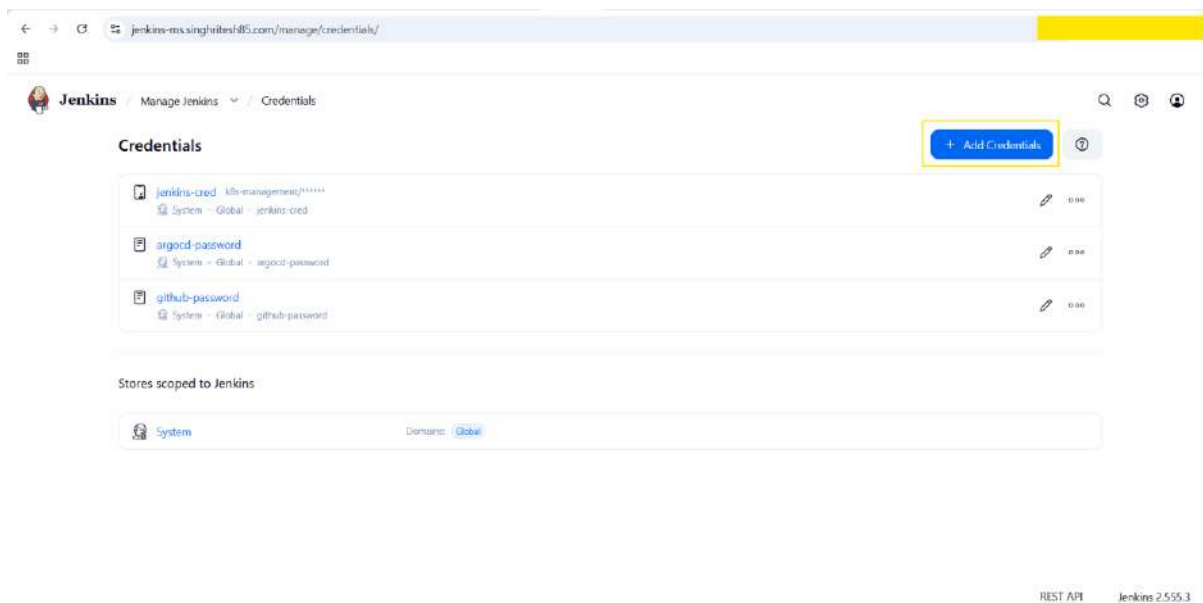
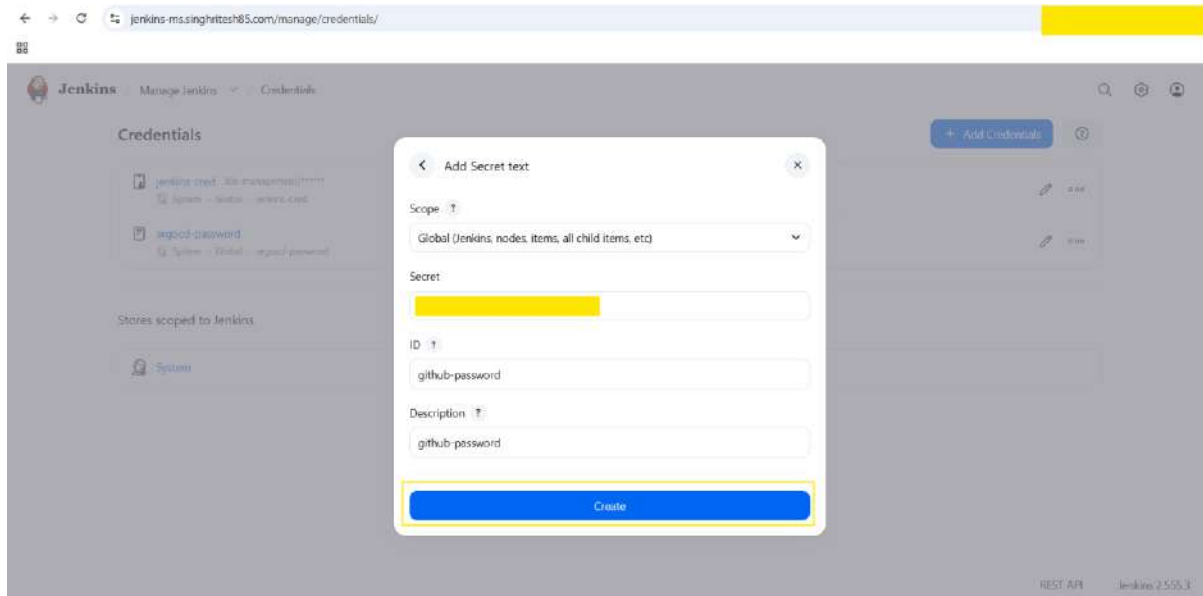
- Security: Secure Jenkins; define who is allowed to access/use the system.
- Credentials**: Configure credentials.
- Credential Providers: Configure the credential providers and types.
- Users: Create/delete/modify users that can log in to this Jenkins.

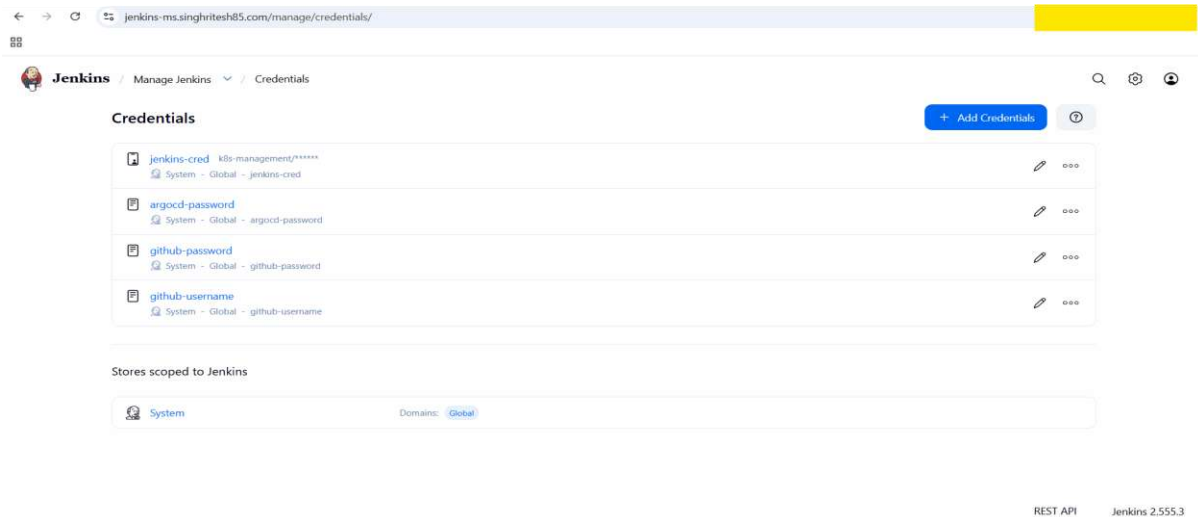
The image displays three sequential screenshots of the Jenkins web interface, specifically the 'Credentials' management page. The browser address bar shows 'jenkins-ms.singhritesh85.com/manage/credentials/'.

Top Screenshot: The 'Credentials' page is shown with a list of existing credentials (one named 'jenkins-cred') and a section for 'Stores scoped to Jenkins' (one named 'System'). A yellow box highlights the '+ Add Credentials' button in the top right corner.

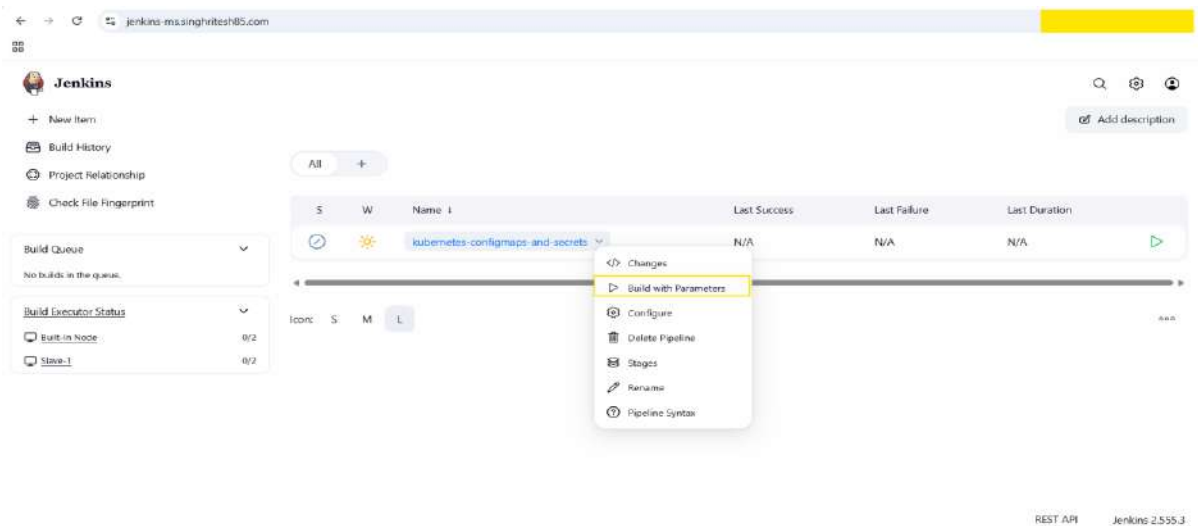
Middle Screenshot: The 'Add Credentials' modal is open. It prompts the user to 'Select a type of credential'. The options are: 'Username with password', 'GitHub App', 'SSH Username with private key', 'Secret file', 'Secret text', and 'Certificate'. The 'Secret text' option is highlighted with a yellow box. A 'Next' button is visible at the bottom of the modal.

Bottom Screenshot: The 'Add Secret text' modal is open. It contains the following fields: 'Scope' (set to 'Global (Jenkins, nodes, items, all child items, etc)'), 'Secret' (a text input field with a yellow box), 'ID' (set to 'argocd-password'), and 'Description' (set to 'argocd-password'). A 'Create' button is at the bottom of the modal.

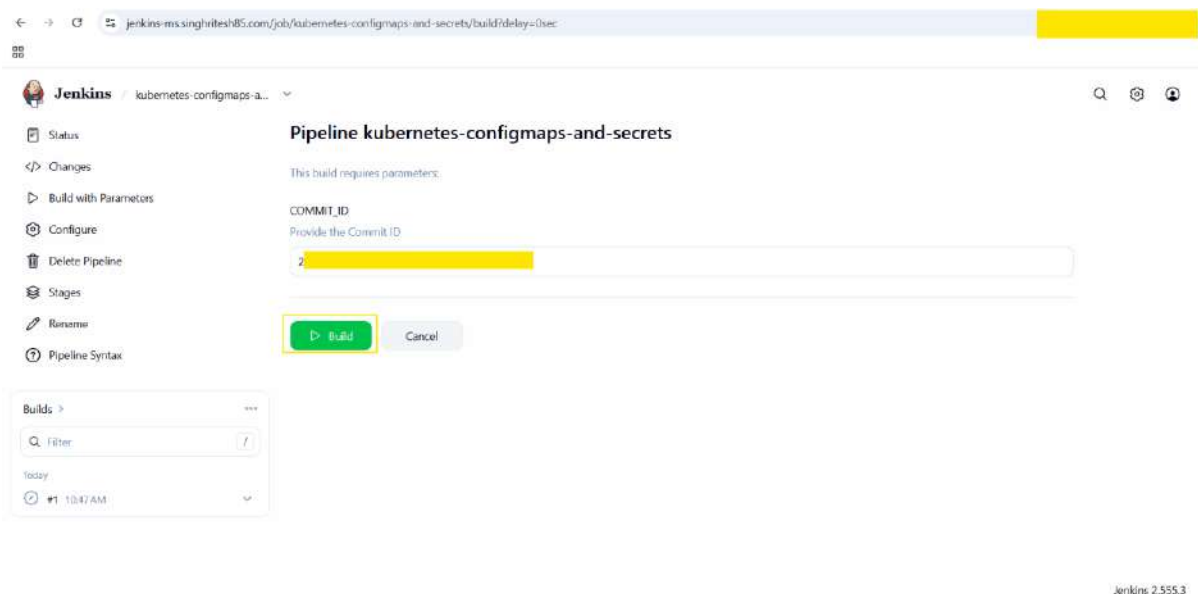




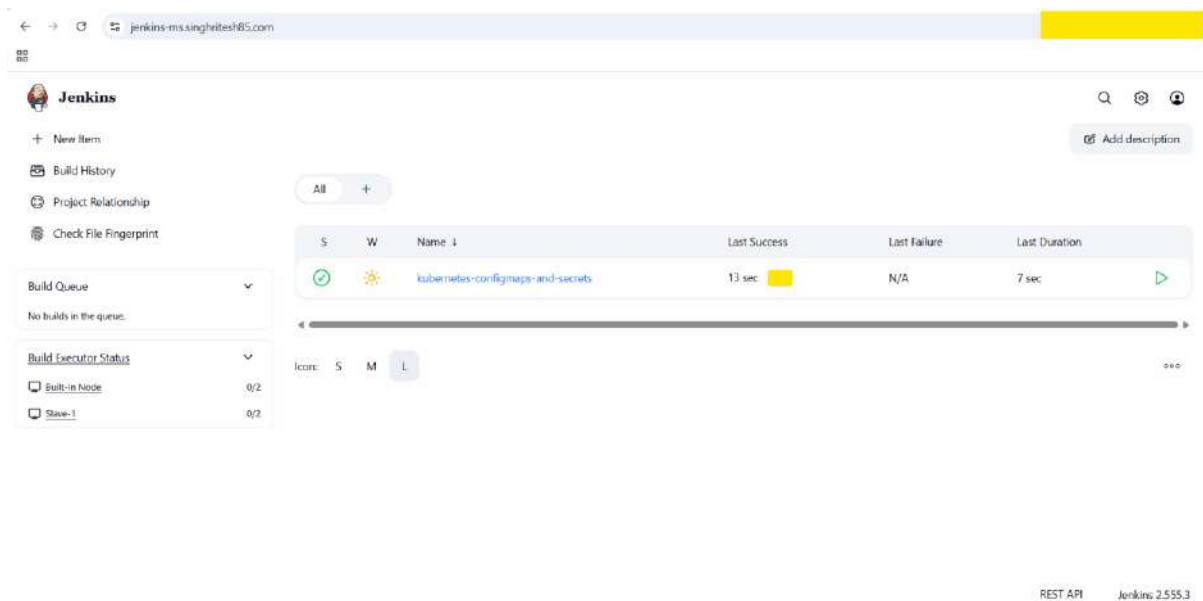
Then ran the pipeline as shown in the screenshot attached below.



To run the Jenkins Pipeline, you need to provide the commit id as shown in the screenshot attached below.



The screenshot of the Jenkins Job after successfully running the Jenkins Job is as shown in the screenshot attached below.



Then created another Jenkins Job, to create AWS MSK Kafka Topic. The steps followed to create the Jenkins Job is as discussed above. The Jenkinsfile for this Jenkins Job is present in the same GitHub Repo at the path **module-3/Section-1/Jenkinsfile-kafka-topic**. The screenshot of the Jenkins Job after successfully running the Jenkins Job is as shown in the screenshot attached below. Before running the Jenkins Job for kafka topic, create Jenkins Credentials named as github-cred as shown in the screenshot attached below. Then ran this Jenkins Job for Kafka Topic.

< Add Username with password ×

Scope ?
Global (Jenkins, nodes, items, all child items, etc)

Username

☐ Treat username as secret ?

Password

ID ?
github-cred

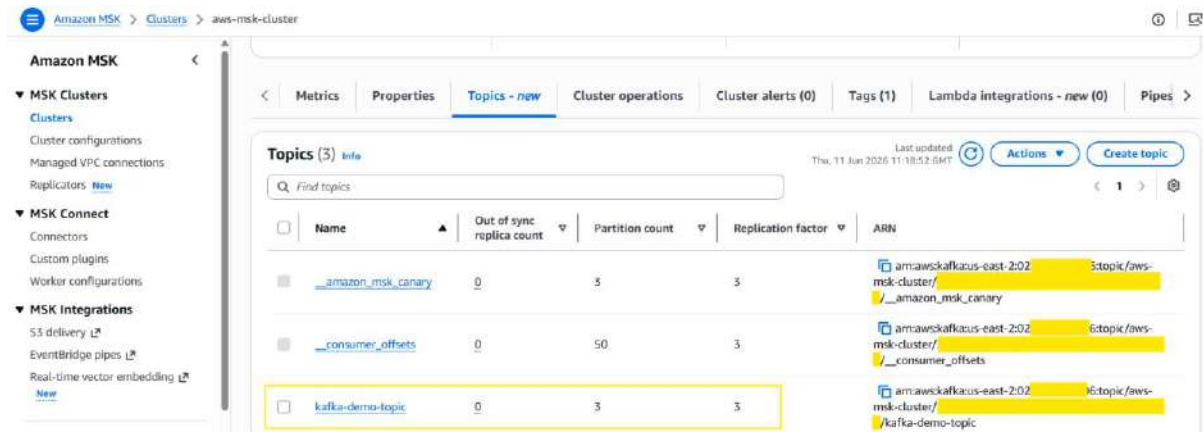
Description ?
github-cred

Create

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☁	kafka-topic	4 min 25 sec		20 sec
✓	☀	kubernetes-configmaps-and-secrets	21 min #6	N/A	7 sec

```
[k8s-management@k8s-management aws-msk-kafka-producer-consumer]$ kubectl get pods -n producer-consumer
NAME                                READY   STATUS    RESTARTS   AGE
create-kafka-topic-7tdj6            0/1     Completed 0           23s
```

After successful execution of Jenkins Job, I checked AWS Console for MSK and found Kafka Topic had been created as shown in the screenshot attached below.



Then created the Jenkins Job to deploy Kafka-Producer and Kafka-Consumer. The steps followed while creating the Jenkins Job is as discussed earlier. The Jenkinsfile to create the Jenkins Job is present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-3/Section-1/Jenkinsfile-Producer-Consumer**.

The screenshot of Jenkins Job after successfully running it is as shown in the screenshot attached below.

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	Kafka-Producer-consumer			7.2 sec
✓	☁	kafka-topic			20 sec
✓	☀	kubernetes-configmaps-and-secrets			7 sec

Successfully deployed Kafka Producer and Kafka Consumer pods. Verified the Kafka Consumer logs and confirmed that messages published by the Kafka Producer were being consumed successfully from the Kafka topic.

```
[k8s-management@k8s-management aws-msk-kafka-producer-consumer]$ kubectl get pods -n producer-consumer
NAME                                READY   STATUS    RESTARTS   AGE
create-kafka-topic-7tdj6            0/1     Completed 0           40m
kafka-consumer-6b9c87679c-qjp4f     1/1     Running   0           26s
kafka-producer-54f8bb9dccc-qv3pb     1/1     Running   0           26s
[k8s-management@k8s-management aws-msk-kafka-producer-consumer]$ kubectl logs -f deployment/kafka-consumer -n producer-consumer
Hello Kafka from Kubernetes Thu Jun 11 11:58:50 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:58:58 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:06 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:14 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:21 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:29 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:36 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:44 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:51 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 11:59:59 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 12:00:08 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 12:00:16 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 12:00:24 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 12:00:32 UTC 2026
Hello Kafka from Kubernetes Thu Jun 11 12:00:39 UTC 2026
```




Installation and configuration of KEDA (Kubernetes Event Driven Autoscaling)



KEDA stands for Kubernetes Event Driven Autoscaling. KEDA will scale the workload automatically based on the volumes of events. To install KEDA using helm you can use the command as shown below.

```
helm repo add kedacore https://kedacore.github.io/charts
```

```
helm repo update
```

```
helm upgrade --install keda kedacore/keda --namespace keda --create-namespace --set
serviceAccount.operator.create=true --set serviceAccount.operator.name=keda-operator --set-
json 'serviceAccount.operator.annotations={"eks.amazonaws.com/role-arn":"arn:aws:iam::02XXXXXXX6:role/aws-irsa-role"}'
```

```
[demo@demo-azure-vm ~]$ helm repo add kedacore https://kedacore.github.io/charts
"kedacore" has been added to your repositories
[demo@demo-azure-vm ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "secrets-store-csi-driver" chart repository
...Successfully got an update from the "kedacore" chart repository
...Successfully got an update from the "istio" chart repository
Update Complete. 🎉Happy Helming!🎉
```

```
[k8s-management@k8s-management ~]$ helm upgrade --install keda kedacore/keda --namespace keda --create-namespace --set serviceAccount.operator.create=true --set
serviceAccount.operator.name=keda-operator --set-json 'serviceAccount.operator.annotations={"eks.amazonaws.com/role-arn":"arn:aws:iam::02XXXXXXX6:role/aws-irsa-role"}'
```

```
[k8s-management@k8s-management ~]$ kubectl get pods -n keda
```

NAME	READY	STATUS	RESTARTS	AGE
keda-admission-webhooks-d6dc4d56f-2mx4x	1/1	Running	0	80s
keda-operator-69f7467655-kdzhx	1/1	Running	1 (30s ago)	80s
keda-operator-metrics-apiserver-589688768f-nf9n8	1/1	Running	0	80s

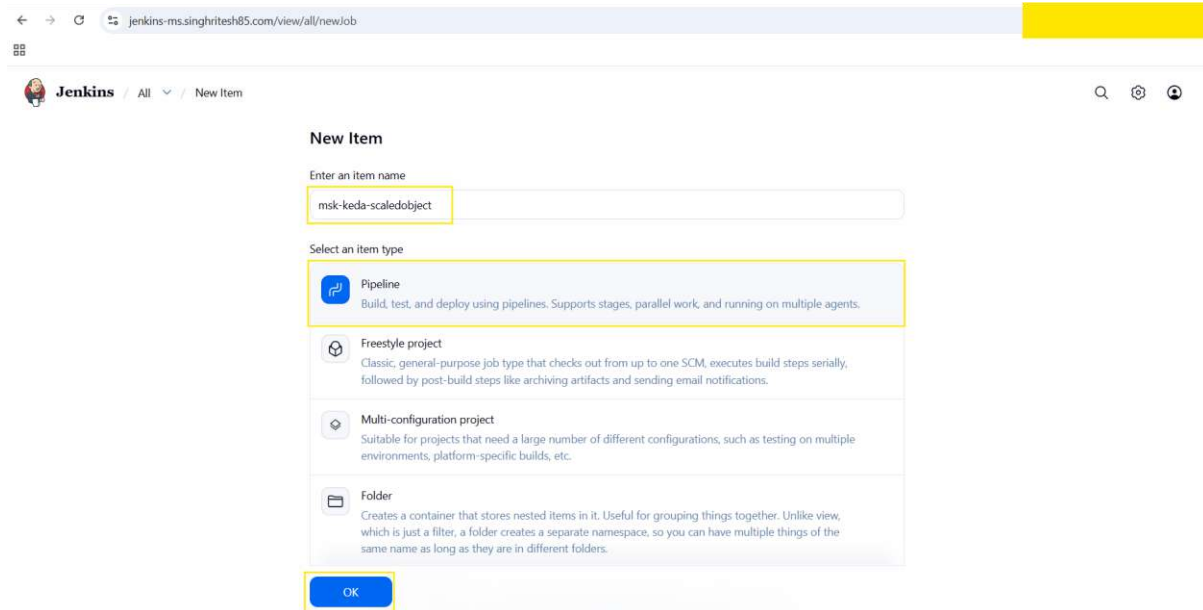
The above screenshot shows KEDA had been installed successfully in EKS Cluster. In the current project KEDA's job is to watch AWS MSK Kafka lag and automatically increase or decrease the number of kafka-consumer pods between 1 and 5 based on how many messages are waiting to be processed.

To do so I configured KEDA Trigger Authentication and Scaled Object using Jenkins Declarative Pipeline and ArgoCD as shown in the screenshot attached below.

The screenshot shows the Jenkins dashboard with a table of build history. The table has columns for Status (S), Warnings (W), Name, Last Success, Last Failure, and Last Duration. The builds are for 'Kafka-Producer-consumer', 'kafka-topic', and 'kubernetes-configmaps-secrets'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀️	Kafka-Producer-consumer	6 min 25 sec #3	N/A	7.2 sec
✓	☁️	kafka-topic	8 min 2 sec #2	8 min 55 sec #1	38 sec
✓	☁️	kubernetes-configmaps-secrets	10 min #3	10 min #2	7.8 sec

REST API Jenkins 2.555.3



jenkins-ms.singhritesh85.com/view/all/newJob

Jenkins / All / New Item

New Item

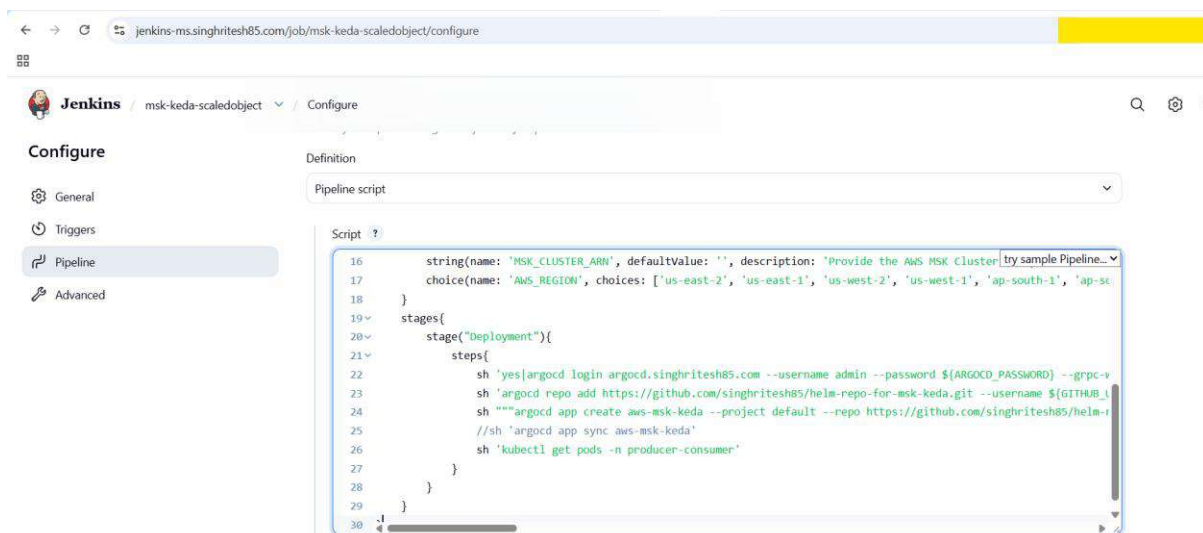
Enter an item name

msk-keda-scaledobject

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK



jenkins-ms.singhritesh85.com/job/msk-keda-scaledobject/configure

Jenkins / msk-keda-scaledobject / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Definition

Pipeline script

Script ?

```

16 string(name: 'MSK_CLUSTER_ARN', defaultValue: '', description: 'Provide the AWS MSK Cluster')
17 choice(name: 'AWS_REGION', choices: ['us-east-2', 'us-east-1', 'us-west-2', 'us-west-1', 'ap-south-1', 'ap-se
18 }
19 stages{
20   stage("Deployment"){
21     steps{
22       sh 'yes|argocd login argocd.singhritesh85.com --username admin --password ${ARGOCD_PASSWORD} --grpc-v
23       sh 'argocd repo add https://github.com/singhritesh85/helm-repo-for-msk-keda.git --username ${GITHUB_
24       sh 'argocd app create aws-msk-keda --project default --repo https://github.com/singhritesh85/helm-r
25       //sh 'argocd app sync aws-msk-keda'
26       sh 'kubectl get pods -n producer-consumer'
27     }
28   }
29 }
30
  
```

To Run this Jenkins Job, you need to pass the parameter as shown in the screenshot attached below.



jenkins-ms.singhritesh85.com/job/msk-keda-scaledobject/build?delay=0sec

Jenkins / msk-keda-scaledobject

Pipeline msk-keda-scaledobject

This build requires parameters:

MSK_CLUSTER_ARN
Provide the AWS MSK Cluster ARN

arn:aws:kafka:us-east-2:025553189191:cluster/aws-msk-cluster/

AWS_REGION
Select the AWS Region

us-east-2

Builds

Filter

Today

Build Cancel

After successfully Running the Jenkins Job, the screenshot is as shown below.

The screenshot shows the Jenkins web interface at `jenkins-ms.singhritesh85.com`. The main view displays a table of builds with columns: S, W, Name, Last Success, Last Failure, and Last Duration. The build `msk-keda-scaledobject` is highlighted in yellow, showing a last success of 2 min 49 sec and a last failure of 6 min 10 sec.

Below the table, there is a terminal output showing the following commands and results:

```
[k8s-management@k8s-management helm-repo-for-msk-keda]$ kubectl get scaledobject -n producer-consumer
NAME                                SCALETARGETKIND  SCALETARGETNAME  MIN  MAX  READY  ACTIVE  FALLBACK  PAUSED  TRIGGERS  AUTHENTICATIONS  AGE
kafka-consumer-scaler  apps/v1.Deployment  kafka-consumer  1    5    True   False   False    False   apache-kafka  keda-msk-trigger-auth  7m21s
[k8s-management@k8s-management helm-repo-for-msk-keda]$ kubectl describe scaledobject kafka-consumer-scaler -n producer-consumer
```

Events:

Type	Reason	Age	From	Message
Normal	KEDAScalersStarted	8m37s	keda-operator	Scaler apache-kafka is built
Normal	KEDAScalersStarted	8m37s	keda-operator	Started scalers watch
Normal	ScaledObjectReady	8m37s	keda-operator	ScaledObject is ready for scaling
Normal	KEDAScalersStarted	7m8s (x2 over 7m8s)	keda-operator	Scaler apache-kafka is built

For your reference, the Jenkins file to create this Jenkins Job is present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-3/Section-1/Jenkinsfile-aws-msk-keda**. While creating AWS resources using Terraform, we configured **AWS IRSA (IAM Roles for Service Accounts)** to securely map Kubernetes service accounts to IAM roles, ensuring that only authorized workloads can assume specific AWS permissions.

```
resource "aws_iam_role" "irsa_role" {
  name = "${var.prefix}-irsa-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = {
          Federated = aws_iam_openid_connect_provider.eksopidc.arn
        }
        Action = ["sts:AssumeRoleWithWebIdentity"]
        Condition = {
          StringEquals = {
            "${replace(aws_iam_openid_connect_provider.eksopidc.url, "https://", "")}:sub" = ["system:serviceaccount:producer-consumer:msk-sa", "system:serviceaccount:keda:keda-operator"],
            "system:serviceaccount:monitoring:kafka-exporter-prometheus-kafka-exporter"
          }
          "${replace(aws_iam_openid_connect_provider.eksopidc.url, "https://", "")}:aud" = ["sts.amazonaws.com"]
        }
      }
    ]
  })
}
```

Installation of Prometheus, Grafana, Prometheus Kafka Exporter and Monitoring AWS MSK using Prometheus and Grafana

To install Prometheus, Grafana, Prometheus Exporter for Kafka, helm had been used and to create Service Monitor Kubernetes Yaml Manifests File had been used which is present at the path **module-3/Section-1/kafka-exporter-sm.yaml** in the GitHub Repo <https://github.com/singhritesh85/DevOps->

[Project-using-Kafka-as-MessageQueue-the-GitOps-way.git](#). While creating AWS resources using Terraform, we configured **AWS IRSA (IAM Roles for Service Accounts)** to securely map Kubernetes service accounts to IAM roles, ensuring that only authorized workloads can assume specific AWS permissions.

```
resource "aws_iam_role" "irsa_role" {
  name = "${var.prefix}-irsa-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = {
          Federated = aws_iam_openid_connect_provider.eksopidc.arn
        }
        Action = ["sts:AssumeRoleWithWebIdentity"]
        Condition = {
          StringEquals = {
            "${replace(aws_iam_openid_connect_provider.eksopidc.url, "https://", "")}:sub" = ["system:serviceaccount:producer-consumer:msk-sa", "system:serviceaccount:keda:keda-operator"],
            "${replace(aws_iam_openid_connect_provider.eksopidc.url, "https://", "")}:aud" = ["sts.amazonaws.com"]
          }
        }
      }
    ]
  })
}
```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

helm repo update

helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace

helm upgrade --install kafka-exporter prometheus-community/prometheus-kafka-exporter -n monitoring --set kafkaServer[0]=b-1.awsmskcluster.XXXXXX.XX.kafka.us-east-2.amazonaws.com:9098 --set kafkaServer[1]=b-2.awsmskcluster.XXXXXX.XX.kafka.us-east-2.amazonaws.com:9098 --set kafkaServer[2]=b-3.awsmskcluster.XXXXXX.XX.kafka.us-east-2.amazonaws.com:9098 --set sasl.enabled=true --set sasl.securityProtocol=SASL_SSL --set serviceMonitor.enabled=true --set extraArgs[0]="--sasl.mechanism=awsiam" --set extraArgs[1]="--tls.enabled" --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"="arn:aws:iam::02XXXXXXXXXX6:role/aws-irsa-role"

```
[k8s-management@k8s-management ~]$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
[k8s-management@k8s-management ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "keda-core" chart repository
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. 🎉Happy Helming!🎉
[k8s-management@k8s-management ~]$ helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace
```

```
[k8s-management@k8s-management ~]$ helm upgrade --install kafka-exporter prometheus-community/prometheus-kafka-exporter -n monitoring --set kafkaServer[0]=b-1.awsmskcluster.XXXXXX.XX.kafka.us-east-2.amazonaws.com:9098 --set kafkaServer[1]=b-2.awsmskcluster.XXXXXX.XX.kafka.us-east-2.amazonaws.com:9098 --set kafkaServer[2]=b-3.awsmskcluster.XXXXXX.XX.kafka.us-east-2.amazonaws.com:9098 --set sasl.enabled=true --set sasl.securityProtocol=SASL_SSL --set serviceMonitor.enabled=true --set extraArgs[0]="--sasl.mechanism=awsiam" --set extraArgs[1]="--tls.enabled" --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"="arn:aws:iam::027330342406:role/aws-irsa-role"
```

```
[k8s-management@k8s-management ~]$ kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-kube-prometheus-stack-alertmanager-0	2/2	Running	0	4m19s
kafka-exporter-prometheus-kafka-exporter-XXXXXX	1/1	Running	0	40s
kube-prometheus-stack-grafana-XXXXXX	3/3	Running	0	4m22s
kube-prometheus-stack-kube-state-metrics-XXXXXX	1/1	Running	0	4m22s
kube-prometheus-stack-operator-XXXXXX	1/1	Running	0	4m22s
kube-prometheus-stack-prometheus-node-exporter-XXXXXX	1/1	Running	0	3m57s
kube-prometheus-stack-prometheus-node-exporter-XXXXXX	0/1	Pending	0	4m22s
kube-prometheus-stack-prometheus-node-exporter-XXXXXX	0/1	Pending	0	4m22s
kube-prometheus-stack-prometheus-node-exporter-XXXXXX	0/1	Pending	0	4m22s
kube-prometheus-stack-prometheus-node-exporter-XXXXXX	1/1	Running	0	4m22s
prometheus-kube-prometheus-stack-prometheus-0	2/2	Running	0	4m18s


```
[k8s-management@k8s-management ~]$ kubectl get svc -n monitoring
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP,9094/TCP,9094/UDP	5m12s
kafka-exporter-prometheus-kafka-exporter	ClusterIP	10.100.118.48	<none>	9308/TCP	93s
kube-prometheus-stack-alertmanager	ClusterIP	10.100.80.53	<none>	9093/TCP,8080/TCP	5m15s
kube-prometheus-stack-grafana	ClusterIP	10.100.28.98	<none>	80/TCP	5m15s
kube-prometheus-stack-kube-state-metrics	ClusterIP	10.100.63.20	<none>	8080/TCP	5m15s
kube-prometheus-stack-operator	ClusterIP	10.100.40.90	<none>	443/TCP	5m15s
kube-prometheus-stack-prometheus	ClusterIP	10.100.218.62	<none>	9090/TCP,8080/TCP	5m15s
kube-prometheus-stack-prometheus-node-exporter	ClusterIP	10.100.93.179	<none>	9100/TCP	5m15s
prometheus-operated	ClusterIP	None	<none>	9090/TCP	5m12s

Then create Gateway-API for Grafana using the kubernetes manifests file present at the path **module-3/Section-1/gateway-api-grafana.yaml** in GitHub Repo

<https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>.

Before applying this Yaml Manifests file created Kubernetes Secrets to Access the URL with HTTPS Protocol as shown in the screenshot attached below.

```
kubectl create secret tls tls-grafana-ingress --cert=STAR_singhritesh85_com.crt --key=mykey.key -n monitoring
```

```
[k8s-management@k8s-management ~]$ kubectl apply -f gateway-api-grafana.yaml
gateway.gateway.networking.k8s.io/grafana-gateway created
httproute.gateway.networking.k8s.io/grafana-route created
[k8s-management@k8s-management ~]$ kubectl get gateway -n monitoring
```

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
grafana-gateway	nginx	a [REDACTED] 5.us-east-2.elb.amazonaws.com	True	66s

```
[k8s-management@k8s-management ~]$ kubectl get httproute -n monitoring
```

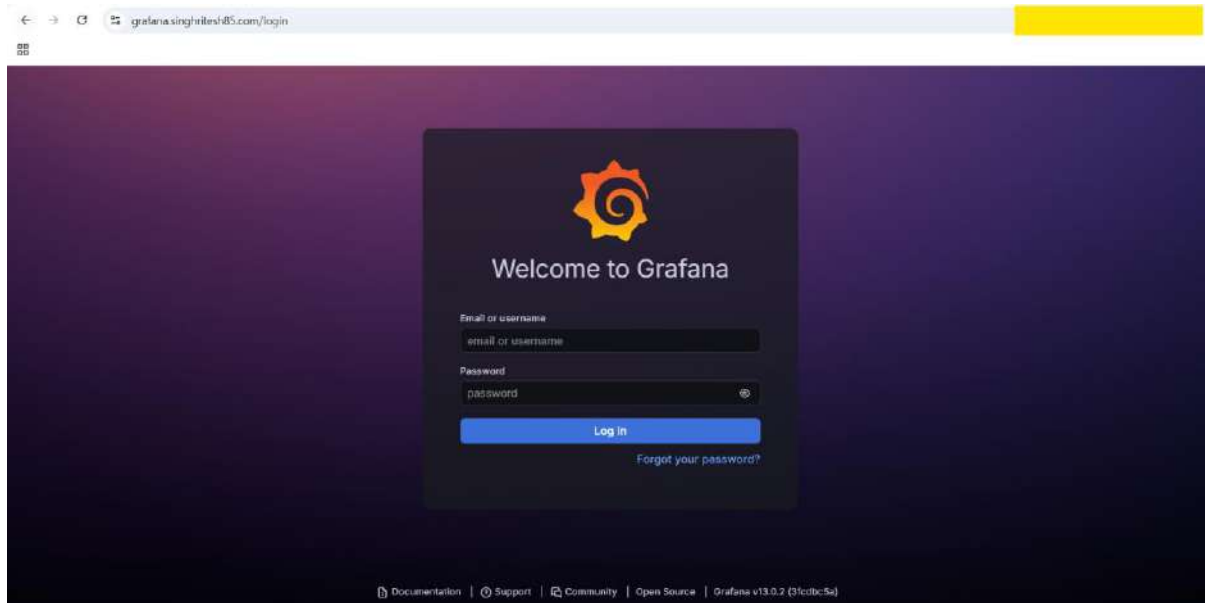
NAME	HOSTNAMES	AGE
grafana-route	["grafana.singhritesh85.com"]	76s

Did the entry in Route53 to create the Record-Set of A-Type with Alias using URL as obtained, which is shown in the screenshot attached above.

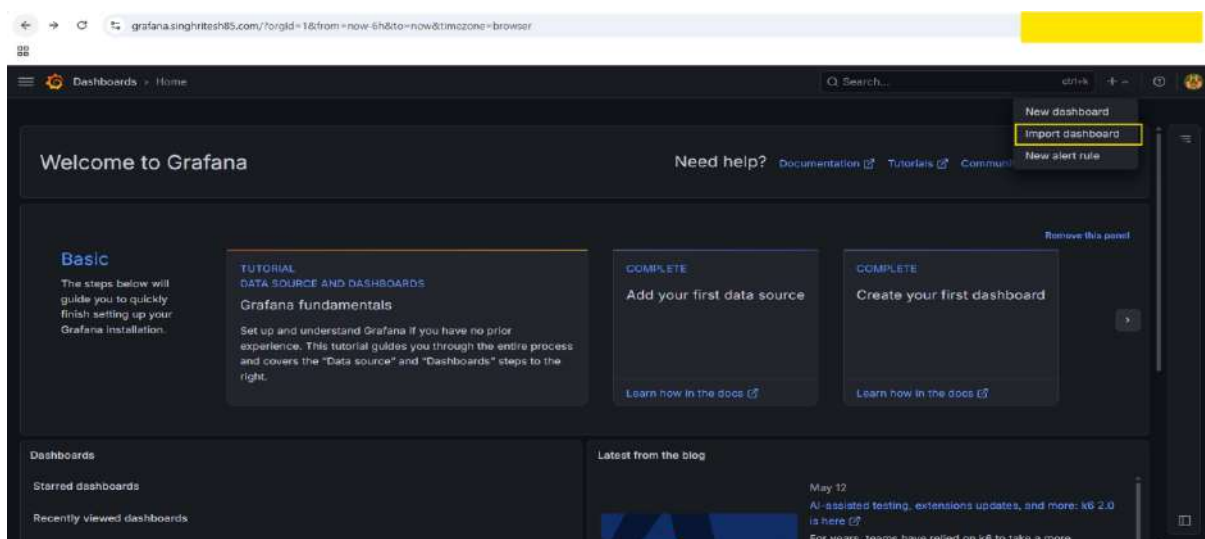
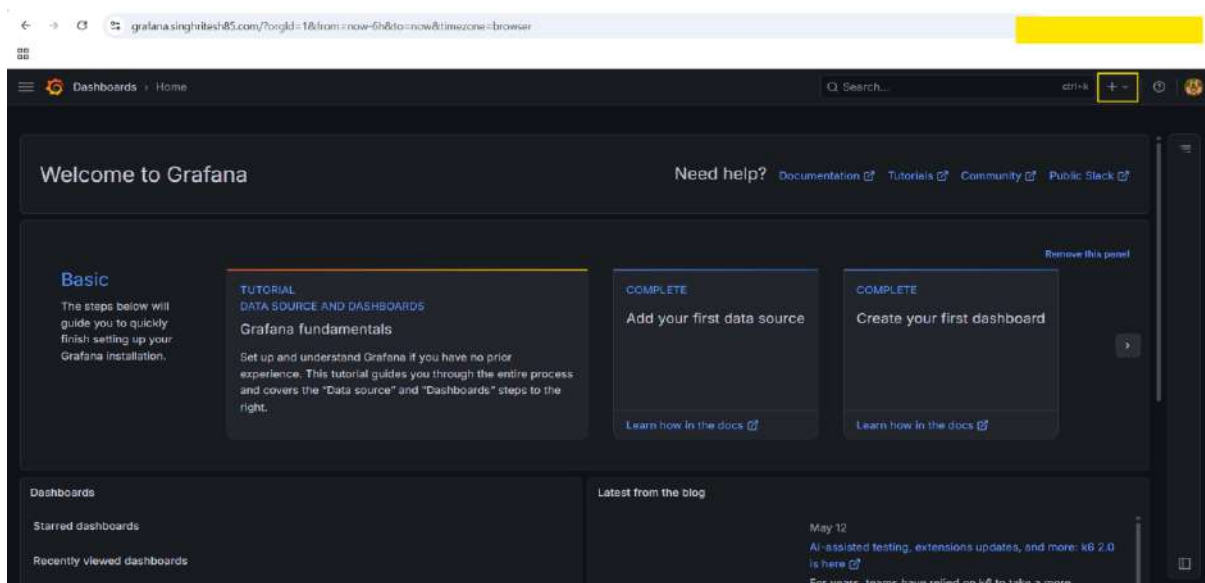
Finally, I was able to access Grafana as shown in the screenshot attached below. You can get the password to login into Grafana Dashboard using the admin user s shown in the screenshot attached below.

```
kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 --decode; echo
```

```
[k8s-management@k8s-management ~]$ kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 --decode; echo
f [REDACTED]
```

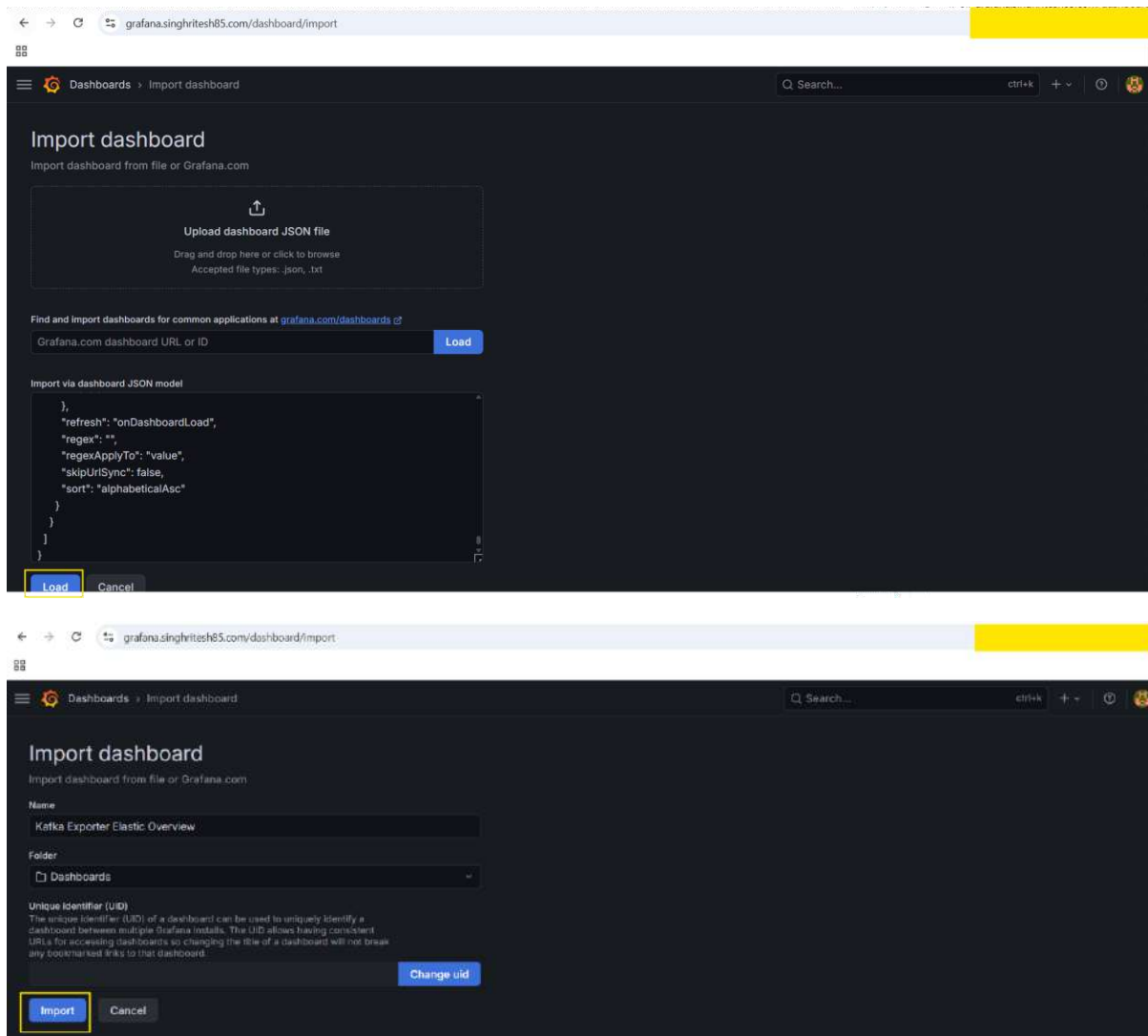



Finally, I was able to Login into Grafana Dashboard, then, I created the dashboard in Grafana by importing the Grafana JSON model, as shown in the screenshot below.



Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>

For your reference I kept the Grafana JSON Model at the path **module-3/Section-1/grafana-json-model** in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>.



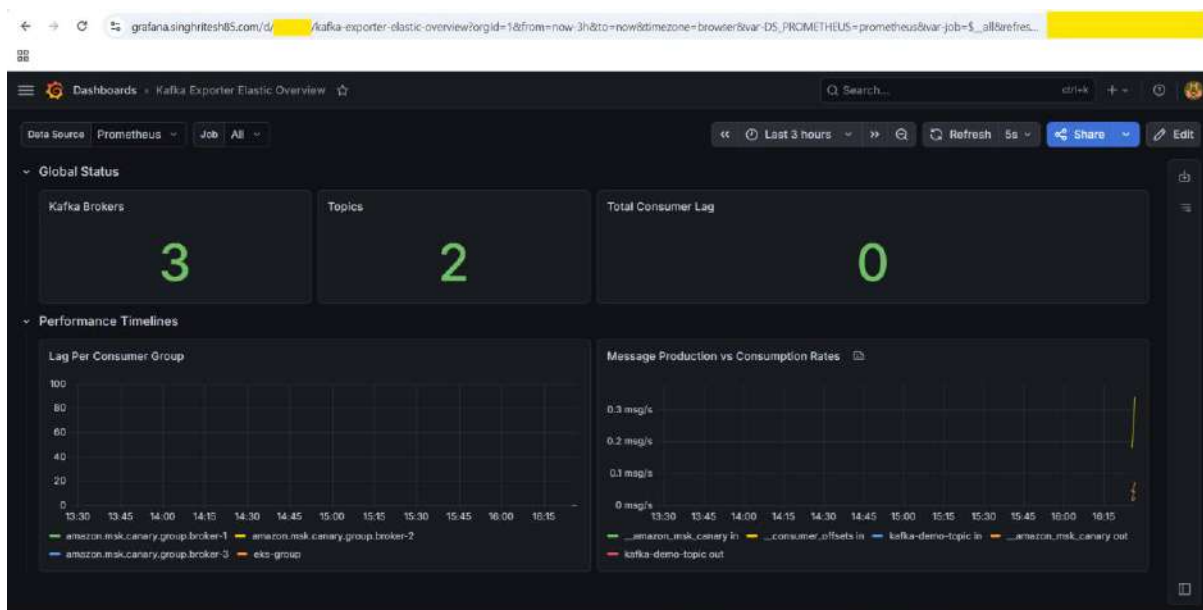
Then applied the kubernetes Manifests file for service monitor as shown in the screenshot attached below.

```
[k8s-management@k8s-management ~]$ cat kafka-exporter-sm.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: kafka-exporter-custom
  namespace: monitoring
  labels:
    release: kube-prometheus-stack
spec:
  selector:
    matchLabels:
      app: prometheus-kafka-exporter
  endpoints:
    - port: exporter-port
      interval: 30s
      path: /metrics
```

```
[k8s-management@k8s-management ~]$ kubectl apply -f kafka-exporter-sm.yaml
servicemonitor.monitoring.coreos.com/kafka-exporter-custom created
[k8s-management@k8s-management ~]$ kubectl get servicemonitor -n monitoring
```

NAME	AGE
kafka-exporter-custom	16s
kube-prometheus-stack-alertmanager	40m
kube-prometheus-stack-apiserver	40m
kube-prometheus-stack-coredns	40m
kube-prometheus-stack-grafana	40m
kube-prometheus-stack-kube-controller-manager	40m
kube-prometheus-stack-kube-etcd	40m
kube-prometheus-stack-kube-proxy	40m
kube-prometheus-stack-kube-scheduler	40m
kube-prometheus-stack-kube-state-metrics	40m
kube-prometheus-stack-kubelet	40m
kube-prometheus-stack-operator	40m
kube-prometheus-stack-prometheus	40m
kube-prometheus-stack-prometheus-node-exporter	40m

Checked after 2 minutes after applying the kubernetes manifest file for system monitor, the Grafana Dashboard is as shown in the screenshot attached below.



GitHub Repo: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>

Terraform Script: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> path:- module-3/Section-1

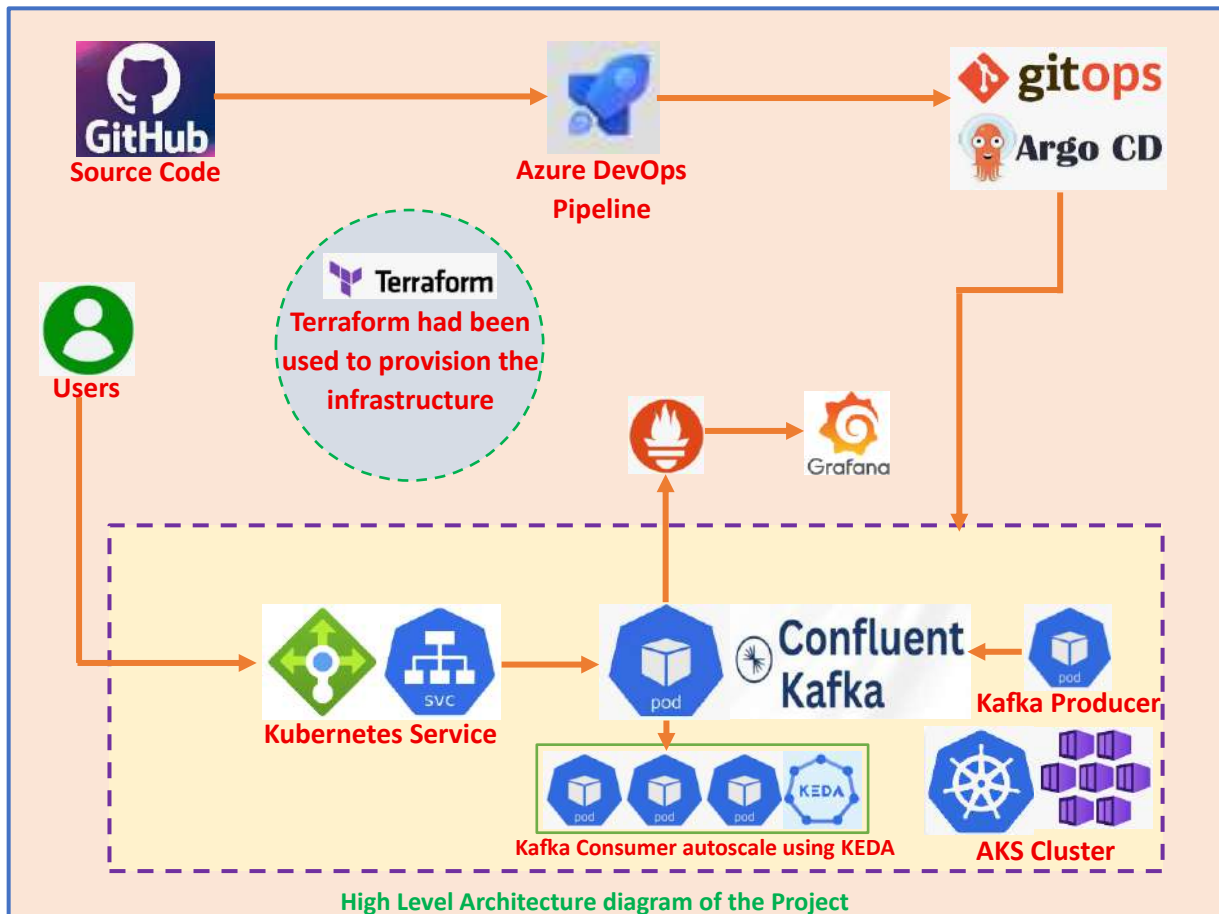
Helm Chart: - <https://github.com/singhritesh85/helm-repo-for-msk-keda.git>

Kustomize Repo: - <https://github.com/singhritesh85/aws-msk-producer-consumer.git>

Dockerfile: - <https://github.com/singhritesh85/aws-msk-producer-consumer.git>

Module-3

Section-2: - DevOps Project using Confluent Kafka as Message Queue with GitOps



This DevOps Project involves creation of infrastructure using Terraform, the Terraform script to provision the infrastructure is kept in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-3/Section-2/terraform-private-aks-azure-vm-with-backups**. Kubernetes Manifests files and YAML files to configure CI/CD Azure DevOps Pipelines for this project was kept in the GitHub Repo <https://github.com/singhritesh85/confluent-kubernetes.git>. In this project for Kafka, Confluent kafka had been used which was deployed using Azure DevOps Pipeline through ArgoCD. Producer and Consumer for Kafka was deployed using Azure DevOps Pipeline through ArgoCD. In this project for CI/CD Pipeline Azure DevOps Pipeline had been used and for GitOps ArgoCD had been used. KEDA was used to autoscale Kafka consumers based on consumer lag. Prometheus, configured with Service Monitor, was used to scrape metrics from Confluent Kafka, and Grafana was used to visualize those metrics.

Commands as written below had been used to provision the infrastructure using Terraform.

terraform init -----> initializes a working directory containing configuration files and installs plugins for required providers.

terraform validate -----> verify that terraform configuration file is correct or not

terraform plan -----> Check which resources are going to be created.

Then you can run the command **terraform apply -auto-approve** -----> Finally, Created the resources.

```
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [09m50s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [10m00s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [10m10s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [10m20s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [10m30s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [10m40s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [10m50s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [11m00s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [11m10s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [11m20s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [11m30s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Still creating... [11m40s elapsed]
module.aks_confluent.azure_rm_kubernetes_cluster_extension.aks_cluster_extension: Creation complete after 11m46s [id=/subscriptions/5[REDACTED]
/resourceGroups/confluent-aks-rg/providers/Microsoft.ContainerService/managedClusters/confluent-aks-cluster/providers/Microsoft.KubernetesConfigurat
on/extensions/confluent-aks-extension]
module.aks_confluent.azure_rm_role_assignment.backup_vault_data_contributor_on_storage: Creating...
module.aks_confluent.azure_rm_role_assignment.backup_extension_and_storage_account_permission: Creating...
module.aks_confluent.azure_rm_role_assignment.backup_vault_data_contributor_on_storage: Still creating... [00m10s elapsed]
module.aks_confluent.azure_rm_role_assignment.backup_extension_and_storage_account_permission: Still creating... [00m10s elapsed]
module.aks_confluent.azure_rm_role_assignment.backup_vault_data_contributor_on_storage: Still creating... [00m20s elapsed]
module.aks_confluent.azure_rm_role_assignment.backup_extension_and_storage_account_permission: Still creating... [00m20s elapsed]
module.aks_confluent.azure_rm_role_assignment.backup_vault_data_contributor_on_storage: Creation complete after 26s [id=/subscriptions/5[REDACTED]
/resourceGroups/confluent-aks-rg/providers/Microsoft.Storage/storageAccounts/confluentaksbackupsa2026/providers/Microsoft.Authorization/roleAssign
ments/[REDACTED]]
module.aks_confluent.azure_rm_role_assignment.backup_extension_and_storage_account_permission: Creation complete after 27s [id=/subscriptions/5[REDACTED]
/resourceGroups/confluent-aks-rg/providers/Microsoft.Storage/storageAccounts/confluentaksbackupsa2026/providers/Microsoft.Authorization/rol
eAssignments/[REDACTED]]
module.aks_confluent.azure_rm_data_protection_backup_instance_kubernetes_cluster.aks_backup_instance: Creating...
module.aks_confluent.azure_rm_data_protection_backup_instance_kubernetes_cluster.aks_backup_instance: Still creating... [00m10s elapsed]
module.aks_confluent.azure_rm_data_protection_backup_instance_kubernetes_cluster.aks_backup_instance: Creation complete after 17s [id=/subscriptions/5[REDACTED]
/resourceGroups/confluent-aks-rg/providers/Microsoft.DataProtection/backupVaults/confluent-aks-backup-vault/backupInstances/confluen
t-aks-backup-instance]

Apply complete! Resources: 46 added, 0 changed, 0 destroyed.

Outputs:
aks_k8s_management_and_azure_vm_instance_details = <sensitive>
```

```
[root@Terraform-Server main]# terraform output -json
{
  "aks_k8s_management_and_azure_vm_instance_details": {
    "sensitive": true,
    "type": [
      "object",
      {
        "aks_cluster_endpoint": "string",
        "aks_cluster_name": "string",
        "azure_dns_zone_name": "string",
        "azure_dns_zone_nameservers": [
          "set",
          "string"
        ],
        "k8s_management_instance_ip": "string",
        "k8s_management_public_ip": "string"
      }
    ],
    "value": {
      "aks_cluster_endpoint": "",
      "aks_cluster_name": "confluent-aks-cluster",
      "azure_dns_zone_name": "singhritesh85.com",
      "azure_dns_zone_nameservers": [
        "[REDACTED]",
        "[REDACTED]",
        "[REDACTED]",
        "[REDACTED]"
      ],
      "k8s_management_instance_ip": "10.224.0.4",
      "k8s_management_public_ip": "20.[REDACTED]"
    }
  }
}
```


After creating the resources, update the name server records at your domain registrar with the name servers shown in the screenshot above.

To install ArgoCD and to change the password of admin user you can refer the project present in **module-3/Section-1** in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>.

In this project our first aim is to deploy confluent kafka 7.9.0 then to create kafka topic and producer-consumer. Then upgraded Confluent-Kafka-7.9.0 to Confluent-Kafka-8.1.3 followed by deployment of Prometheus-Grafana and service monitor then deployment of keda to autoscale kafka consumer as stated above. As explained earlier in this project Azure DevOps Pipeline (with Self-Hosted Agent) had been used as CI/CD Tool in this Project. Below screenshot shows, how to configure the Self-Hosted Agent for Azure DevOps Pipeline.

```
[root@demo-azure-vm ~]# cd /opt/
[root@demo-azure-vm opt]# mkdir myagent && cd myagent
[root@demo-azure-vm myagent]# wget https://download.agent.dev.azure.com/agent/5.275.0/vsts-agent-linux-x64-5.275.0.tar.gz
--2026-07-09 07:30:37-- https://download.agent.dev.azure.com/agent/5.275.0/vsts-agent-linux-x64-5.275.0.tar.gz
Resolving download.agent.dev.azure.com (download.agent.dev.azure.com)... 
Connecting to download.agent.dev.azure.com (download.agent.dev.azure.com)|... connected.
HTTP request sent, awaiting response... 200 OK
Length: 216871406 (207M) [application/octet-stream]
Saving to: 'vsts-agent-linux-x64-5.275.0.tar.gz'

vsts-agent-linux-x64-5.275.0.tar.gz 100%[=====] 206.82M 192MB/s in 1.1s

2026-07-09 07:30:38 (192 MB/s) - 'vsts-agent-linux-x64-5.275.0.tar.gz' saved [216871406/216871406]

[root@demo-azure-vm myagent]# tar -xvf vsts-agent-linux-x64-5.275.0.tar.gz
```

```
[root@demo-azure-vm myagent]# rm -f vsts-agent-linux-x64-5.275.0.tar.gz
[root@demo-azure-vm myagent]# chown -R demo:demo /opt/myagent/
[root@demo-azure-vm myagent]# logout
[demo@demo-azure-vm ~]$ cd /opt/myagent/
[demo@demo-azure-vm myagent]$ ./config.sh
```



```
Azure Pipelines
agent v5.275.0 (commit 19f077a)

>> End User License Agreements:
Building sources from a TFVC repository requires accepting the Team Explorer Everywhere End User License Agreement. This step is not required for building sources from Git repositories.
A copy of the Team Explorer Everywhere license agreement can be found at:
/opt/myagent/license.html
Enter (Y/N) Accept the Team Explorer Everywhere license agreement now? (press enter for N) > N

>> Connect:
Enter server URL > https://dev.azure.com/
Enter authentication type (press enter for PAT) >
Enter personal access token >
Connecting to server ...

>> Register Agent:
```

```
Enter agent pool (press enter for default) > demo
Enter agent name (press enter for demo-azure-vm) > demo
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) >
2026-07-09 07:37:21Z: Settings Saved.
```

```
[demo@demo-azure-vm myagent]$ sudo ./svc.sh install
Creating launch agent in /etc/systemd/system/vsts.agent. [redacted].demo.demo.service
Run as user: demo
Run as uid: 1001
gid: 1001
/sbin/sestatus
SELinux status: enabled
Created symlink /etc/systemd/system/multi-user.target.wants/vsts.agent. [redacted].demo.demo.service → /etc/systemd/system/vsts.agent. [redacted].demo.demo.service.

[demo@demo-azure-vm myagent]$ sudo ./svc.sh start
/etc/systemd/system/vsts.agent. [redacted].demo.demo.service
• vsts.agent. [redacted].demo.demo.service - Azure Pipelines Agent ([redacted].demo.demo)
  Loaded: loaded (/etc/systemd/system/vsts.agent. [redacted].demo.demo.service; enabled; vendor preset: disabled)
  Active: active (running) since Thu 2026-07-09 07:39:20 UTC; 16ms ago
  Main PID: 16899 (runsvch)
  Tasks: 2 (limit: 24588)
  Memory: 1.0M
  CGroup: /system.slice/vsts.agent. [redacted].demo.demo.service
          └─16899 /bin/bash /opt/myagent/runsvch
          └─16905 /bin/bash /opt/myagent/runsvch

Jul 09 07:39:20 demo-azure-vm systemd[1]: Started Azure Pipelines Agent ([redacted].demo.demo).
Jul 09 07:39:20 demo-azure-vm runsvch[16899]: .pathw/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/bin:/opt/apache-maven/bin:/opt/node-v16.0.0/local/bin
Jul 09 07:39:20 demo-azure-vm runsvch[16902]: v20.20.2
Hint: Some lines were ellipsized, use -l to show in full.
```

```
[demo@demo-azure-vm myagent]$ ./env.sh
```

At this stage, Azure DevOps Agent (Self-Hosted) had been configured and in online state. Which was verified with the below screenshot.

demo					Update all agents	New agent
Jobs Agents Details Security Settings Maintenance History Analytics						
Name	Last run	Current status	Agent version	Enabled		
demo ● Online		Idle	5.275.0	☒ On		

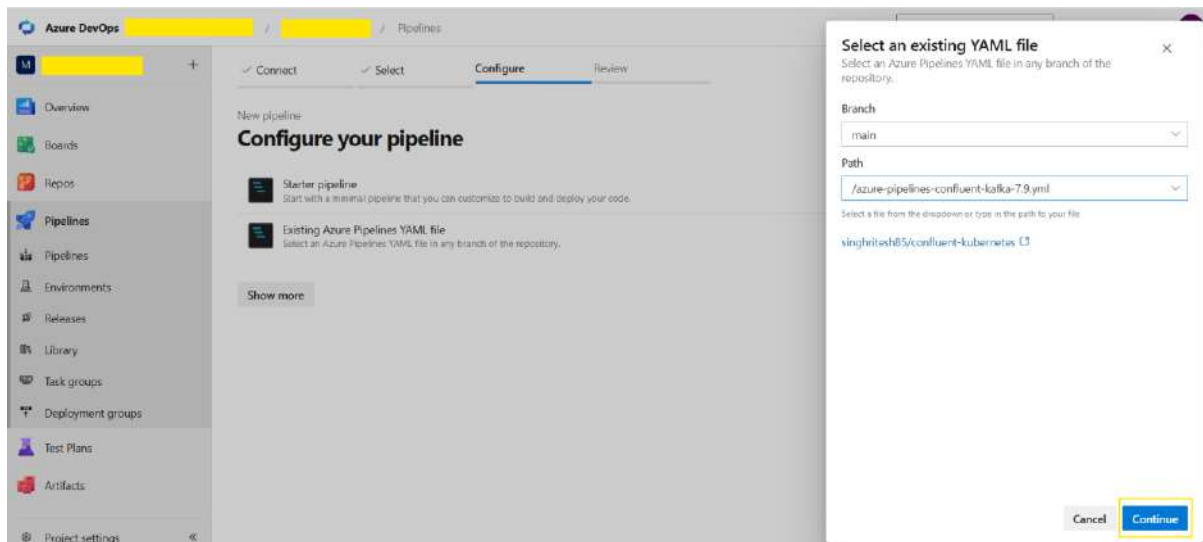
Then configured the Azure DevOps Pipeline to deploy the Confluent-Kafka-7.9.0.

The screenshot shows the Azure DevOps web interface. On the left, a sidebar contains a navigation menu with items like 'Overview', 'Boards', 'Repos', 'Pipelines' (highlighted), 'Environments', 'Releases', 'Library', 'Task groups', 'Deployment groups', 'Test Plans', and 'Artifacts'. The main content area features a large illustration of a robot and a person working on a laptop, with the heading 'Create your first Pipeline'. Below this, a message states: 'Automate your build and release processes using our wizard, and go from code to cloud-hosted within minutes.' A prominent blue button labeled 'Create Pipeline' is centered at the bottom of this section.

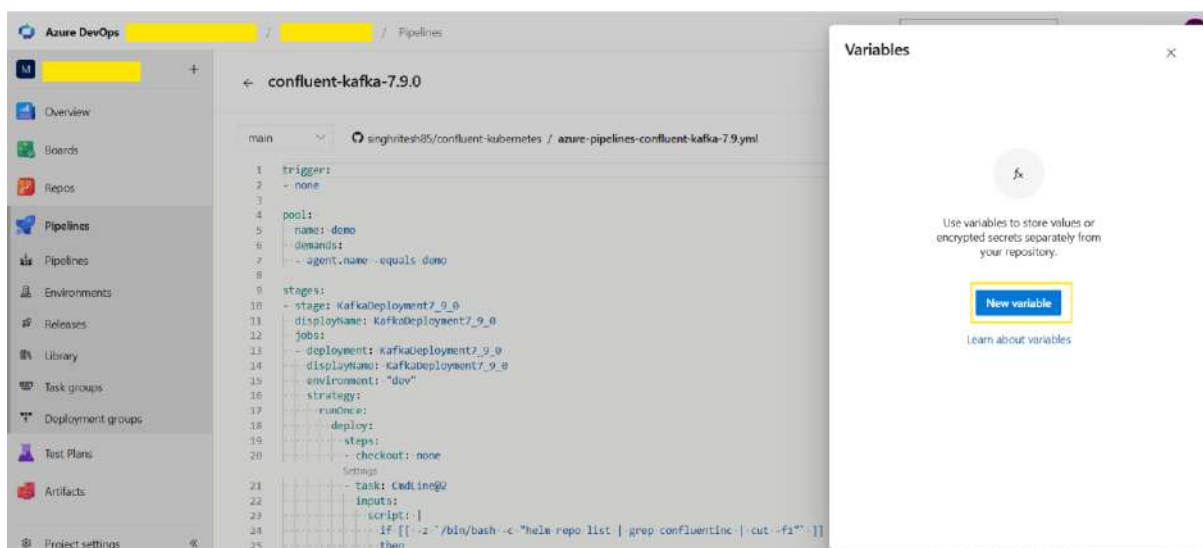
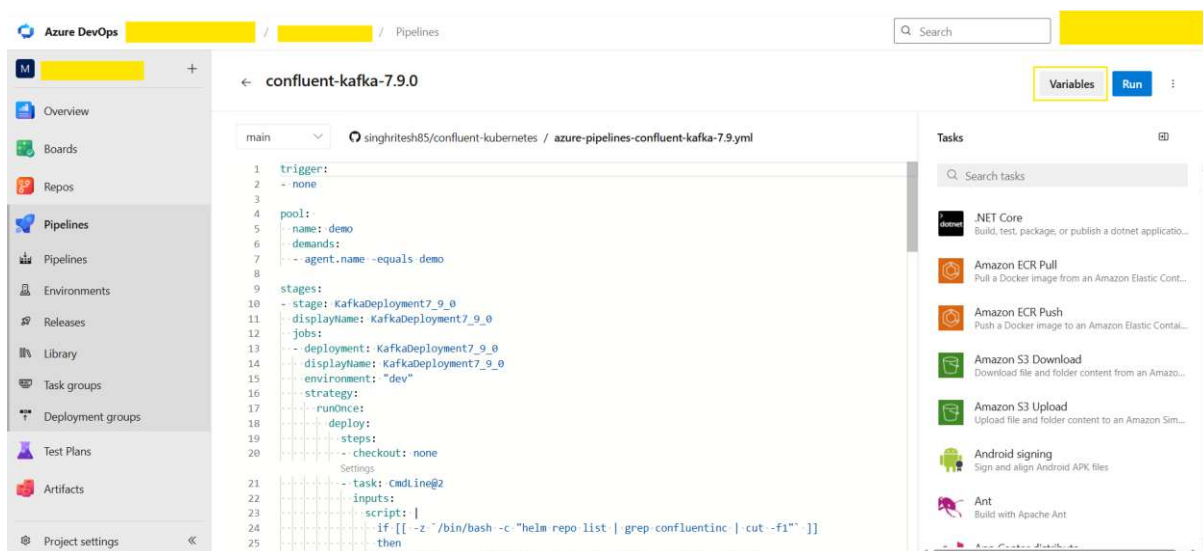
This screenshot shows the 'Select a repository' step in the Azure DevOps pipeline configuration interface. The left sidebar contains navigation links for Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, Artifacts, and Project settings. The main area has tabs for Connect, Select (active), Configure, and Review. Under the 'Select' tab, the 'New pipeline' section is titled 'Select a repository'. A search bar with the text 'Filter by keywords' and a dropdown for 'My repositories' is present. Below this, a list of repositories is displayed, including 'singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way', 'singhritesh85/confluent-kubernetes' (highlighted with a yellow box), 'singhritesh85/helm-repo-for-msk-keda', 'singhritesh85/aws-msk-producer-consumer', 'singhritesh85/DevOps-Project-Netflix-Clone-Aws', 'singhritesh85/Kubernetes-headlamp-and-k9s', 'singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator', and 'singhritesh85/strimzi-kafka-demo'.

This screenshot shows the 'Configure your pipeline' step in the Azure DevOps pipeline configuration interface. The left sidebar is the same as the previous screenshot. The main area has tabs for Connect, Select, Configure (active), and Review. Under the 'Configure' tab, the 'New pipeline' section is titled 'Configure your pipeline'. It offers two options: 'Starter pipeline' (described as a minimal pipeline for customization) and 'Existing Azure Pipelines YAML file' (described as selecting an existing YAML file from a repository). The 'Existing Azure Pipelines YAML file' option is highlighted with a yellow box. A 'Show more' button is located at the bottom of the options.

This screenshot shows the 'Select an existing YAML file' dialog box overlaid on the 'Configure your pipeline' step. The dialog has a title bar with a close button. It contains a 'Branch' dropdown set to 'main' and a 'Path' dropdown. The 'Path' dropdown is open, showing a list of files including '/azure-pipelines-confluent-kafka-7.9.yml' (highlighted with a yellow box), '/azure-pipelines-confluent-kafka-7.0.yml', '/azure-pipelines-confluent-kafka-8.1.3.yml', '/azure-pipelines-confluent-kafka-topic.yml', '/azure-pipelines-keda-triggerauthentication-scaledobject.yml', '/azure-pipelines-prometheus-grafana-confluent-kafka-service-monitor.yml', '/confluent-kafka-7.9.0/confluent-kafka-7.9.0.yml', '/confluent-kafka-8.1.3/confluent-kafka-8.1.3.yml', '/confluent-kafka-producer-consumer/confluent-kafka-producer-consumer.yml', '/confluent-kafka-topic/confluent-kafka-topic.yml', '/keda-triggerauthentication-scaledobject/trigerauthentication-scaledobject.yml', and '/service-monitor/service-monitor-alertmanager.yml'.



Before deploying Azure DevOps Pipeline created three variables as shown in the screenshot attached below.



The screenshot shows the Azure DevOps Pipelines interface. On the left, the 'Pipelines' tab is selected. The main area displays a YAML file named 'confluent-kafka-7.9.0'. The YAML content is as follows:

```

1 trigger:
2   - none
3
4 pool:
5   - name: demo
6     demands:
7       - agent.name -equals demo
8
9 stages:
10  - stage: KafkaDeployment7_9_0
11    displayName: KafkaDeployment7_9_0
12    jobs:
13      - deployment: KafkaDeployment7_9_0
14        displayName: KafkaDeployment7_9_0
15        environment: "dev"
16        strategy:
17          runOnce:
18            deploy:
19              steps:
20                - checkout: none
21                  settings:
22                    task: Outline@2
23                    inputs:
24                      script: |
25                        if [[ -z "/bin/bash -c 'helm repo list | grep confluentinc | cut -f1'" ]]
26                          then

```

On the right, the 'New variable' dialog box is open. The 'Name' field is 'ARGOCD_PASSWORD' and the 'Value' field is empty. The 'Keep this value secret' checkbox is checked. The dialog includes instructions on how to reference variables in YAML and scripts, and examples for using variables in different scripts.

The screenshot shows the Azure DevOps Pipelines interface. On the left, the 'Pipelines' tab is selected. The main area displays a YAML file named 'confluent-kafka-7.9.0'. The YAML content is as follows:

```

1 trigger:
2   - none
3
4 pool:
5   - name: demo
6     demands:
7       - agent.name -equals demo
8
9 stages:
10  - stage: KafkaDeployment7_9_0
11    displayName: KafkaDeployment7_9_0
12    jobs:
13      - deployment: KafkaDeployment7_9_0
14        displayName: KafkaDeployment7_9_0
15        environment: "dev"
16        strategy:
17          runOnce:
18            deploy:
19              steps:
20                - checkout: none
21                  settings:
22                    task: Outline@2
23                    inputs:
24                      script: |
25                        if [[ -z "/bin/bash -c 'helm repo list | grep confluentinc | cut -f1'" ]]
26                          then

```

On the right, the 'New variable' dialog box is open. The 'Name' field is 'GITHUB_PASSWORD' and the 'Value' field is empty. The 'Keep this value secret' checkbox is checked. The dialog includes instructions on how to reference variables in YAML and scripts, and examples for using variables in different scripts.

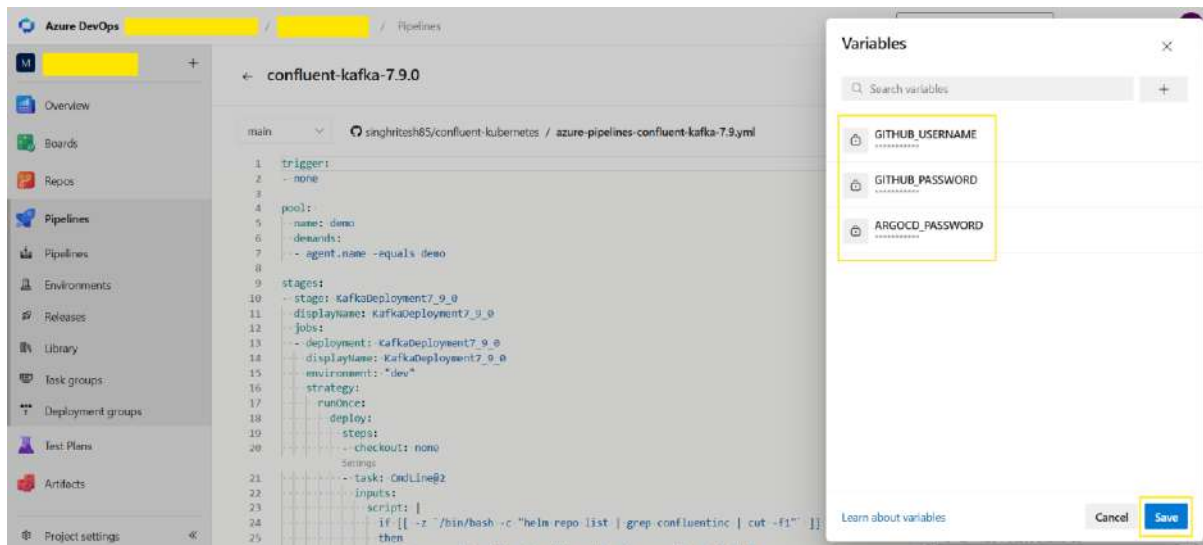
The screenshot shows the Azure DevOps Pipelines interface. On the left, the 'Pipelines' tab is selected. The main area displays a YAML file named 'confluent-kafka-7.9.0'. The YAML content is as follows:

```

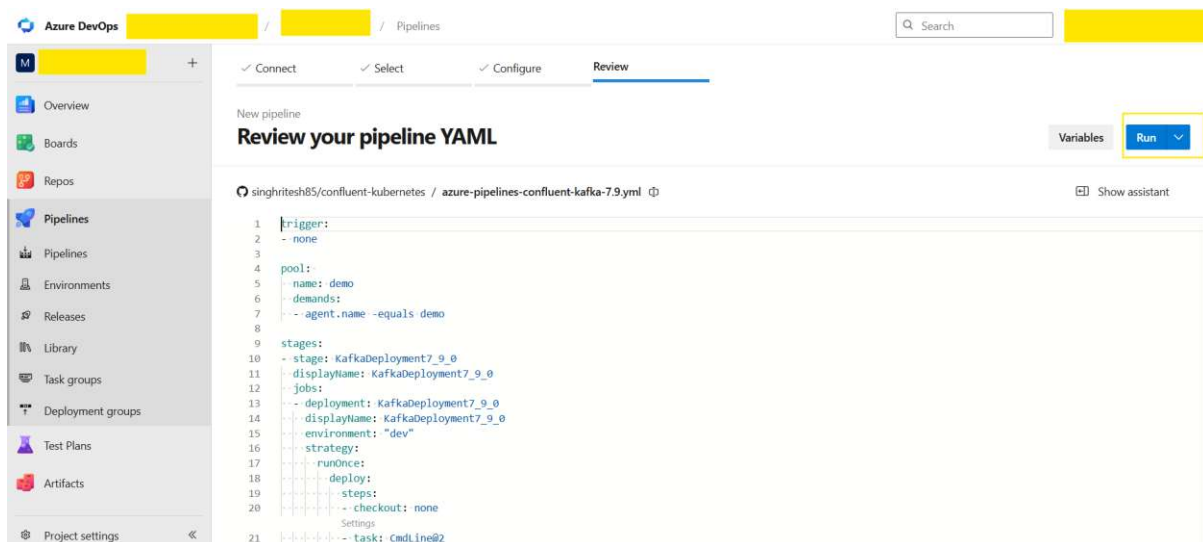
1 trigger:
2   - none
3
4 pool:
5   - name: demo
6     demands:
7       - agent.name -equals demo
8
9 stages:
10  - stage: KafkaDeployment7_9_0
11    displayName: KafkaDeployment7_9_0
12    jobs:
13      - deployment: KafkaDeployment7_9_0
14        displayName: KafkaDeployment7_9_0
15        environment: "dev"
16        strategy:
17          runOnce:
18            deploy:
19              steps:
20                - checkout: none
21                  settings:
22                    task: Outline@2
23                    inputs:
24                      script: |
25                        if [[ -z "/bin/bash -c 'helm repo list | grep confluentinc | cut -f1'" ]]
26                          then

```

On the right, the 'New variable' dialog box is open. The 'Name' field is 'GITHUB_PASSWORD' and the 'Value' field is empty. The 'Keep this value secret' checkbox is checked. The dialog includes instructions on how to reference variables in YAML and scripts, and examples for using variables in different scripts.



Finally, Run the Azure DevOps Pipeline as shown in the screenshot attached below.



After successfully Running the Pipeline, the screenshot is as shown below.



After successful execution of Azure DevOps Pipeline Pods and Services had been created in the namespace **confluent** as shown below.

```
[demo@demo-azure-vm ~]$ kubectl get pods -n confluent
```

NAME	READY	STATUS	RESTARTS	AGE
confluent-operator-f66c6465d-jnpch	1/1	Running	0	92m
connect-0	1/1	Running	0	4m54s
controlcenter-0	1/1	Running	3 (4m49s ago)	8m2s
kafka-0	1/1	Running	0	5m1s
kafka-1	1/1	Running	0	5m1s
kafka-2	1/1	Running	0	5m1s
kafkarestproxy-0	1/1	Running	0	5m31s
kraftcontroller-0	1/1	Running	0	8m1s
kraftcontroller-1	1/1	Running	0	110s
kraftcontroller-2	1/1	Running	0	4m53s
ksqldb-0	1/1	Running	4 (3m29s ago)	8m1s
ksqldb-1	1/1	Running	4 (3m28s ago)	8m1s
schemaregistry-0	1/1	Running	1 (3m1s ago)	8m1s

```
[demo@demo-azure-vm ~]$ kubectl get svc -n confluent
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
confluent-operator	ClusterIP	10.0.16.244	<none>	7778/TCP	94m
connect	ClusterIP	None	<none>	8083/TCP,7203/TCP,7777/TCP,7778/TCP	9m47s
connect-0-internal	ClusterIP	10.0.97.125	<none>	8083/TCP,7203/TCP,7777/TCP,7778/TCP	9m47s
controlcenter	ClusterIP	None	<none>	9021/TCP,7203/TCP,7777/TCP,7778/TCP	9m47s
controlcenter-0-internal	ClusterIP	10.0.85.213	<none>	9021/TCP,7203/TCP,7777/TCP,7778/TCP	9m47s
controlcenter-bootstrap-lb	LoadBalancer	10.0.177.18	20.1.1.1	443:30567/TCP	9m47s
kafka	ClusterIP	None	<none>	9074/TCP,9092/TCP,8090/TCP,9071/TCP,7203/TCP,7777/TCP,7778/TCP,9072/TCP	6m46s
kafka-0-internal	ClusterIP	10.0.96.5	<none>	9074/TCP,9092/TCP,8090/TCP,9071/TCP,7203/TCP,7777/TCP,7778/TCP,9072/TCP	6m46s
kafka-1-internal	ClusterIP	10.0.191.52	<none>	9074/TCP,9092/TCP,8090/TCP,9071/TCP,7203/TCP,7777/TCP,7778/TCP,9072/TCP	6m45s
kafka-2-internal	ClusterIP	10.0.196.140	<none>	9074/TCP,9092/TCP,8090/TCP,9071/TCP,7203/TCP,7777/TCP,7778/TCP,9072/TCP	6m45s
kafkarestproxy	ClusterIP	None	<none>	8082/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
kafkarestproxy-0-internal	ClusterIP	10.0.29.136	<none>	8082/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
kraftcontroller	ClusterIP	None	<none>	9074/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
kraftcontroller-0-internal	ClusterIP	10.0.191.218	<none>	9074/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
kraftcontroller-1-internal	ClusterIP	10.0.162.52	<none>	9074/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
kraftcontroller-2-internal	ClusterIP	10.0.36.173	<none>	9074/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
ksqldb	ClusterIP	None	<none>	8088/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
ksqldb-0-internal	ClusterIP	10.0.166.125	<none>	8088/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
ksqldb-1-internal	ClusterIP	10.0.227.59	<none>	8088/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
schemaregistry	ClusterIP	None	<none>	8081/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s
schemaregistry-0-internal	ClusterIP	10.0.30.73	<none>	8081/TCP,7203/TCP,7777/TCP,7778/TCP	9m46s

Finally did the entry for External IP Address obtained from the Kubernetes Service **controlcenter-bootstrap-lb** as shown in the screenshot attached above in the Azure DNS Zone, to create the Record-Set of A-Type to access confluent kafka using the URL as shown in the screenshot attached below.

The screenshot shows the 'Add record set' form in the Azure Portal. The DNS zone is 'singhritesh85.com'. The record set name is 'confluent'. The type is 'A - IPv4 Address records'. The alias record set is 'No'. The TTL is '1' hour. The IP address is '20.1.1.1'. The 'Add' button is highlighted with a yellow box.

1 Healthy clusters 0 Unhealthy clusters

controlcenter.cluster

Running

Overview

Brokers	3
Partitions	---
Topics	58
Production	39.31KB/s
Consumption	29.51KB/s

Connected services

localDB-sql-entire	1
--------------------	---

Cluster overview

- Brokers
- Topics
- Connect
- ksqlDB
- Consumers
- Replicators
- Cluster settings

Health+

Brokers		
3	39.31K	29.51K
Total	Production (bytes / second)	Consumption (bytes / second)

Cluster overview

- Brokers
- Topics
- Connect
- ksqlDB
- Consumers
- Replicators
- Cluster settings

Health+

The dashboard displays four metrics in a grid layout. The 'Production' and 'Consumption' metrics are shown in light blue boxes, while 'Partitioning and replication' and 'KRaft leader' are in light orange boxes. Each metric includes a large numerical value, a unit, and a small green upward arrow indicating a positive trend.

Metric	Value	Unit
Production	39.58K	bytes / second
Consumption	30.15K	bytes / second
Partitioning and replication	58	Topics
	58	Partitions
	58	Replicas
KRaft leader	1	

Now, do the deployment to create Kafka-Topic and Kafka Producer-Consumer using the Azure DevOps Pipeline though ArgoCD as shown in the screenshot attached below.

The top screenshot shows the Azure DevOps 'Pipelines' page. A 'New pipeline' button is highlighted in the top right corner. Below it, a table lists recently run pipelines. The bottom screenshot shows the 'Where is your code?' step in a new pipeline. The 'GitHub' option is highlighted, indicating it is the selected code source.

Recently run pipelines

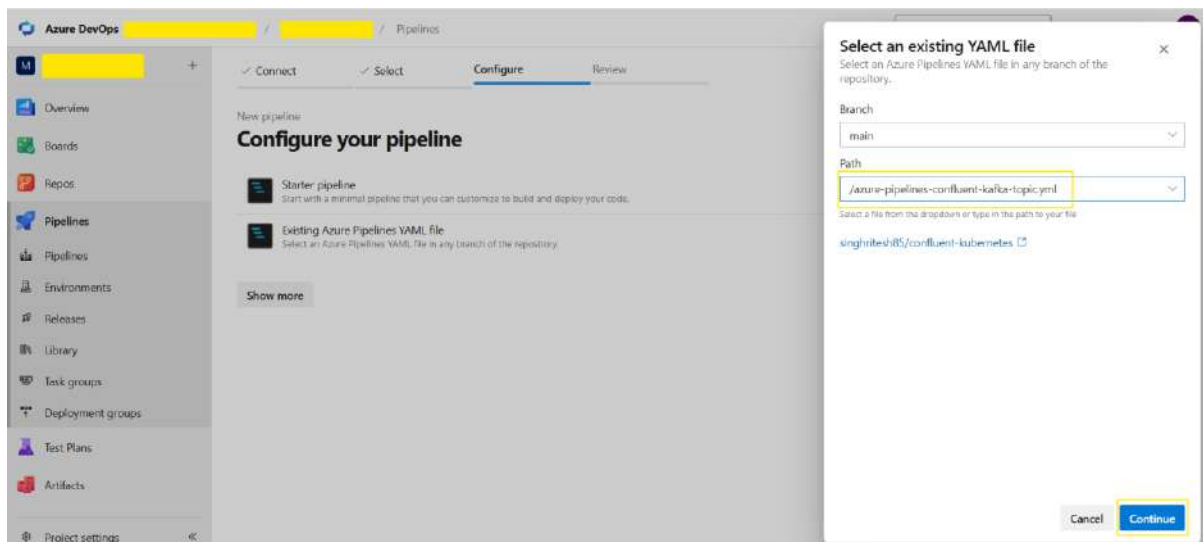
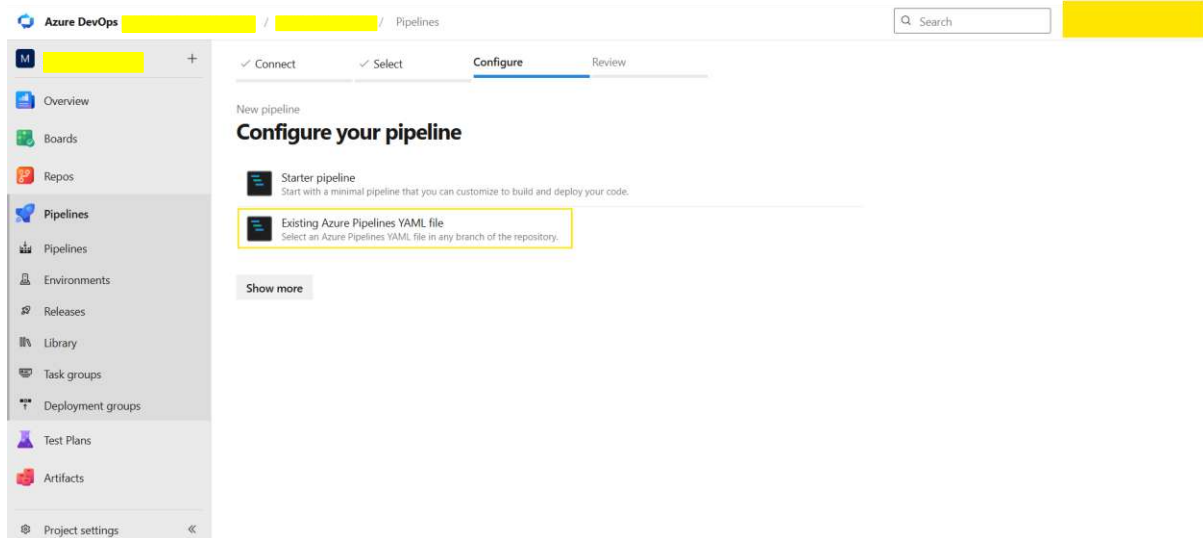
Pipeline	Last run
confluent-kafka-7.9.0	#20260709 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by main 47m ago 12s

Where is your code?

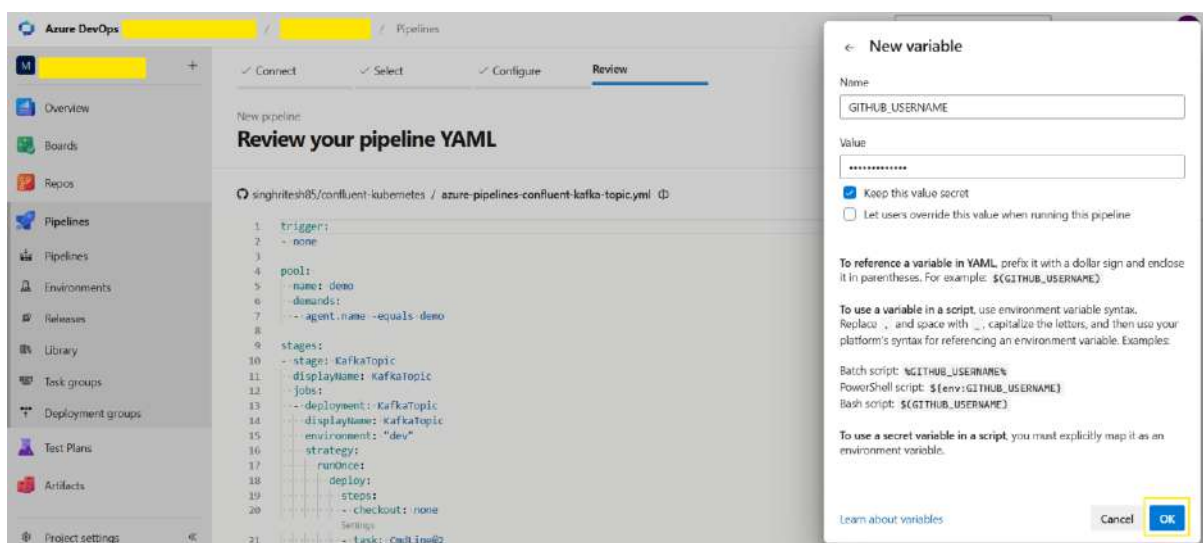
- Azure Repos Git (YAML)
- GitHub (YAML) - Home to the world's largest community of developers
- Bitbucket Cloud (YAML)
- Other Git
- Subversion

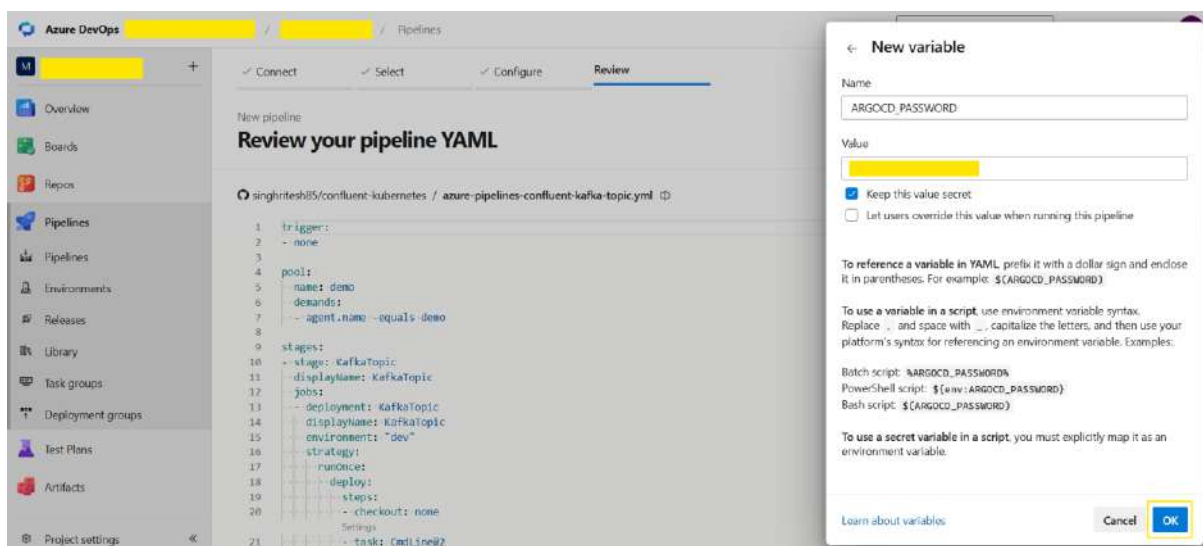
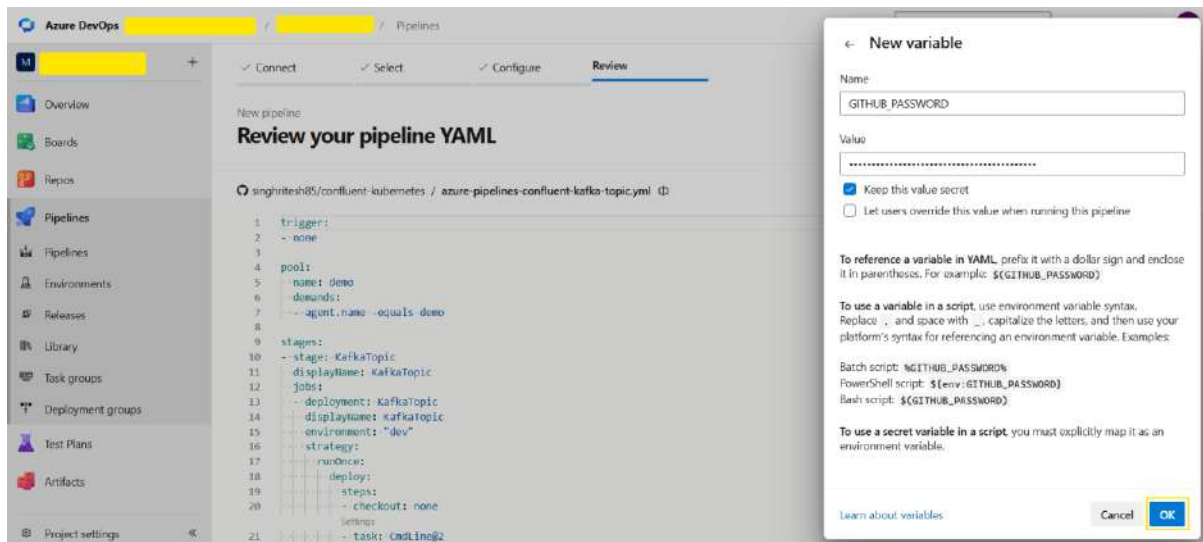
Select a repository

- singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way
- singhritesh85/confluent-kubernetes
- singhritesh85/helm-repo-for-msk-keda
- singhritesh85/eks-msk-producer-consumer
- singhritesh85/DevOps-Project-Netflix-Clone-Aws
- singhritesh85/Kubernetes-headlamp-and-k8s
- singhritesh85/kafka-cluster-with-kraft-using-strimzi-operator
- singhritesh85/strimzi-kafka-demo

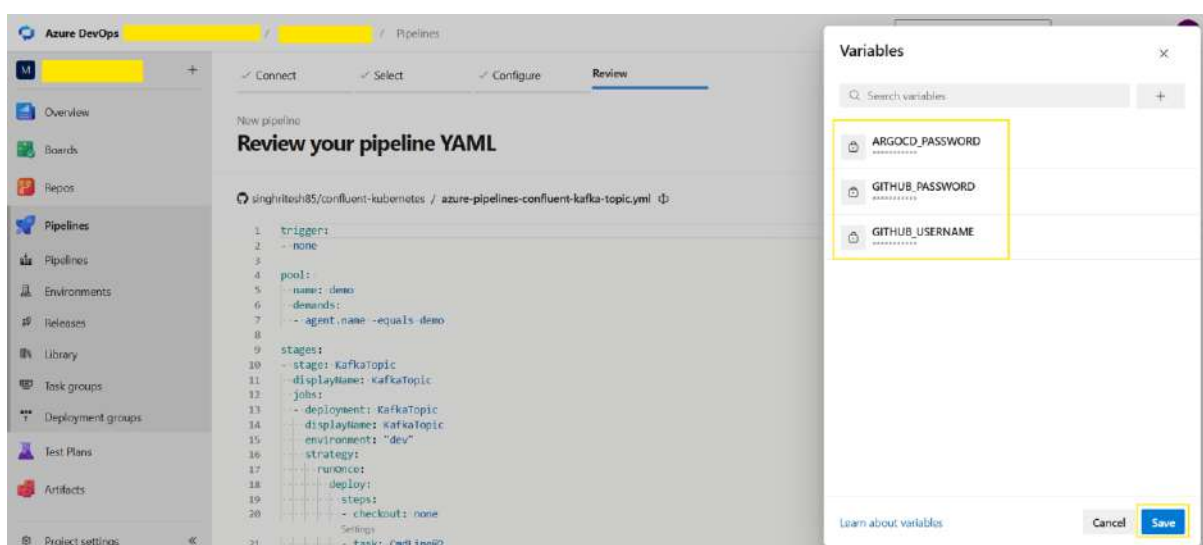


Before Running the Pipeline created three pipeline variables as shown in the screenshot attached below.

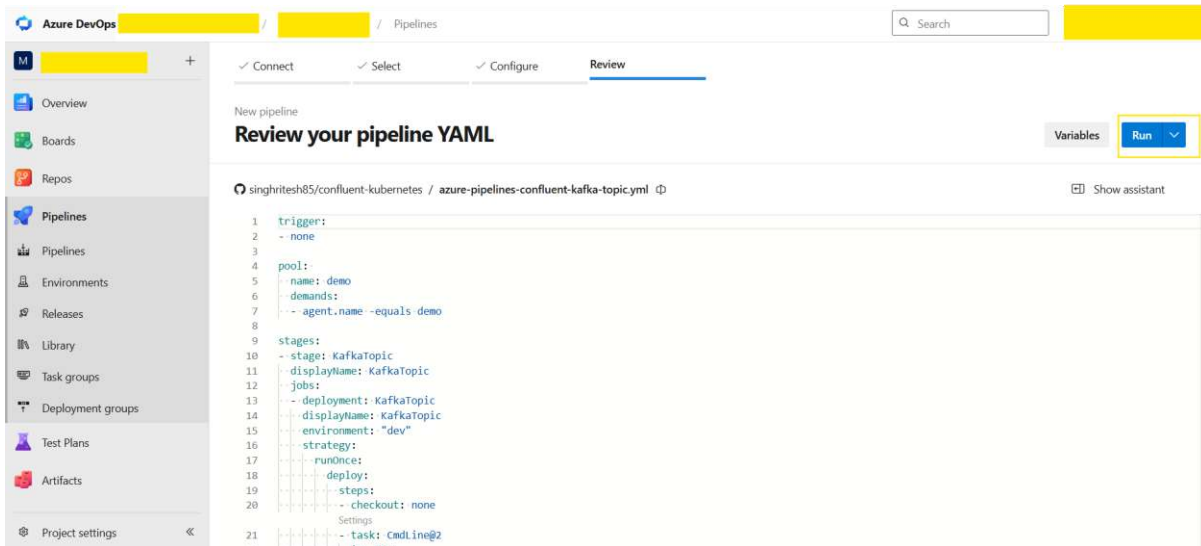




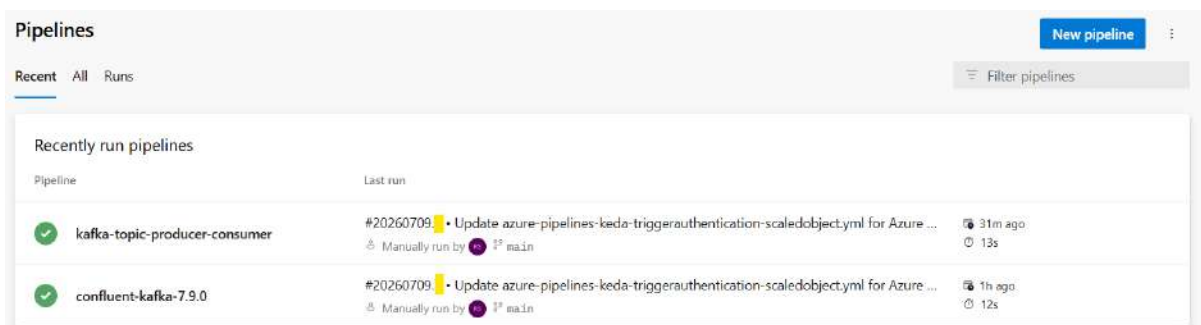
Below screenshot shows the three created variables before running the pipeline.



Finally, Run the Pipeline as shown in the screenshot attached below.



The screenshot of the pipeline after its successful execution and Kubernetes Job and pods in the namespace confluent and producer-consumer respectively is as shown below.



```
[demo@demo-azure-vm ~]$ kubectl get jobs -n confluent
NAME                  STATUS      COMPLETIONS   DURATION   AGE
create-demo-topic    Complete    1/1            33s        2m51s
[demo@demo-azure-vm ~]$ kubectl get pods -n producer-consumer
NAME                                READY   STATUS    RESTARTS   AGE
kafka-consumer-app-56dcddc94b-d7flg 1/1     Running   0           3m29s
kafka-producer-app-5d6687c59f-dt8f2 1/1     Running   0           3m29s
```

Then, I checked the Confluent Control Centre dashboard and confirmed that the Kafka topic had been created and that messages had been published to the topic, where they would be consumed by the consumer.

Topics

Topic name	Status	Partitions	Production (last 5 mins)	Consumption (last 5 mins)	Followers	Obs
_schemas_schemaregistry_confluent	Healthy	1	0B/s	0B/s	3	0
confluent.connect-configs	Healthy	1	0B/s	0B/s	3	0
confluent.connect-offsets	Healthy	25	--	0B/s	75	0
confluent.connect-status	Healthy	5	--	0B/s	15	0
demo-topic	Healthy	3	14B/s	14B/s	3	0

demo-topic

Overview Messages Schema Configuration

Production: 7 B/s | Consumption: 7 B/s | Total messages: 65 | Retention time: 1 week

Filter by timestamp, offset, key or value | All partitions | Latest | Max 50 results

50 messages shown | Auto-refresh on

Timestamp	Offset	Partition	Key	Value
1783593210724	19	1	**	"Message sent at Thu Jul 9 10:33:28 UTC 2026"
1783593203360	18	1	**	"Message sent at Thu Jul 9 10:33:21 UTC 2026"
1783593195929	25	2	**	"Message sent at Thu Jul 9 10:33:14 UTC 2026"
1783593189670	19	0	**	"Message sent at Thu Jul 9 10:33:06 UTC 2026"

Confluent Kafka will be upgraded from version 7.9.0 to 8.1.3 using an Azure DevOps Pipeline. The pipeline will trigger the upgrade, while Confluent will handle the rolling upgrade process automatically.

The screenshot of the pipeline after its successful execution is as shown below. Create the three variables named as ARGODC_PASSWORD, GITHUB_USERNAME and GITHUB_PASSWORD as discussed earlier before running the pipeline.

Recently run pipelines		
Pipeline	Last run	
confluent-kafka-8.1.3	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 37 min	3m ago 12s
kafka-topic-producer-consumer	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 37 min	45m ago 13s
confluent-kafka-7.9.0	#20260709.3 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 37 min	2h ago 12s

```
[demo@demo-azure-vm ~]$ kubectl get pods -n confluent --watch
NAME                                READY   STATUS    RESTARTS   AGE
confluent-operator-f66c6465d-jnpch 1/1     Running   0           3h12m
connect-0                            1/1     Running   0           6m56s
controlcenter-0                     1/1     Running   2 (4m33s ago) 6m55s
create-demo-topic-c6xwk             0/1     Completed 0           48m
kafka-0                             1/1     Running   0           41s
kafka-1                             1/1     Running   0           2m53s
kafka-2                             1/1     Running   0           6m44s
kafkarestproxy-0                   1/1     Running   0           6m56s
kraftcontroller-0                   1/1     Running   0           80s
kraftcontroller-1                   1/1     Running   0           3m21s
kraftcontroller-2                   1/1     Running   0           6m53s
ksqldb-0                           1/1     Running   0           4m2s
ksqldb-1                           1/1     Running   0           6m56s
schemaregistry-0                   1/1     Running   0           6m56s
```

Before running this pipeline make sure you had taken the backup, in present scenario I had taken the backup manually using Azure Backup vaults before running the pipeline for upgrade confluent from 7.9.0 to 8.1.3.

Home > Backup vaults > confluent-aks-backup-vault | Backup instances

confluent-aks-backup-instance

Backup instance

[Backup Now](#) [Restore](#) [Stop Backup](#) [Resume Backup](#) [Fix protection error](#) [Delete](#) [UnDelete](#) [Edit backup instance](#) [Refresh](#) [Feedback](#)

Essentials

Datasource	: confluent-aks-backup-instance	Backup Vault	: confluent-aks-backup-vault
Datasource type	: Kubernetes Services	Backup Policy	: confluent-aks-backup-policy
Disk Snapshot Resource	: confluent-aks-rg	Backup storage redundancy	: Locally-redundant
Subscription	: Pay-As-You-Go	Backup instance configuration	: view
Subscription ID	: 5 [redacted]	Storage account	: confluentaksbackupsa2026
Resource group	: confluent-aks-rg	Blob container	: confluent-aks-container
Location	: East US	Managed identity type	: System assigned
Status	: Protection configured	Managed identity id	: [redacted]

Jobs (last 7 days) [View all](#)

Operation	Failed	In progress	Completed
Scheduled Backup	0	0	0
On-demand backup	0	0	0
Restore	0	0	0
Tiering	0	0	0

Here I had taken backup of all the namespaces. If you need you can restrict your backup to a specific namespace.

Home > Backup vaults > confluent-aks-backup-vault | Backup instances

confluent-aks-backup-instance

Backup instance

[Backup Now](#) [Restore](#) [Stop Backup](#) [Resume Backup](#) [Delete](#) [UnDelete](#) [Edit backup instance](#) [Refresh](#) [Feedback](#)

Essentials

Datasource	: confluent-aks-backup-instance	Backup Vault	: confluent-aks-backup-vault
Datasource type	: Kubernetes Services	Backup Policy	: confluent-aks-backup-policy
Disk Snapshot Resource	: confluent-aks-rg	Backup storage redundancy	: Locally-redundant
Subscription	: Pay-As-You-Go	Backup instance configuration	: view
Subscription ID	: 5 [redacted]	Storage account	: confluentaksbackupsa2026
Resource group	: confluent-aks-rg	Blob container	: confluent-aks-container
Location	: East US	Managed identity type	: System assigned
Status	: Protection configured	Managed identity id	: [redacted]

Jobs (last 7 days) [View all](#)

Operation	Failed	In progress	Completed
Scheduled Backup	0	0	0
On-demand backup	0	1	0
Restore	0	0	0
Tiering	0	0	0

Home > Backup vaults > confluent-aks-backup-vault | Backup instances

confluent-aks-backup-instance

Backup instance

[Backup Now](#)
[Restore](#)
[Stop Backup](#)
[Resume Backup](#)
[Delete](#)
[UnDelete](#)
[Edit backup instance](#)
[Refresh](#)
[Feedback](#)
Essentials

Datasource : confluent-aks-backup-instance

Datasource type : Kubernetes Services

Disk Snapshot Resource ... : confluent-aks-rg

Subscription [\[move\]](#) : [Pay-As-You-Go](#)

Subscription ID : 5

Resource group [\[move\]](#) : [confluent-aks-rg](#)

Location : East US

Status : Protection configured

Backup Vault : [confluent-aks-backup-vault](#)

Backup Policy : [confluent-aks-backup-policy](#)

Backup storage redunda... : Locally-redundant

Backup instance configur... : [view](#)

Storage account : confluentaksbackupsa2026

Blob container : confluent-aks-container

Managed identity type : System assigned

Managed identity id :

Jobs (last 7 days)[View all](#)

Operation	Failed	In progress	Completed
Scheduled Backup	0	0	0
On-demand backup	0	0	1
Restore	0	0	0
Tiering	0	0	0

After successful upgrade of Confluent Kafka from 7.9.0 to 8.1.3 opened the URL <https://confluent.singhritesh85.com> and found the Confluent Kafka Graphical User Interface also got upgraded.

The screenshot shows the Confluent Control Center interface. At the top, there's a navigation bar with the Confluent logo and a trial status. The main heading is "Welcome to Control Center". Below this, a "Resource Overview" section displays various metrics: 1 Cluster (Healthy), 3 Brokers (Healthy), 109 Topics, 0 Partitions, 1 Connect cluster (Healthy), and 1 ksqlDB cluster (Healthy). A "Clients" section with a link to "Configure clients" is also present. On the right, there's a promotional banner for "Confluent Platform for Apache Flink". At the bottom, a section titled "All clusters (1)" is partially visible.

The first screenshot shows the Confluent Kafka 'Overview' page for a cluster named 'controlcenter.cluster'. The page displays the following metrics:

- Brokers:** 3 Total, 59.69K Production (bytes / second), 0 Consumption (bytes / second).
- Topics:** 109 Total, 0 Partitions, 0 Under replicated partitions, 0 Out of Sync Replicas (observers and followers).
- Connect:** 0 Total, 0 Under replicated partitions, 0 Out of Sync Replicas (observers and followers).

The second screenshot shows the 'demo-topic' message viewer. The page displays the following metrics:

- Production:** 14 B/s
- Consumption:** 14 B/s
- Total messages:** 446
- Retention time:** 1 week

The message viewer also includes a search bar and a table of messages. The table has the following columns: Timestamp, Offset, Partition, Key, and Value. The messages are as follows:

Timestamp	Offset	Partition	Key	Value
1783596137350	142	0	null	"Message sent at Thu Jul 9 11:22:15 UTC 2026"
1783596129937	141	0	null	"Message sent at Thu Jul 9 11:22:07 UTC 2026"
1783596115150	140	0	null	"Message sent at Thu Jul 9 11:21:53 UTC 2026"

Above screenshots verified that Confluent Kafka had been upgraded from 7.9.0 to 8.1.3.

Then ran the Azure DevOps Pipeline for Keda installation and creation of trigger authentication with scaled object and Azure DevOps Pipeline for Prometheus-Grafana with Service Monitor.

The screenshot shows the Azure DevOps Pipelines interface. The 'Recently run pipelines' section displays the following information:

Pipeline	Last run
keda	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by aa1n 2m ago 12s
confluent-kafka-8.1.3	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by aa1n 1h ago 12s
kafka-topic-producer-consumer	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by aa1n 2h ago 13s
confluent-kafka-7.9.0	#20260709.3 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by aa1n 3h ago 12s

```
[demo@demo-azure-vm ~]$ kubectl get scaledobjects -n producer-consumer
```

NAME	SCALETARGETKIND	SCALETARGETNAME	MIN	MAX	READY	ACTIVE	FALLBACK	PAUSED	TRIGGERS	AUTHENTIFICATIONS	AGE
kafka-consumer-scaler	apps/v1.Deployment	kafka-consumer-app	1	5	True	True	False	Unknown	kafka	kafka-mtls-auth	31m

```
[demo@demo-azure-vm ~]$ kubectl get pods -n producer-consumer
```

NAME	READY	STATUS	RESTARTS	AGE
kafka-consumer-app-	1/1	Running	0	13m
kafka-consumer-app-	1/1	Running	0	139m
kafka-producer-app-	1/1	Running	0	139m

As shown in the above screenshot consumer pod had been auto scaled.

Pipelines New pipeline

Recent All Runs Filter pipelines

Recently run pipelines

Pipeline	Last run
prometheus-grafana-servicemonitor	#20260709.2 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 5m ago 1m 4s
keda	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 11m ago 22s
confluent-kafka-8.1.3	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 1h ago 12s
kafka-topic-producer-consumer	#20260709.1 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 2h ago 13s
confluent-kafka-7.9.0	#20260709.3 • Update azure-pipelines-keda-triggerauthentication-scaledobject.yml for Azure ... Manually run by 3h ago 12s

```
[demo@demo-azure-vm ~]$ kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-kube-prometheus-stack-alertmanager-0	2/2	Running	0	5m15s
alertmanager-main-0	2/2	Running	0	5m2s
kube-prometheus-stack-grafana-569d647b65-7qp7h	3/3	Running	0	5m23s
kube-prometheus-stack-kube-state-metrics-5865f47984-5tzzk	1/1	Running	0	5m23s
kube-prometheus-stack-operator-844bf84759-m4z22	1/1	Running	0	5m23s
kube-prometheus-stack-prometheus-node-exporter-kxnbx	1/1	Running	0	5m23s
kube-prometheus-stack-prometheus-node-exporter-l66rt	1/1	Running	0	5m23s
prometheus-kube-prometheus-stack-prometheus-0	2/2	Running	0	5m14s

```
[demo@demo-azure-vm ~]$ kubectl get svc -n monitoring
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP, 9094/TCP, 9094/UDP	5m24s
kube-prometheus-stack-alertmanager	ClusterIP	10.0.142.239	<none>	9093/TCP, 8080/TCP	5m32s
kube-prometheus-stack-grafana	ClusterIP	10.0.148.178	<none>	80/TCP	5m32s
kube-prometheus-stack-kube-state-metrics	ClusterIP	10.0.10.165	<none>	8080/TCP	5m32s
kube-prometheus-stack-operator	ClusterIP	10.0.200.215	<none>	443/TCP	5m32s
kube-prometheus-stack-prometheus	ClusterIP	10.0.63.166	<none>	9090/TCP, 8080/TCP	5m32s
kube-prometheus-stack-prometheus-node-exporter	ClusterIP	10.0.140.227	<none>	9100/TCP	5m32s
prometheus-operated	ClusterIP	None	<none>	9090/TCP	5m23s

```
[demo@demo-azure-vm ~]$ kubectl get servicemonitor.monitoring.coreos.com -n monitoring
```

NAME	AGE
kafka	6m17s
kube-prometheus-stack-alertmanager	6m38s
kube-prometheus-stack-apiserver	6m38s
kube-prometheus-stack-coredns	6m38s
kube-prometheus-stack-grafana	6m38s
kube-prometheus-stack-kube-controller-manager	6m38s
kube-prometheus-stack-kube-etcd	6m38s
kube-prometheus-stack-kube-proxy	6m38s
kube-prometheus-stack-kube-scheduler	6m38s
kube-prometheus-stack-kube-state-metrics	6m38s
kube-prometheus-stack-kubelet	6m38s
kube-prometheus-stack-operator	6m38s
kube-prometheus-stack-prometheus	6m38s
kube-prometheus-stack-prometheus-node-exporter	6m38s

To access Grafana using the URL used Gateway API for Grafana with the kubernetes manifests file present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-3/Section-2/gateway-api-grafana.yaml**.

```
[demo@demo-azure-vm ~]$ kubectl create secret tls tls-grafana-ingress --cert=STAR_singhritesh85_com.crt --key=mykey.key -n monitoring
```

```
secret/tls-grafana-ingress created
```

```
[demo@demo-azure-vm ~]$ kubectl apply -f gateway-api-grafana.yaml
gateway.gateway.networking.k8s.io/grafana-gateway created
httproute.gateway.networking.k8s.io/grafana-route created
```

```
[demo@demo-azure-vm ~]$ kubectl get httproute -n monitoring
NAME          HOSTNAMES          AGE
grafana-route ["grafana.singhritesh85.com"] 54s
```

```
[demo@demo-azure-vm ~]$ kubectl get gateway -n monitoring
NAME          CLASS    ADDRESS    PROGRAMMED    AGE
grafana-gateway  nginx    48.        True          71s
```

Did the entry for this IP Address as shown in the screenshot attached above in Azure DNS Zone to create the Record-Set of A-Type. Finally, I was able to access Grafana as shown in the screenshot attached below.

The screenshot shows the Azure DNS Zone management interface for the domain 'singhritesh85.com'. The 'Recordsets' tab is selected, displaying a table of existing record sets. A dialog box 'Add record set' is open, showing the configuration for a new record set named 'grafana' of type 'A - IPv4 Address records'. The IP address '48' is entered in the 'IP address' field. The browser address bar shows 'grafana.singhritesh85.com/login'.

Name	Type	TTL	Value
@	NS	172800	
@	SOA	3600	
argocd	A	3600	57

Record set configuration details:

- Name: grafana
- Type: A - IPv4 Address records
- Alias record set: No
- TTL: 1
- TTL unit: Hours
- IP address: 48

Browser address bar: grafana.singhritesh85.com/login

Grafana login page:

Welcome to Grafana

Email or username: email or username

Password: password

Log in

Forgot your password?

Documentation | Support | Community | Open Source | Grafana v13.1.0 (b309c9bb3b)

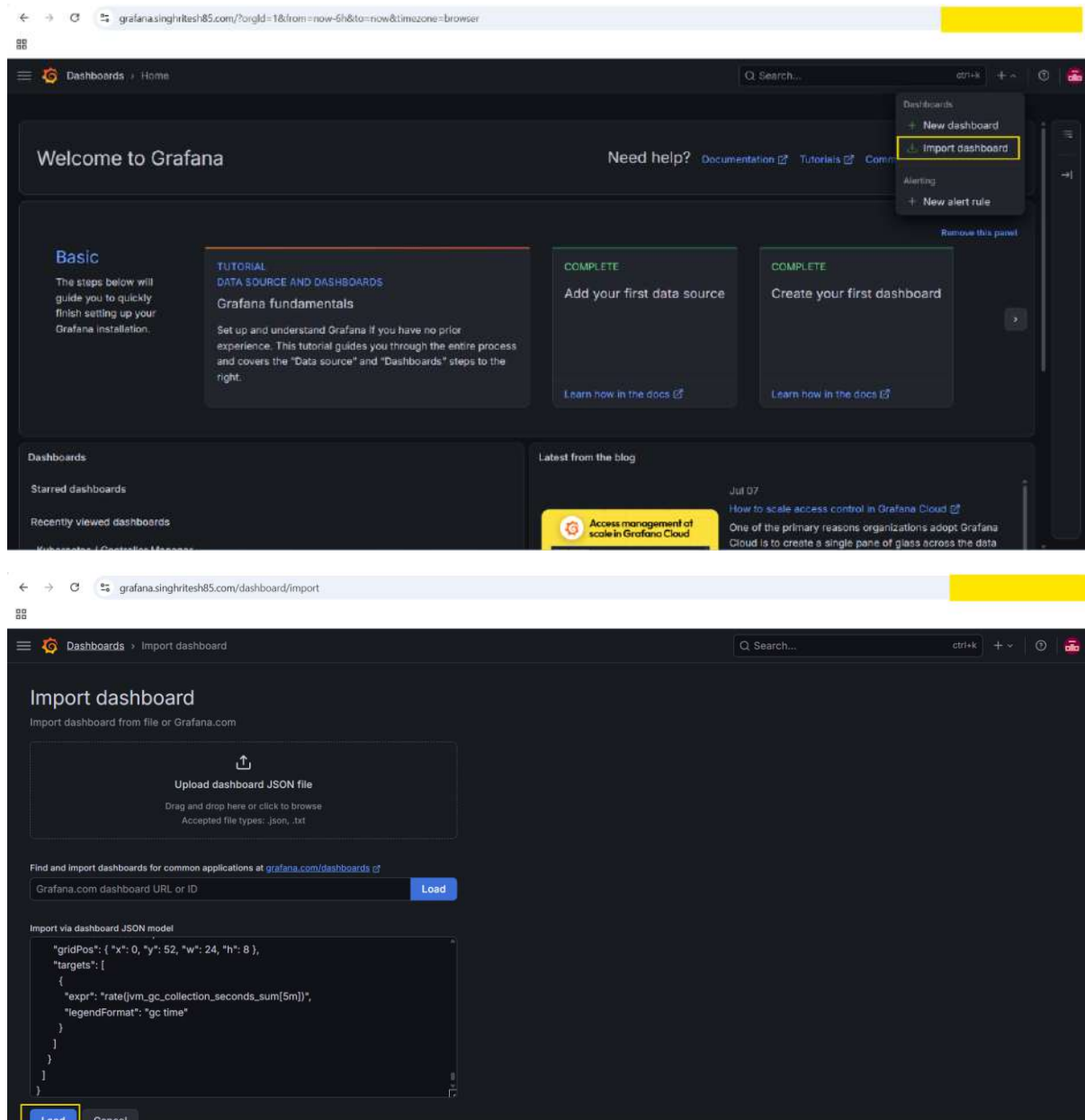
Obtained the admin user password for Grafana using the command as shown below.

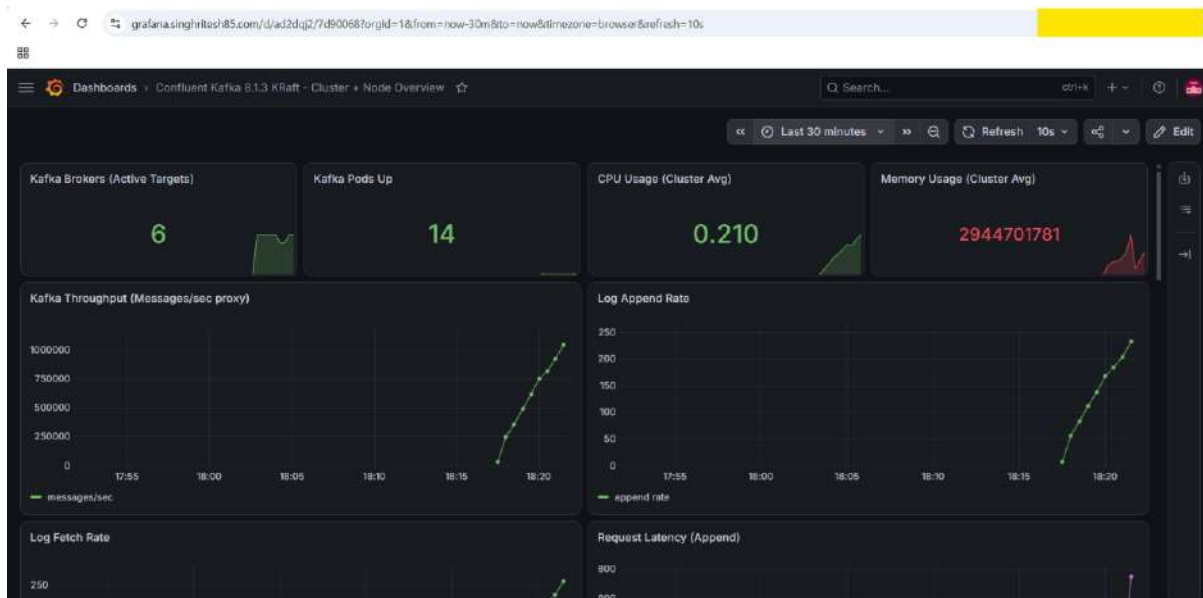
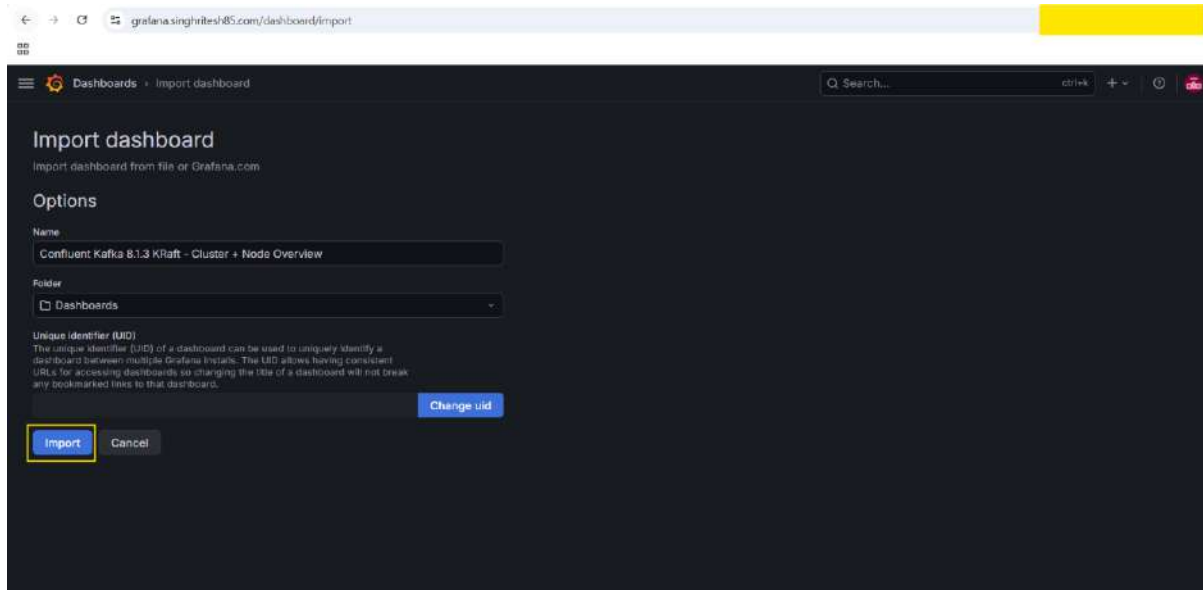
```
[demo@demo-azure-vm ~]$ kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 --decode; echo
```

Ritesh Kumar Singh || Email Address: - riteshkumarsingh9559@gmail.com || LinkedIn: - <https://www.linkedin.com/in/ritesh-kumar-singh-41113128b/> || GitHub: - <https://github.com/singhritesh85>

```
kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 --decode; echo
```

Then created the Dashboard in Grafana by importing the Json Model present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-3/Section-2/grafana-dashboard.json**.





GitHub Repo: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> **Project Path:** - module-3/Section-2/

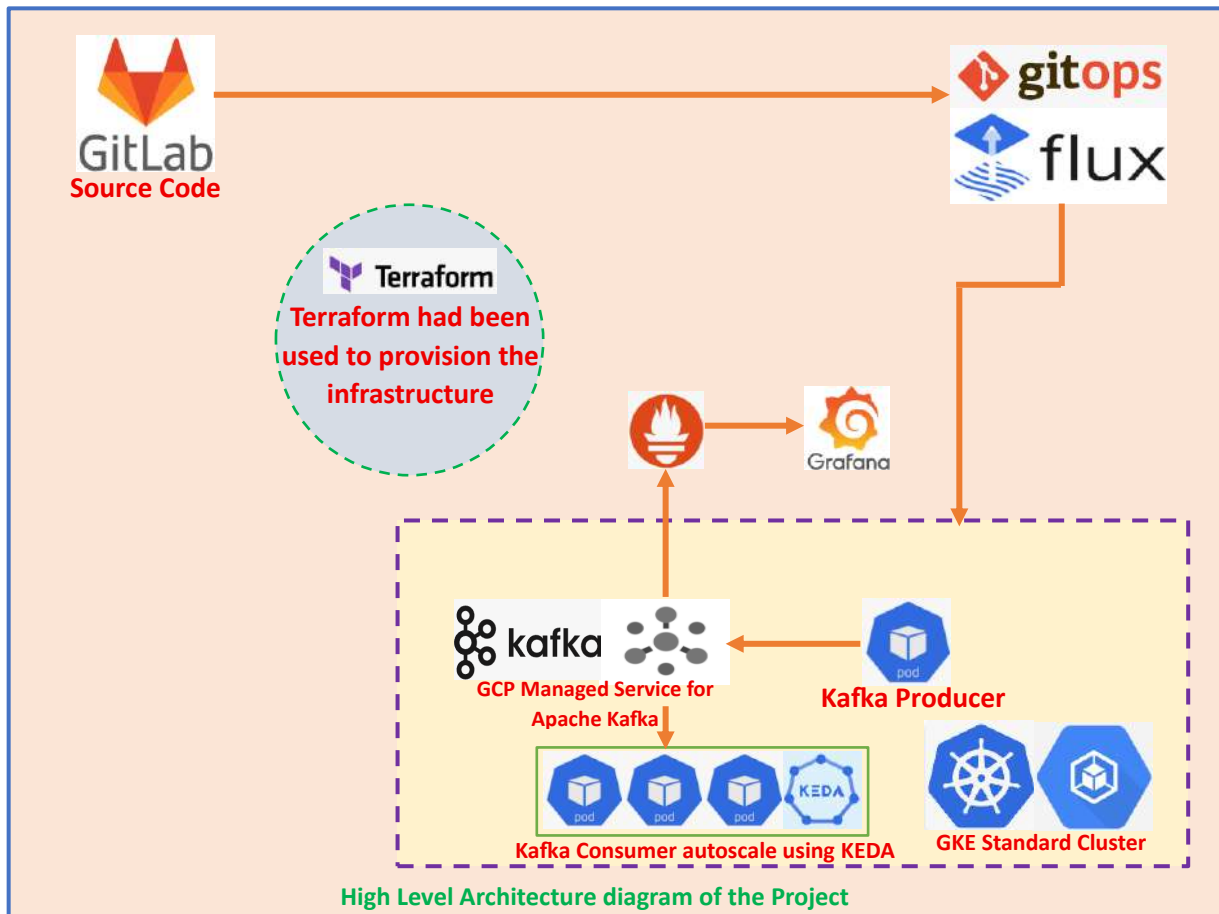
Terraform Script: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git>

Kubernetes Manifests file: - <https://github.com/singhritesh85/confluent-kubernetes.git>

Azure DevOps Pipeline YAML configuration files: - <https://github.com/singhritesh85/confluent-kubernetes.git>

Module-3 (Section-3): -

DevOps Project using GCP Managed Service for Apache Kafka as Message Queue with GitOps



This DevOps Project deals with provision of infrastructure using Terraform Script present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path **module-3/Section-3/terraform-gcp-managed-apache-kafka-gitlab-server**. For GitOps in this project FluxCD had been used as shown in the high-level architecture diagram of the project drawn above. Depending on the organization's operating model, the ownership of Kubernetes manifests for Kafka-related deployments may vary. In some organizations, the application development team maintains the Kubernetes manifests and deployment configurations. In others, the DevOps or Platform Engineering team manages these manifests in close collaboration with the application team.

Typically, the DevOps/Platform Engineering team is responsible for:

- Managing the Kubernetes manifests for platform components such as KEDA.
- Establishing and maintaining the monitoring and observability stack using Prometheus and Grafana.
- Implementing and maintaining the GitOps workflow using FluxCD together with GitLab (or another Git repository such as GitHub) while collaborating with application teams for application-specific deployment configurations.

Commands as written below had been used to provision the infrastructure using Terraform.

- **terraform init** -----> initializes a working directory containing configuration files and installs plugins for required providers.
- **terraform validate** -----> verify that terraform configuration file is correct or not
- **terraform plan** -----> Check which resources are going to be created.

Then you can run the command **terraform apply -auto-approve** -----> Finally, Created the resources.

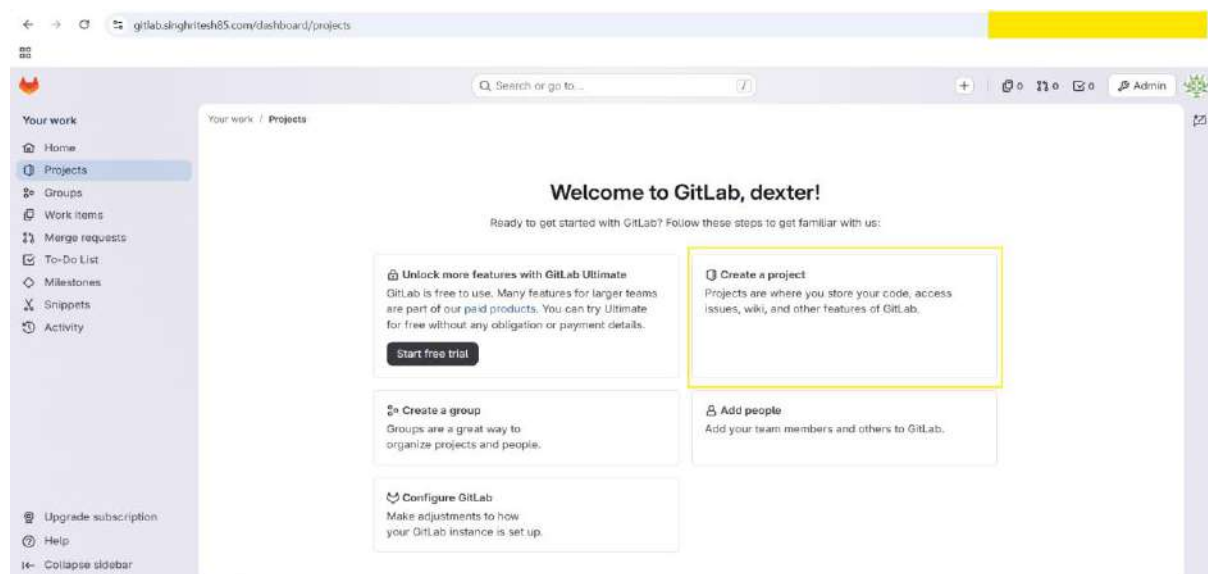
```
module.gcp_cloud_dns_zone.null_resource.gitlab_server (remote-exec): run: puma: (pid 92469) 238s; run: log: (pid 92497) 235s
module.gcp_cloud_dns_zone.null_resource.gitlab_server (remote-exec): run: redis: (pid 91556) 413s; run: log: (pid 91572) 409s
module.gcp_cloud_dns_zone.null_resource.gitlab_server (remote-exec): run: redis-exporter: (pid 93585) 89s; run: log: (pid 93660) 189s
module.gcp_cloud_dns_zone.null_resource.gitlab_server: Still creating... [18m18s elapsed]
module.gcp_cloud_dns_zone.null_resource.gitlab_server: Creation complete after 18m12s [id=XXXXXXXXXX]
module.gcp_cloud_dns_zone.google_dns_record_set_a_record: Creating...
module.gcp_cloud_dns_zone.google_dns_record_set_a_record: Creation complete after 2s [id=projects/XXXXXXXXXX/managedZones/gitlab-public-zone/rrsats/g
itlab.singhritesh85.com/A]

Apply complete! Resources: 57 added, 0 changed, 0 destroyed.

Outputs:
gke_cluster_gcp_cloud_dns_zone_vm_instances_and_gcp_alb_details = {
  "cloud_dns_zone_id" = "projects/XXXXXXXXXX/managedZones/gitlab-public-zone"
  "gcp_alb_public_ip" = "136.XXXXXXX"
  "gcp_cloud_dns_zone_name_servers" = tolist([
    "XXXXXXXXXX",
    "XXXXXXXXXX",
    "XXXXXXXXXX",
    "XXXXXXXXXX",
    "XXXXXXXXXX",
    "XXXXXXXXXX"
  ])
  "gcp_vm_instances_name" = [
    "gitlab-server",
    "gitlab-gitlab-runner",
  ]
  "gcp_vm_private_ip_address" = [
    "10.20.15.100",
    "10.20.15.101",
  ]
  "gcp_vm_public_ip_address" = [
    "34.XXXXXXX",
    "34.XXXXXXX",
  ]
  "gke_cluster_endpoint" = "172.16.0.2"
  "gke_cluster_name" = "managed-kafka-gke-cluster"
}
```

After running **terraform apply -auto-approve** and successfully creating the Cloud DNS managed zone in Google Cloud, update the nameserver (NS) records at your domain registrar with the nameservers assigned to the Google Cloud DNS managed zone. This delegates DNS authority for your domain to Google Cloud DNS.

To Login into GitLab for the very first time and to perform initial configuration (creating an Administrative user apart from root) you can refer the project present in the GitHub Repository <https://github.com/singhritesh85/DevOps-Project-BankApp-GCP-ALB-Autoscaling.git>. After Login into the GitLab I had created two GitLab Private Repository as shown in the screenshot attached below.



gitlab.singhritesh85.com/projects/new/blank_project

Search or go to...

Your work / Projects / New project / Create blank project

Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Project name
gcp-managed-apache-kafka-fluxcd
Start with a letter, digit, emoji, or underscore.

Project URL
http://gitlab.singhritesh85.com/ dexter

Project slug
gcp-managed-apache-kafka-fluxcd

Visibility Level

- ☒ Private
Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.
- ☐ Internal
The project can be accessed by any logged in user except external users.
- ☐ Public
The project can be accessed without any authentication.

Project Configuration

- ☒ Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.
- ☐ Enable Static Application Security Testing (SAST)
Analyze your source code for known security vulnerabilities. Learn more.

gitlab.singhritesh85.com/projects/new/blank_project

Search or go to...

Your work / Projects / New project / Create blank project

Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Project name
gcp-managed-apache-kafka
Start with a letter, digit, emoji, or underscore.

Project URL
http://gitlab.singhritesh85.com/ dexter

Project slug
gcp-managed-apache-kafka

Visibility Level

- ☒ Private
Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.
- ☐ Internal
The project can be accessed by any logged in user except external users.
- ☐ Public
The project can be accessed without any authentication.

Project Configuration

- ☒ Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.
- ☐ Enable Static Application Security Testing (SAST)
Analyze your source code for known security vulnerabilities. Learn more.

gitlab.singhritesh85.com/dashboard/projects

Search or go to...

Your work / Projects

Projects

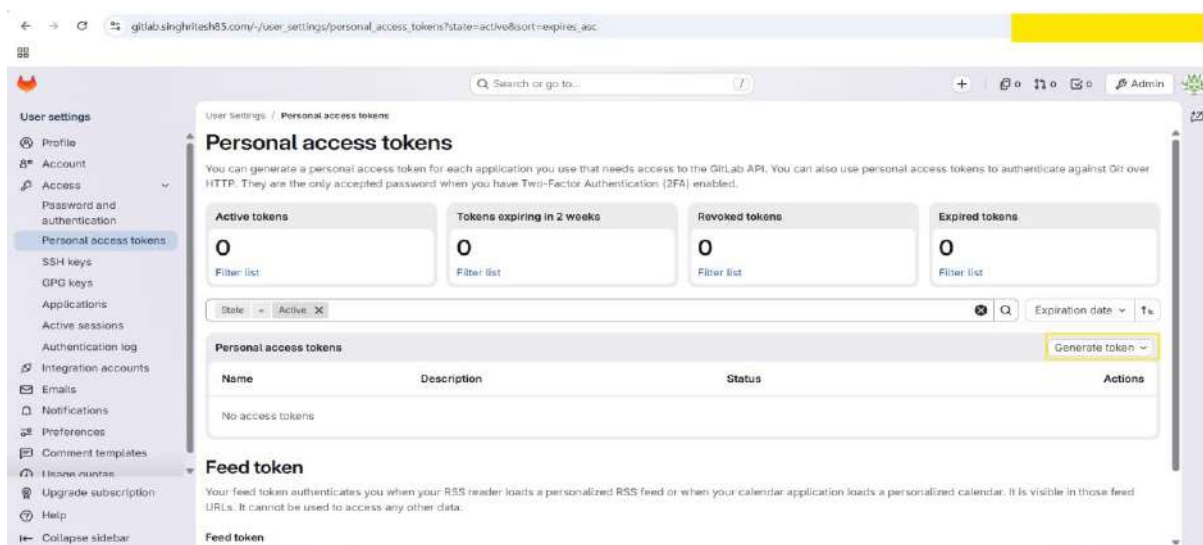
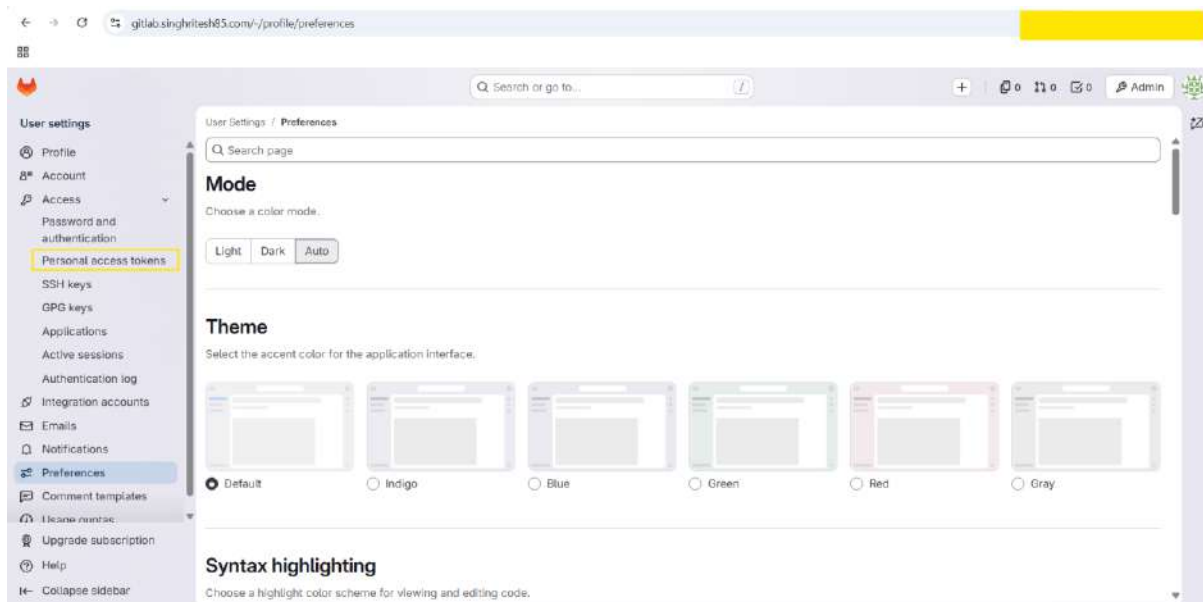
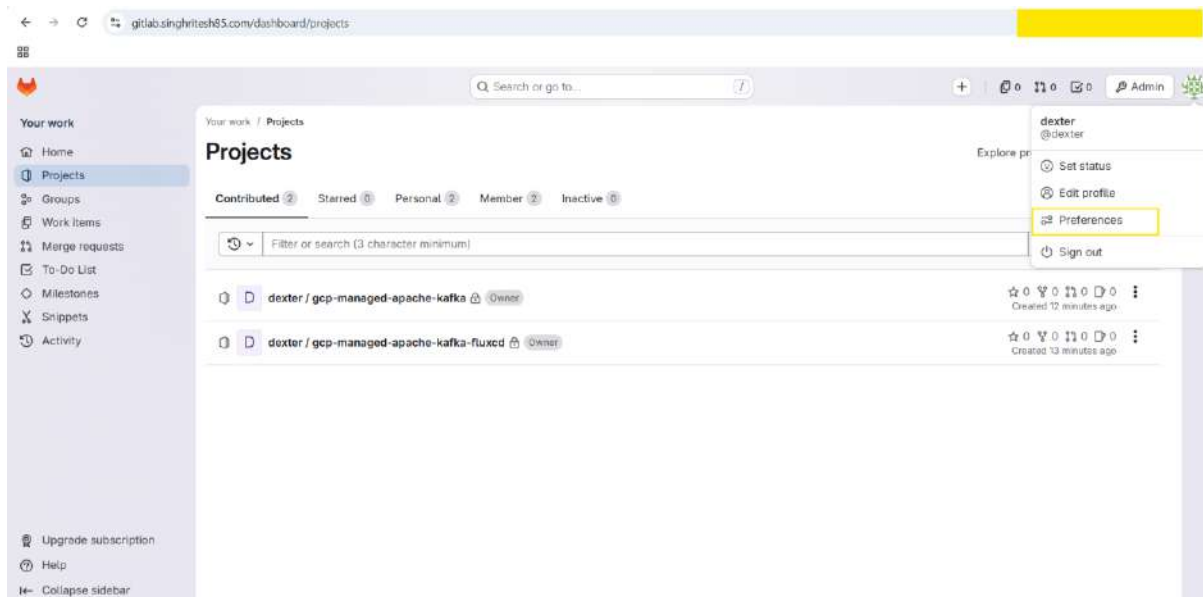
Explore projects [New project](#)

Contributed 2 Starred 0 Personal 2 Member 2 Inactive 0

Filter or search (3 character minimum)

dexter / gcp-managed-apache-kafka	Owner	☆ 0 🗨 0 📄 0 📁 0	Created 15 seconds ago
dexter / gcp-managed-apache-kafka-fluxcd	Owner	☆ 0 🗨 0 📄 0 📁 0	Created 1 minute ago

Created the Personal Access Token (PAT) in GitLab as shown in the screenshot attached below.



gitlab.singhritesh85.com/-/user_settings/personal_access_tokens?state=active&sort=expires_asc

User settings

- Profile
- Account
- Access
- Password and authentication
- Personal access tokens**
- SSH keys
- GPG keys
- Applications
- Active sessions
- Authentication log
- Integration accounts
- Emails
- Notifications
- Preferences
- Comment templates
- Upgrade subscription
- Help

User Settings / Personal access tokens

Personal access tokens

You can generate a personal access token for each application you use that needs access to the GitLab API. You can also use personal access tokens to authenticate against Git over HTTP. They are the only accepted password when you have Two-Factor Authentication (2FA) enabled.

Active tokens: 0
Tokens expiring in 2 weeks: 0
Revoked tokens: 0
Expired tokens: 0

Generate token

Feed token

Your feed token authenticates you when your RSS reader loads a personalized RSS feed or when your calendar application loads a personalized calendar. It is visible in those feed URLs. It cannot be used to access any other data.

gitlab.singhritesh85.com/-/user_settings/personal_access_tokens/legacy/new

User settings

- Profile
- Account
- Access
- Password and authentication
- Personal access tokens**
- SSH keys
- GPG keys
- Applications
- Active sessions
- Authentication log
- Integration accounts
- Emails
- Notifications
- Preferences
- Comment templates
- Upgrade subscription
- Help

User Settings / Personal access tokens / Generate legacy token

Generate legacy token

Legacy personal access tokens are scoped to all groups and projects with broad permissions to resources.

Token name: dexter

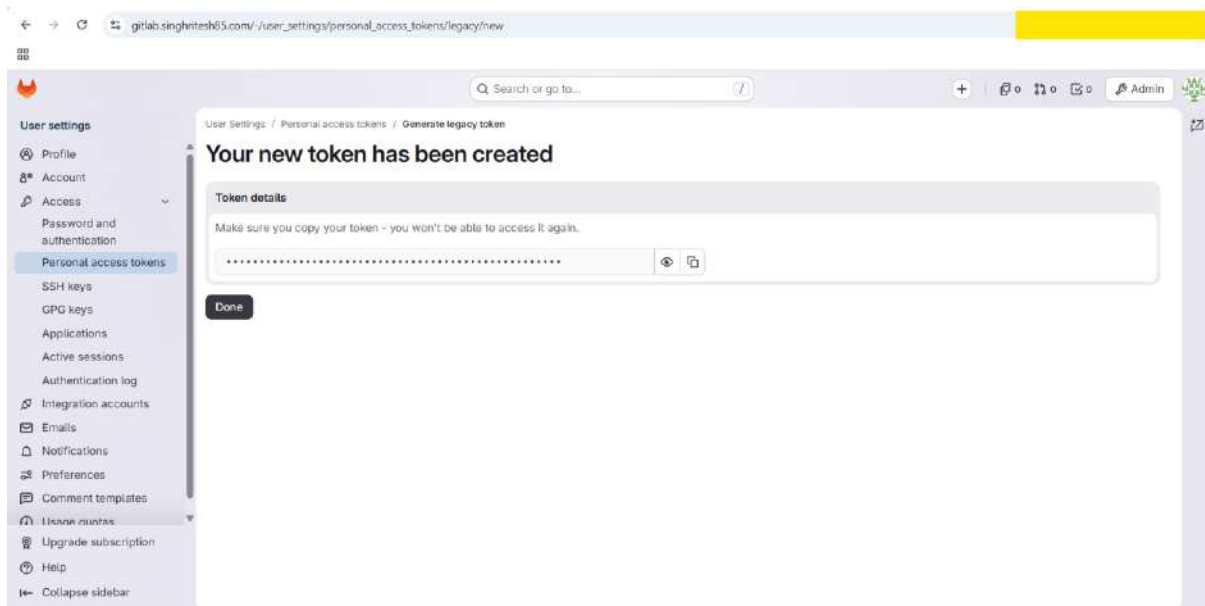
Description (optional): dexter

Expiration date: 2026-08-27

Select scopes

- ☒ read_service_ping: Grants access to download Service Ping payloads through the API when authenticated as an administrator user.
- ☒ read_user: Grants read-only access to your profile through the /user API endpoint, which includes username, public email, and full name. Also grants access to read-only API endpoints under /users.
- ☒ read_repository: Grants read-only access to repositories on private projects using Git-over-HTTP or the Repository Files API.
- ☒ read_virtual_registry: Grants read-only access to container images through the dependency proxy in private projects and virtual registries.
- ☒ read_api: Grants read access to the API, including all groups and projects, the container registry, and the package registry.
- ☐ self_rotate: Grants permission for token to rotate itself.
- ☒ write_repository: Grants read-write access to repositories on private projects using Git-over-HTTP (not using the API).
- ☒ write_virtual_registry: Grants read, write, and delete access to container images through the dependency proxy in private projects.
- ☐ api: Grants complete read/write access to the API, including all groups and projects, the container registry, the dependency proxy, and the package registry.
- ☐ all_features: Grants access to GitLab Duo related API endpoints.
- ☐ create_runner: Grants create access to the runners.
- ☐ manage_runner: Grants access to manage the runners.
- ☐ k8s_proxy: Grants permission to perform Kubernetes API calls using the agent for Kubernetes.
- ☐ admin_mode: Grants access to perform administrative actions.

Copy the below Token and keep it safely for future uses.



To initialize Flux CD on your Kubernetes cluster and connect it to a Git Repository and enabling GitOps ran the command on GitLab-Runner as shown in the screenshot attached below.

```
flux bootstrap git --url=https://gitlab.singhritesh85.com/dexter/gcp-managed-apache-kafka-fluxcd.git -  
-branch=main --path=cluster/gcp-managed-apache-kafka --username=XXXXXX --  
password=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX --token-auth
```

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
gke-managed-kafka-gk-managed-kafka-1i-025f1cf0-lqdn    Ready    cnone    3h49m    v1.35.6-gke.1250000
gke-managed-kafka-gk-managed-kafka-1i-9f8afe6b-u6kc    Ready    cnone    3h49m    v1.35.6-gke.1250000
gke-managed-kafka-gk-managed-kafka-1i-af936a8c-17ps    Ready    cnone    3h49m    v1.35.6-gke.1250000
[gitlab-runner@gitlab-gitlab-runner ~]$ flux bootstrap git --url=https://gitlab.singhritesh85.com/dexter/gcp-managed-apache-kafka-fluxcd.git --branch=main --path=cluster/gcp-managed-apache-kafka --username=dexter --password=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX --token-auth
> cloning branch "main" from git repository "https://gitlab.singhritesh85.com/dexter/gcp-managed-apache-kafka-fluxcd.git"
✓ cloned repository
> generating component manifests
✓ generated component manifests
✓ committed component manifests to "main" ("XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX")
> pushing component manifests to "https://gitlab.singhritesh85.com/dexter/gcp-managed-apache-kafka-fluxcd.git"
> installing components in "flux-system" namespace
✓ installed components
✓ reconciled components
> determining if source secret "flux-system/flux-system" exists
> generating source secret
> applying source secret "flux-system/flux-system"
✓ reconciled source secret
> generating sync manifests
✓ generated sync manifests
✓ committed sync manifests to "main" ("XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX")
> pushing sync manifests to "https://gitlab.singhritesh85.com/dexter/gcp-managed-apache-kafka-fluxcd.git"
> applying sync manifests
✓ reconciled sync configuration
@ waiting for GitRepository "flux-system/flux-system" to be reconciled
✓ GitRepository reconciled successfully
@ waiting for Kustomization "flux-system/flux-system" to be reconciled
✓ Kustomization reconciled successfully
> confirming components are healthy
✓ helm-controller: deployment ready
✓ kustomize-controller: deployment ready
✓ notification-controller: deployment ready
✓ source-controller: deployment ready
✓ all components are healthy
```

After running the above command, you will see the below files and directory will be created in the GitLab Repository **gcp-managed-apache-kafka-fluxcd**.

Created the kubernetes secrets as shown in the screenshot attached below.

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl create secret generic gcp-managed-kafka-secret --from-literal=username=dexter --from-literal=password=[REDACTED] --namespace=flux-system
secret/gcp-managed-kafka-secret created
```

You can check the Live Logs for FluxCD using the command **flux logs -f** as shown in the screenshot attached below.

To view the status of all Flux CD resources across all namespaces **flux get all -A** had been used as shown in the screenshot attached below. This command provides a quick overview of whether your GitOps pipeline is healthy and whether Flux is successfully synchronizing your Kubernetes cluster with the Git repository.

```
[gitlab-runner@gitlab-gitlab-runner ~]$ flux get all -A
```

NAMESPACE	NAME	REVISION	SUSPENDED	READY	MESSAGE
Flux-system	gitrepository/flux-system	main@sha1:618f35b5	False	True	stored artifact for revision 'main@sha1:618f35b5'
Flux-system	gitrepository/source-gcp-managed-kafka-git	main@sha1:e15007f0	False	True	stored artifact for revision 'main@sha1:e15007f0'

NAMESPACE	NAME	REVISION	SUSPENDED	READY	MESSAGE
keda	helmrepository/kedacore	sha256:1e14d662	False	True	stored artifact: revision 'sha256:1e14d662'
monitoring	helmrepository/prometheus-community	sha256:d17c0694	False	True	stored artifact: revision 'sha256:d17c0694'

NAMESPACE	NAME	REVISION	SUSPENDED	READY	MESSAGE
keda	helmchart/keda-keda	2.20.1	False	True	pulled 'keda' chart with version '2.20.1'
monitoring	helmchart/monitoring-kube-prometheus-stack	87.21.0	False	True	pulled 'kube-prometheus-stack' chart with version '87.21.0'

NAMESPACE	NAME	REVISION	SUSPENDED	READY	MESSAGE
keda	helmrelease/keda	2.20.1	False	True	Helm install succeeded for release keda/keda.v1 with chart keda
monitoring	helmrelease/kube-prometheus-stack	87.21.0	False	True	Helm install succeeded for release monitoring/kube-prometheus-stack.v1 with chart kube-prometheus-stack@87.21.0

NAMESPACE	NAME	REVISION	SUSPENDED	READY	MESSAGE
Flux-system	kustomization/flux-system	main@sha1:610f35b5	False	True	Applied revision: main@sha1:610f35b5
Flux-system	kustomization/kustomization-gcp-managed-kafka	main@sha1:e15007f0	False	True	Applied revision: main@sha1:e15007f0

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get pods -n kafka
```

NAME	READY	STATUS	RESTARTS	AGE
kafka-consumer-844b455f7-12hdd	1/1	Running	0	5m45s
kafka-lag-exporter-66f67b8bc8-srwqf	1/1	Running	0	5m45s
kafka-producer-779bd6765f-sj7lk	1/1	Running	0	5m45s

Below screenshot confirms Kafka Consumers consuming the messages from GCP Managed Apache Kafka Topic.

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl logs -f kafka-consumer-844b455f7-12hdd -n kafka
```

Defaulted container "consumer" out of: consumer, download-auth-libs (init)

```
msg-key:Hello from GKE Tue Jul 28 13:52:42 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:52:57 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:53:07 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:53:16 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:53:26 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:53:37 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:53:46 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:53:56 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:54:06 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:54:16 UTC 2026
msg-key:Hello from GKE Tue Jul 28 13:54:26 UTC 2026
```

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get pods -n keda
```

NAME	READY	STATUS	RESTARTS	AGE
keda-admission-webhooks-d6dc4d56f-tsfgk	1/1	Running	1 (21m ago)	21m
keda-operator-69f7467655-ktwwg	1/1	Running	1 (21m ago)	21m
keda-operator-metrics-apiserver-589688760f-22bmd	1/1	Running	1 (21m ago)	21m

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get scaledobjects -n kafka
```

NAME	SCALETARGETKIND	SCALETARGETNAME	MIN	MAX	READY	ACTIVE	FALLBACK	PAUSED	TRIGGERS	AUTHENTICATIONS	AGE
kafka-consumer	apps/v1.Deployment	kafka-consumer	1	5	True	False	False	False	prometheus		15m

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl describe scaledobjects kafka-consumer -n kafka
```

Events:

Type	Reason	Age	From	Message
Normal	KEDAScalersStarted	16m	keda-operator	Scaler prometheus is built
Normal	KEDAScalersStarted	16m	keda-operator	Started scalers watch
Normal	ScaledObjectReady	16m	keda-operator	ScaledObject is ready for scaling
Normal	ScaledObjectReady	16m	keda-operator	ScaledObject is ready for scaling

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-kube-prometheus-stack-alertmanager-0	2/2	Running	0	23m
kube-prometheus-stack-grafana-74fddd756b-zfqf8	3/3	Running	0	23m
kube-prometheus-stack-kube-state-metrics-7cfd6d45cc-p4jkv	1/1	Running	0	23m
kube-prometheus-stack-operator-879479c5c-zhmzc	1/1	Running	0	23m
kube-prometheus-stack-prometheus-node-exporter-5d7p2	1/1	Running	0	23m
kube-prometheus-stack-prometheus-node-exporter-fnq9l	1/1	Running	0	23m
kube-prometheus-stack-prometheus-node-exporter-l24b5	1/1	Running	0	23m
prometheus-kube-prometheus-stack-prometheus-0	2/2	Running	0	23m

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get svc -n monitoring
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP,9094/TCP,9094/UDP	23m
kube-prometheus-stack-alertmanager	ClusterIP	172.19.33.64	<none>	9093/TCP,8080/TCP	23m
kube-prometheus-stack-grafana	ClusterIP	172.19.51.236	<none>	80/TCP	23m
kube-prometheus-stack-kube-state-metrics	ClusterIP	172.19.60.166	<none>	8080/TCP	23m
kube-prometheus-stack-operator	ClusterIP	172.19.168.27	<none>	443/TCP	23m
kube-prometheus-stack-prometheus	ClusterIP	172.19.9.192	<none>	9090/TCP,8080/TCP	23m
kube-prometheus-stack-prometheus-node-exporter	ClusterIP	172.19.185.137	<none>	9100/TCP	23m
prometheus-operated	ClusterIP	None	<none>	9090/TCP	23m

For Grafana UI first of all created kubernetes secret as shown in the screenshot attached below.

```
kubectl create secret tls tls-grafana-ingress --cert=437ee3bf40dd4901.crt --key=server.key -n monitoring
```

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl create secret tls tls-grafana-ingress --cert=437ee3bf40dd4901.crt --key=server.key -n monitoring
secret/tls-grafana-ingress created
```

To access Grafana using URL with the help of Kubernetes Gateway API, I used the gateway-api-grafana.yaml as present in the GitHub Repo <https://github.com/singhritesh85/gcp-managed-apache-kafka.git> at the path **gateway-api-grafana.yaml**.

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get gateway -n monitoring
```

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
grafana-gateway	nginx	34. [REDACTED]	True	23m

```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get httproute -n monitoring
```

NAME	HOSTNAMES	AGE
grafana-route	["grafana.singhritesh85.com"]	23m

Then created Record-Set of A-Type in GCP Cloud DNS Zone with the External IP Address as obtained in the screenshot attached above.

Network Services / Zones / Zone: gitlab-public-zone / Create record set

← Create record set

DNS name: grafana.singhritesh85.com

Resource record type: A

TTL *: 5

TTL unit: minutes

IPv4 Address

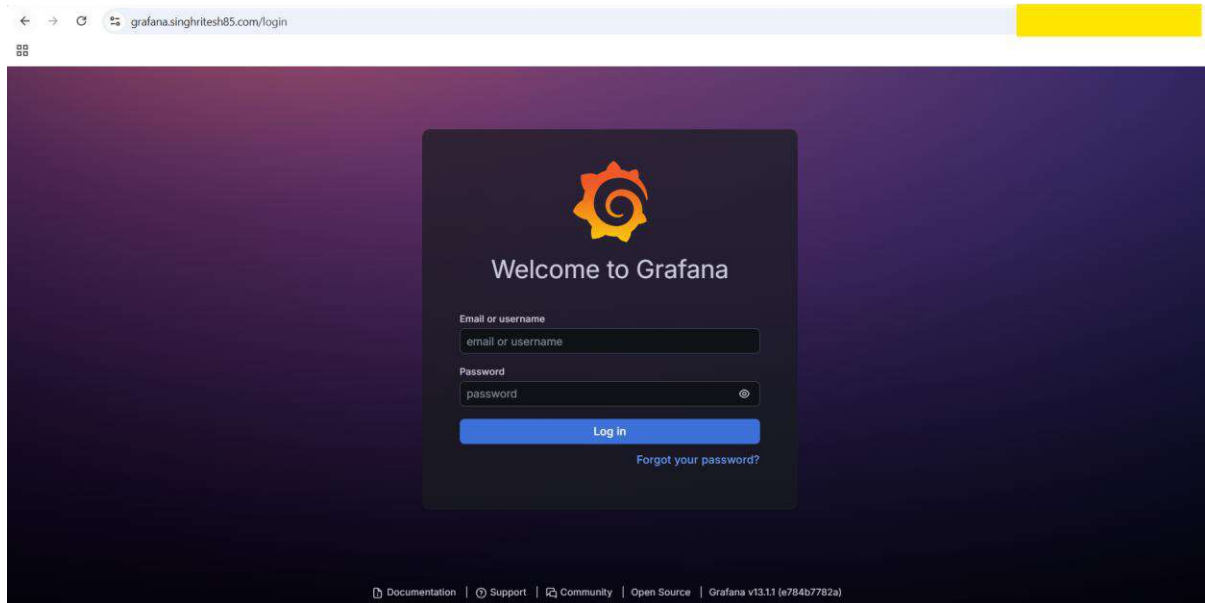
IPv4 Address 1 *: 34. [REDACTED] [Select](#)

Example: 192.0.2.91

[+ Add item](#)

[Create](#) [Cancel](#)

Finally, I was able to access Grafana using the URL <https://grafana.singhritesh85.com> as shown in the screenshot attached below.

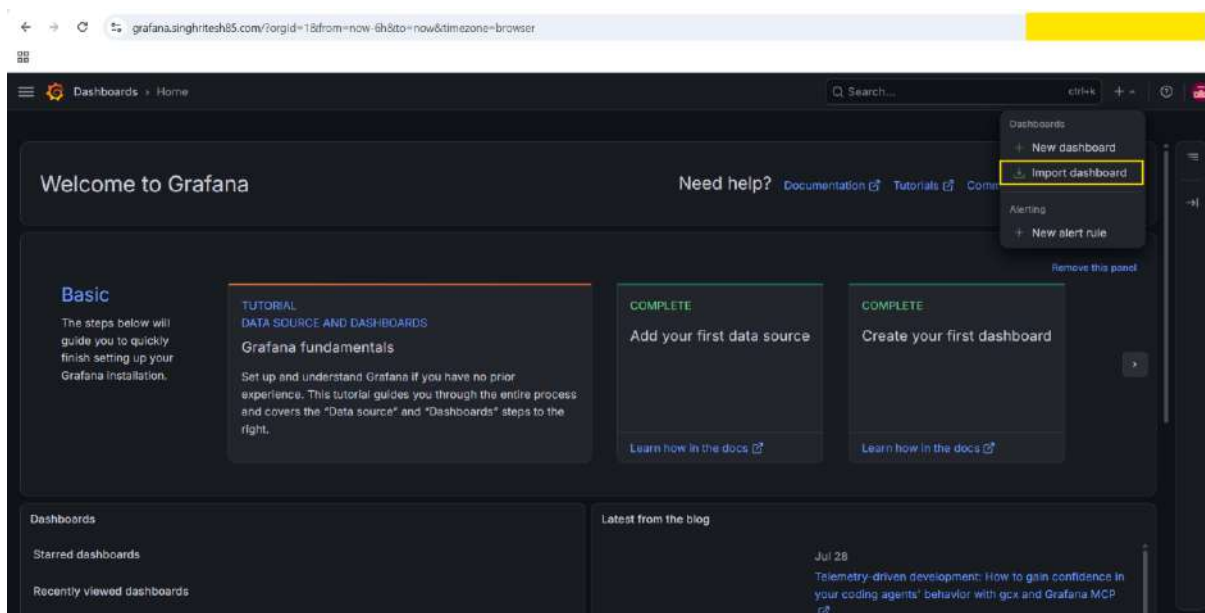


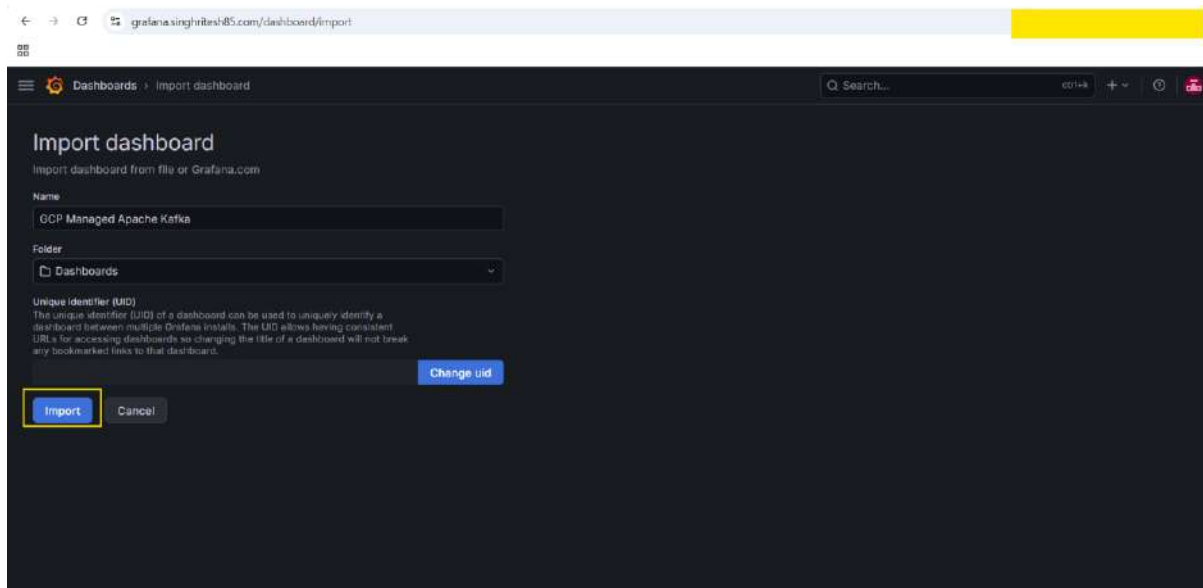
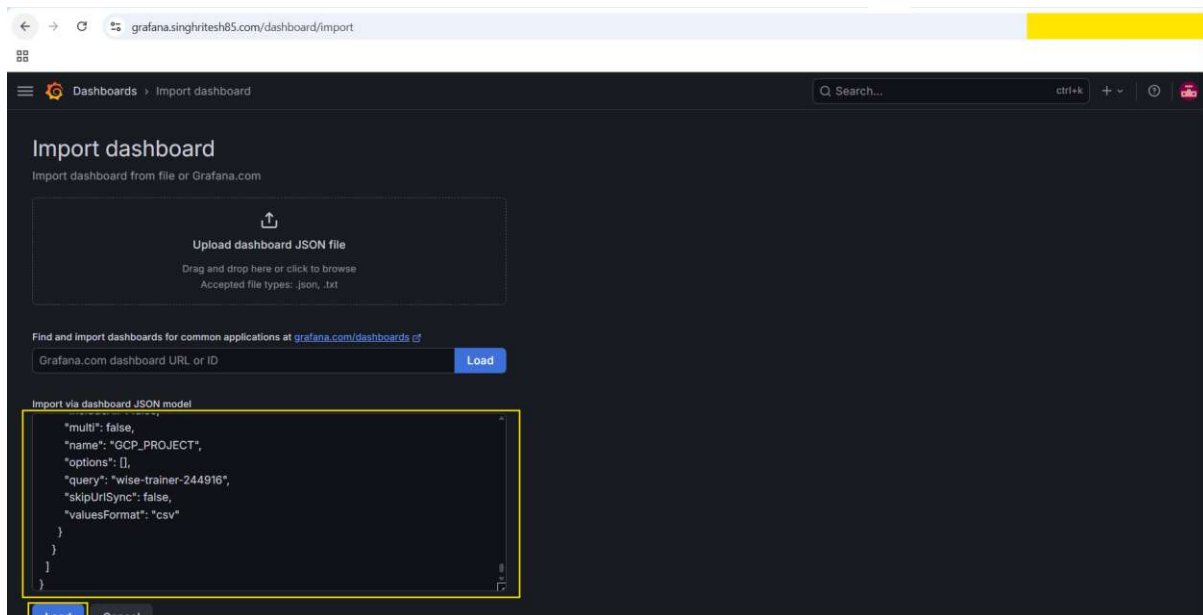
For the very first-time login into Grafana UI, you can obtain the admin user password using the command as shown in the screenshot attached below.

```
kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 --decode; echo
```

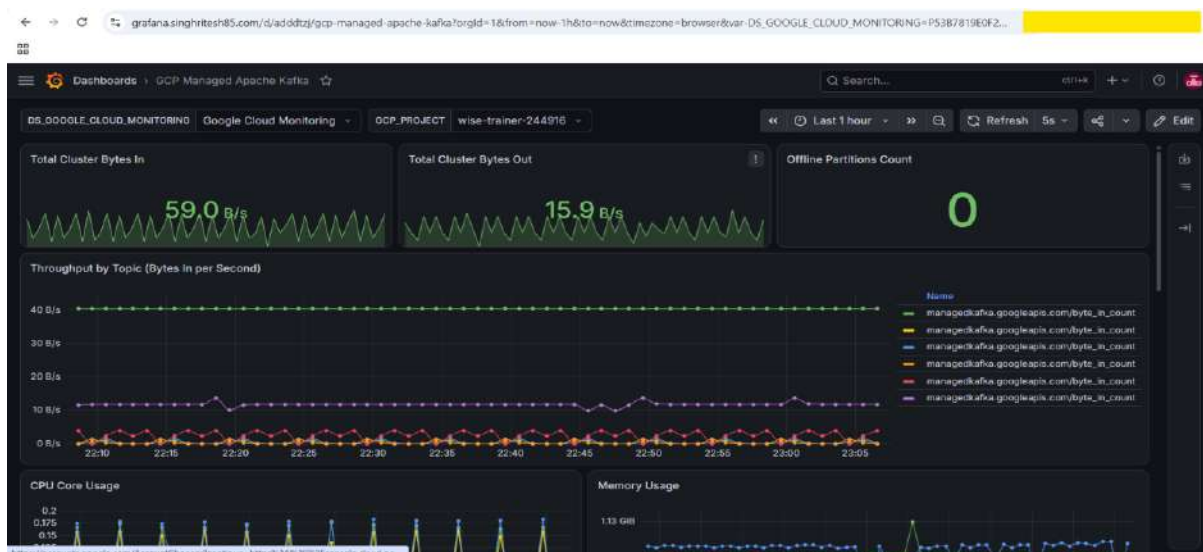
```
[gitlab-runner@gitlab-gitlab-runner ~]$ kubectl get secret --namespace monitoring kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 --decode; echo
```

After logged-in into Grafana UI, created the dashboard for GCP managed Apache Kafka using the Grafana JSON Model present in the GitHub Repo <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> at the path module-3/Section-3/grafana-json-model.json as shown the screenshot attached below.





Finally, you will see the Dashboard in Grafana UI as shown in the screenshot attached below.



GitHub Project Repo: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> (Project Path: - module-3/Section-3/)

Terraform Script: - <https://github.com/singhritesh85/DevOps-Project-using-Kafka-as-MessageQueue-the-GitOps-way.git> (Path: - module-3/Section-3/terraform-gcp-managed-apache-kafka-gitlab-server)

FluxCD GitHub Repo: - <https://github.com/singhritesh85/gcp-managed-apache-kafka-fluxcd.git>

Kubernetes Manifests file: - <https://github.com/singhritesh85/gcp-managed-apache-kafka.git>

A Note to Readers

Dear Readers,

First and foremost, thank you for your continued support and encouragement. Over the years, I have always enjoyed bringing new projects and practical learning resources to you. This time, however, it took me a little longer to publish a new project.

The reason is a joyful one. By God's grace, I was blessed with one of life's greatest gifts—I became a father. Embracing this new chapter brought wonderful moments, along with new responsibilities that naturally required my time and attention. As a result, the completion of this publication took longer than I had originally planned.

I sincerely appreciate your patience and understanding. Your support has always motivated me to continue sharing knowledge and practical experiences. I hope this publication proves to be informative, insightful, and valuable in your learning journey.

Thank you once again for being a part of this journey. I look forward to bringing you many more exciting projects in the future.

With gratitude,

Ritesh Kumar Singh